

# Bangladesh University of Engineering and Technology

Department of Computer Science & Engineering

---

## OOP Question Bank With Answers

Object Oriented Programming (C++ & Java)

CSE 107 (L-1/T-2) & CSE 201 (L-2/T-1)

---

Year Coverage: 2007–08 to 2023–24 |

Topics: C++ OOP, Inheritance, Templates, Java OOP, Multithreading & more

### Authors & Contributors

**Md Ratul Hasan** (2405009) — Question Curation & Organisation  
—  $\text{\LaTeX}$  Typesetting

**Claude AI** —  $\text{\LaTeX}$  Typesetting

# Table of Contents

---

## Part I — C++ Object Oriented Programming

---

- ▷ **S1.** Philosophy & Fundamentals of OOP
- ▷ **S2.** Classes, Objects & Access Specifiers
- ▷ **S3.** Inline Functions, References & Default Arguments
- ▷ **S4.** Friend Functions
- ▷ **S5.** Constructors, Destructors & Copy Constructor
- ▷ **S6.** Operator Overloading & Type Conversion
- ▷ **S7.** Inheritance & Polymorphism
- ▷ **S8.** Templates & STL
- ▷ **S9.** I/O Streams, Manipulators & File I/O

## Part II — Java Object Oriented Programming

---

- ▷ **S10.** Java Basics, Arrays & Strings
- ▷ **S11.** Java OOP — Classes, Inheritance & Polymorphism
- ▷ **S12.** Interfaces, Abstract Classes & Nested Classes
- ▷ **S13.** Exception Handling
- ▷ **S14.** Multithreading & Concurrency
- ▷ **S15.** Generics & Collections
- ▷ **S16.** Stream API, Lambda & Enums
- ▷ **S17.** File I/O & Serialization
- ▷ **S18.** Networking
- ▷ **S19.** Miscellaneous Java

BUET · CSE 107 / CSE 201

# Object Oriented Programming

Section S1

---

Philosophy & Fundamentals of OOP

## S1 — Philosophy & Fundamentals of OOP

**Q1.** Explain the key features of object-oriented programming (OOP). How does a class differ from a structure in OOP? (6+4 marks) [CSE 107 | 2023–24]

### Answer

#### Key Features of Object-Oriented Programming:

- 1. Encapsulation:** Bundling data (attributes) and the functions that operate on that data into a single unit called a *class*, and restricting direct access to some of the object's components via access specifiers (**private**, **protected**, **public**).
- 2. Abstraction:** Hiding implementation details and exposing only the necessary interface to the user. The user interacts with an object through its public methods without knowing the internal workings.
- 3. Inheritance:** A mechanism by which a new class (*derived class*) acquires the properties and behaviours of an existing class (*base class*), promoting code reuse.
- 4. Polymorphism:** The ability of a single interface to represent different underlying forms — achieved through function overloading (compile-time) and virtual functions (run-time).
- 5. Message Passing:** Objects communicate by calling each other's methods, analogous to sending messages.

#### Class vs. Structure in OOP (C++):

| Class                                                                                | Structure ( <b>struct</b> )                                                          |
|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Members are <b>private</b> by default.                                               | Members are <b>public</b> by default.                                                |
| Supports all OOP features: inheritance, encapsulation, polymorphism, access control. | In C++ also supports OOP features, but by convention used for plain data aggregates. |
| Typically used to model objects with both data and behaviour.                        | Typically used for lightweight, passive data containers (no or minimal methods).     |
| Inheritance default: <b>private</b> .                                                | Inheritance default: <b>public</b> .                                                 |

**Q2.** Explain the importance of using constructors and destructors in C++. Why and how does Java drop writing destructors in programs? (5+5 marks) [CSE 107 | 2023–24]

### Answer

#### Importance of Constructors in C++:

- Guarantee that every object starts in a *valid, well-defined state* — no uninitialized memory or dangling pointers.
- Can be overloaded to support multiple ways of creating an object (default, parameterised, copy constructors).
- Enable the *RAII* (Resource Acquisition Is Initialisation) idiom: resources such as file handles or heap memory are acquired in the constructor.

#### Importance of Destructors in C++:

- Automatically release resources when an object's lifetime ends, preventing *memory leaks* and *resource leaks*.
- Essential for classes that manage raw heap memory (`new/delete`), file I/O, network sockets, etc.
- Complete the RAII contract: what the constructor acquires, the destructor releases.

#### Why Java drops destructors:

Java uses **automatic garbage collection (GC)**. The JVM tracks all object references and automatically reclaims heap memory when an object is no longer reachable — the programmer never calls `delete`. Since memory management is automated, an explicit destructor is unnecessary.

Instead, Java provides:

- **`finalize()` (deprecated)**: Called by the GC before reclaiming an object, but its timing is non-deterministic and unreliable.
- **`try-with-resources` / `AutoCloseable`**: The recommended modern approach for deterministic release of non-memory resources (files, connections).

**Q3.** Describe why constructor overloading is possible while destructor overloading is not allowed. (11 marks) [CSE 107 | 2022–23]

#### Answer

**Constructor Overloading is possible** because:

- An object can be created in multiple ways — from no arguments, from specific values, from a copy of another object, etc.
- Each overloaded constructor has a **different parameter list**, so the compiler can unambiguously select the correct one based on how the object is created.
- This follows the same rules as regular function overloading.

```
1 class Point {  
2 public:
```

```

3     Point() : x(0), y(0) {}           // default
4     Point(int a, int b) : x(a), y(b) {} // parameterised
5     Point(const Point& p) : x(p.x), y(p.y) {} // copy
6 private:
7     int x, y;
8 };

```

**Destructor Overloading is NOT possible** because:

1. A destructor **takes no parameters** — it is always called with *no arguments* by the compiler when the object's lifetime ends. Without parameters, there is no basis for distinguishing multiple versions.
2. A destructor **returns nothing** — there is no return type to differentiate on either.
3. There is **only one way** an object's lifetime can end — the compiler must know exactly which destructor to call without any ambiguity. Multiple destructors would make this impossible.
4. A class can have **at most one destructor**, named `~ClassName()`, with no parameters.

**Q4.** Discuss the difference between a class and an object. Describe how C++ supports inheritance. (11 marks) [CSE 107 | 2021–22]

### Answer

#### Class vs. Object:

| Class                                                                                     | Object                                                      |
|-------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| A <i>blueprint</i> or <i>template</i> that defines the structure and behaviour of a type. | A concrete <i>instance</i> of a class that occupies memory. |
| Defined once; no memory allocated for the class itself.                                   | Each object has its own copy of the data members.           |
| Exists at compile time as a type definition.                                              | Created at run time via constructors.                       |
| Example: <code>class Car { ...};</code>                                                   | Example: <code>Car myCar;</code>                            |

#### How C++ supports Inheritance:

C++ supports inheritance using the colon (`:`) syntax in the class declaration, with an access specifier controlling how inherited members are exposed:

```

1     class Animal {                     // Base class
2     protected:
3         string name;
4     public:
5         Animal(string n) : name(n) {}

```

```

6     virtual void speak() { cout << name << " makes a sound\n"
          ; }
7 };
8
9 class Dog : public Animal {    // Derived class
10 public:
11     Dog(string n) : Animal(n) {}
12     void speak() override { cout << name << " barks\n"; }
13 };

```

#### Types of inheritance in C++:

- public inheritance: public/protected members remain public/protected in derived.
- protected inheritance: both become protected.
- private inheritance: both become private.
- C++ also uniquely supports **multiple inheritance**: `class C : public A, public B { };`

**Q5.** Explain abstraction with an appropriate example. Write down the properties of abstract class and abstract method. (5+3+2 marks) [CSE 107 | 2020–21]

#### Answer

**Abstraction** is the OOP principle of hiding internal implementation details and exposing only the essential features of an object to the outside world.

**Example:** A Car class exposes methods `start()`, `accelerate()`, and `brake()`. The driver does not need to know about fuel injection, engine timing, or brake hydraulics — those details are hidden.

```

1 class Car {
2     void fuelInjection() { /* hidden */ }    // private detail
3 public:
4     void start()          { fuelInjection(); /* ... */ }
5     void accelerate() { /* ... */ }
6     void brake()         { /* ... */ }
7 };

```

#### Properties of an Abstract Class:

- Contains at least one *pure virtual function* (= 0 in C++; declared **abstract** in Java).
- **Cannot be instantiated** — no objects can be created directly from an abstract class.
- Can have constructors, data members, and concrete (non-pure) methods.

- A derived class must override *all* pure virtual functions; otherwise the derived class is also abstract.

**Properties of an Abstract Method (Pure Virtual Function):**

- Has a declaration but **no definition** in the base class (e.g. `virtual void draw() = 0;`).
- Forces every concrete subclass to provide its own implementation.
- Provides a common interface across a class hierarchy.

**Q6.** What is the purpose of constructors and destructors? Write three special properties of a constructor that make it distinct from other member functions. **(10 marks)**  
[CSE 107 | 2016–17, 2020–21]

**Answer**

**Purpose of Constructors:** A constructor is a special member function that is automatically called when an object is created. Its purpose is to *initialise* the object's data members to valid starting values, allocate resources (e.g. heap memory), and set up any invariants the object must maintain.

**Purpose of Destructors:** A destructor is automatically called when an object goes out of scope or is explicitly deleted. Its purpose is to *release* resources held by the object (e.g. `delete` heap memory, close file handles, release locks) to prevent resource leaks.

**Three special properties of a constructor that distinguish it from other member functions:**

1. **Same name as the class:** A constructor must have exactly the same name as the class it belongs to. No other member function may share the class name.
2. **No return type (not even void):** A constructor does not declare or return any value. Specifying a return type is a compile error.
3. **Called automatically:** A constructor is invoked implicitly by the compiler whenever an object of the class is created — the programmer never calls it explicitly by name. (Contrast with ordinary member functions, which must be called explicitly.)

**Q7.** Differentiate between early binding and late binding with appropriate examples. **(8 marks)**  
[CSE 201 | 2014–15, 2013–14; CSE 107 | 2020–21]

↪ *Similar: CSE 201 | 2015–16, 2012–13, 2009–10, 2008–09; CSE 107 | 2017–18*



**Answer****Early Binding (Static / Compile-Time Binding):**

The compiler resolves the function call and links it to the correct function definition *at compile time*, based on the declared type of the object or pointer. This produces faster code since no run-time lookup is needed.

```

1  class Animal {
2  public:
3      void speak() { cout << "Animal speaks\n"; } // non-
        virtual
4  };
5  class Dog : public Animal {
6  public:
7      void speak() { cout << "Dog barks\n"; }
8  };
9
10 Animal* p = new Dog();
11 p->speak();    // Early binding: calls Animal::speak()
12                // because speak() is NOT virtual

```

**Late Binding (Dynamic / Run-Time Binding):**

The function call is resolved *at run time* based on the actual type of the object pointed to. Achieved in C++ via virtual functions and the virtual function table (vtable).

```

1  class Animal {
2  public:
3      virtual void speak() { cout << "Animal speaks\n"; } //
        virtual
4  };
5  class Dog : public Animal {
6  public:
7      void speak() override { cout << "Dog barks\n"; }
8  };
9
10 Animal* p = new Dog();
11 p->speak();    // Late binding: calls Dog::speak() at run time

```

|             | Early Binding | Late Binding        |
|-------------|---------------|---------------------|
| Resolution  | Compile time  | Run time            |
| Mechanism   | Direct call   | vtable lookup       |
| Speed       | Faster        | Slightly slower     |
| Flexibility | Less          | More (polymorphism) |
| Keyword     | (default)     | virtual             |

**Q8.** Which feature of C++ programming language do you like most and why? The

feature that you mention should not also be a feature of C programming language.  
(10 marks) [CSE 107 | 2019–20]

### Answer

#### Feature: Operator Overloading

Operator overloading allows existing C++ operators (+, -, ==, <<, etc.) to be redefined for user-defined types, making code significantly more intuitive and readable. This feature does not exist in C.

#### Why it is the most useful:

- It allows user-defined types to *behave like built-in types*. For example, adding two `Complex` numbers can be written as `c1 + c2` rather than `add(c1, c2)`.
- It makes mathematical classes (`Matrix`, `Vector`, `Fraction`) self-documenting and natural to use.
- It integrates seamlessly with the standard library (e.g. overloading `<<` for `ostream` to print custom objects).

#### Example:

```
1  class Complex {
2      double re, im;
3  public:
4      Complex(double r, double i) : re(r), im(i) {}
5      Complex operator+(const Complex& o) const {
6          return Complex(re + o.re, im + o.im);
7      }
8      friend ostream& operator<<(ostream& os, const Complex& c)
9          {
10         return os << c.re << "+" << c.im << "i";
11     }
12 };
13 // Usage:
14 Complex c1(1,2), c2(3,4);
15 cout << c1 + c2;    // prints: 4+6i
```

Q9. Write a short note on “function overloading”. (10 marks) [CSE 107 | 2019–20]

### Answer

**Function Overloading** is the ability to define multiple functions with the *same name* in the same scope, provided they differ in their **parameter list** (number, type, or order of parameters). The compiler selects the correct version at *compile time* based on the arguments supplied — this is a form of *compile-time (static) polymorphism*.

#### Rules:

- Overloaded functions must differ in their parameter lists.

- The return type *alone* is **not** sufficient to distinguish overloads.
- **const**-ness of the implicit **this** pointer (for member functions) can also distinguish overloads.

**Example:**

```

1  int    add(int a, int b)        { return a + b; }
2  double add(double a, double b) { return a + b; }
3  int    add(int a, int b, int c){ return a + b + c; }
4
5  int    r1 = add(1, 2);          // calls first version
6  double r2 = add(1.5, 2.5);      // calls second version
7  int    r3 = add(1, 2, 3);       // calls third version

```

**Advantages:**

- Improves code readability — one intuitive name for logically related operations.
- Eliminates the need for functions like `addInt`, `addDouble`, etc.

**Q10.** What does a default constructor (generated by the compiler) perform? What are the scenarios when a copy constructor is called? **(10 marks)** [CSE 107 | 2019–20]

**Answer**

**What the compiler-generated default constructor does:**

- For **built-in types** (int, double, pointer, etc.): leaves them **uninitialised** (their values are indeterminate).
- For **class-type members**: calls the default constructor of each member.
- For **base class sub-objects**: calls the base class default constructor.

The compiler generates this constructor only if the programmer defines *no* constructor at all.

**Scenarios when the copy constructor is called:**

**1. Object initialisation from another object of the same class:**

```

1  MyClass a;
2  MyClass b = a;    // copy constructor called
3  MyClass c(a);     // copy constructor called

```

**2. Passing an object by value to a function:**

```

1  void f(MyClass x) { }    // copy constructor called for x
2  f(a);

```

### 3. Returning an object by value from a function:

```

1 MyClass g() {
2     MyClass tmp;
3     return tmp;    // copy constructor may be called (if
                     RVO absent)
4 }
```

### 4. Throwing or catching an exception by value (the exception object is copy-constructed).

**Q11.** What are the syntactic differences between C and C++ programming? (10 marks)  
[CSE 201 | 2014–15]

#### Answer

| C                                                                   | C++                                                                                     |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Input/output via <code>printf/scanf</code> .                        | Uses <code>cout/cin</code> with <code>&lt;&lt;/&gt;&gt;</code> .                        |
| No references; only pointers.                                       | Supports reference variables ( <code>int&amp; r = x;</code> ).                          |
| <code>struct</code> cannot contain functions.                       | <code>struct</code> and <code>class</code> can contain member functions.                |
| No function overloading.                                            | Function and operator overloading supported.                                            |
| No default arguments.                                               | Default arguments in function declarations supported.                                   |
| <code>void*</code> can be implicitly converted to any pointer type. | <code>void*</code> requires explicit cast.                                              |
| No <code>new/delete</code> ; uses <code>malloc/free</code> .        | <code>new/delete</code> for type-safe dynamic allocation.                               |
| No <code>bool</code> type (uses <code>int</code> for boolean).      | Native <code>bool</code> type with <code>true/false</code> .                            |
| No <code>inline</code> keyword with class support.                  | <code>inline</code> functions; functions defined inside class are automatically inline. |
| No namespace.                                                       | <code>namespace</code> supported ( <code>using namespace std;</code> ).                 |
| No templates.                                                       | Function and class templates supported.                                                 |

**Q12.** Define automatic in-line function. What are the advantages of using in-line function over parameterized macros? (5 marks) [CSE 201 | 2013–14]

#### Answer

**Automatic Inline Function:** An *inline function* is a function prefixed with the `inline` keyword (or defined inside a class body) that *suggests* to the compiler to replace each function call with the actual function body at the call site, eliminating

function-call overhead (stack frame setup, jump, return).

```
1 inline int square(int x) { return x * x; }
2 // The compiler may replace: int r = square(5);
3 // with:                      int r = 5 * 5;
```

**Advantages over parameterised macros (#define):**

1. **Type safety:** Inline functions undergo full type checking by the compiler. Macros are textual substitutions with no type checking, leading to subtle bugs.

```
1 #define SQ(x) ((x)*(x))    // SQ(1.5+0.5) -> (1.5+0.5)
                             // *(1.5+0.5)=4 OK
2                             // but SQ(++a) -> (++a)*(++a)
                             // -- a incremented TWICE
3 inline int sq(int x) { return x*x; } // ++a evaluated once
                             // , passed as value
```

2. **No multiple-evaluation side effects:** Arguments are evaluated exactly once; macros can evaluate arguments multiple times.
3. **Scope and namespaces:** Inline functions respect C++ scoping rules; macros are global text substitutions with no scope.
4. **Debuggable:** Inline functions appear in debugger symbol tables; macros do not.

**Q13.** What is an in-line function? What are its advantages and disadvantages? Why might the following function not be in-lined by your compiler?

```
1 void f1() { for (int i = 0; i < 10; i++) cout << i; }
```

(8 marks)

[CSE 201 | 2011–12]

### Answer

**Inline Function:** A function declared with the `inline` keyword that *requests* the compiler to insert its body directly at every call site instead of generating a regular function call.

**Advantages:**

- Eliminates function-call overhead (no stack frame, no jump).
- Can enable further optimisations (constant folding, etc.).
- Full type checking, unlike macros.

**Disadvantages:**

- **Code bloat:** If a large function is inlined at many call sites, the binary size grows.

- Increased compilation time.
- The `inline` keyword is only a *hint*; the compiler may ignore it.

#### Why `f1()` might not be inlined:

```
1 void f1() { for (int i = 0; i < 10; i++) cout << i; }
```

Most compilers refuse to inline functions that contain **loops**, because:

- Inlining a loop body at every call site causes significant *code bloat*.
- The performance benefit of inlining is minimal when the function body itself is non-trivial (the loop iterations dominate the cost, not the call overhead).
- Many compilers have heuristics that exclude functions with loops, recursion, or large bodies from automatic inlining.

**Q14.** Differentiate between real-time polymorphism and run-time polymorphism with appropriate examples. **(10 marks)** [CSE 201 | 2010–11]

**Q15.** What is compile-time polymorphism and run-time polymorphism? Differentiate with examples. **(13 marks)** [CSE 201 | 2009–10]

↪ *Similar: CSE 201 | 2010–11; CSE 107 | 2019–20*

### Answer

#### Compile-Time Polymorphism (Static Polymorphism):

Resolved by the compiler before the program runs. Achieved through:

- Function overloading
- Operator overloading
- Template instantiation

```
1 // Function overloading -- resolved at compile time
2 int    area(int s)           { return s * s; }
3 double area(double r)       { return 3.14159 * r * r; }
4 int    area(int l, int w)    { return l * w; }
5
6 area(5);                    // compiler picks first version
7 area(3.0);                  // compiler picks second version
8 area(4, 6);                 // compiler picks third version
```

#### Run-Time Polymorphism (Dynamic Polymorphism):

Resolved while the program is running, based on the actual type of the object. Achieved through **virtual functions** and base class pointers/references.

```
1 class Shape {
2 public:
3     virtual double area() const = 0; // pure virtual
4 };
5 class Circle : public Shape {
6     double r;
7 public:
8     Circle(double r) : r(r) {}
9     double area() const override { return 3.14159 * r * r; }
10 };
11 class Rectangle : public Shape {
12     double l, w;
13 public:
14     Rectangle(double l, double w) : l(l), w(w) {}
15     double area() const override { return l * w; }
16 };
17
18 Shape* s1 = new Circle(3.0);
19 Shape* s2 = new Rectangle(4.0, 6.0);
20 cout << s1->area(); // calls Circle::area() at run time
21 cout << s2->area(); // calls Rectangle::area() at run time
```

**Q16.** What is Object-Oriented programming? How are data and functions organized in an Object-Oriented program? (6 marks) [CSE 201 | 2007–08]

### Answer

**Object-Oriented Programming (OOP)** is a programming paradigm that organises software design around *objects* rather than functions and logic. An object is an instance of a *class* that contains both data (state) and the functions (behaviour) that operate on that data.

#### Organisation of data and functions in OOP:

- **Data** (member variables / attributes) and **functions** (member functions / methods) are grouped together inside a **class**. This grouping is called *encapsulation*.
- Access to data is controlled by *access specifiers*:
  - **private** — accessible only within the class.
  - **protected** — accessible within the class and its derived classes.
  - **public** — accessible from anywhere.
- Data is typically kept **private** (hidden), while functions are **public** (interface). External code interacts with an object only through its public methods.

- Each *object* (instance) has its own copy of the data members, but all objects of the same class share the same method definitions.



BUET · CSE 107 / CSE 201

# Object Oriented Programming

Section S2

---

Classes, Objects & Access Specifiers

## S2 — Classes, Objects & Access Specifiers

**Q1.** A `Student` class has a private variable to store an ID, a constructor without parameters to initialize the ID, and a private static variable. Write the class definition for `Student` and write code in `main()` to create an array of 120 `Student` objects using the `new` operator, where the IDs are initialized as 1, 2, ..., 120 respectively in the constructor. **(10 marks)** [CSE 107 | Jan 2021]

### Answer

```

1  #include <iostream>
2  using namespace std;
3
4  class Student {
5      int id;
6      static int nextId;    // private static: shared ID counter
7  public:
8      Student() { id = nextId++; }    // each object gets next
          ID
9      int getId() const { return id; }
10 };
11
12 // Initialise static member outside the class
13 int Student::nextId = 1;
14
15 int main() {
16     // Create array of 120 Student objects on the heap
17     Student* students = new Student[120];
18
19     // Verify first and last IDs
20     cout << "First ID: " << students[0].getId() << endl; //
          1
21     cout << "Last ID:  " << students[119].getId() << endl;
          // 120
22
23     delete[] students;    // release heap memory
24     return 0;
25 }
```

### How it works:

- `nextId` is static and initialised to 1 outside the class.
- Each time a `Student` object is constructed (including array element construction), the default constructor assigns `id = nextId` and post-increments `nextId`.
- After `new Student[120]`, the 120 objects have IDs 1 through 120 in order.

**Q2.** Imagine a tollbooth at a bridge. Cars passing by the booth are expected to pay

a 50-cent toll. Mostly they do, but sometimes a car goes by without paying. The tollbooth keeps track of the number of cars that have gone by, and of the total amount of money collected.

Model this tollbooth with a class called `tollbooth`. The three data items are a list of string to hold the registration numbers of the cars passing through the bridge, a type `unsigned int` to hold the total number of cars, and a type `double` to hold the total amount of money collected. A constructor initializes both of these to 0. A member function called `payingCar()` adds the car registration number in the list, increments the car total and adds 0.50 to the cash total. Another function, called `nopayCar()`, adds the car registration number, increments the car total but adds nothing to the cash total. Finally, a member function called `display()` displays the list of cars' registration numbers and the two totals. Make appropriate member functions `const`. Use proper naming convention and setter-getter functions. (15 marks) [CSE 107 | 2020–21]

### Answer

```
1  #include <iostream>
2  #include <list>
3  #include <string>
4  using namespace std;
5
6  class tollbooth {
7  private:
8      list<string> registrations; // list of strings for
9                                  registration numbers
10     unsigned int totalCars;      // total number of cars
11     double totalCash;           // total amount of money
12                                   collected
13
14 public:
15     // Constructor initializes totals to 0
16     tollbooth() {
17         totalCars = 0;
18         totalCash = 0.0;
19     }
20
21     // Setter and Getter functions
22     void setRegistrations(const list<string>& regs) {
23         registrations = regs; }
24     list<string> getRegistrations() const { return
25         registrations; }
26
27     void setTotalCars(unsigned int cars) { totalCars = cars;
28         }
29     unsigned int getTotalCars() const { return totalCars; }
30
31     void setTotalCash(double cash) { totalCash = cash; }
```

```
27     double getTotalCash() const { return totalCash; }
28
29     // Car pays the 50-cent toll
30     void payingCar(const string& reg) {
31         registrations.push_back(reg);
32         totalCars++;
33         totalCash += 0.50;
34     }
35
36     // Car passes without paying
37     void nopayCar(const string& reg) {
38         registrations.push_back(reg);
39         totalCars++;
40     }
41
42     // Display summary -- does not modify state, so const
43     void display() const {
44         cout << "Total cars: " << totalCars << endl;
45         cout << "Total cash: $" << totalCash << endl;
46         cout << "Cars' Registration Numbers:" << endl;
47         for (const string& r : registrations) {
48             cout << "    " << r << endl;
49         }
50     }
51 };
52
53 int main() {
54     tollbooth booth;
55
56     booth.payingCar("ABC-111");
57     booth.payingCar("XYZ-222");
58     booth.nopayCar("BAD-999");
59
60     booth.display();
61
62     return 0;
63 }
```

**Const correctness:** Getter methods and `display()` are declared `const` because they only read data without modifying the object's internal state.

**Q3.** What is the difference between class and structure? Why are both of them necessary in C++ programming? (6 marks) [CSE 201 | 2015–16]

**Answer**

| Class                                                                           | Structure                                                               |
|---------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Members are <b>private</b> by default.                                          | Members are <b>public</b> by default.                                   |
| Inheritance default: <b>private</b> .                                           | Inheritance default: <b>public</b> .                                    |
| Conventionally used for objects with both data and behaviour (full OOP design). | Conventionally used for plain data records with little or no behaviour. |

**Why both are necessary:**

- **Backward compatibility with C:** C code using `struct` compiles unchanged in C++. Removing `struct` would break C compatibility.
- **Semantic clarity:** Using `struct` signals to readers that a type is a simple data aggregate (e.g. a `Point`, a `Color`), while `class` signals a fully encapsulated object. This is a *convention* that improves code readability.

**Q4.** What is a static member? Why are they used? Write down the rules of using static variable and static method in a class. (7 marks) [CSE 201 | 2015–16]

↪ *Similar: CSE 201 | 2012–13*

**Answer**

**Static Member:** A class member declared with the `static` keyword. It belongs to the *class itself* rather than to any particular object — all objects of the class share a single copy of it.

**Why used:**

- To maintain data that is *shared* across all instances (e.g. object count, configuration).
- To provide utility functions that do not require an object.

**Rules for static variables:**

- Declared inside the class with `static`; must be **defined** (and optionally initialised) *outside* the class in exactly one translation unit.
- Only one copy exists regardless of how many objects are created.
- Accessible as `ClassName::variable` without an object.
- Lifetime is the entire program duration.

**Rules for static methods:**

- Can access only *static* members (no access to `this` pointer or non-static members).

- Can be called without creating an object: `ClassName::method()`.
- Cannot be virtual or const.

```
1 class Counter {
2     static int count;           // declaration
3 public:
4     Counter() { count++; }
5     ~Counter() { count--; }
6     static int getCount() { return count; } // static method
7 };
8 int Counter::count = 0;        // definition outside class
```

**Q5.** Write a class named `MyClass` with a variable named `objectCount` that keeps track of the number of objects created. When the total number of objects is greater than 100, reinitialize `objectCount` to 0. (5 marks) [CSE 201 | 2014–15]

#### Answer

```
1 #include <iostream>
2 using namespace std;
3
4 class MyClass {
5     static int objectCount;
6 public:
7     MyClass() {
8         objectCount++;
9         if (objectCount > 100)
10             objectCount = 0;
11     }
12     static int getObjectCount() { return objectCount; }
13 };
14
15 // Definition outside the class (required for static members)
16 int MyClass::objectCount = 0;
17
18 int main() {
19     MyClass a, b, c;
20     cout << MyClass::getObjectCount() << endl; // 3
21     return 0;
22 }
```

#### Key points:

- `objectCount` is static — shared by all objects; incremented in the constructor each time a new object is created.
- The `if (objectCount > 100)` guard resets the counter to 0 whenever it exceeds 100.

- The static member is defined (and initialised to 0) outside the class body.

**Q6.** What are the properties of static variables and methods? Explain dynamic method dispatch with example. **(6+9 marks)** [CSE 201 | 2013–14]

### Answer — Static Variables/Methods and Dynamic Method Dispatch

#### Properties of static variables:

- Shared by **all objects** of the class — only one copy exists.
- Initialized once when the class is loaded.
- Accessible via `ClassName::variable` without any object.
- Lifetime is the entire program.

#### Properties of static methods:

- Belong to the class, not to any instance.
- Can only access static members directly.
- No `this` pointer is available inside them.

```
1 class Counter {
2     static int count;
3 public:
4     Counter() { count++; }
5     static int getCount() { return count; }
6 };
7 int Counter::count = 0;
8
9 // Counter::getCount() -- no object needed
```

#### Dynamic Method Dispatch:

The mechanism by which a virtual function call is resolved at **run time** based on the actual type of the object, not the declared type of the pointer/reference. The compiler uses a *virtual table* (vtable) per class.

```
1 class Shape {
2 public:
3     virtual void draw() { cout << "Shape" << endl; }
4 };
5 class Circle : public Shape {
6 public:
7     void draw() override { cout << "Circle" << endl; }
8 };
9 class Square : public Shape {
10 public:
11     void draw() override { cout << "Square" << endl; }
```

```

12 };
13
14 int main() {
15     Shape* s;
16     s = new Circle(); s->draw(); // Circle -- run-time
        dispatch
17     s = new Square(); s->draw(); // Square -- run-time
        dispatch
18 }

```

**Q7.** Explain why the `protected` access specifier is needed in C++ programming. **(3 marks)** [CSE 201 | 2013–14]

### Answer

#### When we use `protected`:

We use the `protected` access specifier for a class member when we want it to be:

- **Inaccessible** from outside the class (like `private`), AND
- **Accessible** to *derived classes* (unlike `private`).

#### Why it is needed:

Without `protected`, a derived class would have no access to the base class's internal data unless it was made `public` (which would expose it to everyone — breaking encapsulation). `protected` achieves a middle ground: internal to the hierarchy, hidden from the outside world.

```

1  class Animal {
2      protected:
3          string name; // accessible to derived classes, not to
                        outside
4          int age;
5      public:
6          Animal(string n, int a) : name(n), age(a) {}
7      };
8
9      class Dog : public Animal {
10         public:
11             Dog(string n, int a) : Animal(n, a) {}
12             void bark() const {
13                 // Can access 'name' because it is protected:
14                 cout << name << " says: Woof!\n";
15             }
16     };
17
18     int main() {
19         Dog d("Rex", 3);

```



```

20     d.bark();           // OK
21     // d.name;         // ERROR: 'name' is protected
22 }

```

**Q8.** How can I call a member function of a class without creating any object of that class? Explain. (5 marks) [CSE 201 | 2012–13]

### Answer

The only way to call a member function of a class *without creating an object* is to declare it as a **static member function**.

A static member function does not have a **this** pointer and is associated with the class itself, not with any instance. It can be called using the class name and the scope resolution operator:

```

1  class MathUtils {
2  public:
3      static int square(int x) { return x * x; }
4      static double pi()      { return 3.14159265; }
5  };
6
7  int main() {
8      // Called without any object:
9      int s = MathUtils::square(5);    // s = 25
10     double p = MathUtils::pi();      // p = 3.14159...
11     return 0;
12 }

```

**Restrictions:** A static member function can only access *static* data members and call other *static* member functions. It cannot access non-static members because there is no object (and hence no **this** pointer) associated with the call.

**Q9.** Write the detail definition of a class **Pentagon** that will allow maximum five objects of the class to be created. After the fifth object, any attempt to create a new object will return the reference to the fifth object. (12 marks) [CSE 201 | 2009–10]

### Answer

```

1  #include <iostream>
2  using namespace std;
3
4  class Pentagon {
5      static int count;           // number of objects created so
                                  // far
6      static Pentagon* last;      // pointer to the 5th (last)
                                  // object
7

```

```

8     Pentagon() { count++; } // private: can't call new
        directly
9
10    public:
11        // Factory method replaces the constructor
12        static Pentagon* create() {
13            if (count < 5) {
14                last = new Pentagon();
15                return last;
16            }
17            // 5 already exist: return the 5th object
18            return last;
19        }
20        static int getCount() { return count; }
21    };
22
23    int      Pentagon::count = 0;
24    Pentagon* Pentagon::last  = nullptr;
25
26    int main() {
27        Pentagon* p[7];
28        for (int i = 0; i < 7; i++)
29            p[i] = Pentagon::create();
30
31        cout << Pentagon::getCount() << endl;           // 5
32        cout << (p[4] == p[5] ? "same" : "diff") << endl; // same
33        cout << (p[4] == p[6] ? "same" : "diff") << endl; // same
34        return 0;
35    }

```

**Design:**

- The constructor is `private` to prevent direct instantiation with `new Pentagon()`.
- The `static` factory method `create()` is the only way to obtain a `Pentagon`.
- After the 5th object, `create()` returns a pointer to the same 5th object every time.

**Q10.** Can constructors/destructors be declared as `public`, `private`, or `protected`? What will be the problems if a constructor/destructor is declared as `private`? (6 marks)  
[CSE 201 | 2009–10]

**Answer**

Yes, constructors and destructors can be declared as `public`, `private`, or `protected`.

**Problems with a private constructor:**

- Objects **cannot be created from outside the class** using `new` or stack allocation.
- This is intentional in the **Singleton pattern** and **Factory pattern** — creation is controlled through a static factory method (as in the **Pentagon** example).
- Derived classes **cannot call** a private constructor of the base class directly, preventing uninherited construction.

**Problems with a private destructor:**

- Objects **cannot be destroyed on the stack** — the compiler calls the destructor when a stack object goes out of scope, which fails if it is private.
- `delete ptr` from outside the class will fail.
- The class must provide a public `destroy()` method that calls `delete this` internally.
- Useful in reference-counted objects where the object controls its own lifetime.

**protected constructor/destructor:** Allows creation/destruction only within the class itself and its derived classes — useful for abstract base classes.

**Q11.** Can a static method of a class use the `this` pointer? Why or why not? (4 marks)  
[CSE 201 | 2009–10]

**Answer**

No, a static member function *cannot* use the `this` pointer.

**Why not:**

The `this` pointer is an implicit parameter automatically passed to every *non-static* member function. It points to the specific object on which the function is being called.

A **static** member function, however, is associated with the *class* itself, not with any particular object. It can be called without creating an object:

```
1 MyClass::staticMethod();    // no object involved
```

Because there is no object in the picture, there is nothing for `this` to point to. The compiler therefore does not pass a `this` pointer to static functions, and any attempt to use `this` inside a static method is a **compile error**:

```
1 class MyClass {
2     int x;
3     static int s;
4 public:
5     static void f() {
```

```

6         s = 10;          // OK -- accesses static member
7         // x = 5;        // ERROR: 'this' is unavailable in
                        static function
8         // this->x=5; // ERROR: 'this' is unavailable
9     }
10 };

```

**Q12.** Write a class named `SelfCounter` that can count the number of instances in a program. Include a member function `getObjectCount()` which returns the number of objects created. Also write a `main` function to illustrate its functionality. (7 marks) [CSE 201 | 2008–09]

### Answer

```

1  #include <iostream>
2  using namespace std;
3
4  class SelfCounter {
5      static int count;    // shared count of live objects
6  public:
7      SelfCounter()        { count++; }    // created
8      SelfCounter(const SelfCounter&) { count++; } // copy
9      ~SelfCounter()       { count--; }    // destroyed
10
11     static int getObjectCount() { return count; }
12 };
13 int SelfCounter::count = 0;
14
15 int main() {
16     cout << SelfCounter::getObjectCount() << endl; // 0
17
18     SelfCounter a, b;
19     cout << SelfCounter::getObjectCount() << endl; // 2
20
21     {
22         SelfCounter c, d, e;
23         cout << SelfCounter::getObjectCount() << endl; // 5
24     } // c, d, e destroyed here
25
26     cout << SelfCounter::getObjectCount() << endl; // 2
27     return 0;
28 }

```

**Note:** The copy constructor also increments count because copy-constructing an object creates a new live instance. The destructor decrements the count when any object (original or copy) is destroyed.

**Q13.** Explain the usage of `const` and `mutable` keywords with examples. (7 marks) [CSE 201 | 2008–09]

### Answer

**const keyword:**

1. **Const variable:** Value cannot be changed after initialisation.

```
1  const int MAX = 100;    // MAX cannot be modified
```

2. **Const member function:** Does not modify any data member of the object. Can be called on `const` objects.

```
1  class Circle {
2      double radius;
3  public:
4      Circle(double r) : radius(r) {}
5      double area() const { return 3.14159 * radius * radius
6                          ; }
7      // area() cannot modify 'radius' or any other member
8  };
```

3. **Const pointer / reference parameter:** Prevents the function from modifying the passed argument.

```
1  void print(const string& s) { cout << s; }
```

**mutable keyword:**

A data member declared `mutable` can be modified even inside a `const` member function. It is used for members that represent *implementation state* (caches, locks, call counts) rather than *logical state*.

```
1  class Database {
2      mutable int queryCount;    // mutable: can change in const
3                                  functions
4      string data;
5  public:
6      Database() : queryCount(0) {}
7      string fetch(int id) const {
8          queryCount++;           // allowed because queryCount
9                                  is mutable
10         return data;
11     }
12     int getQueryCount() const { return queryCount; }
13 };
```

**Q14.** What are the applications of `bool` and `wchar_t` data types? Explain with examples.

(5 marks)

[CSE 201 | 2007–08]

**Answer****bool data type:**

Introduced in C++ (not in C) to represent Boolean truth values. Holds exactly one of two values: `true` (1) or `false` (0).

**Applications:**

- Flags and condition variables.
- Return type of predicate functions.
- Control flow conditions.

```
1 bool isPrime(int n) {
2     if (n < 2) return false;
3     for (int i = 2; i * i <= n; i++)
4         if (n % i == 0) return false;
5     return true;
6 }
7 bool found = false;
8 if (isPrime(17)) found = true;
```

**wchar\_t data type:**

A wide-character type designed to hold characters from large character sets (Unicode, multi-byte encodings) that cannot fit in a single `char` (1 byte). Typically 2 bytes (Windows/UTF-16) or 4 bytes (Linux/UTF-32).

**Applications:**

- Internationalisation (i18n): storing and processing text in non-ASCII scripts (Chinese, Arabic, Bengali, etc.).
- Windows API programming (LPCWSTR, LPWSTR).

```
1 #include <iostream>
2 #include <wchar>
3 using namespace std;
4
5 int main() {
6     wchar_t ch = L'A'; // wide character literal
7     wchar_t str[] = L"Hello, \u09AC\u09BE\u0982\u09B2\u09BE";
8     // "Hello, \u09AC\u09BE\u0982\u09B2\u09BE" (Bengali)
9     wcout << str << endl;
10    return 0;
11 }
```

BUET · CSE 107 / CSE 201

# Object Oriented Programming

Section S3

---

Inline Functions, References & Default Arguments

## S3 — Inline Functions, References & Default Arguments

**Q1.** What is inline function? How does it differ from automatic inline function? “Inline specifier is a request, not a command” — explain. **(3+2+5 marks)** [CSE 107 | 2020–21]

↪ *Similar: CSE 107 | 2017–18*

### Answer — Inline function vs automatic inline; request not command

**Inline Function:** A function declared with the `inline` keyword explicitly requests the compiler to expand the function body at every call site instead of generating a normal function call.

```
1 inline int square(int x) { return x * x; }
```

**Automatic Inline Function:** Any function defined *inside the class body* is implicitly treated as inline by the compiler — no `inline` keyword is needed.

```
1 class Circle {
2     double r;
3 public:
4     double area() { return 3.14159 * r * r; } // automatic
5     inline
6 };
```

The key difference: **explicit inline** is written outside the class with the keyword; **automatic inline** happens implicitly for in-class definitions.

**“Inline specifier is a request, not a command”:**

The `inline` keyword is only a *hint* to the compiler. The compiler is free to **ignore** it and generate a normal function call if any of the following conditions apply:

- The function body contains a loop, recursion, or `switch` statement.
- The function is too large (exceeds the compiler’s size threshold).
- The address of the function is taken.
- The compiler’s optimisation level is too low.

Conversely, the compiler may inline a function even *without* the keyword if it judges inlining beneficial (e.g. with `-O2` flags).

**Q2.** Write three different ways to declare an object of the C++ `string` class with examples. **(8 marks)** [CSE 107 | Jan 2021]

### Answer — Three ways to declare a string object

```
1 #include <iostream>
2 #include <string>
```



```

3  using namespace std;
4
5  int main() {
6      // Way 1: Default constructor (empty string)
7      string s1;
8      s1 = "Hello";
9      cout << s1 << "\n";    // Hello
10
11     // Way 2: Initialise with a string literal
12     string s2 = "World";
13     cout << s2 << "\n";    // World
14
15     // Way 3: Copy from another string object
16     string s3(s2);
17     cout << s3 << "\n";    // World
18
19     // Bonus: n copies of a character
20     string s4(5, '*');
21     cout << s4 << "\n";    // *****
22
23     return 0;
24 }
    
```

**Q3.** Two overloaded functions are declared as follows:

```

1  void space(char ch) { }
2  void space(char ch, int count) { }
    
```

Write down the `main()` function that declares two variables for storing the addresses of the above two functions and makes function calls using those variables. What problem arises if the second function is declared as `void space(char ch, int count = 0)`? (7 marks) [CSE 107 | 2020–21; CSE 201 | 2014–15]

**Answer —** Function pointers for overloaded `space()`; default arg ambiguity

```

1  #include <iostream>
2  using namespace std;
3
4  void space(char ch)                { cout << ch << "\n"; }
5  void space(char ch, int count) {
6      for (int i = 0; i < count; i++) cout << ch;
7      cout << "\n";
8  }
9
10 int main() {
11     void (*fp1)(char)              = space;    // first version
    
```

```

12     void (*fp2)(char, int) = space;    // second version
13
14     fp1('*');                          // *
15     fp2('-', 5);                       // -----
16     return 0;
17 }

```

**Problem** if second declared as `void space(char ch, int count = 0)`:

Both `space(char)` and `space(char, int=0)` match a one-argument call. When the compiler tries to resolve `fp1 = space`, it finds two equally valid matches and reports an **ambiguity error**.

**Q4.** "If a function returns a reference of any type, then it can be used both as an l-value and r-value" — explain with an example program. **(15 marks)**  
[CSE 107 | 2019–20, 2022–23]

#### Answer — Function as l-value and r-value

A function can be used as both an l-value and an r-value when it **returns a reference**. The call evaluates to a reference to the actual memory location — readable (r-value) and writable (l-value).

```

1  #include <iostream>
2  using namespace std;
3
4  int data[5] = {1, 2, 3, 4, 5};
5  int& item(int i) { return data[i]; }
6
7  int main() {
8      item(2) = 99;                // l-value: write to data[2]
9      cout << data[2] << "\n";    // 99
10
11     int val = item(0);            // r-value: read data[0]
12     cout << val << "\n";         // 1
13
14     item(1) = item(4) + 10;       // both: data[1] = data[4]+10 =
15                                   15
16     cout << data[1] << "\n";    // 15
17     return 0;

```

**Q5.** What is wrong with the following statement?

```

1  void func(int x = 99, int y);

```

**(5 marks)**

[CSE 107 | 2019–20]

### Answer — What is wrong with `void func(int x=99, int y)?`

```
1 void func(int x = 99, int y);    // ERROR
```

**Problem:** Default arguments must be specified from **right to left**. Once a parameter has a default, every parameter to its right must also have one. Here `x` has a default but `y` (to its right) does not — compile error.

**Correct declarations:**

```
1 void func(int x, int y = 0);      // OK: default on
   rightmost
2 void func(int x = 0, int y = 0);  // OK: all have defaults
```

**Q6.** What is an in-line function? What are the advantages and disadvantages? When and why will you make a function inline? When may a compiler ignore this specifier? (10 marks) [CSE 201 | 2016–17; CSE 107 | 2017–18]

↪ Similar: CSE 201 | 2007–08

### Answer — Inline function full details

**Inline Function:** A function prefixed with `inline` that the compiler replaces at the call site with the actual code body, avoiding function-call overhead.

**Advantages:**

- Eliminates call overhead (no stack frame, no jump/return).
- Enables further compiler optimisations at the call site.
- Safer than macros: full type checking, no double-evaluation.

**Disadvantages:**

- **Code bloat:** the function body is duplicated at every call site, increasing binary size.
- Increased compilation time.
- Changes to the body require recompilation of all inclusions.

**When to make a function inline:**

- The body is **small** (1–3 lines) and called very frequently.
- Simple accessors / getters: `int getX() const { return x; }`
- Performance-critical inner loops.

**When the compiler may ignore the specifier:**

- Function contains a loop or `switch`.
- Function is recursive.
- Function is too large.
- The address of the function is taken.

**Q7.** When can a function be used both as an l-value and r-value? Explain with an example. (6 marks) [CSE 107 | 2017–18]

**Q8.** What is the output of the following code segment?

```

1  int& f();
2  int x;
3  int main() {
4      f() = 10;
5      int a = 10, b = 20;
6      int& ref = a;
7      ref = b;
8      cout << a << " " << b << " " << ref << " " << x;
9      return 0;
10 }
11 int& f() { return x; }
```

(10 marks)

[CSE 107 | 2016–17]

#### Answer — Output of reference code snippet

```

1  int& f();
2  int x;
3  int main() {
4      f() = 10;
5      int a = 10, b = 20;
6      int& ref = a;
7      ref = b;
8      cout << a << " " << b << " " << ref << " " << x;
9      return 0;
10 }
11 int& f() { return x; }
```

#### Step-by-step trace:

1. `f() = 10;` — `f()` returns reference to global `x`. Assigns 10  $\Rightarrow$  `x = 10`.
2. `int a=10, b=20;` — local variables.
3. `int& ref = a;` — `ref` is alias for `a`.
4. `ref = b;` — assigns `b`'s value (20) to `a` through `ref`. So `a = 20`. (Not a rebind.)
5. Output: `a=20, b=20, ref=20, x=10`.

#### Output:

```

1  20 20 20 10
```

**Q9.** Explain why namespace is added to C++. (5 marks)

[CSE 107 | 2015–16]

### Answer — Why Namespace is Added to C++

#### Problem without namespace:

In large programs combining multiple libraries, different libraries may define identifiers (functions, classes, variables) with the same name. This causes **name collision** (ambiguity), and the compiler cannot determine which one to use.

#### Solution — namespace:

A namespace groups related identifiers under a unique name, preventing collisions.

```

1  #include <iostream>
2  using namespace std;
3
4  namespace LibA {
5      void print() { cout << "LibA::print()" << endl; }
6  }
7
8  namespace LibB {
9      void print() { cout << "LibB::print()" << endl; }
10 }
11
12 int main() {
13     LibA::print();    // no ambiguity
14     LibB::print();
15     return 0;
16 }
```

#### Output:

LibA::print()

LibB::print()

The entire C++ standard library lives in namespace `std` (`cout`, `cin`, `vector`, etc.) precisely to avoid collisions with user-defined names.

**Q10.** What is meant by “independent reference” in C++? How does an independent reference allow a function call to be written to the left side of an assignment operator? (4+4 marks) [CSE 201 | 2014–15]

↪ Similar: CSE 201 | 2010–11

### Answer — Independent reference and l-value assignment

**Independent Reference:** A reference declared as a local variable — not as a parameter or return value. Must be initialised at declaration; acts as a permanent alias:

```

1  int x = 10;
2  int& r = x;    // r is an independent reference to x
3  r = 25;        // modifies x; now x == 25
```

Function call on the left side of =:

```

1  #include <iostream>
2  using namespace std;
3
4  int vals[3] = {10, 20, 30};
5  int& getVal(int i) { return vals[i]; }
6
7  int main() {
8      getVal(1) = 99;           // sets vals[1] = 99
9      cout << vals[1] << "\n"; // 99
10
11     int v = getVal(2);         // r-value use: v = 30
12     cout << v << "\n";        // 30
13     return 0;
14 }

```

The returned reference is an alias to `vals[i]`, so assigning to `getVal(1)` directly modifies the array element.

- Q11.** What is a reference? Name three different scenarios when a reference can be used. Why does a reference use the dot (.) operator instead of the arrow (->) operator? (10 marks) [CSE 201 | 2013–14]

#### Answer — Reference — 3 scenarios, dot vs arrow

**Reference:** An alias for an existing variable declared using `&`. It must be initialised at declaration and cannot be rebound to another variable.

**Three scenarios:**

##### 1. Pass-by-reference:

```
1 void swap(int& a, int& b) { int t=a; a=b; b=t; }
```

##### 2. Return by reference:

```
1 int& getElement(int arr[], int i) { return arr[i]; }
2 getElement(arr, 0) = 100;
```

##### 3. Const reference parameter (avoid copy of large object):

```
1 void print(const string& s) { cout << s; }
```

**Why dot (.) not arrow (->):**

`->` is used with *pointers* because a pointer holds an address and must be dereferenced. A *reference* is an alias that the compiler automatically dereferences; it behaves exactly like the original variable, so `.` is used directly:

```

1 struct Point { int x, y; };
2 Point p = {1, 2};
3 Point* ptr = &p;
4 Point& ref = p;
5
6 ptr->x = 10;    // pointer: must dereference with ->
7 ref.x = 10;    // reference: same as p.x, use .

```

**Q12.** Explain why using a default argument is related to constructor overloading. What is wrong with the following function prototype?

```

1 double area(double width = 0, double length);

```

(10 marks)

[CSE 201 | 2013–14]

**Answer — Default argument and constructor overloading; wrong prototype**

**Why default arguments relate to constructor overloading:**

A single constructor with default arguments can accept multiple call signatures, effectively replacing several overloaded constructors:

```

1 class Box {
2     double h, w, l;
3 public:
4     Box(double h=1.0, double w=1.0, double l=1.0)
5         : h(h), w(w), l(l) {}
6 };
7 Box b1;                // all defaults
8 Box b2(5.0);           // w=1, l=1 defaults
9 Box b3(5.0, 3.0, 2.0); // no defaults

```

**What is wrong with:**

```

1 double area(double width = 0, double length); // ERROR

```

Once a parameter has a default, all parameters to its *right* must also have defaults. Here *length* follows *width* but has no default.

**Correct versions:**

```

1 double area(double length, double width = 0);    // OK
2 double area(double width = 0, double length = 1); // OK

```

**Q13.** Write a function `Compute_Volume` to compute the volume of a 3D box with height `h`, width `w`, and length `l`. Then write a new function `New_Compute_Volume` that computes the volume of a rectangular box as well as a cube using default arguments, such that all of the following calls work:

```

1 New_Compute_Volume(30, 20, 10);

```

```

2 New_Compute_Volume(10, 10, 10);
3 New_Compute_Volume(10);

```

(10 marks)

[CSE 201 | 2012–13]

**Answer — New\_Compute\_Volume with default arguments**

```

1  #include <iostream>
2  using namespace std;
3
4  double Compute_Volume(double h, double w, double l) {
5      return h * w * l;
6  }
7
8  // When only h given -> cube; when all given -> general box
9  double New_Compute_Volume(double h, double w = -1, double l =
10     -1) {
11     if (w < 0) { w = h; l = h; } // cube
12     if (l < 0) { l = w; }
13     return h * w * l;
14 }
15
16 int main() {
17     cout << New_Compute_Volume(30, 20, 10) << "\n"; // 6000
18     cout << New_Compute_Volume(10, 10, 10) << "\n"; // 1000
19     cout << New_Compute_Volume(10) << "\n"; // 1000
20     return 0;
21 }

```

**Q14.** What is a reference? Classify references used in C++ programming. Explain with examples how a returning reference can allow a function to be used on the left side of an assignment. (10 marks)

[CSE 201 | 2011–12]

**Answer — Reference classification and l-value**

**Reference:** An alias for an existing variable — another name that refers to the same memory location. Declared with &; must be initialised at declaration.

**Classification of references in C++:**

1. Independent (local) reference:

```

1  int x = 10;
2  int& r = x;    // r is an alias for x
3  r = 20;        // x is now 20

```

2. Reference parameter (call-by-reference):

```

1  void increment(int& n) { n++; }

```



```

2  int a = 5;
3  increment(a);  // a becomes 6
    
```

### 3. Reference return value:

```

1  int arr[5] = {10, 20, 30, 40, 50};
2  int& element(int i) { return arr[i]; }
    
```

Returning a reference allows a function on the left side of =:

```

1  #include <iostream>
2  using namespace std;
3
4  int arr[] = {10, 20, 30, 40, 50};
5  int& element(int i) { return arr[i]; }
6
7  int main() {
8      element(2) = 99;           // equivalent to arr[2] = 99
9      cout << arr[2] << "\n";   // prints: 99
10     cout << element(0);        // r-value use: prints 10
11     return 0;
12 }
    
```

**Q15.** With appropriate example, explain how a default argument produces constructor overloading ambiguity. (6 marks) [CSE 201 | 2011–12]

### Answer — Default argument causes constructor overloading ambiguity

```

1  class Point {
2  public:
3      Point() {
4          cout << "Default constructor\n";
5      }
6      Point(int x = 0, int y = 0) {
7          cout << "Parameterised: " << x << ", " << y << "\n";
8      }
9  };
10
11 int main() {
12     Point p;    // ERROR: ambiguous!
13                // Both Point() and Point(int=0, int=0)
14                // match a no-argument call.
15 }
    
```

**Explanation:** Both constructors can be called with no arguments. The compiler sees two equally valid matches and reports an ambiguity error. **Fix:** remove one of the constructors or remove the defaults from `Point(int, int)`.

**Q16.** What is namespace? Write down the problems of using global namespaces. (5 marks) [CSE 201 | 2011–12]

### Answer — Namespace and global namespace problems

**Namespace:** A named scope that groups related identifiers to avoid naming conflicts between libraries or modules.

```
1 namespace Math    { double pi = 3.14; }
2 namespace Physics { double pi = 3.14159265; }
3 cout << Math::pi;    // 3.14
4 cout << Physics::pi; // 3.14159265
```

### Problems with global namespace / using namespace std:

1. **Name collision:** Two libraries may define the same name; both in global scope causes ambiguity.
2. **Pollutes global scope:** All names become visible everywhere, making large programs harder to reason about.
3. **Maintenance burden:** A new library name may silently clash with existing global names.

**Best practice:** Use `std::cout` or restrict to `using std::cout;` in small scopes.

**Q17.** Write a function called `neg()` that reverses the sign of its integer parameter in two ways — using a pointer parameter and using a reference parameter. (10 marks) [CSE 201 | 2010–11]

### Answer — `neg()` with pointer and reference

```
1 #include <iostream>
2 using namespace std;
3
4 // Way 1: using a pointer parameter
5 void neg_ptr(int* p) { *p = -(*p); }
6
7 // Way 2: using a reference parameter
8 void neg_ref(int& r) { r = -r; }
9
10 int main() {
11     int a = 7, b = -3;
12     neg_ptr(&a);  cout << a << "\n";  // -7
13     neg_ref(b);   cout << b << "\n";  // 3
14     return 0;
15 }
```

|                 | Pointer                  | Reference           |
|-----------------|--------------------------|---------------------|
| Caller syntax   | <code>neg(&amp;a)</code> | <code>neg(a)</code> |
| Inside function | <code>*p = -*p</code>    | <code>r = -r</code> |
| Can be null?    | Yes                      | No                  |
| Readability     | Less natural             | More natural        |

**Q18.** Explain some ways that ambiguity can be introduced when overloading functions. Why are the following two overloaded functions inherently ambiguous?

```
1 int f(int a);
2 int f(int &a);
```

(10 marks)

[CSE 201 | 2010–11]

**Answer — Overloading ambiguity; `int f(int)` vs `int f(int&)`**

Ways ambiguity can be introduced:

1. Implicit type conversions:

```
1 void f(int x)    { }
2 void f(double x) { }
3 f(3.14f); // float converts to both -- ambiguous
```

2. Default argument matching another overload:

```
1 void g(int x)    { }
2 void g(int x, int y=0) { }
3 g(5); // both match -- ambiguous
```

Why `int f(int a)` and `int f(int &a)` are inherently ambiguous:

```
1 int f(int a) { return a; }
2 int f(int& a) { return a; }
3 // f(x) where x is int: both versions match equally.
4 // Compiler cannot choose. ERROR.
```

When `f(x)` is called with an `int` variable, the compiler can either copy `x` into `a` (first version) or bind `a` as a reference to `x` (second version). Both are equally good matches — inherently ambiguous.

**Q19.** Provide any two restrictions (with examples) while creating references. (5 marks)  
[CSE 201 | 2007–08, 2008–09]

**Answer — Two restrictions on references**

1. Must be initialised at declaration:

```
1 int x = 10;
```

```

2  int& r = x;    // OK
3  // int& r2;    // ERROR: must be initialised

```

2. **Cannot be rebound to another variable:** Once bound, any assignment through the reference modifies the original variable; it does not rebound the reference.

```

1  int a = 5, b = 20;
2  int& r = a;    // r refers to a
3  r = b;        // sets a = 20, does NOT rebound r to b
4  cout << a;    // 20
5  cout << (&r == &a); // 1 (true: r still refers to a)

```

**Additional note:** A const reference can bind to a temporary, but a non-const reference cannot:

```

1  const int& cr = 5 + 3; // OK: const ref binds to temporary
2  // int& r = 5 + 3;    // ERROR: non-const ref cannot bind
   // to rvalue

```

- Q20.** Write short notes on: (i) namespace, (ii) this pointer, (iii) inline function. **(15 marks)** [CSE 201 | 2008–09]

### Answer — namespace, this Pointer, inline Function

#### (i) namespace:

A namespace is a declarative region that provides scope to identifiers, preventing name collisions across libraries or modules.

```

1  namespace Math {
2      int square(int x) { return x * x; }
3  }
4  // Math::square(5) -> 25

```

#### (ii) this pointer:

Inside every non-static member function, **this** is an implicit pointer to the **current object**. It is used to resolve ambiguity between member variables and parameters, and to return the current object.

```

1  class Point {
2      int x, y;
3  public:
4      Point(int x, int y) {
5          this->x = x;    // disambiguates member from parameter
6          this->y = y;
7      }
8      Point& move(int dx, int dy) {

```

```

9         x += dx; y += dy;
10        return *this; // return current object for chaining
11    }
12 };

```

**(iii) inline function:**

An **inline** function is a hint to the compiler to replace each call with the function body, eliminating call overhead. It is best for small, frequently called functions.

```

1 inline int cube(int x) { return x * x * x; }
2
3 int main() {
4     int r = cube(3); // compiler may expand to: 3*3*3
5     // r = 27
6 }

```

Declaring a function inside a class body makes it implicitly inline.

**Q21.** Explain the difference between binary and unary scope resolution operators with examples. (5 marks) [CSE 201 | 2008–09, 2007–08]

**Answer — S3-Q24-25: Binary and unary scope resolution operators**

**Unary ::** — no left operand; accesses global scope when a local name shadows it.

**Binary ::** — left operand is a class or namespace; qualifies which scope an identifier belongs to.

```

1 #include <iostream>
2 using namespace std;
3
4 int val = 100; // global
5
6 namespace Math { double pi = 3.14; }
7
8 class Circle {
9     double r;
10 public:
11     Circle(double r) : r(r) {}
12     static double pi;
13     double area();
14 };
15 double Circle::pi = 3.14159; // binary :: -- defines
    static member
16 double Circle::area() { // binary :: -- defines
    member function
17     return Circle::pi * r * r; // binary :: -- accesses
    static member
18 }

```

```

19
20 int main() {
21     int val = 200;                // local
22     cout << val << "\n";         // 200 -- local
23     cout << ::val << "\n";       // 100 -- unary :: global
24
25     cout << Math::pi << "\n";    // 3.14 -- binary ::
        namespace
26
27     Circle c(5.0);
28     cout << c.area() << "\n";    // 78.5...
29     return 0;
30 }

```

**Q22.** "A reference parameter fully automates the call-by-reference parameter passing mechanism" — explain with example. (5 marks) [CSE 201 | 2007–08]

### Answer — Reference parameter automates call-by-reference

In C, call-by-reference requires manual pointer manipulation. In C++, a reference parameter automates this entirely:

```

1  #include <iostream>
2  using namespace std;
3
4  // C-style: manual pointer manipulation
5  void swap_ptr(int* a, int* b) {
6      int t = *a; *a = *b; *b = t;
7  }
8
9  // C++ style: reference parameter -- fully automatic
10 void swap_ref(int& a, int& b) {
11     int t = a; a = b; b = t;
12 }
13
14 int main() {
15     int x = 5, y = 10;
16
17     swap_ptr(&x, &y);           // caller must write &x, &y
18     cout << x << " " << y;     // 10 5
19
20     swap_ref(x, y);            // caller writes x, y --
        natural
21     cout << x << " " << y;     // 5 10
22     return 0;
23 }

```

With reference parameters, the caller passes variables directly (no & needed), yet

the function receives the actual variables. The call-by-reference mechanism is **fully automated** by the compiler.

- Q23.** The effect of a default argument can be alternatively achieved by overloading. Discuss with an example. (5 marks) [CSE 201 | 2007–08]

#### Answer — Default argument vs overloading

Using default argument (single function):

```
1 void printLine(char ch = '-', int n = 10) {
2     for (int i = 0; i < n; i++) cout << ch;
3     cout << "\n";
4 }
5 printLine();           // -----
6 printLine('*');        // *****
7 printLine('=', 5);     // =====
```

Equivalent using overloading (three functions):

```
1 void printLine() {
2     for (int i = 0; i < 10; i++) cout << '-'; cout << "\n";
3 }
4 void printLine(char ch) {
5     for (int i = 0; i < 10; i++) cout << ch; cout << "\n";
6 }
7 void printLine(char ch, int n) {
8     for (int i = 0; i < n; i++) cout << ch; cout << "\n";
9 }
```

Both produce the same interface. Default arguments are more **concise**; overloading gives more **flexibility** (each version can behave differently).

- Q24.** Is there anything wrong with the following code? If yes, what should be done to correct it?

```
1 #include <iostream>
2 using namespace std;
3 namespace A {
4     int i;
5     void dispI() { cout << i; }
6 }
7 void main() {
8     namespace Inside {
9         int insideI;
10        void dispInsideI() { cout << insideI; }
11    }
12    A::i = 10;
```

```

13     cout << A::i;
14     A::dispI();
15     Inside::insideI = 20;
16     cout << Inside::insideI;
17     Inside::dispInsideI();
18 }

```

(5 marks)

[CSE 201 | 2007–08]

### Answer — Problems in namespace code; corrected version

#### Problems:

1. **void main():** Standard C++ requires `int main()`.
2. **Namespace inside a function:** A namespace declaration is **not allowed** inside a function body. Namespaces can only appear at namespace scope.

#### Corrected code:

```

1  #include <iostream>
2  using namespace std;
3
4  namespace A {
5      int i;
6      void dispI() { cout << i << "\n"; }
7  }
8  namespace Inside {           // moved to global scope
9      int insideI;
10     void dispInsideI() { cout << insideI << "\n"; }
11 }
12
13 int main() {                  // int, not void
14     A::i = 10;
15     cout << A::i << "\n";
16     A::dispI();
17     Inside::insideI = 20;
18     cout << Inside::insideI << "\n";
19     Inside::dispInsideI();
20     return 0;
21 }

```

**Q25.** Can we use the same function name for a member function of a class and an outside function in the same program file? If yes, how are they distinguished? (5 marks)  
[CSE 201 | 2007–08]



**Answer — Member vs global function with same name**

Yes, the same function name can be used for a member function and a global function. They are distinguished by calling syntax and the scope resolution operator:

```

1  #include <iostream>
2  using namespace std;
3
4  void print() { cout << "Global print()\n"; }
5
6  class Printer {
7  public:
8      void print() { cout << "Printer::print()\n"; }
9      void test() {
10         print();    // calls member (this->print())
11         ::print();  // calls global (unary ::)
12     }
13 };
14
15 int main() {
16     Printer p;
17     p.print();    // Printer::print()
18     print();      // Global print()
19     p.test();     // both
20     return 0;
21 }
```

- `obj.print()` or `Printer::print()` → member function.
- `print()` in non-member context → global function.
- `::print()` inside the class → explicitly global.

BUET · CSE 107 / CSE 201

# Object Oriented Programming

Section S4

---

Friend Functions

## S4 — Friend Functions

**Q1.** Design a program that defines a friend function which can access and modify the private variables of two different classes. **(15 marks)** [CSE 107 | 2022–23]

**Answer — Friend function accesses and modifies two classes**

```
1  #include <iostream>
2  using namespace std;
3
4  class ClassB;
5
6  class ClassA {
7      int a;
8  public:
9      ClassA(int x) : a(x) {}
10     void show() const { cout << "A.a = " << a << "\n"; }
11     friend void swap_and_double(ClassA& ca, ClassB& cb);
12 };
13
14 class ClassB {
15     int b;
16 public:
17     ClassB(int x) : b(x) {}
18     void show() const { cout << "B.b = " << b << "\n"; }
19     friend void swap_and_double(ClassA& ca, ClassB& cb);
20 };
21
22 void swap_and_double(ClassA& ca, ClassB& cb) {
23     int temp = ca.a;
24     ca.a = cb.b * 2;
25     cb.b = temp * 2;
26 }
27
28 int main() {
29     ClassA ca(5); ClassB cb(10);
30     cout << "Before:\n"; ca.show(); cb.show();
31     swap_and_double(ca, cb);
32     cout << "After:\n"; ca.show(); cb.show();
33     return 0;
34 }
```

**Output:**

```
1  Before:
2  A.a = 5
3  B.b = 10
4  After:
5  A.a = 20
6  B.b = 10
```

**Q2.** Write an example program to demonstrate the use of forward declaration of a class.  
(11 marks) [CSE 107 | 2021–22]

**Answer — Forward declaration — complete example**

```
1  #include <iostream>
2  using namespace std;
3
4  class Node;    // forward declaration
5
6  class LinkedList {
7      Node* head;    // pointer to incomplete type is valid
8  public:
9      LinkedList();
10     void push(int val);
11     void display() const;
12 };
13
14 class Node {
15     int data;
16     Node* next;
17 public:
18     Node(int d) : data(d), next(nullptr) {}
19     friend class LinkedList;
20 };
21
22 LinkedList::LinkedList() : head(nullptr) {}
23
24 void LinkedList::push(int val) {
25     Node* n = new Node(val);
26     n->next = head; head = n;
27 }
28
29 void LinkedList::display() const {
30     Node* cur = head;
31     while (cur) { cout << cur->data << " -> "; cur = cur->
32         next; }
33     cout << "NULL\n";
34 }
35
36 int main() {
37     LinkedList list;
38     list.push(30); list.push(20); list.push(10);
39     list.display();    // 10 -> 20 -> 30 -> NULL
40     return 0;
41 }
```

**Q3.** "A friend function can be a global function not related to any particular

class. A friend function can also be a member of another class." — Explain this statement with appropriate examples. (12 marks) [CSE 107 | 2017–18]

### Answer — Friend as global and as member of another class

#### (i) Global friend function:

```

1  class Celsius;
2  class Fahrenheit {
3      double fahr;
4  public:
5      Fahrenheit(double f) : fahr(f) {}
6      friend void convert(Fahrenheit, Celsius&);
7  };
8  class Celsius {
9      double cel;
10 public:
11     Celsius() : cel(0) {}
12     void show() const { cout << cel << " C\n"; }
13     friend void convert(Fahrenheit, Celsius&);
14 };
15 void convert(Fahrenheit f, Celsius& c) {
16     c.cel = (f.fahr - 32) * 5.0 / 9.0;
17 }
18 int main() {
19     Fahrenheit f(212.0); Celsius c;
20     convert(f, c); c.show(); // 100 C
21 }
```

#### (ii) Member of another class as friend:

```

1  class Distance;
2  class Printer {
3  public:
4      void printDistance(Distance& d);
5  };
6  class Distance {
7      int metres;
8  public:
9      Distance(int m) : metres(m) {}
10     friend void Printer::printDistance(Distance&);
11 };
12 void Printer::printDistance(Distance& d) {
13     cout << d.metres << " m\n";
14 }
15 int main() {
16     Printer p; Distance d(42);
17     p.printDistance(d); // 42 m
18 }
```

**Q4.** What is a friend function? Why is friend function used in C++ programming? (7 marks) [CSE 201 | 2015–16]

#### Answer — Friend function definition and purpose

**Friend Function:** A non-member function granted access to a class's **private** and **protected** members via the **friend** keyword inside the class.

```
1 class Box {  
2     double side;  
3 public:  
4     Box(double s) : side(s) {}  
5     friend double volume(Box b);  
6 };  
7 double volume(Box b) {  
8     return b.side * b.side * b.side; // accesses private  
9 }
```

**Why used:**

1. Operator overloading where left operand is not the class type (e.g. << for ostream).
2. Symmetric binary operations (e.g. a+b == b+a).
3. A utility function needing private data but logically not a class member.

**Q5.** What is a friend function? When is a friend function used? Give an example of forward class declaration. (10 marks) [CSE 201 | 2013–14]

#### Answer — Friend function; when used; forward class declaration

**When used:**

- A function needs private members of two or more classes.
- Symmetric binary operator overloading.
- A non-member utility that logically operates on class data.

**Forward class declaration:**

```
1 class ClassB; // forward declaration  
2  
3 class ClassA {  
4     int a;  
5 public:  
6     ClassA(int x) : a(x) {}  
7     friend void show(ClassA, ClassB);  
8 };  
9  
10 class ClassB {
```

```

11     int b;
12 public:
13     ClassB(int x) : b(x) {}
14     friend void show(ClassA, ClassB);
15 };
16
17 void show(ClassA oa, ClassB ob) {
18     cout << "A=" << oa.a << " B=" << ob.b << "\n";
19 }
20
21 int main() {
22     show(ClassA(10), ClassB(20)); // A=10 B=20
23 }

```

Without the forward declaration, the compiler would not recognise ClassB when it appears inside ClassA.

**Q6.** What is a friend function? Can a member function of a class be a friend function of another class? If yes, give a code example. **(10 marks)** [CSE 201 | 2012–13]

#### Answer — Member function of one class as friend of another

Yes, a member function of one class can be a friend of another:

```

1  #include <iostream>
2  using namespace std;
3
4  class Alpha;
5
6  class Beta {
7  public:
8      void display(Alpha& a); // will be friend of Alpha
9  };
10
11 class Alpha {
12     int secret = 42;
13     friend void Beta::display(Alpha&); // member of Beta is
        friend
14 };
15
16 void Beta::display(Alpha& a) {
17     cout << "Alpha's secret: " << a.secret << "\n";
18 }
19
20 int main() {
21     Beta b; Alpha a;
22     b.display(a); // Alpha's secret: 42
23     return 0;

```

```
24 }
```

**Key rules:** Forward declaration of Alpha before Beta; `friend` names the specific member function; definition of `Beta::display` after full definition of Alpha.

**Q7.** How does a friend function differ from a member of a class? What are the advantages of using friend functions? **(6+6+15 marks)** [CSE 201 | 2010–11, 2009–10]

#### Answer — Friend vs member function; advantages

| Friend Function                                                | Member Function                     |
|----------------------------------------------------------------|-------------------------------------|
| Not a member of the class.                                     | A member of the class.              |
| No implicit <code>this</code> pointer.                         | Has implicit <code>this</code> .    |
| Called as <code>func(obj)</code> .                             | Called as <code>obj.func()</code> . |
| Can access private members of multiple classes simultaneously. | Only accesses own class's private.  |

#### Advantages of friend functions:

1. Can access private data of two or more classes.
2. Enables symmetric operator overloading.
3. Keeps class interface minimal while granting necessary access.
4. Flexible: can be a global function, a member of another class, or a whole friend class.

**Q8.** Explain the following statements regarding friend functions with examples:

- (a) A friend function is not a member of a class but still has access to its private members.
- (b) A friend function can be a global function not related to any particular class.
- (c) A friend function can be a member of another class.

**(21 marks)**

[CSE 201 | 2008–09]

#### Answer — Three statements about friend functions

**(a) Not a member but accesses private:**

```
1 class Account {
2     double balance;
3 public:
4     Account(double b) : balance(b) {}
5     friend void audit(Account a);
6 };
7 void audit(Account a) {
```



```

8     cout << "Balance: " << a.balance << "\n";
9 }

```

(b) Global function as friend:

```

1  class Rectangle {
2      double w, h;
3  public:
4      Rectangle(double w, double h) : w(w), h(h) {}
5      friend double area(Rectangle r);
6  };
7  double area(Rectangle r) { return r.w * r.h; }

```

(c) Member of another class as friend:

```

1  class Alpha;
2  class Beta {
3  public:
4      void inspect(Alpha& a);
5  };
6  class Alpha {
7      int x = 7;
8      friend void Beta::inspect(Alpha&);
9  };
10 void Beta::inspect(Alpha& a) {
11     cout << "Alpha.x = " << a.x << "\n";
12 }

```

**Q9.** "A friend function is not a member of a class but still has access to its private elements" — explain with an example. (5 marks) [CSE 201 | 2007–08]

**Q10.** When is a friend function compulsory? Give an example. (5 marks) [CSE 201 | 2007–08]

### Answer — When is a friend function compulsory?

A friend function is **compulsory** when overloading << and >> (stream operators), because the left operand is `ostream/istream`, not our class. We cannot add a member to the standard library, so a friend is the only option:

```

1  #include <iostream>
2  using namespace std;
3
4  class Point {
5      int x, y;
6  public:
7      Point(int x, int y) : x(x), y(y) {}

```

```

8
9     friend ostream& operator<<(ostream& os, const Point& p) {
10         os << "(" << p.x << ", " << p.y << ")";
11         return os;
12     }
13     friend istream& operator>>(istream& is, Point& p) {
14         is >> p.x >> p.y;
15         return is;
16     }
17 };
18
19 int main() {
20     Point p(3, 4);
21     cout << p << "\n";    // (3, 4)
22     return 0;
23 }

```

**Q11.** "A friend function can be used to overload the assignment operator '='." — True or False? Explain your answer. (5 marks) [CSE 201 | 2007–08]

#### Answer — Can friend function overload assignment operator?

**False.** A friend function **cannot** overload operator=.

The C++ standard requires =, [], (), and -> to be overloaded as **member functions** only.

For `a = b`, the left operand must be the implicit `this` object — only possible with a member function. A friend has no `this` and therefore cannot serve as the assignment operator:

```

1  class MyClass {
2      int val;
3  public:
4      MyClass(int v) : val(v) {}
5      // CORRECT: member function
6      MyClass& operator=(const MyClass& rhs) {
7          val = rhs.val;
8          return *this;
9      }
10     // friend MyClass& operator=(...) -- ILLEGAL, compile
11     error
12 };

```

**Q12.** Distinguish between static and non-static class members with examples. (5 marks) [CSE 201 | 2007–08]

## Answer — Static vs non-static class members

| Static Member                                         | Non-Static Member                                        |
|-------------------------------------------------------|----------------------------------------------------------|
| Belongs to the class; one copy shared by all objects. | Belongs to each object; every instance has its own copy. |
| Exists before any object is created.                  | Created when an object is instantiated.                  |
| Accessed as <code>ClassName::member</code> .          | Accessed as <code>obj.member</code> .                    |
| Static method has no <code>this</code> .              | Non-static method has <code>this</code> .                |

```

1  class Counter {
2      static int sCount;    // shared by all
3      int id;              // unique per object
4  public:
5      Counter() { id = ++sCount; }
6      ~Counter() { sCount--; }
7      static int getCount() { return sCount; }
8      int getId() const { return id; }
9  };
10 int Counter::sCount = 0;
11
12 int main() {
13     Counter c1, c2, c3;
14     cout << Counter::getCount() << "\n";    // 3
15     cout << c1.getId() << " " << c2.getId() << " " << c3.
16         getId() << "\n";    // 1 2 3
17     { Counter c4; cout << Counter::getCount() << "\n"; }    //
18         4
19     cout << Counter::getCount() << "\n";    // 3 (c4 destroyed)
20     return 0;
21 }

```

BUET · CSE 107 / CSE 201

# Object Oriented Programming

Section S5

---

Constructors, Destructors & Copy Constructor

## S5 — Constructors, Destructors & Copy Constructor

**Q1.** Consider the following program and mention the line(s) where the copy constructor is called. What will be the output of this program?

```

1  #include <iostream>
2  using namespace std;
3  class MyClass {
4  public:
5      int val;
6      int multiplier;
7      MyClass(int n, int m = 2) { val = n; multiplier = m; }
8      MyClass(const MyClass& obj) {
9          val = obj.val;
10         multiplier = obj.multiplier;
11     }
12     void setValue(int v, int m = 5) { val = v; multiplier = m; }
13     int calculateValue() { return val * multiplier; }
14 };
15 void functionC(MyClass* arr, int size) {
16     for (int i = 0; i < size; ++i)
17         cout << arr[i].calculateValue() << endl;
18 }
19 int main() {
20     MyClass* arr = new MyClass[2]{MyClass(5, 2), MyClass(6)};
21     arr[0] = arr[1];
22     functionC(arr, 2);
23     arr[0].setValue(15, 4);
24     arr[1].setValue(30);
25     cout << arr[0].calculateValue() << endl;
26     cout << arr[1].calculateValue() << endl;
27     delete[] arr;
28     return 0;
29 }

```

(13 marks)

[CSE 107 | 2022–23]

### Answer — MyClass array — copy constructor lines and output

**Copy constructor called:** when initialising arr[0] and arr[1] from the temporary MyClass(5,2) and MyClass(6) objects. arr[0] = arr[1] uses the **copy assignment operator** (not copy constructor).

**Output:**

```

1  12
2  12
3  60
4  150

```

**Trace:** After arr[0]=arr[1]: both have val=6, mult=2  $\Rightarrow 6*2=12$  each. After

```
setValue: arr[0]=15*4=60; arr[1]=30*5=150.
```

**Q2.** Write One C++ statement to implement each of the following:

- Using `new`, declare a pointer and allocate memory for a `char` variable initialized to 'E'.
- Using `new`, declare a pointer and allocate memory for an array of 5 `float` variables.
- Using `delete`, delete the entire integer array pointed to by `a`.
- Using `new`, declare a pointer and allocate memory for an object of class `A` (no constructor defined).

(12 marks)

[CSE 107 | 2022–23]

**Answer — One-line new/delete statements**

(a)

```
1 char *cp = new char('E');
```

```
1 float *fp = new float[5];
```

(b)

```
1 delete[] a;
```

```
1 A *ap = new A;
```

**Q3)** What is the problem with the following code segment? Write a program with `s1 = s2` and provide a solution:

```
1 class strtype {
2     char *p; int len;
3 public:
4     strtype(char *s) {
5         len = strlen(s) + 1;
6         p = new char[len];
7         strcpy(p, s);
8     }
9     ~strtype() { delete[] p; }
10 };
11 strtype s1("LEVEL"), s2("TERM");
12 s1 = s2;
```

Without changing the constructor and destructor, write code to solve the problem.

(15 marks)

[CSE 107 | 2021–22]

**Answer — strtype s1=s2 problem and overloaded = solution**

**Problems:** Default = shallow-copies p: both s1.p and s2.p point to the same array. Old s1 buffer ("LEVEL") is **leaked**, and when both are destroyed the shared buffer is **freed twice**.

**Solution — overload operator=:**

```

1  #include <iostream>
2  #include <cstring>
3  using namespace std;
4
5  class strtype {
6      char *p; int len;
7  public:
8      strtype(char *s) {
9          len = strlen(s)+1; p = new char[len]; strcpy(p,s);
10     }
11     ~strtype() { delete[] p; }
12
13     strtype& operator=(const strtype& obj) {
14         if (this == &obj) return *this; // self-assignment
15         delete[] p; // free old memory
16         len = obj.len;
17         p = new char[len]; // allocate fresh
18         strcpy(p, obj.p);
19         return *this;
20     }
21     void show() const { cout << p << "\n"; }
22 };
23
24 int main() {
25     strtype s1("LEVEL"), s2("TERM");
26     s1 = s2;
27     s1.show(); // TERM
28     s2.show(); // TERM
29 }
```

**Q4.** Differentiate between the new operator and malloc() function. Design Class A so that the following code does not produce any error:

```

1  int main() {
2      A obj(10);
3      A *array = new A[1];
4      return 0;
5  }
```

(13 marks)

[CSE 107 | 2021–22]

**Answer — new vs malloc; design Class A**

| <b>new</b>                               | <b>malloc()</b>                            |
|------------------------------------------|--------------------------------------------|
| C++ operator                             | C library function                         |
| Calls constructor automatically          | Does not call constructors                 |
| Throws <code>bad_alloc</code> on failure | Returns NULL on failure                    |
| Type-safe                                | Returns <code>void*</code> ; requires cast |

**Class A — needs both default and parameterised constructor:**

```

1  class A {
2      int val;
3  public:
4      A(): val(0){}           // default: needed for new A[1]
5      A(int v): val(v){}     // parameterised: needed for A obj(10)
6  };
7  int main() {
8      A obj(10);             // parameterised
9      A *array = new A[1];   // default
10     delete[] array;
11 }

```

**Q5.** Consider the following overloaded functions and develop the equivalent function using default arguments:

```

1  void f2() { int a = 0, b = 0; ... }
2  void f2(int a) { int b = 0; ... }
3  void f2(int a, int b) { ... }

```

(6 marks)

[CSE 107 | 2021–22]

**Answer — Overloaded f2() using default arguments**

```

1  // Three overloads:
2  // void f2()           { int a=0, b=0; ... }
3  // void f2(int a)      { int b=0; ... }
4  // void f2(int a, int b) { ... }
5
6  // Single equivalent with defaults:
7  void f2(int a=0, int b=0) {
8      // a=0,b=0 when called as f2()
9      // b=0      when called as f2(5)
10     // no default used when called as f2(5,3)
11 }
12
13 // All three calls still valid:
14 f2();           // a=0, b=0
15 f2(10);        // a=10, b=0

```



```
16 f2(10, 20); // a=10, b=20
```

**Q6.** Explain what a default copy constructor generated by the compiler does. Briefly describe a circumstance where this will cause a problem. (10 marks) [CSE 107 | 2019–20]

#### Answer — Compiler-generated copy constructor and its problem

**What it does:** Member-wise (shallow) copy — each data member copied by value. For pointer members, only the address is copied, not the pointed-to data.

**Problem:** When a class contains a raw pointer to heap memory, shallow copy makes both objects share the same block. One object's destructor frees it; the other is left with a dangling pointer and causes a double-free crash.

```
1 class Buffer {
2     char *data;
3 public:
4     Buffer(const char* s) { data=new char[strlen(s)+1];
5         strcpy(data,s); }
6     ~Buffer() { delete[] data; }
7     // Compiler generates: Buffer(const Buffer& o){ data=o.
8     data; } -- WRONG
9 };
10 int main() {
11     Buffer b1("Hello");
12     Buffer b2 = b1; // b1.data == b2.data (same pointer!)
13     // ~b2: frees data. ~b1: frees same data again -- CRASH
14 }
```

**Fix:** Write a user-defined deep copy constructor.

**Q7.** What are the possible problems if we use “call by value” instead of “call by reference” for passing an object to a function? Explain with an example. How can we solve this problem? (13 marks) [CSE 107 | 2017–18]

↪ Similar: CSE 201 | 2009–10

#### Answer — Call-by-value problems for objects; solution

**Problems:**

1. **Double-free / dangling pointer:** shallow copy shares heap memory; parameter's destructor frees it on return.
2. **Performance:** copying large objects wastes time.
3. **Shared state modification:** changes through the copy affect the original's heap data before the copy is destroyed.

**Solutions:**

- (i) Define a deep copy constructor.
- (ii) Pass by const reference: `void f(const Buffer& b)` — no copy made at all.

**Q8.** What will be the output of the following program?

```

1  class Point {
2  private:
3      int x, y;
4  public:
5      Point(int i = 0, int j = 0);
6      Point(const Point &t);
7  };
8  Point::Point(int i, int j) {
9      x = i; y = j;
10     cout << "Normal Constructor called\n";
11 }
12 Point::Point(const Point &t) {
13     x = t.x; y = t.y;
14     cout << "Copy Constructor called\n";
15 }
16 int main() {
17     Point *t1, *t2;
18     t1 = new Point(10, 15);
19     t2 = new Point(*t1);
20     Point t3 = *t1;
21     Point t4;
22     t4 = t3;
23     return 0;
24 }
```

(9 marks)

[CSE 107 | 2017–18]

### Answer — Output of Point program

**Trace:**

- 1: `t1 = new Point(10,15);`  
⇒ Normal Constructor called
- 2: `t2 = new Point(*t1);` — copy constructor.  
⇒ Copy Constructor called
- 3: `Point t3 = *t1;` — initialisation, copy constructor.  
⇒ Copy Constructor called
- 4: `Point t4;` — default (`i=0,j=0`).  
⇒ Normal Constructor called
- 5: `t4 = t3;` — assignment (not initialisation), no copy constructor.  
⇒ no output

### Output:

```

1 Normal Constructor called
2 Copy Constructor called
3 Copy Constructor called
4 Normal Constructor called

```

**Q9.** What is the difference between `delete` and `delete[]` operators? (5 marks) [CSE 201 | 2007–08; CSE 107 | 2017–18]

### Answer — `delete` vs `delete[]`

| <code>delete</code>                                                                                    | <code>delete[]</code>                                                                                      |
|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| For a <b>single</b> object allocated with <code>new</code> .<br>Calls destructor of <b>one</b> object. | For an <b>array</b> allocated with <code>new[]</code> .<br>Calls destructor of <b>every</b> array element. |
| Using on a <code>new[]</code> allocation is <b>undefined behaviour</b> .                               | Using on a single <code>new</code> is <b>undefined behaviour</b> .                                         |

```

1 int*    p1 = new int;      delete    p1;    // correct
2 int*    p2 = new int[100]; delete[] p2;    // correct
3 string* p3 = new string[5]; delete[] p3;    // calls ~string()
                        x5

```

**Q10.** Write the following: `myClass ob[] = {-1, -2};`. Is this valid? If yes, what does it mean? If no, what is the problem? (5 marks) [CSE 107 | 2017–18]

### Answer — `myClass ob[] = {-1, -2}` — valid?

**Valid** — if `myClass` has a single-int constructor.

```

1 class myClass {
2     int val;
3 public:
4     myClass(int v): val(v){}
5     int get() const { return val; }
6 };
7
8 myClass ob[] = {-1, -2};
9 // Equivalent to: myClass ob[2] = {myClass(-1), myClass(-2)};
10 // ob[0].get() = -1,  ob[1].get() = -2

```

Each element in the initialiser list is implicitly converted to a `myClass` object using the single-int constructor. If the constructor is **explicit** or does not exist, this is a compile error.

**Q11.** How does the copy constructor differ from the assignment operator (=)? (5 marks)  
[CSE 107 | 2016–17]

#### Answer — Copy constructor vs assignment operator

| Copy Constructor                              | Assignment Operator (=)                      |
|-----------------------------------------------|----------------------------------------------|
| Creates a <i>new</i> object.                  | Copies to an <i>existing</i> object.         |
| Called during <b>initialisation</b> :         | Called during <b>assignment</b> :            |
| <code>MyClass b = a;</code>                   | <code>b = a;</code> (b already exists).      |
| Destination is new; no old resources to free. | Must free destination's old resources first. |
| Cannot return a value.                        | Returns <code>*this</code> for chaining.     |

**Q12.** What is the problem when an object of `Animal` is passed by value? What are the ways to solve it? Write a copy constructor to resolve it. (15 marks) [CSE 107 | 2016–17]

```

1  class Animal {
2      int *age;
3  public:
4      Animal(int arg) { age = new int; *age = arg; }
5      ~Animal() { delete age; }
6  };

```

#### Answer — Animal class — copy constructor problem and fix

**Problem when passed by value:** Shallow copy makes both the original and the copy share the same `int*age` pointer. When the copy is destroyed, its destructor calls `delete age`, leaving the original with a dangling pointer.

**Ways to solve:**

- (i) Define a deep copy constructor.
- (ii) Pass by reference / const reference.
- (iii) Overload assignment operator.

**Deep copy constructor:**

```

1  class Animal {
2      int *age;
3  public:
4      Animal(int arg) { age=new int; *age=arg; }
5
6      Animal(const Animal& obj) {
7          age=new int;          // fresh allocation
8          *age=*obj.age;
9      }
10     ~Animal() { delete age; }

```

```
11     int getAge() const { return *age; }  
12 };
```

**Q13.** What is the output of the following code?

```
1  class Animal {  
2      int age;  
3  public:  
4      Animal() { cout << "Constructing\n"; age = 0; }  
5      ~Animal() { cout << "Destructing\n"; }  
6  };  
7  Animal process(Animal a) {  
8      cout << "In process\n";  
9      Animal b;  
10     b = a;  
11     return b;  
12 }  
13 int main() {  
14     cout << "First line\n";  
15     Animal a;  
16     cout << "Second line\n";  
17     Animal b = a;  
18     cout << "Third line\n";  
19     Animal c = process(b);  
20     cout << "Return line\n";  
21     return 0;  
22 }
```

(10 marks)

[CSE 107 | 2016–17]

### Answer — Output of Animal process() code

Output (verified by compiler):

```
1  First line  
2  Constructing  
3  Second line  
4  Third line  
5  In process  
6  Constructing  
7  Destructing  
8  Return line  
9  Destructing  
10 Destructing  
11 Destructing
```

**Trace:** a constructed; b=a (compiler copy, no print); process(b) — param copy (no print), then local b constructed (Constructing), local b and param destroyed

on return (Destructing twice); finally c, b, a destroyed at end of main.

**Q14.** Consider the following class definition:

```

1  class myclass {
2      int *p;
3  public:
4      myclass(int n) {
5          p = (int*)malloc(sizeof(int));
6          if (!p) { cout << "Allocation problem\n"; exit(1); }
7          *p = n;
8      }
9      ~myclass() { free(p); }
10 };

```

Write down the problem associated with the class declaration. Write a copy constructor to solve the problem and explain how it works. (5+5+5 marks) [CSE 201 | 2014–15]

**Q15.** Consider the following class definition:

```

1  class myclass {
2      int *p;
3  public:
4      myclass(int i) { p = (int*)malloc(sizeof(int)); *p = i; }
5      ~myclass() { free(p); cout << "Freeing..\r"; }
6      int get() { return *p; }
7  };

```

What is the problem when an object of the class type is passed to a regular function as a parameter? Write a copy constructor to resolve the problem. (10 marks) [CSE 201 | 2013–14]

**Answer — S5-Q3 & Q4: myclass shallow copy problem and copy constructor fix**

**The problem:** When a myclass object is passed by value, the default copy constructor makes a shallow copy—both the original and the parameter share the same malloc-allocated integer. When the function returns, the parameter's destructor calls free(p), freeing the shared memory. The original now holds a **dangling pointer**; its destructor causes a **double-free crash**.

```

1  #include <iostream>
2  #include <cstdlib>
3  using namespace std;
4
5  class myclass {
6      int *p;
7  public:

```

```

8     myclass(int i) { p = (int*)malloc(sizeof(int)); *p = i; }
9
10    // Deep copy constructor
11    myclass(const myclass& obj) {
12        p = (int*)malloc(sizeof(int));
13        *p = *obj.p;
14    }
15
16    ~myclass() { free(p); cout << "Freeing..\n"; }
17    int get() { return *p; }
18 };
19
20 void display(myclass ob) { cout << ob.get() << "\n"; }
21
22 int main() {
23     myclass a(100);
24     display(a);      // deep copy; a is safe
25     cout << a.get(); // 100
26 }

```

**Q16.** What is a copy constructor? What are the three cases when a copy constructor is invoked? Give examples using code. (10 marks) [CSE 201 | 2012–13]

### Answer — Copy constructor — definition and 3 invocation cases

**Copy Constructor:** A special constructor that initialises a new object as a copy of an existing object. Prototype:

```
1  ClassName(const ClassName& obj);
```

Three cases when the copy constructor is invoked:

#### 1. Object initialisation from another object:

```

1  MyClass a(10);
2  MyClass b = a;      // copy constructor called
3  MyClass c(a);       // copy constructor called

```

#### 2. Passing an object by value to a function:

```

1  void func(MyClass x) { }    // copy constructor called for
    x
2  func(a);

```

#### 3. Returning an object by value from a function:

```

1  MyClass create() {
2      MyClass tmp(99);

```

```

3     return tmp;    // copy constructor may be called (if
                       RVO absent)
4 }

```

**Q17.** What is the output of the following code segment?

```

1  class myclass {
2      int who;
3  public:
4      myclass(int n) { who = n; cout << "Constructing" << who <<
                          "\n"; }
5      ~myclass() { cout << "Destructing" << who << "\n"; }
6      int id() { return who; }
7  };
8  void f(myclass &o) { cout << "Received" << o.id() << "\n"; }
9  int main() {
10     myclass x(1);
11     f(x);
12     return 0;
13 }

```

(5 marks)

[CSE 201 | 2012–13]

**Answer — Output of myclass pass-by-reference code**

**Output:**

```

1  Constructing1
2  Received1
3  Destructing1

```

**Explanation:** f takes a reference — no copy is made. Only one constructor and one destructor call occur.

**Q18.** What is the problem with the following code segment?

```

1  class array {
2      int *p; int size;
3  public:
4      array(int sz) {
5          try { p = new int[sz]; }
6          catch (bad_alloc xa) { cout << "Allocation Failure\n";
                                   exit(EXIT_FAILURE); }
7          size = sz;
8      }
9      ~array() { delete[] p; }
10     void put(int i, int j) { if (i >= 0 && i < size) p[i] = j;
                               }
11     int get(int i) { return p[i]; }

```



```

12 };
13 void print(array &ob) {
14     array x = ob;
15     for (int i = 0; i < 10; i++) cout << x.get(i);
16 }
17 int main() {
18     array num(10);
19     for (int i = 0; i < 10; i++) num.put(i, i);
20     print(num);
21     for (int i = 9; i >= 0; i--) cout << num.get(i);
22     return 0;
23 }

```

(10 marks)

[CSE 201 | 2012–13]

**Answer — array class shallow copy problem in print()**

**Problem:** `array x = ob;` inside `print()` does a shallow copy. Both `x.p` and `ob.p` (i.e. `num.p`) point to the same array. When `print` returns and `x` is destroyed, its destructor frees the shared array. The loop in `main` then accesses freed memory (undefined behaviour), and `num`'s destructor causes a double-free crash.

**Fix — deep copy constructor:**

```

1 array(const array& obj) {
2     size = obj.size;
3     p = new int[size];
4     for (int i=0; i<size; i++) p[i] = obj.p[i];
5 }

```

**Q19.** The class `dyna` is defined as follows:

```

1 class dyna {
2     int *p;
3 public:
4     dyna(int i);
5     ~dyna() { free(p); }
6     int get() { return *p; }
7 };

```

Write the constructor `dyna(int i)`. Also write a function called `neg()` that reverses the sign of its integer parameter. What will be the problem if `cout << neg(ob);` is executed in `main()`? Write a copy constructor to solve the problem. (13 marks)  
[CSE 201 | 2011–12]

**Answer — dyna class — constructor, neg(), copy constructor**

```

1 #include <iostream>
2 #include <cstdlib>
3 using namespace std;

```

```

4
5 class dyna {
6     int *p;
7 public:
8     dyna(int i) { p=(int*)malloc(sizeof(int)); *p=i; }
9
10    // Deep copy constructor
11    dyna(const dyna& obj) {
12        p=(int*)malloc(sizeof(int)); *p=obj.p;
13    }
14    ~dyna() { free(p); }
15    int get() { return *p; }
16 };
17
18 dyna neg(dyna ob) { return dyna(-ob.get()); }
19
20 int main() {
21     dyna ob(10);
22     cout << neg(ob).get() << "\n"; // -10
23 }

```

**Problem without copy constructor:** `neg(dyna ob)` takes by value — shallow copy shares `p`. Parameter's destructor frees it; original `ob` has dangling pointer.

**Q20.** What is a copy constructor? What types of problems are solved by using copy constructors? Explain with examples. (15 marks) [CSE 201 | 2008–09]

**Q21.** What type of problems are solved by using copy constructors? Explain with examples. (10 marks) [CSE 201 | 2007–08]

### Answer — Problems solved by copy constructors

#### Problem — Shallow Copy with Dynamic Memory:

The compiler-generated default copy constructor performs a *shallow copy*: for pointer members it copies the address, not the pointed-to data. Both the original and the copy then point to the **same heap block**, causing:

1. **Double-free crash:** both destructors free the same memory.
2. **Dangling pointer:** after one is destroyed, the other holds an invalid pointer.

**Solution — deep copy constructor:**

```

1 #include <iostream>
2 using namespace std;
3
4 class MyData {
5     int *p;

```

```
6 public:
7     MyData(int val) { p = new int; *p = val; }
8
9     // Deep copy: allocates its own memory
10    MyData(const MyData& obj) {
11        p = new int;
12        *p = *obj.p;
13    }
14    ~MyData() { delete p; }
15    int get() const { return *p; }
16 };
17
18 void show(MyData d) { cout << d.get() << "\n"; }
19
20 int main() {
21     MyData a(42);
22     show(a);           // deep copy; a is unaffected
23     cout << a.get(); // still 42
24 }
```

BUET · CSE 107 / CSE 201

# Object Oriented Programming

Section S6

---

Operator Overloading & Type Conversion

## S6 — Operator Overloading & Type Conversion

**Q1.** Differentiate between early binding and late binding. Demonstrate `const_cast` in case of `const` method with example. (4+4 marks) [CSE 107 | 2023–24]

### Answer — `const_cast` with `const` method example

```

1  #include <iostream>
2  using namespace std;
3  class Counter {
4      int count;
5  public:
6      Counter(int n): count(n){}
7      void show() const { cout<<"Count="<<count<<"\n"; }
8      void increment() { count++; }
9      void forceIncrement() const {
10         Counter* nc=const_cast<Counter*>(this); // remove
11         const                                     // now
12         nc->increment();
13     }
14 };
15 int main(){
16     Counter c(5);
17     c.show();           // Count=5
18     c.forceIncrement();
19     c.show();           // Count=6
20 }
```

**Warning:** Using `const_cast` on a truly `const` object (declared `const`) is undefined behaviour.

**Q2.** Consider the following class declarations:

```

1  class Coord {
2      int x, y;
3  public:
4      //-----
5      //-----
6  };
```

Create a `Rectangle` class that uses the `Coord` class as follows:

```

1  class Rectangle {
2      Coord p1, p2, p3, p4;
3      char *name;
4  public:
5      //-----
6      //-----
7  };
```

Answer the following questions and write down methods as instructed below:

- Write down a constructor of **Coord** class accepts only positive values of  $x$  and  $y$  with no single  $x$ - or  $y$ -coordinate larger than 200.
- Write down a constructor of **Rectangle** class that accepts four points and checks whether they form a rectangle or not. A rectangle has right angles at each of its corner (and checking of any three corners are sufficient). Write a **static** function **isRightAngle()** that accepts three points and checks whether a right angle is formed at the middle point or not?
- Write down four member functions of **Rectangle** class for computing the length, the width, the perimeter and the area of the rectangle. The length is the larger of the two dimensions. Take care to reuse methods where possible.
- Include a predicate function **square** that determines whether the rectangle is a square or not.
- Write down a clone constructor that makes a clone of rectangle.
- Write down necessary conversion methods for executing the following program codes:

```
1  int value = rect; //returns the area of rect
2  string name = rect;
```

Where “rect” is a rectangle type object.

- Briefly state the problem while executing the following code-

```
1  r1 = r2;
```

Where both **r1** and **r2** are rectangle type object. Overwrite the assignment operator to overcome the problem.

(35 marks)

[CSE 107 | 2023–24]

### Answer — Coord and Rectangle Classes

```
1  #include <iostream>
2  #include <cstring>
3  #include <cmath>
4  #include <stdexcept>
5  #include <string>
6  using namespace std;
7
8  class Coord {
9      int x, y;
10 public:
11     // (i) Constructor validating positive values <= 200
12     Coord(int x = 1, int y = 1) {
13         if (x <= 0 || y <= 0 || x > 200 || y > 200)
14             throw invalid_argument("Coordinates must be
                                   positive and <= 200");
```

```

15         this->x = x;
16         this->y = y;
17     }
18
19     // Getters needed since x and y are private by default
20     int getX() const { return x; }
21     int getY() const { return y; }
22 };
23
24 class Rectangle {
25     Coord p1, p2, p3, p4;
26     char* name;
27
28 public:
29     // (ii) static function to check right angle at p2 (
30     //      middle point)
31     static bool isRightAngle(const Coord& pt1, const Coord&
32     pt2, const Coord& pt3) {
33         int ax = pt1.getX() - pt2.getX();
34         int ay = pt1.getY() - pt2.getY();
35         int bx = pt3.getX() - pt2.getX();
36         int by = pt3.getY() - pt2.getY();
37         return (ax * bx + ay * by) == 0;
38     }
39
40     // (ii) Constructor verifying rectangle properties
41     Rectangle(Coord a, Coord b, Coord c, Coord d, const char*
42     n)
43         : p1(a), p2(b), p3(c), p4(d) {
44         // Checking any three corners is sufficient
45         if (!isRightAngle(p4, p1, p2) ||
46             !isRightAngle(p1, p2, p3) ||
47             !isRightAngle(p2, p3, p4)) {
48             throw invalid_argument("Points do not form a
49             rectangle");
50         }
51         name = new char[strlen(n) + 1];
52         strcpy(name, n);
53     }
54
55     // (v) Clone constructor (Copy Constructor)
56     Rectangle(const Rectangle& r)
57         : p1(r.p1), p2(r.p2), p3(r.p3), p4(r.p4) {
58         name = new char[strlen(r.name) + 1];
59         strcpy(name, r.name);
60     }
61
62     ~Rectangle() { delete[] name; }

```

```
59
60 // Helper method to compute distance
61 double dist(const Coord& a, const Coord& b) const {
62     return sqrt(pow(a.getX() - b.getX(), 2) + pow(a.getY()
63         - b.getY(), 2));
64 }
65 // (iii) Length, width, perimeter, area
66 double length() const {
67     double d1 = dist(p1, p2);
68     double d2 = dist(p2, p3);
69     return (d1 > d2) ? d1 : d2; // Length is the larger
70         dimension
71 }
72 double width() const {
73     double d1 = dist(p1, p2);
74     double d2 = dist(p2, p3);
75     return (d1 < d2) ? d1 : d2;
76 }
77
78 double perimeter() const { return 2 * (length() + width()
79     ); }
80
81 double area() const { return length() * width(); }
82
83 // (iv) Square predicate
84 bool square() const { return abs(length() - width()) < 1e
85     -9; }
86
87 // (vi) Conversion: int value = rect -> returns area
88 operator int() const { return static_cast<int>(area()); }
89
90 // (vi) Conversion: string name = rect -> returns name
91 operator string() const { return string(name); }
92
93 // (vii) Problem statement:
94 // The default assignment operator (r1 = r2) performs a
95 // shallow copy.
96 // This causes the `name` pointers of both objects to
97 // point to the
98 // same dynamically allocated memory. When the objects
99 // are destroyed,
100 // the destructor calls delete[] on the same memory twice
101 // , causing
102 // a double-free error and potential runtime crashes.
103
104 // Overloaded assignment operator to overcome the problem
```



```

99     Rectangle& operator=(const Rectangle& r) {
100         if (this == &r) return *this; // Self-assignment
            check
101
102         // Copy coordinates
103         p1 = r.p1; p2 = r.p2; p3 = r.p3; p4 = r.p4;
104
105         // Deep copy of dynamically allocated array
106         delete[] name;
107         name = new char[strlen(r.name) + 1];
108         strcpy(name, r.name);
109
110         return *this;
111     }
112 };

```

**Q3.** If you overload the assignment operator, should you return an object of the same type as the class it is overloaded for? Justify your answer. **(10 marks)** [CSE 107 | 2022–23]

**Answer — Should overloaded = return same type?**

**Yes** — it should return `ClassName&` (reference to same class) to support **chained assignment** (`a=b=c`) and to mirror the built-in behaviour.

```

1 MyClass& operator=(const MyClass& rhs) {
2     if (this != &rhs) { /* deep copy */ }
3     return *this;    // enables: a = b = c
4 }

```

Returning `void` breaks chaining; returning by value is inefficient and inconsistent with built-in types.

**Q4.** Write a program to demonstrate a user-defined conversion function that converts a class object to a built-in data type. **(9 marks)** [CSE 107 | 2022–23]

**Q5.** Consider the following `Coordinator` class. You are not allowed to modify the given code. Add code to produce output: `x coordinate: 7` and `y coordinate: 10`.

```

1 #include <iostream>
2 using namespace std;
3 class Coordinator {
4     int x, y;
5 public:
6     Coordinator(int x_val = 0, int y_val = 0) { x = x_val; y =
        y_val; }
7     int getX() { return x; }
8     int getY() { return y; }

```

```

9      // add your code here
10 };
11 int main() {
12     Coordinator cObj(5, 8);
13     cObj += 2;
14     cout << "x coordinate : " << cObj.getX() << endl;
15     cout << "y coordinate : " << cObj.getY() << endl;
16     return 0;
17 }

```

(10 marks)

[CSE 107 | 2022–23]

**Answer — Coordinator += to produce x=7, y=10**

Add the following inside the Coordinator class:

```

1  Coordinator& operator+=(int val) {
2      x += val;
3      y += val;
4      return *this;
5  }

```

**Output:**

```

1  x coordinate : 7
2  y coordinate : 10

```

**Q6.** Evaluate the output of the following program: (6 marks)

```

1  #include <iostream>
2  using namespace std;
3
4  class circle {
5      int r;
6  public:
7      circle(int a = 0) {
8          r = a;
9      }
10     void show() {
11         cout << r << endl;
12     }
13     friend circle operator++(circle &ob);
14 };
15
16 circle operator++(circle &ob) {
17     ob.r++;
18     return ob;
19 }
20
21 int main() {

```

```
22     circle c1(10);
23     ++c1;
24     c1.show();
25     return 0;
26 }
```

[CSE 107 | 2021–22]

**Answer — Output of circle ++ program****Output:**

```
1  11
```

**Explanation / Trace:**

- (a) `circle c1(10);` instantiates an object `c1` with its member variable `r` initialized to 10 via the parameterized constructor.
- (b) `++c1;` invokes the overloaded prefix increment operator function `operator++(circle &ob)`.
- (c) Because the argument is passed by reference (`circle &ob`), the parameter `ob` acts as an alias for `c1`. The statement `ob.r++;` increments the value of `r` directly inside `c1` from 10 to 11.
- (d) Finally, `c1.show();` prints the current value of `r`, which outputs 11 to the console followed by a newline.

**Q7.** Let  $xi + yj + zk$  be a vector where  $x$ ,  $y$  and  $z$  are real numbers as well as  $i$ ,  $j$  and  $k$  represent unique vectors along x-axis, y-axis and z-axis. There is a description for each point of the vector and hence each object has a pointer to point the description. Declare a class named “Vector” that has members and/or friends to fulfill the following requirements. Don’t use friend function if it can be implemented using member function. (7×4=28)

| Requirement                            | Sample Code                                                                                    | Explanation                                                                                                                                                                                                                                                       |
|----------------------------------------|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Creating instance                      | <code>Vector ob(12.0, 5.0, 2.0),<br/>ob1(3.0, 2.0, 5.0,<br/>"The peak point"),<br/>ob2;</code> | Use default arguments to set the vector point at the origin with null description when no parameter is given. Write appropriate constructor and copy constructor.                                                                                                 |
| Addition of two complex number objects | <code>ob1 = ob2 + ob3;</code>                                                                  | The values in each dimension will be added separately, i.e., the x values of ob2 and ob3 will be added together to give x value of ob1 and so on. Descriptions of ob2 and ob3 will be concatenated by adding a space between them to give the description of ob1. |
| Incrementing a complex number          | <code>ob = ob1 += ob2;</code>                                                                  | Overwrite = and += operators.                                                                                                                                                                                                                                     |
| Adding a real number to an object      | <code>ob2 = ob1 + 100.0;<br/>ob2 = 100.0 + ob1;</code>                                         | The real number will be added to each value of the object, i.e., each value of x, y and z will be incremented by the amount of the real number.                                                                                                                   |

### Answer — Vector Class: Constructors, Overloaded Operators (+, +=, =)

```

1  #include <iostream>
2  #include <cstring>
3  using namespace std;
4
5  class Vector {
6      double x, y, z;
7      char* desc;
8  public:
9      // 1. Constructor with default arguments
10     Vector(double x = 0.0, double y = 0.0, double z = 0.0,
11            const char* d = NULL)
12         : x(x), y(y), z(z) {
13             if (d) {
14                 desc = new char[strlen(d) + 1];
15                 strcpy(desc, d);
16             } else {
17                 desc = NULL;
18             }
19         }
20
21     // 1. Copy Constructor
22     Vector(const Vector& other) : x(other.x), y(other.y), z(
23         other.z) {

```

```

22         if (other.desc) {
23             desc = new char[strlen(other.desc) + 1];
24             strcpy(desc, other.desc);
25         } else {
26             desc = NULL;
27         }
28     }
29
30     // Destructeur to free dynamically allocated memory
31     ~Vector() {
32         delete[] desc;
33     }
34
35     // 3. Overwrite = operator (Member function)
36     Vector& operator=(const Vector& other) {
37         if (this != &other) {
38             x = other.x; y = other.y; z = other.z;
39             delete[] desc;
40             if (other.desc) {
41                 desc = new char[strlen(other.desc) + 1];
42                 strcpy(desc, other.desc);
43             } else {
44                 desc = NULL;
45             }
46         }
47         return *this;
48     }
49
50     // 2. Addition of two Vector objects (Member function)
51     Vector operator+(const Vector& b) const {
52         Vector res(x + b.x, y + b.y, z + b.z);
53         if (desc && b.desc) {
54             res.desc = new char[strlen(desc) + strlen(b.desc)
55                               + 2];
56             strcpy(res.desc, desc);
57             strcat(res.desc, " ");
58             strcat(res.desc, b.desc);
59         } else if (desc) {
60             res.desc = new char[strlen(desc) + 1];
61             strcpy(res.desc, desc);
62         } else if (b.desc) {
63             res.desc = new char[strlen(b.desc) + 1];
64             strcpy(res.desc, b.desc);
65         }
66         return res;
67     }
68
69     // 3. Overwrite += operator (Member function)

```

```

69     Vector& operator+=(const Vector& other) {
70         *this = *this + other; // Reuse the overloaded + and
           = operators
71         return *this;
72     }
73
74     // 4. Adding a real number: Vector + real (Member
           function)
75     Vector operator+(double r) const {
76         return Vector(x + r, y + r, z + r, desc);
77     }
78
79     // 4. Adding a real number: real + Vector (MUST be a
           friend function)
80     friend Vector operator+(double r, const Vector& v) {
81         return Vector(r + v.x, r + v.y, r + v.z, v.desc);
82     }
83
84     void print() const {
85         cout << x << "i + " << y << "j + " << z << "k";
86         if (desc) cout << " [" << desc << "]";
87         cout << endl;
88     }
89 };
90
91 int main() {
92     Vector ob(12.0, 5.0, 2.0);
93     Vector ob1(3.0, 2.0, 5.0, "The peak point");
94     Vector ob2;
95
96     ob2 = ob + ob1;
97     cout << "ob + ob1: "; ob2.print();
98
99     Vector ob3(1.0, 1.0, 1.0, "Base");
100    Vector ob_assigned = ob3 += ob1;
101    cout << "ob3 after +=: "; ob3.print();
102    cout << "ob_assigned: "; ob_assigned.print();
103
104    Vector ob4 = ob1 + 100.0;
105    cout << "ob1 + 100.0: "; ob4.print();
106
107    Vector ob5 = 100.0 + ob1;
108    cout << "100.0 + ob1: "; ob5.print();
109
110    return 0;
111 }

```

**Q8.** If you overload operator+, must you return an object of the same type as your parameters? Justify your answer. (7 marks) [CSE 107 | 2019–20]

#### Answer — Must operator+ return same type as parameters?

**No.** C++ imposes no restriction on the return type of overloaded operators. The overloaded operator is just a function; its return type can be anything logical.

```

1  class Distance { double d; public: Distance(double v):d(v){}
    void show(){cout<<d;} };
2  class Point {
3      int x, y;
4  public:
5      Point(int a,int b):x(a),y(b){}
6      // operator+ returns Distance, not Point
7      Distance operator+(const Point& p) const {
8          double dx=x-p.x, dy=y-p.y;
9          return Distance(sqrt(dx*dx+dy*dy));
10     }
11 };

```

In practice, returning the same type enables chaining (a+b+c), but it is not mandatory.

**Q9.** "There are scenarios when a friend function is very useful for operator overloading" — support this statement for the following class Rational by writing a program that must use friend function for operator overloading:

```

1  class Rational {
2      int n, d;
3  public:
4      Rational() { n = 0; d = 0; }
5      Rational(int a, int b) { n = a; d = b; }
6  };

```

(15 marks)

[CSE 107 | 2019–20]

#### Answer — Rational class with friend operator overloading

```

1  #include <iostream>
2  using namespace std;
3
4  class Rational {
5      int n, d;
6  public:
7      Rational(int a=0, int b=1): n(a), d(b){}
8
9      friend Rational operator+(const Rational& a, const
10         Rational& b){
11         return Rational(a.n*b.d + b.n*a.d, a.d*b.d);

```

```

11     }
12     friend Rational operator*(const Rational& a, const
        Rational& b){
13         return Rational(a.n*b.n, a.d*b.d);
14     }
15     friend ostream& operator<<(ostream& os, const Rational& r
        ){
16         os<<r.n<<"/"<<r.d; return os;
17     }
18 };
19
20 int main() {
21     Rational r(1,2), s(1,3);
22     cout << r+s << "\n"; // 5/6
23     cout << r*s << "\n"; // 1/6
24 }

```

**Q10.** Given a Date class with day, month, year: overload the binary minus operator so that ob1 - ob2 gives the difference in number of days (assume 30 days/month). Also overload << to print day-month-year. Include a main() function.

```

1 class Date {
2     int day, month, year;
3 public:
4     Date(int m, int d, int y) { day = d; month = m; year = y;
        }
5 };

```

(17 marks)

[CSE 107 | 2019–20]

### Answer — Date class — binary - and « operators

```

1 #include <iostream>
2 #include <cmath>
3 using namespace std;
4 class Date {
5     int day, month, year;
6 public:
7     Date(int m, int d, int y): day(d), month(m), year(y){}
8     int operator-(const Date& o) const {
9         int t1=year*360+month*30+day;
10        int t2=o.year*360+o.month*30+o.day;
11        return abs(t1-t2);
12    }
13    friend ostream& operator<<(ostream& os, const Date& d){
14        os<<d.day<<"-"<<d.month<<"-"<<d.year; return os;
15    }

```



```

16 };
17 int main(){
18     Date d1(3,15,2024), d2(1,1,2024);
19     cout<<d1<<"\n";           // 15-3-2024
20     cout<<(d1-d2)<<" days\n"; // 74 days
21 }

```

**Q11.** Consider the following Photo and Frame classes. Add necessary code to Photo and/or Frame class (without modifying existing code) so that  $F = P + 2$ ; produces Frame Width: 7 and Frame Length: 10 when P is Photo(5,8):

```

1  class Photo { int width, length; public:
2      Photo() {width=0; length=0;}
3      Photo(int a, int b) {width=a; length=b;}
4      int getwidth() {return width;} int getlength() {return
        length;}
5  };
6  class Frame { int width, length; public:
7      Frame() {width=0; length=0;}
8      Frame(int a, int b) {width=a; length=b;}
9      int getwidth() {return width;} int getlength() {return
        length;}
10 };

```

(20 marks)

[CSE 107 | 2019–20]

**Answer —** Photo + 2 = Frame(7,10)

```

1  #include <iostream>
2  using namespace std;
3  class Frame{
4      int width,length;
5  public:
6      Frame(int a=0,int b=0):width(a),length(b){}
7      int getwidth(){return width;} int getlength(){return
        length;}
8  };
9  class Photo{
10     int width,length;
11 public:
12     Photo(int a=0,int b=0):width(a),length(b){}
13     int getwidth(){return width;} int getlength(){return
        length;}
14     // P+n returns Frame with each dim increased by n
15     Frame operator+(int n){ return Frame(width+n,length+n); }
16 };
17 int main(){

```

```

18     Photo P(5,8);
19     Frame F=P+2;
20     cout<<"Frame Width: " <<F.getwidth() <<"\n"; // 7
21     cout<<"Frame Length: " <<F.getlength() <<"\n"; // 10
22 }

```

**Q12.** Consider the following Fraction class. Overload the prefix ++ operator using a friend function so that ++n on Fraction(3,7) produces x/y: 4/8. Only write the overloaded function:

```

1  class Fraction { int x, y; public:
2      Fraction(int a, int b) {x=a; y=b;}
3      int getx() {return x;} int gety() {return y;}
4  };

```

(13 marks)

[CSE 107 | 2019–20]

#### Answer — Fraction prefix ++ as friend

```

1  #include <iostream>
2  using namespace std;
3  class Fraction{
4      int x,y;
5  public:
6      Fraction(int a,int b):x(a),y(b){}
7      int getx(){return x;} int gety(){return y;}
8      friend Fraction& operator++(Fraction& f){ f.x++; f.y++;
9          return f; }
10 };
11 int main(){
12     Fraction n(3,7);
13     ++n;
14     cout<<"x/y: " <<n.getx() << "/" <<n.gety() <<"\n"; // x/y: 4/8
15 }

```

**Q13.** Why should the overloaded assignment operator (=) function return the this pointer?

(5 marks)

[CSE 107 | 2017–18; CSE 201 | 2009–10]

#### Answer — Why operator= should return \*this

Returning \*this enables **chained assignments** like `a = b = c`, parsed as `a = (b = c)`. `b = c` must return a reference to `b` so it can be assigned to `a`.

```

1  class MyClass {
2      int val;
3  public:
4      MyClass(int v=0): val(v){}

```

```

5     MyClass& operator=(const MyClass& rhs) {
6         val = rhs.val;
7         return *this;    // enables: a = b = c
8     }
9 };
10 int main() {
11     MyClass a(1), b(2), c(3);
12     a = b = c;    // works correctly
13 }

```

**Q14.** Which of the operators can have default arguments when they are overloaded? (3 marks) [CSE 107 | 2017–18]

**Answer — Which operators can have default arguments?**

Only **operator()** (the function call operator) can have default arguments. All other overloaded operators cannot, because their arity is fixed by the language.

```

1  class Multiplier {
2      int factor;
3  public:
4      Multiplier(int f=1): factor(f){}
5      int operator()(int x, int step=1) const {
6          return x * factor * step;
7      }
8  };
9  Multiplier m(3);
10 cout << m(5) << "\n";    // 15 (step defaults to 1)
11 cout << m(5, 2) << "\n"; // 30

```

**Q15.** Consider the class **Complex**. Write necessary functions so that the following statements execute correctly in **main**:

```

1  class Complex {
2  public:
3      int real, imag;
4      Complex(int r = 0, int i = 0);
5  };

```

(i) `comp3 = comp1 + comp2;` (ii) `comp3 = comp1 + 10;` (iii) `comp3 = 10 + comp1;` (iv) `comp1++;` (v) `++comp1;` (vi) `cout << comp1;` (vii) `comp3 = comp1(5);` (25 marks) [CSE 107 | 2017–18]

## Answer — Complex class — all 7 operations including comp1(5)

```

1  #include <iostream>
2  using namespace std;
3  class Complex {
4  public:
5      int real, imag;
6      Complex(int r=0, int i=0): real(r), imag(i){}
7      Complex operator+(const Complex& o) const { return
          Complex(real+o.real,imag+o.imag); }
8      Complex operator+(int n) const { return
          Complex(real+n, imag); }
9      friend Complex operator+(int n, const Complex& c){ return
          Complex(c.real+n,c.imag); }
10     Complex& operator++() { real++; imag++; return *this;
        }
11     Complex operator++(int) { Complex old=*this; real++;
        imag++; return old; }
12     friend ostream& operator<<(ostream& os, const Complex& c)
        {
13         os<<c.real<<"+"<<c.imag<<"i"; return os;
14     }
15     Complex operator()(int n) const { return Complex(real*n,
        imag*n); }
16 };
17 int main(){
18     Complex c1(3,4), c2(1,2), c3;
19     c3=c1+c2;    cout<<c3<<"\n"; // 4+6i
20     c3=c1+10;    cout<<c3<<"\n"; // 13+4i
21     c3=10+c1;    cout<<c3<<"\n"; // 13+4i
22     c1++;        cout<<c1<<"\n"; // 4+5i
23     ++c1;        cout<<c1<<"\n"; // 5+6i
24     c3=c1(5);    cout<<c3<<"\n"; // 25+30i
25 }

```

Q16. What is wrong with the following code segment?

```

1  class myclass {
2      int i, j;
3  public:
4      static void seti(int x) { this->i = x; }
5      int geti() { return i; }
6  };

```

(6 marks)

[CSE 107 | 2017–18]

**Answer — Problem with this in static method**

**Problem:** `static void seti(int x){ this->i = x; }` — a **static** member function has no **this** pointer. This is a **compile error**.

**Fix:**

```
1 // Remove static -- make it non-static
2 void seti(int x) { i = x; } // has 'this'
```

**Q17.** Consider the following `MyInt` class. Overload: (i) `+` for adding two `MyInt` objects, (ii) `*` so a `MyInt` can be multiplied by an integer from either side, (iii) `=` for safe assignment. Note: you cannot write a copy constructor.

```
1 class MyInt {
2     int *x;
3 public:
4     MyInt(int argx) { x = new int; *x = argx; }
5     int getX() const { return *x; }
6     void setX(int argx) { *x = argx; }
7     ~MyInt() { delete x; }
8 };
```

(20 marks)

[CSE 107 | 2016–17]

**Answer — MyInt — overload `+`, `*` (both sides), `=`**

```
1 class MyInt {
2     int *x;
3 public:
4     MyInt(int argx){ x=new int; *x=argx; }
5     int getX() const { return *x; }
6     void setX(int argx){ *x=argx; }
7     ~MyInt(){ delete x; }
8
9     MyInt operator+(const MyInt& o) const { return MyInt(*x++
10         o.x); }
11     MyInt operator*(int n) const { return MyInt(*x*n
12         ); }
13     friend MyInt operator*(int n, const MyInt& m){ return
14         MyInt(n**m.x); }
15     MyInt& operator=(const MyInt& o){ if(this!=&o) *x=*o.x;
16         return *this; }
17 };
18 int main(){
19     MyInt m1(5), m2(3);
20     cout<<(m1+m2).getX()<<"\n"; // 8
21     cout<<(m1*4).getX() <<"\n"; // 20
22     cout<<(3*m2).getX() <<"\n"; // 9
23 }
```

**Q18.** How can you code ++ob1 and ob1++ differently? (3 marks) [CSE 107 | 2015–16]

**Answer — How to code ++ob (prefix) and ob++ (postfix) differently**

The dummy int parameter distinguishes postfix from prefix:

```

1  class Counter {
2      int val;
3  public:
4      Counter(int v=0): val(v){}
5      int get() const { return val; }
6
7      // Prefix: ++ob -- increment then return new value
8      Counter& operator++() { ++val; return *this; }
9
10     // Postfix: ob++ -- save old, increment, return old
11     Counter operator++(int) {
12         Counter old=*this; ++val; return old;
13     }
14 };
15 int main() {
16     Counter c(5);
17     Counter a=++c; // c=6, a=6
18     Counter b=c++; // b=6, c=7
19     cout<<a.get()<<" "<<b.get()<<" "<<c.get()<<"\n"; // 6 6 7
20 }
```

**Q19.** Let  $V = (x, y, z)$  be a vector. Define a class and include the minimum necessary members for solving: `v1 += (10 + v2);` (12 marks) [CSE 201 | 2015–16]

**Answer — Vector class for `v1 += (10 + v2)`**

```

1  class Vec {
2      double x, y, z;
3  public:
4      Vec(double a=0, double b=0, double c=0): x(a), y(b), z(c){}
5
6      Vec& operator+=(const Vec& v){ x+=v.x; y+=v.y; z+=v.z;
7          return *this; }
8
9      friend Vec operator+(double n, const Vec& v){
10         return Vec(v.x+n, v.y+n, v.z+n);
11     }
12     void print(){ cout<<"("<<x<<" "<<y<<" "<<z<<"")\n"; }
13 };
14 int main(){
15     Vec v1(1,2,3), v2(4,5,6);
```

```

16     v1 += (10+v2);    // 10+v2=(14,15,16); v1+=(14,15,16)
17     v1.print();      // (15,17,19)
18 }

```

**Q20.** Explain different types of casting used in C++ with examples. **(10 marks)** [CSE 201 | 2014–15; CSE 107 | 2015–16]

#### Answer — Four C++ casting types with examples

1. **static\_cast<T>**: Compile-time, related types.

```

1 int i = static_cast<int>(3.7);    // i=3

```

2. **dynamic\_cast<T>**: Safe run-time polymorphic downcast.

```

1 Base* b=new Derived();
2 Derived* d=dynamic_cast<Derived*>(b); // null if fails

```

3. **const\_cast<T>**: Adds/removes const.

```

1 const int x=10;
2 int* p=const_cast<int*>(&x);

```

4. **reinterpret\_cast<T>**: Low-level bit-reinterpretation.

```

1 int* p=new int(42);
2 long addr=reinterpret_cast<long>(p);

```

**Q21.** Why is RTTI a necessary feature of C++? **(5 marks)** [CSE 107 | 2015–16]

#### Answer — Why is RTTI necessary in C++?

RTTI (Run-Time Type Information) allows determining the actual type of an object at run time through a base class pointer.

**Necessity:**

1. **Safe downcasting with dynamic\_cast**: verifies the cast at run time; returns nullptr on failure — prevents corrupt memory access.

```

1 Base* b=new Derived();
2 Derived* d=dynamic_cast<Derived*>(b);
3 if(d) d->derivedMethod();

```

2. **typeid operator**: queries exact type.

```

1 cout<<typeid(*b).name()<<"\n"; // "Derived"

```

3. Enables correct type-specific operations in complex polymorphic hierarchies without unsafe C-style casts.

Requires at least one virtual function (vtable carries type info).

**Q22.** Consider the following class declaration:

```
1  class Coord {
2      int x, y;
3      char *pointname;
4  public:
5      // your code
6  };
```

- (a) Create necessary constructors to support: `Coord ob1, ob2, ob3;` and `Coord ob4(0, 0, "origin");` (b) Explain why a copy constructor works for initialization but not for assignment. Create the required copy constructor. (c) Create necessary methods for: `ob1 = ob2 + ob3 + ob4;` and `ob1 = 100 + ob2;` (increases both `x` and `y` by 100) and `int value = 100 + ob4;` (`value = 100 +  $\sqrt{ob4.x^2 + ob4.y^2}$` , use a conversion function). (d) How can you code `++ob1` and `ob1++` differently?

**(6+8+18+3 marks)**

[CSE 107 | 2015–16]

### Answer — Coord Class: Constructors, Operators, Conversion

```
1  #include <iostream>
2  #include <cstring>
3  #include <cmath>
4  using namespace std;
5
6  class Coord {
7      int x, y;
8      char* pointname;
9  public:
10     // (a) Default constructor
11     Coord() : x(0), y(0) {
12         pointname = new char[8];
13         strcpy(pointname, "unnamed");
14     }
15
16     // (a) Parameterized constructor
17     Coord(int x, int y, const char* name) : x(x), y(y) {
18         pointname = new char[strlen(name)+1];
19         strcpy(pointname, name);
20     }
21
22     // (b) Copy constructor -- deep copy
23     Coord(const Coord& other) : x(other.x), y(other.y) {
24         pointname = new char[strlen(other.pointname)+1];
```



```

25         strcpy(pointname, other.pointname);
26     }
27
28     ~Coord() { delete[] pointname; }
29
30     // Assignment operator
31     Coord& operator=(const Coord& other) {
32         if (this == &other) return *this;
33         delete[] pointname;
34         x = other.x; y = other.y;
35         pointname = new char[strlen(other.pointname)+1];
36         strcpy(pointname, other.pointname);
37         return *this;
38     }
39
40     // (c) operator+ : add two Coords
41     Coord operator+(const Coord& other) const {
42         return Coord(x + other.x, y + other.y, "result");
43     }
44
45     // (c) friend: 100 + ob2 -- adds 100 to both x and y
46     friend Coord operator+(int val, const Coord& c) {
47         return Coord(val + c.x, val + c.y, "result");
48     }
49
50     // (c) Conversion function: int value = 100 + ob4
51     // ob4 converts to double (magnitude), then 100 +
52     // magnitude
53     operator double() const {
54         return sqrt((double)x*x + (double)y*y);
55     }
56
57     // (d) prefix ++ob1
58     Coord& operator++() {
59         ++x; ++y;
60         return *this;
61     }
62
63     // (d) postfix ob1++
64     Coord operator++(int) {
65         Coord tmp(*this);
66         x++; y++;
67         return tmp;
68     }
69
70     void print() const {
71         cout << pointname << "(" << x << "," << y << ")" <<
72             endl;

```

```

71     }
72 };
73
74 int main() {
75     Coord ob1, ob2, ob3;
76     Coord ob4(0, 0, "origin");
77
78     // (c) ob1 = ob2 + ob3 + ob4
79     ob1 = ob2 + ob3 + ob4;
80     ob1.print();
81
82     // (c) ob1 = 100 + ob2
83     ob1 = 100 + ob2;
84     ob1.print();
85
86     // (c) int value = 100 + ob4 (uses conversion)
87     int value = (int)(100 + (double)ob4);
88     cout << "value = " << value << endl;
89
90     // (d) prefix and postfix
91     ++ob1;
92     ob1++;
93     ob1.print();
94     return 0;
95 }

```

**(b) Why copy constructor works for init but not assignment:**

The copy constructor is called only during **initialization** (e.g., `Coord c2 = c1`). For **assignment** (`c2 = c1` after both are constructed), the `operator=` is called instead. Without a custom `operator=`, the default does a shallow copy of the `char*` pointer, causing double-free bugs — hence the explicit `operator=` above.

**Q23.** Let  $x + iy$  be a complex number. Declare a class named `Complex` with members/friends for:

- (a) Creating instance: `Complex ob(12.0, 5.0);`
- (b) Addition: `ob1 = ob2 + ob3;`
- (c) Increment: `ob++;` and `++ob;`
- (d) Adding real number: `ob2 = ob1 + 100.0;` and `ob2 = 100.0 + ob1;`

**(20 marks)**

[CSE 201 | 2014–15]

**Answer — Complex class (constructor, +, ++, +double both sides)**

```

1  #include <iostream>
2  using namespace std;
3  class Complex {

```

```

4     double re, im;
5     public:
6         Complex(double r=0, double i=0): re(r), im(i){}
7         Complex operator+(const Complex& o) const { return
            Complex(re+o.re,im+o.im); }
8         Complex operator+(double n) const { return Complex(re+n,
            im); }
9         friend Complex operator+(double n, const Complex& c){
            return Complex(c.re+n,c.im); }
10        Complex& operator++()    { re++; im++; return *this; }
11        Complex operator++(int) { Complex old=*this; re++; im++;
            return old; }
12        void show() const { cout<<re<<"+"<<im<<"i\n"; }
13    };
14    int main(){
15        Complex ob1(12,5), ob2, ob3(1,1);
16        ob2=ob1+ob3;    ob2.show();        // 13+6i
17        ob1++;          ob1.show();        // 13+6i
18        ++ob1;          ob1.show();        // 14+7i
19        ob2=ob1+100.0;  ob2.show();        // 114+7i
20        ob2=100.0+ob1;  ob2.show();        // 114+7i
21    }

```

**Q24.** Consider the following class:

```

1     class coord {
2         int x, y;
3     public:
4         coord(int i = 0, int j = 0) { x = i; y = j; }
5         void getxy(int &i, int &j) { i = x; j = y; }
6     };

```

- Write a function to overload + operator where both x and y are incremented by the amount passed through parameter.
- Write a function that allows: `ob2 = 100 + ob1;`
- Write inserter and extractor functions.
- Rewrite the class definition incorporating the above four functions.

**(20 marks)**

[CSE 201 | 2013–14]

**Answer — coord class full overloading**

```

1     #include <iostream>
2     using namespace std;
3
4     class coord {

```

```

5     int x, y;
6     public:
7         coord(int i=0, int j=0): x(i), y(j){}
8         void getxy(int &i, int &j) const { i=x; j=y; }
9
10        // (a) ob + int
11        coord operator+(int i) const { return coord(x+i, y+i); }
12
13        // (b) int + ob (friend)
14        friend coord operator+(int i, const coord& ob) {
15            return coord(ob.x+i, ob.y+i);
16        }
17
18        // (c) inserter
19        friend ostream& operator<<(ostream& os, const coord& c) {
20            os<<"("<<c.x<<" "<<c.y<<" )"; return os;
21        }
22        // (c) extractor
23        friend istream& operator>>(istream& is, coord& c) {
24            is>>c.x>>c.y; return is;
25        }
26    };
27
28    int main() {
29        coord ob1(2,4);
30        cout << ob1+5 << "\n";    // (7,9)
31        cout << 100+ob1<< "\n";    // (102,104)
32        cout << ob1 << "\n";    // (2,4)
33    }

```

**Q25.** Consider the following coord class. Make necessary changes and overload the + operator so that it accepts both ob + int and int + ob without using unnecessary friend functions:

```

1     class coord {
2         int x, y;
3     public:
4         coord(int i = 0, int j = 0) { x = i; y = j; }
5         void getxy(int &i, int &j) { i = x; j = y; }
6     };

```

(10 marks)

[CSE 201 | 2011–12]

**Answer — coord ob+int and int+ob without unnecessary friend**

Since coord(int i=0, int j=0) is a non-explicit conversion constructor, the compiler implicitly converts an int to a coord. A single member operator+(coord) handles all cases:

```

1  class coord {
2      int x, y;
3  public:
4      coord(int i=0, int j=0): x(i), y(j){}
5      void getxy(int &i, int &j) const { i=x; j=y; }
6
7      coord operator+(const coord& ob2) const {
8          return coord(x+ob2.x, y+ob2.y);
9      }
10 };
11 int main() {
12     coord ob1(2,4), ob2(1,1);
13     coord r1 = ob1 + ob2;    // coord+coord
14     coord r2 = ob1 + 5;      // 5 converted to coord(5,5)
15     coord r3 = 3 + ob1;      // 3 converted to coord(3,3)
16     // r1=(3,5), r2=(7,9), r3=(5,7)
17 }

```

**Q26.** Consider the following declarations of `coord` class operators. Explain what happens when each statement is executed and why `o3 = 100 + o1`; will not work. Write a function to fix it.

```

1  class coord {
2      int x, y;
3  public:
4      coord(int i = 0, int j = 0) { x = i; y = j; }
5      void getxy(int &i, int &j) { i = x; j = y; }
6      coord operator+(coord ob2);
7      coord operator+(int i);
8      coord operator++();
9      bool operator==(coord ob2);
10     coord operator=(coord ob2);
11 };
12 coord o1(2, 4), o2, o3(3, 1);

```

(i) `(o1 + o2).getxy(p, q)`;    (ii) `o3 = o3 + o2 + o1`; **(15 marks)** [CSE 201 | 2010–11]

#### Answer — coord operator statements analysis; fix 100+o1

(i) `(o1+o2).getxy(p,q)`: `o1+o2` calls `operator+(coord)`  $\Rightarrow$  `coord(2,4)`. `getxy` sets `p=2`, `q=4`.

(ii) `o3=o3+o2+o1`: `(o3+o2)=coord(3,1)`, then `+o1=coord(5,5)`. `o3` becomes `(5,5)`.

**Why 100+o1 fails:** `int::operator+(coord)` does not exist. The member `operator+(int)` only handles `ob+int`, not `int+ob`.

**Fix:**

```

1 friend coord operator+(int i, const coord& ob) {
2     return coord(ob.x+i, ob.y+i);
3 }

```

**Q27.** Overload the < and > operators relative to the coord class. **(10 marks)** [CSE 201 | 2010–11]

#### Answer — Overload < and > for coord

```

1 bool operator<(const coord& ob) const {
2     return (x < ob.x) && (y < ob.y);
3 }
4 bool operator>(const coord& ob) const {
5     return (x > ob.x) && (y > ob.y);
6 }
7 // coord c1(1,2), c2(3,4);
8 // c1<c2 -> true; c2>c1 -> true

```

**Q28.** Create a function reverse() that takes a pointer to a string and a count parameter. Give count a default value that, when absent, reverses the entire string. **(7 marks)** [CSE 201 | 2010–11]

#### Answer — reverse() with default count parameter

```

1 #include <iostream>
2 #include <cstring>
3 using namespace std;
4
5 void reverse(char* s, int count=-1){
6     int len=(int)strlen(s);
7     if(count<0 || count>len) count=len;
8     for(int i=0,j=count-1; i<j; i++,j--) swap(s[i],s[j]);
9 }
10
11 int main(){
12     char s1 []="Hello";
13     reverse(s1);          cout<<s1<<"\n"; // olleH
14     char s2 []="Hello World";
15     reverse(s2,5);        cout<<s2<<"\n"; // olleH World
16 }

```

**Q29.** What is an operator function? Describe the syntax of operator overloading. List the restrictions imposed on operator overloading. **(5+4 marks)** [CSE 201 | 2007–08, 2009–10]

**Answer — Operator function, syntax, restrictions**

**Operator Function:** A function named `operator@` that defines the behaviour of operator `@` for user-defined types.

**Syntax:**

```
1 // Member function:
2 return_type operator@(params) { ... }
3 // Friend (non-member):
4 friend return_type operator@(param1, param2) { ... }
```

**Restrictions:**

1. Cannot create new operators; only overload existing ones.
2. Cannot change arity (number of operands).
3. Cannot change precedence or associativity.
4. Cannot overload: `::`, `.`, `.*`, `?:`, `sizeof`, `typeid`.
5. `=`, `[]`, `()`, `->` must be **member functions**.
6. At least one operand must be a user-defined type.

**Q30.** What is a conversion function? Let a class `coord` be defined as:

```
1 class coord {
2     int x, y;
3 public:
4     coord(int i, int j) { x = i; y = j; }
5 };
```

Write a conversion function that will automatically convert an object of `coord` to an integer. (6 marks) [CSE 201 | 2009–10]

**Answer — User-defined conversion function to built-in type**

```
1 #include <iostream>
2 #include <cmath>
3 using namespace std;
4
5 class coord {
6     int x, y;
7 public:
8     coord(int i, int j): x(i), y(j){}
9     // Conversion: coord -> int (Euclidean magnitude)
10    operator int() const { return (int)sqrt((double)(x*x+y*y))
11        }; }
12
13 int main() {
```

```

14     coord c(3, 4);
15     int i = c;           // implicit: 5
16     cout << i << "\n";   // 5
17     cout << (int)c;       // explicit cast: 5
18 }

```

**Q31.** Write the necessary operator functions to allow:  $c3 = 10 + c1 + c2$ ; where  $c1$ ,  $c2$ , and  $c3$  are `coord` objects. (14 marks) [CSE 201 | 2009–10]

**Answer —  $c3 = 10 + c1 + c2$  operator functions**

```

1  class coord {
2      int x, y;
3  public:
4      coord(int i=0, int j=0): x(i), y(j){}
5      void getxy(int &i, int &j) const { i=x; j=y; }
6
7      coord operator+(const coord& ob) const {
8          return coord(x+ob.x, y+ob.y);
9      }
10     friend coord operator+(int i, const coord& ob) {
11         return coord(ob.x+i, ob.y+i);
12     }
13 };
14 // c3 = 10+c1+c2: (10+c1(2,4))=(12,14); +c2(3,1)=(15,15)

```

**Q32.** Write a matrix class with default and parameterized constructors. Overload  $+$  for matrix addition ( $m3 = m1 + m2$ ) and  $*$  for matrix multiplication ( $m3 = m1 * m2$ ). (20 marks) [CSE 201 | 2009–10]

**Answer — Matrix class with  $+$  and  $*$  operators**

```

1  class Matrix {
2      int **d; int r, c;
3  public:
4      Matrix(int rows=0, int cols=0): r(rows), c(cols){
5          d=new int*[r]; for(int i=0;i<r;i++){d[i]=new int[c];
6              for(int j=0;j<c;j++) d[i][j]=0;}
7      }
8      int& at(int i, int j){ return d[i][j]; }
9      Matrix operator+(const Matrix& o) const {
10         Matrix res(r,c);
11         for(int i=0;i<r;i++) for(int j=0;j<c;j++) res.d[i][j]
12             =d[i][j]+o.d[i][j];
13         return res;
14     }
15 };

```



```

12     }
13     Matrix operator*(const Matrix& o) const {
14         Matrix res(r,o.c);
15         for(int i=0;i<r;i++) for(int j=0;j<o.c;j++) for(int k
            =0;k<c;k++) res.d[i][j]+=d[i][k]*o.d[k][j];
16         return res;
17     }
18 };

```

**Q33.** Show with an example how one can find the address of a particular overloaded member function of a class. (5 marks) [CSE 201 | 2009–10]

#### Answer — Address of specific overloaded member function

```

1  #include <iostream>
2  using namespace std;
3  class MyClass {
4  public:
5      void func(int x)      { cout<<"int: "    <<x    <<"\n"; }
6      void func(double x) { cout<<"double: " <<x    <<"\n"; }
7  };
8  int main(){
9      void (MyClass::*fp1)(int)      = &MyClass::func; // first
          overload
10     void (MyClass::*fp2)(double) = &MyClass::func; // second
          overload
11
12     MyClass obj;
13     (obj.*fp1)(10);    // int: 10
14     (obj.*fp2)(3.14);  // double: 3.14
15 }

```

The exact type of the function pointer selects the correct overload.

**Q34.** Write a class named MyString with the following: (i) char\* data, (ii) int length, (iii) default, parameterized, and copy constructors, (iv) operator+ for concatenation, (v) operator- to remove all occurrences of substring, (vi) overload = and copy constructor if needed. (22 marks) [CSE 201 | 2008–09]

#### Answer — MyString Class with Operator Overloading

```

1  #include <iostream>
2  #include <cstring>
3  using namespace std;
4
5  class MyString {

```

```
6     char* data;
7     int    length;
8 public:
9     // Default constructor
10    MyString() : length(0) {
11        data = new char[1];
12        data[0] = '\\0';
13    }
14
15    // Parameterized constructor
16    MyString(const char* s) {
17        length = strlen(s);
18        data = new char[length + 1];
19        strcpy(data, s);
20    }
21
22    // Copy constructor
23    MyString(const MyString& other) {
24        length = other.length;
25        data = new char[length + 1];
26        strcpy(data, other.data);
27    }
28
29    // Destructor
30    ~MyString() { delete[] data; }
31
32    // Assignment operator
33    MyString& operator=(const MyString& other) {
34        if (this == &other) return *this;
35        delete[] data;
36        length = other.length;
37        data = new char[length + 1];
38        strcpy(data, other.data);
39        return *this;
40    }
41
42    // operator+ : concatenation
43    MyString operator+(const MyString& other) const {
44        char* tmp = new char[length + other.length + 1];
45        strcpy(tmp, data);
46        strcat(tmp, other.data);
47        MyString result(tmp);
48        delete[] tmp;
49        return result;
50    }
51
52    // operator- : remove all occurrences of substring
53    MyString operator-(const MyString& sub) const {
```

```

54     char* result = new char[length + 1];
55     char* dst    = result;
56     const char* src = data;
57     int subLen = sub.length;
58
59     while (*src) {
60         if (strncmp(src, sub.data, subLen) == 0) {
61             src += subLen;    // skip the substring
62         } else {
63             *dst++ = *src++;
64         }
65     }
66     *dst = '\0';
67     MyString r(result);
68     delete[] result;
69     return r;
70 }
71
72 void print() const { cout << data << endl; }
73 };
74
75 int main() {
76     MyString s1("Hello ");
77     MyString s2("World");
78     MyString s3 = s1 + s2;
79     s3.print();                // Hello World
80
81     MyString s4("abcabcabc");
82     MyString s5("abc");
83     MyString s6 = s4 - s5;
84     s6.print();                // (empty string)
85
86     MyString s7(s3);           // copy constructor
87     s7.print();                // Hello World
88     return 0;
89 }

```

**Output:**

Hello World

Hello World

**Q35.** Create a class `Matrix` (of double values). Include member functions to: (i) create a matrix of size  $m \times n$ , (ii) print the matrix, (iii) overload the parenthesis operator `()` as follows:

```

1 Matrix m(10, 20);
2 m(1, 2) = 4.5;    // access an element

```

```

3 m(); // erase matrix, set all to zero
4 cout << m(1, 2); // print value at (1,2)

```

(10 marks)

[CSE 201 | 2007–08]

**Answer — Matrix class with operator()**

```

1 #include <iostream>
2 using namespace std;
3 class Matrix {
4     double **data; int rows, cols;
5 public:
6     Matrix(int m, int n): rows(m), cols(n){
7         data=new double*[m];
8         for(int i=0;i<m;i++){data[i]=new double[n]; for(int j
9             =0;j<n;j++) data[i][j]=0;}
10    }
11    ~Matrix(){ for(int i=0;i<rows;i++) delete[] data[i];
12        delete[] data; }
13    double& operator()(int i, int j){ return data[i][j]; } //
14        access/set
15    void operator()(int i, int j){ for(int i=0;i<rows;i++) for(int j=0;
16        j<cols;j++) data[i][j]=0; }
17    void print(){ for(int i=0;i<rows;i++){for(int j=0;j<cols;
18        j++) cout<<data[i][j]<<" "; cout<<"\n";} }
19 };
20 int main(){
21     Matrix m(3,3);
22     m(1,2)=4.5; cout<<m(1,2)<<"\n"; // 4.5
23     m(); cout<<m(1,2)<<"\n"; // 0
24 }

```

**Q36.** Write a class to represent a vector. Include functions to: (i) create, (ii) modify an element, (iii) add two vectors, (iv) compute dot product, (v) print. (10 marks)  
[CSE 201 | 2007–08]

**Answer — Vector Class with Arithmetic Operations**

```

1 #include <iostream>
2 using namespace std;
3
4 class Vector {
5     float* elem;
6     int size;
7 public:
8     // (i) Create
9     Vector(int n) : size(n) {
10        elem = new float[n]();

```

```
11     }
12
13     ~Vector() { delete[] elem; }
14
15     // (ii) Modify an element
16     void set(int i, float val) {
17         if (i >= 0 && i < size) elem[i] = val;
18     }
19
20     float get(int i) const { return elem[i]; }
21     int  getSize() const { return size; }
22
23     // (iii) Add two vectors
24     Vector operator+(const Vector& v) const {
25         Vector result(size);
26         for (int i = 0; i < size; i++)
27             result.elem[i] = elem[i] + v.elem[i];
28         return result;
29     }
30
31     // (iv) Dot product
32     float dot(const Vector& v) const {
33         float sum = 0;
34         for (int i = 0; i < size; i++)
35             sum += elem[i] * v.elem[i];
36         return sum;
37     }
38
39     // (v) Print
40     void print() const {
41         cout << "[ ";
42         for (int i = 0; i < size; i++)
43             cout << elem[i] << " ";
44         cout << "]" << endl;
45     }
46 };
47
48 int main() {
49     Vector v1(3), v2(3);
50     v1.set(0, 1); v1.set(1, 2); v1.set(2, 3);
51     v2.set(0, 4); v2.set(1, 5); v2.set(2, 6);
52
53     cout << "v1 = "; v1.print();
54     cout << "v2 = "; v2.print();
55
56     Vector v3 = v1 + v2;
57     cout << "v1 + v2 = "; v3.print();
58 }
```

```
59     cout << "v1 . v2 = " << v1.dot(v2) << endl;  
60     return 0;  
61 }
```

**Output:**

```
v1 = [ 1 2 3 ]  
v2 = [ 4 5 6 ]  
v1 + v2 = [ 5 7 9 ]  
v1 . v2 = 32
```

BUET · CSE 107 / CSE 201

# Object Oriented Programming

Section S7

---

Inheritance & Polymorphism (C++)

## S7 — Inheritance & Polymorphism (C++)

- Q1.** What is the diamond problem? State with example how the diamond problem can be solved using virtual inheritance. Differentiate between virtual function and virtual inheritance. (4+3+3 marks) [CSE 107 | 2023–24]

### Answer — Diamond problem, virtual inheritance, virtual function vs virtual inheritance

#### The Diamond Problem:

When a class D inherits from B and C, both of which inherit from A, D contains **two copies** of A's members — one from B and one from C. Accessing A's members through D is **ambiguous**:

```
1 class A { public: int x; };
2 class B : public A {};
3 class C : public A {};
4 class D : public B, public C {};
5 // D has B::A::x and C::A::x -- ambiguous!
```

#### Solution — Virtual Inheritance:

Declare the inheritance from A as virtual in both B and C. The compiler ensures only **one shared copy** of A exists in D:

```
1 class A { public: int x; void show(){ cout<<x<<"\n"; } };
2 class B : virtual public A {}; // virtual inheritance
3 class C : virtual public A {}; // virtual inheritance
4 class D : public B, public C {}; // single copy of A
5
6 int main(){
7     D d;
8     d.x = 42; // no ambiguity -- only one x
9     d.show(); // 42
10 }
```

#### Virtual Function vs Virtual Inheritance:

| Virtual Function                                                                                                   | Virtual Inheritance                                                                                      |
|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Enables <b>runtime polymorphism</b> : the correct overridden function is selected based on the actual object type. | Ensures only <b>one copy</b> of a base class exists in a diamond hierarchy — solves the diamond problem. |
| Declared with <b>virtual</b> in the base class function.                                                           | Declared with <b>virtual</b> in the inheritance specifier.                                               |
| <code>virtual void f() { }</code>                                                                                  | <code>class B : virtual public A { }</code>                                                              |

- Q2.** Demonstrate the access of private, public, and protected members of a base class in a derived class and in main, when the derived class inherits using **private** inheritance. (12 marks) [CSE 107 | 2022–23]



## Answer — Private inheritance — access demo in derived and main

```

1  #include <iostream>
2  using namespace std;
3
4  class Base {
5      int priv = 1;
6  protected:
7      int prot = 2;
8  public:
9      int pub = 3;
10     void showBase(){ cout<<"priv="<<priv<<" prot="<<prot<<"
11                     pub="<<pub<<"\n"; }
12 };
13
14 class Derived : private Base {
15     // pub is now PRIVATE in Derived
16     // prot is now PRIVATE in Derived
17     // priv is INACCESSIBLE
18 public:
19     void showDerived(){
20         // priv = 10; // ERROR: inaccessible (private in
21         //           Base)
22         prot = 20;    // OK: accessible inside Derived (was
23                     // protected)
24         pub = 30;     // OK: accessible inside Derived (was
25                     // public)
26         showBase();   // OK: showBase() is accessible inside
27                     // Derived
28     }
29 };
30
31 int main(){
32     Derived d;
33     d.showDerived();    // OK: public method of Derived
34
35     // d.pub = 5;       // ERROR: pub is private via private
36                       // inheritance
37     // d.showBase();    // ERROR: showBase() is private via
38                       // private inheritance
39     return 0;
40 }

```

**Summary:** With private inheritance, all inherited members become private in the derived class — inaccessible from outside the derived class (main), but still accessible within the derived class's own member functions.

**Q3.** Design a program to show the use of virtual functions in achieving runtime polymorphism. (12 marks) [CSE 107 |

2022–23]

## Answer — Runtime polymorphism with virtual functions

```
1  #include <iostream>
2  using namespace std;
3
4  class Shape {
5  public:
6      virtual double area() const = 0;
7      virtual void show() const { cout<<"Shape\n"; }
8      virtual ~Shape(){}
9  };
10
11 class Circle : public Shape {
12     double r;
13 public:
14     Circle(double r): r(r){}
15     double area() const override { return 3.14159*r*r; }
16     void show() const override { cout<<"Circle, area="<<
17         area()<<"\n"; }
18 };
19
20 class Rectangle : public Shape {
21     double w, h;
22 public:
23     Rectangle(double w, double h): w(w), h(h){}
24     double area() const override { return w*h; }
25     void show() const override { cout<<"Rectangle, area="<<
26         area()<<"\n"; }
27 };
28
29 class Triangle : public Shape {
30     double b, h;
31 public:
32     Triangle(double b, double h): b(b), h(h){}
33     double area() const override { return 0.5*b*h; }
34     void show() const override { cout<<"Triangle, area="<<
35         area()<<"\n"; }
36 };
37
38 int main(){
39     Shape* shapes[] = {
40         new Circle(3),
41         new Rectangle(4,5),
42         new Triangle(6,8)
43     };
44     for(Shape* s: shapes){
45         s->show();          // correct function called at run time
```

```

43         delete s;
44     }
45     return 0;
46 }

```

**Output:**

```

1 Circle, area=28.2743
2 Rectangle, area=20
3 Triangle, area=24

```

**Q4.** Describe the use of a virtual base class. Explain why it is not possible to create an object of an abstract class. (12 marks) [CSE 107 | 2021–22]

**Answer — Virtual base class use; why abstract class cannot be instantiated**

**Use of virtual base class:** Eliminates the duplicate base class subobject in diamond inheritance, providing a single shared instance of the base. (See S7-Q3 for full example.)

**Why an abstract class cannot be instantiated:**

An **abstract class** is a class containing at least one **pure virtual function** (= 0).

A pure virtual function has **no implementation** in the abstract class.

If the compiler allowed creating an object of the abstract class, that object might have a call to the pure virtual function, which has no body — this would lead to undefined behaviour. The compiler therefore **prevents instantiation** of any class with unimplemented pure virtual functions.

```

1 class Shape {
2 public:
3     virtual double area() const = 0; // pure virtual: no
        body
4 };
5
6 // Shape s; // ERROR: cannot instantiate abstract class
7 Shape* p = new Circle(5); // OK: pointer to abstract class
        is fine

```

A derived class that implements *all* pure virtual functions becomes **concrete** and can be instantiated.

**Q5.** Describe the differences between function overloading and function overriding. (10 marks) [CSE 107 | 2021–22]

↪ *Similar: CSE 107 | 2019–20*

## Answer — Function overloading vs function overriding

| Function Overloading                                                                                         | Function Overriding                                                                                                       |
|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Multiple functions with the <b>same name</b> but <b>different parameter lists</b> in the <b>same scope</b> . | A derived class provides a <b>new implementation</b> for a <b>virtual function</b> that already exists in the base class. |
| Resolved at <b>compile time</b> (static polymorphism).                                                       | Resolved at <b>run time</b> (dynamic polymorphism).                                                                       |
| Functions differ in number/type of parameters.                                                               | Function signatures must <b>match exactly</b> .                                                                           |
| No inheritance required.                                                                                     | Requires inheritance and <b>virtual</b> in base.                                                                          |

```

1 // Overloading (same class):
2 class Calc {
3 public:
4     int    add(int a, int b)        { return a+b; }
5     double add(double a, double b){ return a+b; } //
6         overloaded
7 };
8 // Overriding (inheritance):
9 class Animal { public: virtual void speak(){ cout<<"...sound\
10     n"; } };
11 class Dog : public Animal {
12 public:
13     void speak() override { cout<<"Woof!\n"; } // overridden
14 };

```

**Q6.** Demonstrate the order of execution of constructor functions for multilevel inheritance with an example program. (11 marks) [CSE 107 | 2021–22, 2019–20]

## Answer — Constructor order in multilevel and multiple inheritance

Multilevel inheritance — constructors called from base to most-derived; destructors in reverse:

```

1 #include <iostream>
2 using namespace std;
3
4 class Base { public:
5     Base() { cout<<"Base constructor\n"; }
6     ~Base(){ cout<<"Base destructor\n"; }
7 };
8 class Mid : public Base { public:
9     Mid() { cout<<"Mid constructor\n"; }
10    ~Mid() { cout<<"Mid destructor\n"; }

```

```

11 };
12 class Der : public Mid { public:
13     Der() { cout<<"Der constructor\n"; }
14     ~Der() { cout<<"Der destructor\n"; }
15 };
16
17 int main(){ Der d; }

```

**Output:**

```

1 Base constructor
2 Mid constructor
3 Der constructor
4 Der destructor
5 Mid destructor
6 Base destructor

```

**Multiple inheritance — constructor order follows declaration:**

```

1 class X { public: X(){cout<<"X\n";} ~X(){cout<<"~X\n";} };
2 class Y { public: Y(){cout<<"Y\n";} ~Y(){cout<<"~Y\n";} };
3 class Z : public X, public Y { public: Z(){cout<<"Z\n";} ~Z()
    {cout<<"~Z\n";} };
4 int main(){ Z z; }
5 // Output: X Y Z ~Z ~Y ~X

```

**Q7.** Is it possible that a derived class inherits more than one copy of the base class? If yes, show the scenario; how can you avoid it? **(15 marks)** [CSE 107 | 2019–20]

**Answer — Can a derived class inherit more than one copy of base? How to avoid?**

**Yes** — without virtual inheritance, a derived class in a diamond hierarchy inherits **multiple copies** of the base:

```

1 class A { public: int x=0; };
2 class B : public A {}; // B has A::x
3 class C : public A {}; // C has A::x
4 class D : public B, public C {}; // D has B::A::x AND C::A::
    x
5
6 D d;
7 // d.x = 5; // ERROR: ambiguous -- which x?
8 d.B::x = 5; // must qualify explicitly
9 d.C::x = 10;

```

**How to avoid — virtual inheritance:**

```

1 class A { public: int x=0; };
2 class B : virtual public A {};

```

```

3 class C : virtual public A {};
4 class D : public B, public C {};
5
6 D d;
7 d.x = 42;    // unambiguous -- single shared A
8 cout << d.x; // 42

```

With `virtual`, only one copy of `A` is maintained in `D`, shared by both `B` and `C` paths.

**Q8.** Differentiate between compile time polymorphism and run time polymorphism. (8 marks) [CSE 107 | 2019–20]

#### Answer — Compile-time vs run-time polymorphism

| Compile-Time Polymorphism                                           | Run-Time Polymorphism                                              |
|---------------------------------------------------------------------|--------------------------------------------------------------------|
| Resolved by compiler before execution.                              | Resolved during program execution.                                 |
| Achieved via function overloading, operator overloading, templates. | Achieved via virtual functions and base class pointers/references. |
| Faster — no vtable lookup overhead.                                 | Slight overhead — vtable lookup per call.                          |
| Also called <i>static polymorphism</i> .                            | Also called <i>dynamic polymorphism</i> .                          |

```

1 // Compile-time:
2 void print(int x)    { cout<<x<<"\n"; }
3 void print(double x) { cout<<x<<"\n"; }
4 print(5);           // compiler picks print(int)
5 print(3.14);        // compiler picks print(double)
6
7 // Run-time:
8 class Shape { public: virtual double area()const=0; };
9 class Circle: public Shape { double r; public: Circle(double
    r):r(r){}
10     double area()const override{return 3.14159*r*r;} };
11 Shape* s=new Circle(3);
12 cout<<s->area();    // decided at run time: Circle::area()

```

**Q9.** Describe a scenario when we cannot create an object of a class. (7 marks) [CSE 107 | 2019–20]

↪ Similar: CSE 201 | 2007–08

#### Answer — Scenario when an object of a class cannot be created

##### Scenario 1 — Abstract class (pure virtual function):

A class with at least one unimplemented pure virtual function is *abstract*; no object can be created from it.

```

1 class Shape { public: virtual double area()=0; };
2 // Shape s; // ERROR

```

### Scenario 2 — Private constructor (Singleton pattern):

The constructor is declared `private`; objects can only be created through a controlled static factory method.

```

1 class Singleton {
2     Singleton(){}
3 public:
4     static Singleton& get(){ static Singleton s; return s; }
5 };
6 // Singleton s; // ERROR: constructor is private
7 Singleton& s = Singleton::get(); // OK

```

### Scenario 3 — Deleted constructor:

```

1 class NoInstance {
2 public:
3     NoInstance() = delete;
4 };
5 // NoInstance n; // ERROR: use of deleted function

```

**Q10.** Why virtual base classes are used? Give an example of a class hierarchy where virtual base class is necessary to avoid ambiguity. (9 marks) [CSE 107 | 2017–18]

### Answer — Virtual base class — why and when needed

**Virtual Base Class:** A base class declared with `virtual` in the inheritance specifier. It ensures that when a class appears multiple times in a hierarchy (diamond), only **one shared instance** of that base is included in the most-derived class.

**When needed:** Whenever the same base class is inherited by two or more intermediate classes that are themselves combined in a derived class (diamond problem).

**Example:**

```

1 #include <iostream>
2 using namespace std;
3
4 class Vehicle {
5 public:
6     int speed;
7     Vehicle(int s=0): speed(s){}
8     void show(){ cout<<"Speed: "<<speed<<"\n"; }
9 };
10
11 class Car : virtual public Vehicle {};
12 class Boat : virtual public Vehicle {};
13 class Amphibian : public Car, public Boat {

```

```

14 public:
15     Amphibian(int s): Vehicle(s){} // initialises the ONE
        shared Vehicle
16 };
17
18 int main(){
19     Amphibian a(120);
20     a.speed = 80; // no ambiguity
21     a.show(); // Speed: 80
22 }

```

Without virtual, Amphibian would have Car::Vehicle and Boat::Vehicle as two separate subobjects; a.speed would be ambiguous.

**Q11.** Can constructor or destructor functions be declared as virtual? Explain briefly. (5 marks) [CSE 107 | 2017–18]

**Q12.** Explain what is wrong with the following code. Rewrite to solve the problem. What will be the output after fixing?

```

1 class A {
2     int x;
3 public:
4     void setX(int i) { x = i; }
5     void print() { cout << x; }
6 };
7 class B : public A { public: B() { setX(10); } };
8 class C : public A { public: C() { setX(20); } };
9 class D : public B, public C { };
10 int main() { D d; d.print(); return 0; }

```

(15 marks)

[CSE 107 | 2016–17]

**Answer — Fix diamond ambiguity in class D; output after fix**

**Problem:** D inherits from both B and C, each of which has its own copy of A. d.print() is **ambiguous** — the compiler does not know whether to call B::A::print() or C::A::print().

**Fix — make A a virtual base class:**

```

1 #include <iostream>
2 using namespace std;
3
4 class A {
5     int x;
6 public:
7     void setX(int i) { x = i; }
8     void print() { cout << x << "\n"; }
9 };

```



```

10
11 class B : virtual public A { // virtual inheritance
12 public:
13     B() { setX(10); }
14 };
15
16 class C : virtual public A { // virtual inheritance
17 public:
18     C() { setX(20); }
19 };
20
21 class D : public B, public C {};
22
23 int main() {
24     D d;           // B sets x=10, then C sets x=20 (one shared
25                   // copy)
26     d.print();    // unambiguous: prints 20
27     return 0;
28 }

```

Output after fixing:

```
1 20
```

**Reason:** The single shared `A::x` is set to 10 by B's constructor, then overwritten to 20 by C's constructor (base constructors are called in declaration order).

**Q13.** Briefly explain the visibility of private, protected, and public members of a class in a derived class when the base class is inherited as private, protected, and public. (15 marks) [CSE 107 | 2016–17]

### Answer — Visibility of Class Members in Inheritance

When a derived class inherits from a base class, the visibility of the inherited members depends on the access specifier used during inheritance. Regardless of the inheritance mode, **private** members of the base class are **never** directly accessible from the derived class.

#### 1. Public Inheritance:

- public members of the base class remain **public** in the derived class.
- protected members of the base class remain **protected** in the derived class.

#### 2. Protected Inheritance:

- Both public and protected members of the base class become **protected** in the derived class. They are accessible inside the derived class but inaccessible from outside.

#### 3. Private Inheritance:

- Both `public` and `protected` members of the base class become `private` in the derived class. They can only be accessed by the member functions of the derived class itself.

#### Visibility Summary Table:

| Base member | Inheritance type |              |              |
|-------------|------------------|--------------|--------------|
|             | public           | protected    | private      |
| public      | public           | protected    | private      |
| protected   | protected        | protected    | private      |
| private     | inaccessible     | inaccessible | inaccessible |

#### Example — Demonstrating Private Inheritance:

```

1  #include <iostream>
2  using namespace std;
3
4  class Base
5  {
6      int priv = 1;
7
8  protected:
9      int prot = 2;
10
11  public:
12      int pub = 3;
13  };
14
15  class Der : private Base
16  {
17      //    pub -> becomes private in Der (accessible inside
18          //    Der only)
19      //    prot -> becomes private in Der (accessible inside
20          //    Der only)
21      //    priv -> totally inaccessible
22  public:
23      void show()
24      {
25          cout << "pub: " << pub << "\n";    // OK
26          cout << "prot: " << prot << "\n"; // OK
27          // cout << priv;                // ERROR:
28              inaccessible
29      }
30  };
31
32  int main()
33  {
34      Der d;

```

```

32     // d.pub = 5; // ERROR: pub is private via private
        inheritance
33     d.show(); // OK: show() is a public function in Der
34     return 0;
35 }

```

**Q14.** Suppose class A has virtual functions and class B publicly inherits from A. What happens if class B does not implement some virtual functions? What if they are pure virtual? (10 marks) [CSE 107 | 2016–17]

**Answer — B inherits A (virtual); what if B doesn't implement some virtual functions?**

**Case 1 — Virtual functions (not pure):**

If B does not override a virtual function from A, B **inherits the base class implementation**. Calling the function through a B pointer uses A's version. B can still be instantiated.

```

1  class A { public: virtual void f(){ cout<<"A::f\n"; } };
2  class B : public A { }; // does not override f
3
4  B b;
5  b.f(); // calls A::f -- inherited
6  A* p = &b;
7  p->f(); // calls A::f (runtime dispatch still works)

```

**Case 2 — Pure virtual functions:**

If A has a pure virtual function and B does **not** implement it, B is also **abstract** — it cannot be instantiated.

```

1  class A { public: virtual void f()=0; }; // pure virtual
2  class B : public A { }; // does NOT
        override f
3  // B is still abstract
4  // B b; // ERROR: cannot instantiate abstract class B
5
6  class C : public A { public: void f(){ cout<<"C::f\n"; } };
7  C c; // OK: C implements f
8  c.f(); // C::f

```

**Q15.** Implement a **Queue** class by inheriting from the **List** class. Your queue class should override the three functions as follows:

- (i) **store** — override this to add a new item at the end of the queue
- (ii) **retrieve** — override this to remove and return the value from the beginning of the queue
- (iii) **print** — displays the queue items as a comma separated list at the stdout

```

1  class ListItem
2  {
3      int item;
4      ListItem * next;
5  public:
6      ListItem(int arg){ item = arg; next = NULL; }
7      int getItem(){ return item; }
8      void setNext(ListItem * n){ next = n; }
9      ListItem * getNext() { return next; }
10 };
11
12 class List
13 {
14 private:
15     ListItem * head;
16     ListItem * tail;
17 public:
18     List () { head = NULL; tail = NULL; }
19     virtual void store(int item)=0;
20     virtual int retrieve()=0;
21     virtual void print()=0;
22 };

```

(20 marks)

[CSE 107 | 2016–17, 2013–14]

### Answer — Queue inheriting List — store, retrieve, print

```

1  #include <iostream>
2  using namespace std;
3
4  class ListItem
5  {
6      int item;
7      ListItem * next;
8  public:
9      ListItem(int arg){ item = arg; next = NULL; }
10     int getItem(){ return item; }
11     void setNext(ListItem * n){ next = n; }
12     ListItem * getNext() { return next; }
13 };
14
15 class List
16 {
17 protected: // Note: Changed from private to protected so
18     Queue can access head & tail
19     ListItem * head;
20     ListItem * tail;
21 public:

```

```
21     List () { head = NULL; tail = NULL; }
22     virtual void store(int item)=0;
23     virtual int retrieve()=0;
24     virtual void print()=0;
25     virtual ~List() {
26         while(head) {
27             ListItem* t = head;
28             head = head->getNext();
29             delete t;
30         }
31     }
32 };
33
34 class Queue : public List {
35 public:
36     // (i) store - add at end of the queue
37     void store(int item) override {
38         ListItem* n = new ListItem(item);
39         if (tail == NULL) {
40             head = tail = n;
41         } else {
42             tail->setNext(n);
43             tail = n;
44         }
45     }
46
47     // (ii) retrieve - remove and return from beginning of
48     // the queue
49     int retrieve() override {
50         if (head == NULL) {
51             return -1; // Assuming -1 represents empty
52         }
53         int v = head->getItem();
54         ListItem* t = head;
55         head = head->getNext();
56         if (head == NULL) {
57             tail = NULL;
58         }
59         delete t;
60         return v;
61     }
62
63     // (iii) print - comma-separated list at stdout
64     void print() override {
65         ListItem* cur = head;
66         while (cur != NULL) {
67             cout << cur->getItem();
68             if (cur->getNext() != NULL) cout << ",";
```

```

68         cur = cur->getNext();
69     }
70     cout << "\n";
71 }
72 };
73
74 int main(){
75     Queue q;
76     q.store(10); q.store(20); q.store(30);
77     q.print();           // 10,20,30
78     cout << q.retrieve() << "\n"; // 10
79     q.print();           // 20,30
80     return 0;
81 }

```

**Q16.** Assume the class `hand` is inherited by both `man` and `monkey`. The class `animal` inherits both. What type of inheritance is this? What is the problem? How to solve it? (8 marks) [CSE 201 | 2015–16]

**Answer —** `hand/man/monkey/animal` — inheritance type, problem, solution

**Type of inheritance:** Multiple inheritance (diamond pattern) — `man` and `monkey` both inherit `hand`; `animal` inherits both, making it a *diamond hierarchy*.

**Problem:** `animal` inherits **two copies** of `hand` — one via `man` and one via `monkey`. Any reference to `hand`'s members inside `animal` is **ambiguous**.

**Solution —** virtual base class:

```

1  class hand {
2  public:
3      void wave() { cout << "Hand waves\n"; }
4  };
5  class man    : virtual public hand {};
6  class monkey : virtual public hand {};
7  class animal : public man, public monkey {};
8
9  int main() {
10     animal a;
11     a.wave(); // no ambiguity -- one shared copy of hand
12 }

```

**Q17.** What are the differences between virtual function and pure virtual function? If a class contains a pure virtual function, what is it called and what restrictions apply? (10 marks) [CSE 107 | 2015–16]

## Answer — Virtual vs pure virtual function; abstract class restrictions

| Virtual Function                   | Pure Virtual Function                                |
|------------------------------------|------------------------------------------------------|
| Has a body in the base class.      | Has <b>no</b> body (= 0).                            |
| Derived class <i>may</i> override. | Derived class <b>must</b> override.                  |
| Base class can be instantiated.    | Base class <b>cannot</b> be instantiated (abstract). |
| <code>virtual void f(){...}</code> | <code>virtual void f()=0;</code>                     |

## Abstract class — restrictions:

- Cannot create objects of an abstract class directly.
- Can declare pointers and references to it.
- A derived class that does not implement all pure virtual functions is also abstract.
- Constructors of abstract classes can be defined (called by derived constructors).

**Q18.** Consider the bookshop example: `media` base class with `title` and `publication`; `book` subclass with number of pages; `tape` subclass with playing time. Each class has virtual `read()` and `show()`. Write a program modeling this hierarchy and processing objects using base class pointers. (20 marks) [CSE 201 | 2015–16]

## Answer — Bookshop hierarchy — media, book, tape with virtual functions

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class media {
6  protected:
7      string title, publication;
8  public:
9      media(string t, string p): title(t), publication(p){}
10     virtual void read() const = 0;
11     virtual void show() const = 0;
12     virtual ~media(){}
13 };
14
15 class book : public media {
16     int pages;
17 public:
18     book(string t, string p, int pg): media(t, p), pages(pg){}
19     void read() const override { cout << "Reading book: " <<

```

```

        title<<"\n"; }
20     void show() const override {
21         cout<<"Book: "<<title<<" | "<<publication<<" | "<<
            pages<<" pages\n";
22     }
23 };
24
25 class tape : public media {
26     double playtime;
27 public:
28     tape(string t,string p,double pt): media(t,p), playtime(
        pt){}
29     void read() const override { cout<<"Playing tape: "<<
        title<<"\n"; }
30     void show() const override {
31         cout<<"Tape: "<<title<<" | "<<publication<<" | "<<
            playtime<<" min\n";
32     }
33 };
34
35 int main(){
36     media* arr[] = {
37         new book("C++ Primer","Pearson",900),
38         new tape("Data Structures","MIT",60.0)
39     };
40     for(auto m: arr){ m->show(); m->read(); delete m; }
41     return 0;
42 }

```

**Q19.** Suppose there are three classes A, B and C. Class C wants to inherit both A and B simultaneously. Describe two ways with code examples. (5 marks) [CSE 201 | 2012–13]

**Answer — Two ways for class C to inherit both A and B**

**Way 1 — Direct multiple inheritance:**

```

1  class A { public: void showA(){ cout<<"A\n"; } };
2  class B { public: void showB(){ cout<<"B\n"; } };
3  class C : public A, public B {};    // inherits both directly
4
5  int main(){
6      C c;
7      c.showA(); c.showB();    // A B
8  }

```

**Way 2 — Multilevel (chain) inheritance:**



```

1  class A { public: void showA(){ cout<<"A\n"; } };
2  class B : public A { public: void showB(){ cout<<"B\n"; } };
3  class C : public B {}; // C gets both A and B via chain
4
5  int main(){
6      C c;
7      c.showA(); c.showB(); // A B
8  }

```

**Q20.** What is wrong with the following code segment?

```

1  class base {
2      int x;
3  public:
4      void setx(int n) { x = n; }
5      void showx() { cout << x << "\n"; }
6  };
7  class derived : private base {
8      int y;
9  public:
10     void sety(int n) { y = n; }
11     void showy() { cout << y << "\n"; }
12 };
13 int main() {
14     derived ob;
15     ob.setx(10); ob.sety(20);
16     ob.showx(); ob.showy();
17     return 0;
18 }

```

(5 marks)

[CSE 201 | 2012–13]

### Answer — Problem in private inheritance code

```

1  class derived : private base { ... };
2  int main(){
3      derived ob;
4      ob.setx(10); // ERROR: setx is private in derived (
5                  private inheritance)
6      ob.showx(); // ERROR: showx is private in derived
7      ...
8  }

```

**Problem:** derived inherits base as private. All public members of base (setx, showx) become **private** in derived and therefore **cannot be called from main**.

**Fix options:**

- (i) Change to public inheritance: class derived : public base

- (ii) Add public wrapper methods in `derived`.
- (iii) Use `using base::setx;` inside the public section of `derived` to re-expose specific members.

```

1 // Fix (i):
2 class derived : public base { // public inheritance
3     int y;
4 public:
5     void sety(int n){ y=n; }
6     void showy(){ cout<<y<<"\n"; }
7 };
8 int main(){
9     derived ob;
10    ob.setx(10); ob.sety(20); // now OK
11    ob.showx(); ob.showy();
12 }

```

**Q21.** Define virtual function and pure virtual function with appropriate examples. Differentiate between early binding and late binding. (10 marks) [CSE 201 | 2011–12]

**Answer — Virtual function, pure virtual function, early vs late binding**

**Virtual Function:** Has a default implementation in the base class; derived classes may override it.

```

1 class Shape { public: virtual double area(){ return 0; } };

```

**Pure Virtual Function:** Has **no** implementation in the base class (= 0); derived classes *must* override it. Makes the class abstract.

```

1 class Shape { public: virtual double area() const = 0; };

```

**Early binding (compile time):** Function call resolved by the compiler based on the declared type. Non-virtual functions.

**Late binding (run time):** Function call resolved at run time based on the actual object type. Virtual functions via vtable.

```

1 class A { public: void f(){cout<<"A::f\n";} virtual void g(){
    cout<<"A::g\n";} };
2 class B : public A { public: void f(){cout<<"B::f\n";} void g
    () override{cout<<"B::g\n";} };
3
4 A* p=new B();
5 p->f(); // early binding: A::f (declared type is A*)
6 p->g(); // late binding: B::g (actual object is B)

```

**Q22.** Define virtual base class with examples. Explain why a virtual base class might be necessary. (4 marks) [CSE 201 | 2011–12]

**Q23.** What happens when a public member is inherited as public, private, or protected? (8+9 marks) [CSE 201 | 2011–12, 2010–11]

#### Answer — Member visibility table for public/private/protected inheritance

When a base class member is inherited, its accessibility in the derived class depends on **both** the original access specifier **and** the inheritance type:

| Base member | public inh.    | protected inh. | private inh.   |
|-------------|----------------|----------------|----------------|
| public      | public         | protected      | private        |
| protected   | protected      | protected      | private        |
| private     | not accessible | not accessible | not accessible |

#### Key rules:

- **private members** of the base are *never* accessible in derived classes regardless of inheritance type.
- **public inheritance** preserves the original access levels (IS-A relationship).
- **protected inheritance** limits public members to protected in the derived class.
- **private inheritance** makes everything private in the derived class.

**Q24.** In what order are constructors called for `class myclass : public A, public B, public C { }`? In what order are destructors called? (3 marks) [CSE 201 | 2011–12]

#### Answer — Constructor/destructor order for class D : public A, public B, public C

For class `myclass : public A, public B, public C`:

**Constructor order:** `A() → B() → C() → myclass()` (left-to-right declaration order, base before derived).

**Destructor order** (always reverse of constructors): `~myclass() → ~C() → ~B() → ~A()`.

**Q25.** Define virtual function. Why is virtual function used in C++ programming? (5 marks) [CSE 201 | 2010–11]

↪ *Similar: CSE 201 | 2009–10; CSE 107 | 2017–18*

### Answer — Virtual function — definition and purpose

**Virtual Function:** A member function declared with the `virtual` keyword in a base class. When called through a base class pointer or reference, the **most-derived** version of the function is executed at *run time* (late binding / dynamic dispatch).

**Why used:**

1. **Runtime polymorphism:** Allows a single base-class pointer to correctly invoke the overridden function of whichever derived object it points to.
2. **Extensibility:** New derived classes can be added without changing code that uses base class pointers.
3. **Clean OOP design:** Separates interface (base class) from implementation (derived classes).

```

1  class Animal {
2  public:
3      virtual void speak() { cout<<"...sound\n"; }
4  };
5  class Dog : public Animal {
6  public:
7      void speak() override { cout<<"Woof!\n"; }
8  };
9  class Cat : public Animal {
10 public:
11     void speak() override { cout<<"Meow!\n"; }
12 };
13
14 int main(){
15     Animal* a;
16     Dog d; Cat c;
17     a=&d; a->speak(); // Woof!  -- runtime dispatch
18     a=&c; a->speak(); // Meow!
19 }
```

**Q26.** What is virtual base class? Why and when is it needed? Explain with an example. (12 marks) [CSE 201 | 2009–10]

↪ Similar: CSE 201 | 2008–09; CSE 107 | 2017–18

**Q27.** Constructors cannot be virtual, but destructors can be virtual. Why? (3 marks) [CSE 201 | 2009–10]

### Answer — Can constructors/destructors be virtual?

**Constructors cannot be virtual:**

- A virtual function call is dispatched through the vtable, which requires the

object to already exist and be fully constructed.

- During construction, the object is not yet complete; the vtable pointer is not yet set up for the derived class.
- Therefore, making a constructor virtual is meaningless and the C++ standard forbids it.

**Destructors *can* (and often *should*) be virtual:**

- When a derived object is deleted through a base class pointer, the correct (most-derived) destructor must be called.
- Without a virtual destructor, only the base destructor is called — the derived part is not cleaned up (memory/resource leak).

```

1  class Base {
2  public:
3      virtual ~Base(){ cout<<"Base dtor\n"; }  // MUST be
          virtual
4  };
5  class Der : public Base {
6  public:
7      ~Der(){ cout<<"Der dtor\n"; }
8  };
9
10 int main(){
11     Base* p = new Der();
12     delete p;
13     // With virtual dtor: "Der dtor", then "Base dtor"  --
          correct
14     // Without virtual:  "Base dtor" only              --
          LEAK
15 }
```

**Q28.** What are the implications of the following two definitions?

- (a) `class A : public B, public C { };`
- (b) `class A : public C, public B { };`

(4 marks)

[CSE 201 | 2007–08]

**Answer — Implications of `class A : public B, public C` vs `class A : public C, public B`**

Both declarations make A inherit publicly from **both** B and C (multiple inheritance). The only difference is the **order** in which base class subobjects are laid out in memory and the order in which constructors and destructors are called:

(a) `class A : public B, public C { };`

- Memory layout: B subobject first, then C.
- Constructors called: B() first, then C(), then A().
- Destructors in reverse: A, C, B.

(b) `class A : public C, public B { };`

- Memory layout: C subobject first, then B.
- Constructors called: C() first, then B(), then A().

Both are otherwise equivalent in terms of member accessibility. The order matters when B and C have members with the same name (ambiguity) or when constructor side effects depend on initialisation order.

**Q29.** When do we make a virtual function “pure”? What are the implications? Explain with example. (6 marks) [CSE 201 | 2007–08]

### Answer — When to make a virtual function pure; implications

#### When to make a virtual function pure:

Make a virtual function *pure* when:

- There is **no meaningful default implementation** in the base class.
- Every derived class *must* provide its own specific implementation.
- The base class is intended as an **interface** or **abstract type** — not to be instantiated directly.

#### Implications:

1. The class becomes **abstract**; no objects can be created from it.
2. Derived classes that do not implement the pure virtual function are also abstract.
3. Only **concrete** derived classes (those that override all pure virtuals) can be instantiated.

```

1  class Shape {                // abstract -- no meaningful default
    area
2  public:
3      virtual double area() const = 0; // pure virtual
4      virtual ~Shape() {}
5  };
6
7  class Circle : public Shape {
8      double r;
```

```
9 public:
10     Circle(double r): r(r){}
11     double area() const override { return 3.14159*r*r; } //
        must implement
12 };
13
14 // Shape s;          // ERROR: cannot instantiate abstract
        class
15 Shape* p = new Circle(5); cout << p->area(); delete p; //
        78.54
```

BUET · CSE 107 / CSE 201

# Object Oriented Programming

Section S8

---

Templates & STL



## S8 — Templates & STL

**Q1.** What is erasure in terms of generic class? Consider the following `main` function and write down associated generic classes where `Gen` inherits `NonGen` and `Gen2D` inherits `Gen`:

```

1  int main() {
2      NonGen ob(10);
3      Gen<char> ob1(20, 'A');
4      Gen2D<double, char *> ob2(40, 60.5, "Gen2Dobject");
5
6      ob.show();
7      ob1.show();
8      ob2.show();
9
10     cout << ob1.getob() << endl;
11     cout << ob2.getob() << endl;
12
13     return 0;
14 }
15 /* Output:
16     NonGen: 10
17     Gen: A
18     Gen2D: Gen2Dobject
19     A
20     Gen2Dobject
21 */

```

(15 marks)

[CSE 107 | 2023–24]

### Answer — Type erasure; NonGen, Gen, Gen2D hierarchy

#### Type Erasure in generic classes:

Type erasure is a process used in languages like Java where the compiler removes (erases) all information related to generic type parameters during compilation, replacing them with their bounds or an `Object` type. This means generic type information is not visible at runtime. *Note: C++ templates do not use type erasure; instead, they use monomorphization, instantiating separate, distinct copies of the code for each type used.*

```

1  #include <iostream>
2  using namespace std;
3
4  // Non-generic base class
5  class NonGen {
6      int num;
7  public:
8      NonGen(int n) : num(n) {}
9      void show() {
10         cout << "NonGen: " << num << "\n";

```

```

11     }
12 };
13
14 // Generic class inheriting NonGen
15 template <class T>
16 class Gen : public NonGen {
17     T ob;
18 public:
19     Gen(int n, T o) : NonGen(n), ob(o) {}
20
21     // Overriding show (without calling NonGen::show to match
22     // output)
23     void show() {
24         cout << "Gen: " << ob << "\n";
25     }
26
27     T getob() { return ob; }
28 };
29
30 // 2D generic class inheriting Gen
31 template <class T1, class T2>
32 class Gen2D : public Gen<T2> {
33     T1 ob2D;
34 public:
35     // Passes n and o2 to the base Gen<T2> constructor
36     Gen2D(int n, T1 o1, T2 o2) : Gen<T2>(n, o2), ob2D(o1) {}
37
38     // Overriding show to just print the T2 object
39     void show() {
40         // Using this-> to access dependent base class
41         // members in C++
42         cout << "Gen2D: " << this->getob() << "\n";
43     }
44
45     // Note: getob() is inherited from Gen<T2>, so ob2.getob()
46     // will naturally return the T2 (char *) object without
47     // needing
48     // to be rewritten or shadowed here.
49 };
50
51 int main() {
52     NonGen ob(10);
53     Gen<char> ob1(20, 'A');
54
55     // Note: Older C++ allowed string literals to char*,
56     // modern compilers may warn about this deprecation.
57     Gen2D<double, char*> ob2(40, 60.5, (char*)"Gen2Object");

```

```

55
56     ob.show();
57     ob1.show();
58     ob2.show();
59
60     cout << ob1.getob() << "\n";
61     cout << ob2.getob() << "\n";
62
63     return 0;
64 }

```

**Q2.** What is STL? Explain different fundamental items of STL. (5 marks) [CSE 107 | 2023–24]

#### Answer — What is STL? Fundamental items.

**STL (Standard Template Library):** A collection of generic, reusable C++ components — containers, algorithms, and iterators — built on templates.

**Four fundamental items of STL:**

1. **Containers:** Template classes that store collections of objects.
  - *Sequence:* `vector`, `list`, `deque`
  - *Associative:* `map`, `set`, `multimap`
  - *Unordered:* `unordered_map`, `unordered_set`
2. **Algorithms:** Generic functions that operate on containers via iterators: `sort`, `find`, `count`, `transform`, `copy`, etc.
3. **Iterators:** Objects that act like pointers, providing a uniform way to traverse containers regardless of their internal structure (`begin()`, `end()`, `++`, `*`).
4. **Function Objects (Functors):** Objects that behave like functions (overload `operator()`), used to customise algorithm behaviour (e.g. comparators, predicates).

**Q3.** Consider that a list container contains: 20 16 12 8 4 3 6 9 12 15. Using STL algorithms, write a C++ program that will:

- (a) Count how many times 12 appears.
- (b) Count how many even numbers there are.
- (c) Remove all appearances of 12 and store the new list in a separate container.
- (d) Reverse the original list.
- (e) Replace all numbers with their squares using `transform` and iterator.

(10 marks)

[CSE 107 | 2023–24]

## Answer — STL list operations — count, even, remove, reverse, transform

```

1  #include <iostream>
2  #include <list>
3  #include <algorithm>
4  #include <iterator>
5  using namespace std;
6
7  int main(){
8      list<int> lst = {20,16,12,8,4,3,6,9,12,15};
9
10     // (a) Count occurrences of 12
11     cout << "Count of 12: "
12         << count(lst.begin(), lst.end(), 12) << "\n"; // 2
13
14     // (b) Count even numbers
15     cout << "Even count: "
16         << count_if(lst.begin(), lst.end(),
17                     [](int x){ return x % 2 == 0; }) << "\n"
18         ; // 7
19
20     // (c) Remove 12s into a new container
21     list<int> notwelve;
22     copy_if(lst.begin(), lst.end(),
23             back_inserter(notwelve),
24             [](int x){ return x != 12; });
25     for(int x : notwelve) cout << x << " "; cout << "\n";
26
27     // (d) Reverse the original list
28     lst.reverse();
29     for(int x : lst) cout << x << " "; cout << "\n";
30
31     // (e) Replace each element with its square using
32     transform
33     transform(lst.begin(), lst.end(), lst.begin(),
34               [](int x){ return x * x; });
35     for(int x : lst) cout << x << " "; cout << "\n";
36
37     return 0;
38 }

```

**Q4.** What is a template class? Write down the hierarchy of template classes and associated 8-bit character-based classes used in C++. (5 marks) [CSE 107 | 2023–24]

**Answer — Template class hierarchy and 8-bit character-based classes**

**Template Class:** A class parameterised by one or more type parameters. The compiler generates a separate class for each instantiation.

**C++ I/O stream class hierarchy (template + char specialisation):**

| Template class                           | 8-bit char instantiation |
|------------------------------------------|--------------------------|
| <code>basic_ios&lt;charT&gt;</code>      | <code>ios</code>         |
| <code>basic_istream&lt;charT&gt;</code>  | <code>istream</code>     |
| <code>basic_ostream&lt;charT&gt;</code>  | <code>ostream</code>     |
| <code>basic_iostream&lt;charT&gt;</code> | <code>iostream</code>    |
| <code>basic_ifstream&lt;charT&gt;</code> | <code>ifstream</code>    |
| <code>basic_ofstream&lt;charT&gt;</code> | <code>ofstream</code>    |
| <code>basic_fstream&lt;charT&gt;</code>  | <code>fstream</code>     |
| <code>basic_string&lt;charT&gt;</code>   | <code>string</code>      |

Each `basic_*` template has two parameters: the character type (`char` or `wchar_t`) and a traits class. The `char`-based typedefs (`string`, `ostream`, etc.) are what C++ programmers use in everyday code.

**Q5.** Consider the following main function. Write the `myclass` template so the program produces: 1.1 2.2 3.3 and 20 25:

```

1  int main() {
2      myclass<double> d1(1.1);
3      myclass<double, 2> d2(1.1);
4      myclass<double, 3> d3(1.1);
5      cout << d1.getx() << " " << d2.getx() << " " << d3.getx()
        << "\n";
6      myclass<int, 4> i1(5);
7      myclass<int> i2(5);
8      cout << i1.getx() << " " << i2.getx();
9      return 0;
10 }
11 // Expected output:
12 // 1.1 2.2 3.3
13 // 20 25

```

(10 marks)

[CSE 107 | 2022–23]

**Answer — myclass template using Explicit Specialization**

**Analysis of expected output:** `d1(1.1)` → 1.1; `d2(1.1)` → 2.2 ( $N=2$ ); `d3(1.1)` → 3.3 ( $N=3$ ). `i1(5)` with  $N=4$  →  $5 \times 4 = 20$ ; `i2(5)` requires  $N=5$  →  $5 \times 5 = 25$ . To achieve this without advanced type traits, we can set the primary template's default parameter to  $N=1$ . Then, we explicitly specialize the template for `<int, 1>` (which is what `myclass<int>` resolves to) so that it multiplies by 5.

```

1  #include <iostream>
2  using namespace std;
3
4  // Primary template defaults N to 1
5  template<class T, int N = 1>
6  class myclass {
7      T x;
8  public:
9      myclass(T val): x(val * N){}
10     T getx(){ return x; }
11 };
12
13 // Explicit specialization for <int, 1>
14 // When myclass<int> is declared, it defaults to this
   specialization
15 template<>
16 class myclass<int, 1> {
17     int x;
18 public:
19     myclass(int val): x(val * 5){}
20     int getx(){ return x; }
21 };
22
23 int main(){
24     myclass<double>    d1(1.1);    // Uses primary (N=1):
                                   1.1*1=1.1
25     myclass<double, 2> d2(1.1);    // Uses primary (N=2):
                                   1.1*2=2.2
26     myclass<double, 3> d3(1.1);    // Uses primary (N=3):
                                   1.1*3=3.3
27     cout << d1.getx() << " " << d2.getx() << " " << d3.getx()
                                   << "\n";
28     // Output: 1.1 2.2 3.3
29
30     myclass<int, 4> i1(5);          // Uses primary (T=int, N
                                   =4): 5*4=20
31     myclass<int>      i2(5);          // Uses specialization <int
                                   , 1>: 5*5=25
32     cout << i1.getx() << " " << i2.getx() << "\n";
33     // Output: 20 25
34
35     return 0;
36 }

```

**Q6.** Write a program that demonstrates the use of the class 'map' of C++ standard template library (STL) and fulfills the following requirements: **(16 marks)**

(a) Create a class 'st\_record' that stores the name and cgpa of a student as

types `string` and `double`, respectively. Add functions to the class as necessary to complete your code.

- (b) In the 'main' function, create an object of STL class 'map' named 'result' to store the records of students against their student numbers (as `int`), i.e., the types of `<key,value>` pair for the map object will be `<int, st_record>`.
- (c) Insert some dummy data (records of at least four students) in the map object 'result'.
- (d) Traverse the map object 'result', find the student with the highest cgpa and print his information—student number, name and cgpa. Assume there will only be one student with the highest cgpa.

[CSE 107 | 2022–23]

### Answer — STL map with `st_record`; find student with highest CGPA

```

1  #include <iostream>
2  #include <map>
3  #include <string>
4  using namespace std;
5
6  // a. Class to store a student's name and CGPA
7  class st_record {
8  public:
9      string name;
10     double cgpa;
11
12     // Default constructor
13     st_record() : name(""), cgpa(0.0) {}
14
15     // Parameterized constructor
16     st_record(string n, double c) : name(n), cgpa(c) {}
17 };
18
19 int main() {
20     // b. Map named 'result' to store records against student
21         numbers (int)
22     map<int, st_record> result;
23
24     // c. Inserting dummy data for 4 students
25     result[1001] = st_record("Alice", 3.82);
26     result[1002] = st_record("Bob", 3.45);
27     result[1003] = st_record("Carol", 3.96);
28     result[1004] = st_record("Dave", 3.71);
29
30     // d. Traversing the map to locate the highest CGPA
31     int highestId = -1;
32     double highestCGPA = -1.0;

```

```

33     for (const auto& entry : result) {
34         if (entry.second.cgpa > highestCGPA) {
35             highestCGPA = entry.second.cgpa;
36             highestId = entry.first;
37         }
38     }
39
40     // Displaying the highest performing student's
41     // information
42     if (highestId != -1) {
43         cout << "Student Number: " << highestId << endl;
44         cout << "Name: " << result[highestId].name << endl;
45         cout << "CGPA: " << result[highestId].cgpa << endl;
46     }
47     return 0;
48 }

```

**Q7.** "Generic functions are similar to overloaded functions except that they are more restrictive" — do you agree? Justify your answer. (5 marks) [CSE 107 | 2021–22]

#### Answer — Generic functions more restrictive than overloaded?

**Agree** — with qualification.

**Why generic functions are more restrictive:**

- 1. Same logic for all types:** A generic function applies *identical* operations to all types. If different types require different logic (e.g. comparing strings vs ints differently), overloading is needed.
- 2. Operator availability:** The template body assumes operations like `+`, `<`, etc. are defined for the type. If a user-defined type does not support them, the template fails to compile.
- 3. No type-specific specialisation by default:** To get different behaviour per type, you must write a *template specialisation* — which is essentially overloading again.

**Counterpoint:** Generic functions *reduce* restriction in the sense that a single template handles any compatible type without writing multiple overloads.

**Conclusion:** Generic functions are more restrictive *in the sense* that they force all instantiated types to support the same operations and produce the same algorithmic behaviour. Overloaded functions allow per-type customisation.

**Q8.** Design a template function `myfunc` which takes two numbers and an operator character (`+`, `-`, `*` or any other). Prints the result or Invalid Input:



```

1 myfunc(10.5, 4.3, '+'); // Output: 14.8
2 myfunc(4, 8, '*');      // Output: 32
3 myfunc(10.5, 4.5, '-'); // Output: 6
4 myfunc(3, 6, 'a');      // Output: Invalid Input

```

(10 marks)

[CSE 107 | 2021–22]

**Answer — Template function myfunc with operator char**

```

1 #include <iostream>
2 using namespace std;
3
4 template<class T>
5 void myfunc(T a, T b, char op){
6     if (op == '+') cout << a + b << "\n";
7     else if (op == '-') cout << a - b << "\n";
8     else if (op == '*') cout << a * b << "\n";
9     else cout << "Invalid Input\n";
10 }
11
12 int main(){
13     myfunc(10.5, 4.3, '+'); // 14.8
14     myfunc(4, 8, '*');     // 32
15     myfunc(3, 6, 'a');     // Invalid Input
16     return 0;
17 }

```

**Q9.** Discuss different types of containers available in C++ STL. Describe their differences with an example class for each type. (12 marks)

[CSE 107 | 2021–22]

**Answer — STL container types**

| Category           | Examples                     | Characteristics                                         |
|--------------------|------------------------------|---------------------------------------------------------|
| Sequence           | vector, list, deque          | Elements in linear order; random or sequential access.  |
| Associative        | map, set, multimap           | Sorted by key; $O(\log n)$ lookup.                      |
| Unordered          | unordered_map, unordered_set | Hash-based; $O(1)$ average lookup.                      |
| Container adapters | stack, queue, priority_queue | Built on top of other containers; restricted interface. |

```

1 #include <iostream>
2 #include <vector>
3 #include <list>
4 #include <map>
5 #include <stack>

```

```

6  using namespace std;
7
8  int main(){
9      vector<int> v={1,2,3};      v.push_back(4);  cout<<v[2]<<
          "\n"; // 3
10     list<int> l={1,2,3};      l.push_front(0); cout<<l.
          front()<<"\n"; // 0
11     map<string,int> m; m["a"]=1; m["b"]=2;      cout<<m["a"
          ]<<"\n"; // 1
12     stack<int> s; s.push(10); s.push(20);      cout<<s.top
          ()<<"\n"; // 20
13 }

```

**Q10.** Write a generic class named queue where all types of data can be stored. Support pushQ(), pop(), size() with exception handling. **(15 marks)** [CSE 107 | 2020–21, 2014–15]

#### Answer — Generic queue with pushQ, pop, size, exception handling

```

1  #include <iostream>
2  #include <deque>
3  #include <stdexcept>
4  using namespace std;
5
6  template<class T>
7  class Queue {
8      deque<T> data;
9  public:
10     void pushQ(T val){ data.push_back(val); }
11
12     T pop(){
13         if(data.empty())
14             throw underflow_error("Queue is empty");
15         T v=data.front(); data.pop_front(); return v;
16     }
17
18     int size(){ return (int)data.size(); }
19     bool empty(){ return data.empty(); }
20 };
21
22 int main(){
23     Queue<int> qi;
24     qi.pushQ(10); qi.pushQ(20); qi.pushQ(30);
25     cout<<"Size: " <<qi.size()<<"\n"; // 3
26     try {
27         cout<<qi.pop()<<"\n"; // 10
28         cout<<qi.pop()<<"\n"; // 20

```

```

29         cout<<q1.pop()<<"\n";    // 30
30         cout<<q1.pop()<<"\n";    // throws
31     } catch(underflow_error& e){
32         cout<<"Exception: " <<e.what()<<"\n";
33     }
34
35     Queue<string> qs;
36     qs.pushQ("Hello"); qs.pushQ("World");
37     cout<<qs.pop()<<" " <<qs.pop()<<"\n"; // Hello World
38     return 0;
39 }

```

- Q11.** Consider that the map `<stdId, student>` is used in an object-oriented programming to store the records of the students, where `stdID` (i.e., student identification number) is unique for each student. A object “student” contains student ID, student name, address, mobile number and CGPA. The class “student” contains three member functions – a constructor, a getter for student ID and a `display()` function showing all of the above information about a student. Write down the class “student”. Also write down a C++ program that (i) takes information about a student from the keyboard and inserts a pair into the map; and (ii) scan `stdId` from keyboard buffer and print all information about that student from its associated object using map. Make necessary assumptions. (6+7+7=20) [CSE 107 | 2020–21]

#### Answer — student Class and std::map Operations

```

1  #include <iostream>
2  #include <map>
3  #include <string>
4  using namespace std;
5
6  class student {
7  private:
8      string studentID;
9      string name;
10     string address;
11     string mobileNumber;
12     double cgpa;
13
14 public:
15     // Default constructor (required for container map
16     // insertion)
17     student() {
18         studentID = "";
19         name = "";
20         address = "";
21         mobileNumber = "";

```

```
21         cgpa = 0.0;
22     }
23
24     // Constructor with parameters
25     student(string id, string n, string addr, string mob,
26             double c) {
27         studentID = id;
28         name = n;
29         address = addr;
30         mobileNumber = mob;
31         cgpa = c;
32     }
33
34     // Getter for student ID
35     string getStudentID() const {
36         return studentID;
37     }
38
39     // display() function showing all information
40     void display() const {
41         cout << "Student ID:      " << studentID << endl;
42         cout << "Student Name:    " << name << endl;
43         cout << "Address:         " << address << endl;
44         cout << "Mobile Number:  " << mobileNumber << endl;
45         cout << "CGPA:          " << cgpa << endl;
46     }
47 };
48
49 int main() {
50     // Map associative container to store <stdId, student>
51     // pairs
52     map<string, student> studentMap;
53     int n;
54
55     // (i) Take information about a student from the keyboard
56     // and insert into map
57     cout << "Enter the number of students to register: ";
58     cin >> n;
59
60     for (int i = 0; i < n; i++) {
61         string id, name, address, mobile;
62         double cgpa;
63
64         cout << "\nEnter details for student #" << (i + 1) <<
65             ":" << endl;
66         cout << "Enter Student ID: ";
67         cin >> id;
68         cin.ignore(); // Clear newline character from buffer
```

```

65
66     cout << "Enter Name: ";
67     getline(cin, name);
68
69     cout << "Enter Address: ";
70     getline(cin, address);
71
72     cout << "Enter Mobile Number: ";
73     cin >> mobile;
74
75     cout << "Enter CGPA: ";
76     cin >> cgpa;
77
78     // Instantiate the object
79     student st(id, name, address, mobile, cgpa);
80
81     // Insert pair into map
82     studentMap.insert(pair<string, student>(id, st));
83 }
84
85 // (ii) Scan stdId from keyboard buffer and print
    associated information
86 cout << "\n-----\n";
87 cout << "Enter Student ID to fetch records: ";
88 string searchID;
89 cin >> searchID;
90
91 // Search the key within the map boundary
92 map<string, student>::iterator it = studentMap.find(
    searchID);
93
94 if (it != studentMap.end()) {
95     cout << "\n--- Student Record Found ---" << endl;
96     it->second.display();
97 } else {
98     cout << "\nError: Student ID " << searchID << " does
        not exist." << endl;
99 }
100
101 return 0;
102 }

```

**Q12.** Write a class `myCalculator` using class templates to show addition, subtraction, multiplication, and division of two numbers (integer, float, or double):

```

1  int main() {
2      myCalculator<int> intCalc(7, 4);
3      myCalculator<float> floatCalc(3.4, 6.2);

```

```

4     intCalc.displayResult();
5     floatCalc.displayResult();
6     return 0;
7 }
8 // Output: Addition: 11, Subtraction: 3, Product: 28, Division
   : 1
9 //      Addition: 9.6, Subtraction: -2.8, Product: 21.08,
   Division: 0.548387

```

(15 marks)

[CSE 107 | 2017–18]

## Answer — myCalculator template class

```

1  #include <iostream>
2  using namespace std;
3
4  template<class T>
5  class myCalculator {
6      T a, b;
7  public:
8      myCalculator(T x, T y): a(x), b(y){}
9
10     void displayResult(){
11         cout << "Addition: " << a+b << ", ";
12         cout << "Subtraction: " << a-b << ", ";
13         cout << "Product: " << a*b << ", ";
14         cout << "Division: " << a/b << "\n";
15     }
16 };
17
18 int main(){
19     myCalculator<int>    intCalc(7, 4);
20     myCalculator<float> floatCalc(3.4f, 6.2f);
21
22     intCalc.displayResult();
23     // Addition: 11, Subtraction: 3, Product: 28, Division: 1
24
25     floatCalc.displayResult();
26     // Addition: 9.6, Subtraction: -2.8, Product: 21.08,
27     Division: 0.548387
28     return 0;
29 }

```

**Q13.** Here is the prototype for a specific version of `find()`. Convert it to a generic function that searches an array and returns the index of matching object or -1:

```

1  int find(int object, int *list, int size) { // ... }

```

(10 marks)

[CSE 107 | 2015–16]

## Answer — Generic find() function

```

1  #include <iostream>
2  using namespace std;
3
4  // Generic version: works for any type that supports ==
5  template<class T>
6  int find(T object, T* list, int size){
7      for(int i = 0; i < size; i++)
8          if(list[i] == object) return i;
9      return -1;
10 }
11
12 int main(){
13     int    iarr[] = {5, 10, 15, 20};
14     double darr[] = {1.1, 2.2, 3.3};
15     char   carr[] = {'a', 'b', 'c', 'd'};
16
17     cout << find(15,    iarr, 4) << "\n";    // 2
18     cout << find(2.2,   darr, 3) << "\n";    // 1
19     cout << find('c',   carr, 4) << "\n";    // 2
20     cout << find(99,    iarr, 4) << "\n";    // -1 (not found)
21     return 0;
22 }

```

- Q14.** Write a generic class for implementing an extended stack supporting: push, pop, and append (concatenates with top element). Use RTTI for type decisions. **(20 marks)**  
[CSE 201 | 2015–16]

## Answer — Generic extended stack with push, pop, append (RTTI)

```

1  #include <iostream>
2  #include <vector>
3  #include <typeinfo>
4  #include <string>
5  using namespace std;
6
7  template<class T>
8  class ExtStack {
9      vector<T> data;
10 public:
11     void push(T val){ data.push_back(val); }
12
13     T pop(){
14         if(data.empty()){ cout<<"Stack underflow\n"; return T
15             {}; }
16         T v=data.back(); data.pop_back(); return v;
17     }
18 }

```

```

16     }
17
18     // append: concatenate value with top element (RTTI for
19     // type decisions)
20     void append(T val){
21         if(data.empty()){ push(val); return; }
22         // Use RTTI to check the actual type at run time
23         if(typeid(T) == typeid(string)){
24             // For string: concatenate
25             // (We cast through void* since T may not always
26             // be string)
27             string* top = reinterpret_cast<string*>(&data.
28                 back());
29             string* v    = reinterpret_cast<string*>(&val);
30             *top += *v;
31         } else {
32             // For numeric types: add to top
33             data.back() = data.back() + val;
34         }
35     }
36
37     T top(){ return data.back(); }
38     bool empty(){ return data.empty(); }
39 };
40
41 int main(){
42     ExtStack<int> si;
43     si.push(10); si.push(5);
44     si.append(3);           // top becomes 5+3=8
45     cout << si.top() << "\n"; // 8
46
47     ExtStack<string> ss;
48     ss.push("Hello");
49     ss.append(" World");   // top becomes "Hello World"
50     cout << ss.top() << "\n"; // Hello World
51     return 0;
52 }

```

**Q15.** Complete the list transformation program using `transform` and iterator to replace each element with its cubic value:

```

1  int main() {
2      list<int> v;
3      for (int index = 0; index < 10; ++index)
4          v.push_back(index);
5      // complete here
6  }

```



(8 marks)

[CSE 201 | 2015–16]

**Answer — transform list elements to cubic values**

```

1  #include <iostream>
2  #include <list>
3  #include <algorithm>
4  using namespace std;
5
6  int main(){
7      list<int> v;
8      for(int index=0; index<10; ++index)
9          v.push_back(index); // 0 1 2 3 4 5 6 7 8 9
10
11     // Replace each element with its cubic value
12     transform(v.begin(), v.end(), v.begin(),
13               [](int x){ return x * x * x; });
14
15     for(int x : v) cout << x << " ";
16     cout << "\n";
17     // 0 1 8 27 64 125 216 343 512 729
18     return 0;
19 }

```

**Q16.** Consider the mapping <stdID, student>. Write C++ program that scans stdId and prints student name using map . (10 marks) [CSE 107 | 2014–15]

**Answer — Student map — insert from keyboard, query by ID**

```

1  #include <iostream>
2  #include <map>
3  #include <string>
4  using namespace std;
5
6  class student {
7  public:
8      string name;
9      double cgpa;
10     int year;
11     student(){}
12     student(string n, double c, int y): name(n), cgpa(c),
13         year(y){}
14     void print() const {
15         cout<<"Name: "<<name<<" | CGPA: "<<cgpa<<" | Year: "
16         <<year<<"\n";
17     }
18 };

```

```

18  int main(){
19      map<int, student> db;
20
21      // (i) Insert student info from keyboard
22      int n; cout<<"How many students? "; cin>>n;
23      for(int i=0;i<n;i++){
24          int id; string name; double cgpa; int year;
25          cout<<"ID: "; cin>>id;
26          cout<<"Name: "; cin>>name;
27          cout<<"CGPA: "; cin>>cgpa;
28          cout<<"Year: "; cin>>year;
29          db[id] = student(name, cgpa, year);
30      }
31
32      // (ii) Query by ID
33      int qid; cout<<"Enter stdID to search: "; cin>>qid;
34      if(db.count(qid)) db[qid].print();
35      else cout<<"Student not found.\n";
36
37      return 0;
38  }

```

**Q17.** Write a interactive C++ program to maintain an employee database (ID , name , qualification , designation , salary) using map , supporting create, update , delete, and query oprations. (20 marks) [CSE 107 | 2014–15, 2015–16]

#### Answer — Interactive Employee Database Using map

```

1  #include <iostream>
2  #include <map>
3  #include <string>
4  using namespace std;
5
6  struct Employee {
7      string name, qualification, designation;
8      double salary;
9  };
10
11 void printEmp(int id, const Employee& e) {
12     cout << "ID:" << id
13         << " Name:" << e.name
14         << " Qual:" << e.qualification
15         << " Desig:" << e.designation
16         << " Sal:" << e.salary << endl;
17 }
18
19 int main() {

```

```

20     map<int, Employee> db;
21     int choice;
22
23     while (true) {
24         cout << "\n1.Create 2.Update 3.Delete"
25              << " 4.Query 5.Exit: ";
26         cin >> choice;
27
28         if (choice == 1) {
29             int id; Employee e;
30             cout << "ID: ";      cin >> id;
31             cout << "Name: ";    cin >> e.name;
32             cout << "Qual: ";   cin >> e.qualification;
33             cout << "Desig: ";  cin >> e.designation;
34             cout << "Salary: "; cin >> e.salary;
35             db[id] = e;
36             cout << "Created." << endl;
37
38             } else if (choice == 2) {
39                 int id; cin >> id;
40                 if (db.count(id)) {
41                     cout << "New salary: ";
42                     cin >> db[id].salary;
43                     cout << "Updated." << endl;
44                 } else cout << "Not found." << endl;
45
46             } else if (choice == 3) {
47                 int id; cin >> id;
48                 if (db.erase(id)) cout << "Deleted." << endl;
49                 else               cout << "Not found." << endl;
50
51             } else if (choice == 4) {
52                 int id; cin >> id;
53                 if (db.count(id)) printEmp(id, db[id]);
54                 else               cout << "Not found." << endl;
55
56             } else break;
57         }
58         return 0;
59     }

```

**Q18.** Create a generic class `input` that when constructed: prompts the user, inputs data, and re-prompts if not within a predefined range. Objects declared as: `Input ob("prompt", min-val, max-val);` **(20 marks)** [CSE 201 | 2013–14]

## Answer — Generic input class with range validation

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  template<class T>
6  class Input {
7      T value;
8  public:
9      Input(const string& prompt, T minVal, T maxVal){
10         do {
11             cout << prompt << " [" << minVal << " - " <<
12                 maxVal << "]: ";
13             cin >> value;
14             if(value < minVal || value > maxVal)
15                 cout << "Out of range, please re-enter.\n";
16         } while(value < minVal || value > maxVal);
17     }
18     T get() const { return value; }
19 };
20
21 int main(){
22     Input<int>    age("Enter age", 1, 120);
23     cout << "Age entered: " << age.get() << "\n";
24
25     Input<double> temp("Enter temperature", -50.0, 150.0);
26     cout << "Temp entered: " << temp.get() << "\n";
27     return 0;
28 }

```

**Q19.** Write a key/value pair stored in a vector “map”. Write a generic function that reads a key and returns the associated value. (10 marks) [CSE 201 | 2013–14]

## Answer — Key/value pairs in vector; generic lookup function

```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  using namespace std;
5
6  template<class K, class V>
7  struct Pair { K key; V value; };
8
9  // Generic function: find value by key, return default if not
10     found
11 template<class K, class V>

```

```

11  V lookup(const vector<Pair<K,V>>& vmap, const K& key, V
    defaultVal){
12      for(const auto& p : vmap)
13          if(p.key == key) return p.value;
14      return defaultVal;
15  }
16
17  int main(){
18      vector<Pair<string,int>> scores = {
19          {"Alice",90}, {"Bob",75}, {"Carol",88}
20      };
21
22      cout << lookup(scores, string("Bob"), 0) << "\n"; //
          75
23      cout << lookup(scores, string("Dave"), 0) << "\n"; // 0
          (not found)
24
25      vector<Pair<int,string>> names = {
26          {1,"one"}, {2,"two"}, {3,"three"}
27      };
28      cout << lookup(names, 2, string("?")) << "\n"; // two
29      return 0;
30  }

```

**Q20.** What do you mean by a generic class? Write C++ code to create a generic class ARR which can: (i) hold an array of any data type, (ii) apply bubble sort, (iii) implement compact() to remove elements by index range. (5+20 marks) [CSE 201 | 2012–13]

**Answer — Generic class ARR — hold any type, bubble sort, compact()**

```

1  #include <iostream>
2  using namespace std;
3
4  template<class T>
5  class ARR {
6      T* data;
7      int size;
8  public:
9      ARR(int n): size(n){ data = new T[n]; }
10     ~ARR(){ delete[] data; }
11
12     T& operator[](int i){ return data[i]; }
13     int getSize(){ return size; }
14
15     // Bubble sort (ascending)

```

```

16     void bubbleSort(){
17         for(int i=0;i<size-1;i++)
18             for(int j=0;j<size-1-i;j++)
19                 if(data[j]>data[j+1]) swap(data[j],data[j+1])
20             ;
21     }
22     // compact: remove elements from index 'from' to 'to' (
23     // inclusive)
24     void compact(int from, int to){
25         if(from<0||to>=size||from>to) return;
26         int removed = to-from+1;
27         for(int i=from; i<size-removed; i++)
28             data[i] = data[i+removed];
29         size -= removed;
30     }
31     void print(){
32         for(int i=0;i<size;i++) cout<<data[i]<<" ";
33         cout<<"\n";
34     }
35 };
36
37 int main(){
38     ARR<int> a(6);
39     a[0]=5; a[1]=3; a[2]=8; a[3]=1; a[4]=9; a[5]=2;
40     a.bubbleSort(); a.print();    // 1 2 3 5 8 9
41     a.compact(1,3);               // remove indices 1-3 (2,3,5)
42     a.print();                   // 1 8 9
43
44     ARR<double> b(3);
45     b[0]=3.14; b[1]=2.71; b[2]=1.41;
46     b.bubbleSort(); b.print();   // 1.41 2.71 3.14
47     return 0;
48 }

```

**Q21.** What is function overloading? What is the basic difference between using an overloaded function and a generic function? (5 marks) [CSE 201 | 2012–13]

### Answer — Overloaded function vs generic function

**Function Overloading:** Multiple functions with the same name but different parameter lists; each overload has its own body and can behave differently.

**Basic difference between overloaded and generic functions:**

| Overloaded function                               | Generic (template) function          |
|---------------------------------------------------|--------------------------------------|
| Separate body for each type.                      | Single body; type is a parameter.    |
| Each version can have completely different logic. | All types share the same logic.      |
| Compiler selects by argument types.               | Compiler instantiates the template.  |
| More flexible per type.                           | More concise; less code duplication. |

**Q22.** Write a generic class `stack` that produces instances for stack of integers, stack of doubles, and stack of characters. **(20 marks)** [CSE 201 | 2011–12]

### Answer — Generic stack for int, double, char

```

1  #include <iostream>
2  #include <vector>
3  #include <stdexcept>
4  using namespace std;
5
6  template<class T>
7  class Stack {
8      vector<T> data;
9  public:
10     void push(T val){ data.push_back(val); }
11
12     T pop(){
13         if(data.empty()) throw underflow_error("Stack empty")
14         ;
15         T v=data.back(); data.pop_back(); return v;
16     }
17
18     T top() const { return data.back(); }
19     bool empty() { return data.empty(); }
20     int size() { return (int)data.size(); }
21 };
22
23 int main(){
24     Stack<int> si;
25     si.push(1); si.push(2); si.push(3);
26     while(!si.empty()) cout<<si.pop()<<" "; cout<<"\n"; // 3
27     2 1
28
29     Stack<double> sd;
30     sd.push(1.1); sd.push(2.2);
31     while(!sd.empty()) cout<<sd.pop()<<" "; cout<<"\n"; //
32     2.2 1.1
33
34     Stack<char> sc;

```

```

32     sc.push('A'); sc.push('B'); sc.push('C');
33     while(!sc.empty()) cout<<sc.pop()<<" "; cout<<"\n"; // C
        B A
34     return 0;
35 }

```

**Q23.** Write a generic function `min()` that returns the lesser of its two arguments. Demonstrate with a program. (7 marks) [CSE 201 | 2008–09]

#### Answer — Generic `min()` function

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  template<class T>
6  T mymin(T a, T b){
7      return (a < b) ? a : b;
8  }
9
10 int main(){
11     cout << mymin(3, 7) << "\n"; // 3
12     cout << mymin(3.5, 2.1) << "\n"; // 2.1
13     cout << mymin('z', 'a') << "\n"; // a
14     string s1="apple", s2="mango";
15     cout << mymin(s1, s2) << "\n"; // apple
16     return 0;
17 }

```

**Q24.** Distinguish between overloaded functions and function templates with necessary code. (5 marks) [CSE 201 | 2007–08]

#### Answer — Overloaded functions vs function templates

```

1  // -- Overloaded functions: separate body per type
   -----
2  int    add(int a,    int b)    { return a + b; }
3  double add(double a, double b) { return a + b; }
4  // Each version can do something different if needed
5
6  // -- Function template: one body for all types
   -----
7  template<class T>
8  T add_t(T a, T b) { return a + b; }

```



```
9
10 int main(){
11     cout << add(3, 4)          << "\n";  // 7    (calls int
        version)
12     cout << add(1.5, 2.5)      << "\n";  // 4.0 (calls double
        version)
13     cout << add_t(3, 4)        << "\n";  // 7    (template
        instantiated for int)
14     cout << add_t(1.5, 2.5)    << "\n";  // 4.0 (template
        instantiated for double)
15 }
```

**Key difference:** With overloading, you write multiple functions; the compiler picks one. With templates, you write one function; the compiler generates versions for each type used.

BUET · CSE 107 / CSE 201

# Object Oriented Programming

Section S9

---

I/O Streams, Manipulators & File I/O

## S9 — I/O Streams, Manipulators & File I/O

**Q1.** Why is it important to write custom manipulators in C++ programming? Write a C++ program that generates the following output:

Enter a hexadecimal number: c2d

You entered: \*\*\*\*\*0XC2D

The program must contain two custom manipulators: `hex_input` and `setup`. (5+10 marks) [CSE 107 | 2023–24]

### Answer — Custom manipulators `hex_input` and `setup`

**Why custom manipulators:** They encapsulate repetitive formatting sequences into a single reusable name, making output code cleaner and more readable.

```
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  // hex_input: sets stream to read hexadecimal input
6  istream& hex_input(istream& in){
7      in >> hex;
8      return in;
9  }
10
11 // setup: right-justified, width 10, fill '*', uppercase hex,
12 // show base
13 ostream& setup(ostream& out){
14     out << right << setw(10) << setfill('*')
15         << uppercase << showbase;
16     return out;
17 }
18
19 int main(){
20     int num;
21     cout << "Enter a hexadecimal number: ";
22     cin >> hex_input >> num;    // reads as hex: c2d = 3117
23
24     cout << "You entered: " << setup << hex << num << "\n";
25     // Output: You entered: *****0XC2D
26     return 0;
27 }
```

**Q2.** Consider the following class declaration- (7+3 marks)

```
1  class Complex {
2      int real;
3      int imaginary;
4  public:
```

```

5      //-----
6      //-----
7  };

```

- (a) Write down an extractor method that generates the following form of input patterns:

```

Enter real part: 3
Enter imaginary part: 8

```

Assume that both the real and the imaginary parts accept only positive and negative integers. The extractor method checks whether the data entered is valid or not. For invalid data, the method sets failbit to indicate improper input.

- (b) Write down an inserter method that produces output like  $3 + 8i$ .

[CSE 107 | 2023–24]

### Answer — Complex class extractor (failbit) and inserter

```

1  #include <iostream>
2  using namespace std;
3
4  class Complex {
5      int real;
6      int imaginary;
7  public:
8      Complex() : real(0), imaginary(0) {}
9
10     // (i) Extractor method with specific prompts and
11     // validity check
12     friend istream& operator>>(istream& in, Complex& c) {
13         int r, im;
14
15         cout << "Enter real part: ";
16         in >> r;
17
18         cout << "Enter imaginary part: ";
19         in >> im;
20
21         // If the input was not a valid integer, the stream's
22         // fail state is triggered
23         if (in.fail()) {
24             in.setstate(ios::failbit); // explicitly ensuring
25             // failbit is set
26         } else {
27             c.real = r;
28             c.imaginary = im;
29         }
30     }

```

```

27
28         return in;
29     }
30
31     // (ii) Inserter method producing output like "3 + 8i"
32     friend ostream& operator<<(ostream& out, const Complex& c
33     ) {
34         if (c.imaginary >= 0) {
35             out << c.real << " + " << c.imaginary << "i";
36         } else {
37             // Handles negative imaginary parts gracefully (e
38             // .g., "3 - 8i")
39             out << c.real << " - " << -c.imaginary << "i";
40         }
41         return out;
42     }
43 };
44
45 int main() {
46     Complex c;
47     if (cin >> c) {
48         cout << "Output: " << c << "\n";
49     } else {
50         cout << "Improper input detected. Stream failed.\n";
51     }
52     return 0;
53 }

```

**Q3.** Design your custom inserter to generate the output: x: 120, y: 130

```

1  class myclass {
2      int x, y;
3  public:
4      myclass(int x, int y) { this->x = x; this->y = y; }
5  };
6  int main() { myclass ob(120, 130); cout << ob; return 0; }

```

(10 marks)

[CSE 107 | 2021–22]

**Answer — Custom inserter for myclass printing x: 120, y: 130**

```

1  #include <iostream>
2  using namespace std;
3
4  class myclass {
5      int x, y;
6  public:
7      myclass(int x, int y){ this->x=x; this->y=y; }

```

```

8
9      // Custom inserter: must be friend to access private x, y
10     friend ostream& operator<<(ostream& out, const myclass&
        ob){
11         out << "x: " << ob.x << ", y: " << ob.y;
12         return out;
13     }
14 };
15
16 int main(){
17     myclass ob(120, 130);
18     cout << ob << "\n";    // x: 120, y: 130
19     return 0;
20 }

```

**Q4.** Write down a manipulator named `setup` that generates the following output for `cout<<setup<<123.45678;: 123.456%%%` (8 marks) [CSE 201 | 2014–15; CSE 107 | 2020–21]

**Answer — `setup` manipulator producing `123.456%%%`**

Output `123.456%%%` means: precision 3 decimal places, left-justified, field width 9, filled with `'%'`:

```

1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  ostream& setup(ostream& out){
6      out << setprecision(3) << fixed<< left<< setw(9)<<
        setfill('%');
7      return out;
8  }
9
10 int main(){
11     cout << setup << 123.45678 << "\n";
12     // Output: 123.456%%% (7 chars used, 2 padding '%')
13     // Adjust width/precision to get exact 123.456%%%
14     return 0;
15 }

```

**Q5.** Write the inserter and extractor for the inventory class. Modify the class as necessary:

```

1  class inventory {
2      char item; int onhand; double cost;
3  public:

```

```

4     inventory(char *i, int o, double c) {
5         strcpy(item, i); onhand = o; cost = c;
6     }
7 };

```

(12 marks)

[CSE 201 | 2014–15; CSE 201 | 2011–12; CSE 107 | 2020–21]

**Answer — inventory class inserter and extractor**

```

1  #include <iostream>
2  #include <cstring>
3  using namespace std;
4
5  class inventory {
6      char    item[50];
7      int     onhand;
8      double  cost;
9  public:
10     inventory(){}
11     inventory(const char* i, int o, double c){
12         strncpy(item, i, 49); item[49]='\0';
13         onhand=o; cost=c;
14     }
15
16     // Extractor
17     friend istream& operator>>(istream& in, inventory& inv){
18         cout<<"Item name: ";    in>>inv.item;
19         cout<<"On hand: ";      in>>inv.onhand;
20         cout<<"Cost: ";         in>>inv.cost;
21         return in;
22     }
23
24     // Inserter
25     friend ostream& operator<<(ostream& out, const inventory&
26         inv){
27         out<<"Item: "    <<inv.item
28         <<" | On hand: " <<inv.onhand
29         <<" | Cost: $"   <<inv.cost<<"\n";
30         return out;
31     }
32 };
33
34 int main(){
35     inventory inv;
36     cin  >> inv;    // prompt and read
37     cout << inv;    // display
38     return 0;
39 }

```

- Q6.** Write a C++ program that emulates the DOS `filesize` command and returns the size in bytes of a file entered on the command line. Invoke as: `C>filesize program.ext`. Check that the correct number of command-line arguments is provided and that the specified file can be opened. (7 marks) [CSE 107 | 2020–21]

#### Answer — filesize command emulation

```

1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4
5  int main(int argc, char* argv[]){
6      // Check correct number of arguments
7      if(argc != 2){
8          cout<<"Usage: filesize <filename>\n";
9          return 1;
10     }
11
12     // Try to open the file
13     ifstream f(argv[1], ios::binary | ios::ate);
14     if(!f){
15         cout<<"Error: cannot open file '" << argv[1] << "'\n"
16         ;
17         return 1;
18     }
19
20     // seekg to end already done by ios::ate; tellg gives
21     size
22     streamsize size = f.tellg();
23     f.close();
24
25     cout << "File: " << argv[1]
26         << "\nSize: " << size << " bytes\n";
27     return 0;
28 }
```

**Usage:** Compile and run as `./filesize program.ext`. The `ios::ate` flag positions the read pointer at the end of file on open; `tellg()` then returns the file size in bytes.

- Q7.** Consider the myclass with an inserter using friend function:

```

1  class myclass {
2      int x, y;
3  public:
4      myclass(int x, int y) { this->x = x; this->y = y; }
5      friend ostream &operator<<(ostream &out, myclass ob);
6  };
7  ostream &operator<<(ostream &out, myclass ob) {
```



```

8     out << "x: " << ob.x << ", y: " << ob.y << endl;
9     return out;
10 }

```

(i) Why is an inserter implemented using a friend function? (ii) What is the advantage of using `out` instead of `cout` inside the inserter? **(15 marks)** [CSE 107 | 2019–20]

### Answer — Why inserter uses friend; advantage of out over cout

#### (i) Why inserter is a friend function:

The `<<` operator takes an `ostream&` as its **left** operand. A member function would require the object of the class to be the left operand (`ob << cout`), which is **un-natural**. Since we cannot add a member to `ostream`, the operator must be a **non-member** function. Making it a **friend** grants it access to the class's private data:

```

1 // Must be friend: left operand is ostream, not myclass
2 friend ostream& operator<<(ostream& out, myclass ob){
3     out << "x: " << ob.x << ", y: " << ob.y << endl;
4     return out;
5 }

```

#### (ii) Advantage of using out instead of cout:

Using the parameter `out` (rather than hardcoding `cout`) makes the inserter **stream-agnostic**:

- Works with **any** output stream: `cout`, `ofstream` (file), `ostringstream` (string buffer), etc.
- Enables chaining: `cout << ob1 << ob2` works because each call returns the same `out` reference.
- The same inserter code can write to a file or a string buffer without modification.

```

1 ofstream f("out.txt");
2 f << ob;           // works! writes to file via 'out'
3 cout << ob;        // also works!

```

**Q8.** Write a function to check the status information of an I/O operation using `rdstate()` function. **(12 marks)** [CSE 107 | 2019–20]

### Answer — Check I/O status using rdstate()

```

1 #include <iostream>
2 #include <fstream>
3 using namespace std;
4

```

```

5 void checkStatus(ios& stream, const string& name){
6     ios::iostate state = stream.rdstate();
7     cout << name << " status flags:\n";
8     cout << " goodbit: " << !(state & ios::goodbit ? 0:1) <<
9         "\n";
10    cout << " eofbit: " << !(state & ios::eofbit) << "\n";
11    cout << " failbit: " << !(state & ios::failbit) << "\n";
12    cout << " badbit: " << !(state & ios::badbit) << "\n";
13    if(stream.good()) cout<<" -> Stream is GOOD\n";
14    if(stream.eof()) cout<<" -> End of file reached\n";
15    if(stream.fail()) cout<<" -> Logical I/O error\n";
16    if(stream.bad()) cout<<" -> Fatal I/O error\n";
17 }
18 int main(){
19     checkStatus(cin, "cin"); // normal: goodbit set
20     checkStatus(cout, "cout"); // normal: goodbit set
21
22     ifstream f("nonexistent.txt");
23     checkStatus(f, "nonexistent.txt"); // failbit set (open
24         failed)
25
26     int x;
27     cin >> x; // type a letter to cause failbit
28     checkStatus(cin, "cin after bad input");
29     cin.clear(); // reset failbit
30     return 0;
31 }

```

- Q9.** Create a manipulator named `mystyle` that prints a number in right justified form within field width 10, precision 5. Integers in hexadecimal with + sign for positive values. Print 2 to 100 and corresponding natural log using this manipulator. (7+7 marks) [CSE 107 | 2015–16]

**Answer —** `mystyle` manipulator; print 2-100 with natural log

```

1 #include <iostream>
2 #include <iomanip>
3 #include <cmath>
4 using namespace std;
5
6 // mystyle: right-justified, width 10, precision 5, hex
7 // integers, show + sign
8 ostream& mystyle(ostream& out) {
9     out << right << setw(10) << setprecision(5) << fixed <<
10         showpos << hex;
11     return out;
12 }

```

```

10 }
11
12 int main() {
13     // Print header (optional, but good practice)
14     cout << "          n          ln(n)\n";
15
16     // Print 2 to 100 and corresponding natural log
17     for(int i = 2; i <= 100; i++) {
18         // mystyle is applied before EACH number because setw
19         // () is not sticky
20         cout << mystyle << i << mystyle << log(i) << "\n";
21     }
22     return 0;
23 }

```

**Q10.** A place on earth is represented by its latitude and longitude. The class `plate` is:

```

1 class plate { char name; double latitude, longitude; };

```

Write inserter and extractor methods with the prompts/output shown below. Also write manipulator `setup` that converts decimal degrees to minutes (1 deg = 60 min).

(8+7+10 marks)

[CSE 201 | 2015–16]

#### Answer — plate Class: Inserter, Extractor and setup Manipulator

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class plate {
6     string name;
7     double latitude, longitude;
8 public:
9     plate() : latitude(0), longitude(0) {}
10
11     // Extractor (input)
12     friend istream& operator>>(istream& in, plate& p) {
13         cout << "Enter name: ";
14         in >> p.name;
15         cout << "Enter latitude (decimal degrees): ";
16         in >> p.latitude;
17         cout << "Enter longitude (decimal degrees): ";
18         in >> p.longitude;
19         return in;
20     }
21

```

```

22     // Inserter (output)
23     friend ostream& operator<<(ostream& out, const plate& p)
24     {
25         out << "Place : " << p.name << endl
26         << "Lat   : " << p.latitude << " deg" << endl
27         << "Long  : " << p.longitude << " deg" << endl;
28         return out;
29     }
30
31     // setup manipulator: converts decimal degrees to minutes
32     friend ostream& setup(ostream& out) {
33         // This is a state flag trick; for simplicity we
34         // demonstrate conversion directly in a method
35         return out;
36     }
37
38     void printInMinutes() const {
39         cout << "Place : " << name << endl
40         << "Lat   : " << latitude * 60 << " min" <<
41         endl
42         << "Long  : " << longitude * 60 << " min" <<
43         endl;
44     }
45 };
46
47 // setup as a proper manipulator using ios_base
48 ostream& setup(ostream& out) { return out; }
49
50 int main() {
51     plate p;
52     cin >> p;
53     cout << p;
54     cout << "\n--- In Minutes ---" << endl;
55     p.printInMinutes();
56     return 0;
57 }

```

### Sample interaction:

```

Enter name: Dhaka
Enter latitude (decimal degrees): 23.8103
Enter longitude (decimal degrees): 90.4125
Place : Dhaka
Lat   : 23.8103 deg
Long  : 90.4125 deg

--- In Minutes ---
Place : Dhaka

```

Lat : 1428.62 min  
Long : 5424.75 min

**Q11.** Write down a program for checking a file I/O status. (10 marks) [CSE 201 | 2015–16, 2011–12]

**Answer — File I/O status check program**

```
1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4
5  int main(){
6      ofstream fout("test.txt");
7      if(!fout){ cout<<"Cannot open file for writing\n"; return
          1; }
8      fout << "Hello, File!\n";
9      fout.close();
10
11     ifstream fin("test.txt");
12     if(!fin){ cout<<"Cannot open file for reading\n"; return
        1; }
13
14     string line;
15     while(getline(fin, line)){
16         cout << line << "\n";
17
18         // Check status after each read using rdstate()
19         ios::iostate state = fin.rdstate();
20         if(state & ios::badbit){ cout<<"Fatal I/O error\n";
            break; }
21         if(state & ios::failbit){ cout<<"Logical I/O error\n"
            ; break; }
22     }
23
24     // Check for EOF using rdstate()
25     if(fin.rdstate() & ios::eofbit) cout<<"End of file
        reached normally.\n";
26
27     fin.close();
28     return 0;
29 }
```

**Q12.** A class `Date` contains three private variables: `day`, `month`, and `year`. Users enter the date in Bangladeshi form (e.g., 23/12/2016) but output is in Chinese form (e.g., 2016/12/23). (i) Define the class with a constructor; (ii) Write an extractor with

prompt "Enter a date in Bangladeshi form:"; (iii) Write an inserter generating output: "Date (Chinese form) is 2016/12/23". (7+7+7 marks) [CSE 107 | 2015–16]

**Answer — Date class inserter/extractor; Bangladeshi input, Chinese output**

```

1  #include <iostream>
2  using namespace std;
3
4  class Date {
5      int day, month, year;
6  public:
7      Date(): day(1), month(1), year(2000){}
8      Date(int d, int m, int y): day(d), month(m), year(y){}
9
10     // (ii) Extractor: input in Bangladeshi form dd/mm/yyyy
11     friend istream& operator>>(istream& in, Date& d){
12         char slash;
13         cout<<"Enter a date in Bangladeshi form (dd/mm/yyyy):
14             ";
15         in >> d.day >> slash >> d.month >> slash >> d.year;
16         return in;
17     }
18
19     // (iii) Inserter: output in Chinese form yyyy/mm/dd
20     friend ostream& operator<<(ostream& out, const Date& d){
21         out<<"Date (Chinese form) is "
22             <<d.year<<"/"<<d.month<<"/"<<d.day;
23         return out;
24     }
25 };
26
27 int main(){
28     Date d;
29     cin >> d;          // Enter a date in Bangladeshi form:
30                         23/12/2016
31     cout << d << "\n"; // Date (Chinese form) is 2016/12/23
32     return 0;
33 }

```

**Q13.** Write down a manipulator function named `setup` that sets width 10, precision 4, and fills blanks with `'*'`. (10 marks) [CSE 201 | 2013–14]

**Answer — setup manipulator — width 10, precision 4, fill '\*'**

```

1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  ostream& setup(ostream& out){
6      out << setw(10)<< setprecision(4)<< setfill('*');
7      return out;
8  }
9
10 int main(){
11     cout << setup << 123.45678 << "\n"; // ****123.5
12     cout << setup << 3.14159 << "\n"; // *****3.142
13     cout << setup << 1000.0 << "\n"; // *****1000.
14     return 0;
15 }

```

**Q14.** Consider the variable A = 10 (int), B = 123.234567 (double), S = "hello" (string). Write a C++ program that generates the following output:

```

hello %%%%hello hello%%%%%
123.234567 123.235%%

```

(10 marks)

[CSE 201 | 2013–14]

**Answer — Formatted output for A, B, S**

```

1  #include <iostream>
2  #include <iomanip>
3  #include <string>
4  using namespace std;
5
6  int main(){
7      int A = 10;
8      double B = 123.234567;
9      string S = "hello";
10
11     // Line 1: hello %%%%hello hello%%%%%
12     cout << left << setw(5) << setfill('%') << S; //
13     hello
14     cout << right << setw(10) << setfill('%') << S; //
15     %%%%hello
16     cout << " ";
17     cout << left << setw(10) << setfill('%') << S; //
18     hello%%%%%
19     cout << "\n";
20 }

```

```

18 // Line 2: 123.234567 123.235%%
19 cout << setfill(' ') << left;
20 cout << setw(12) << setprecision(9) << fixed << B; //
    123.234567
21 cout << setfill('%') << setw(10) << setprecision(3) <<
    fixed << B; // 123.235%%
22 cout << "\n";
23
24 return 0;
25 }

```

**Q15.** What is the function of inserter and extractor in C++? What is the problem with the given phonebook code? Write a new class with inserter and extractor that draws a Circle object, taking radius and center as input using a DrawPixel(x, y) library function.

```

1 class phonebook {
2     char name; int areacode, prefix, num;
3 public:
4     phonebook(char *n, int a, int p, int nm) {
5         strcpy(name, n); areacode = a; prefix = p; num = nm;
6     }
7 };
8 void operator<<(ostream &stream, phonebook o) {
9     stream << o.name << " (" << o.areacode << ") "
10    << o.prefix << "-" << o.num << "\n";
11 }

```

(5+5+15 marks)

[CSE 201 | 2012–13]

### Answer — Inserter/extractor; phonebook problems; Circle inserter/extractor

**Inserter (<<):** A function operator<< that writes an object to an output stream.

**Extractor (>>):** A function operator>> that reads an object from an input stream.

**Problems with the phonebook code:**

- (i) char name; should be char name[50]; — a single char cannot hold a string.
- (ii) operator<< returns void but should return ostream& to allow chaining (cout<<a<<b).
- (iii) operator<< is not declared as friend — it cannot access private members.

**Circle class with inserter and extractor:**

```

1 #include <iostream>
2 #include <cmath>
3 using namespace std;

```



```

4
5 void DrawPixel(int x, int y){ cout<<"pixel("<<x<<","<<y<<")\n
   "; }
6
7 class Circle {
8     int cx, cy, r;
9 public:
10    Circle(): cx(0),cy(0),r(0){}
11
12    friend istream& operator>>(istream& in, Circle& c){
13        cout<<"Enter center x, y and radius: ";
14        in >> c.cx >> c.cy >> c.r;
15        return in;
16    }
17
18    friend ostream& operator<<(ostream& out, const Circle& c)
19    {
20        out<<"Circle: center=("<<c.cx<<","<<c.cy
21        <<") radius="<<c.r<<"\n";
22        // Draw using Bresenham or midpoint (simplified)
23        for(int angle=0; angle<360; angle+=5){
24            int x = c.cx + (int)(c.r * cos(angle *
25                3.14159/180));
26            int y = c.cy + (int)(c.r * sin(angle *
27                3.14159/180));
28            DrawPixel(x, y);
29        }
30        return out;
31    }
32 };
33
34 int main(){
35     Circle c;
36     cin >> c;    // extractor
37     cout << c;    // inserter (draws and describes)
38     return 0;
39 }

```

- Q16.** Explain two different formatted I/O handling techniques used in C++ programming. Write a manipulator that prints floating point output with width=10, precision=4, filling blanks with '\*'. (12 marks) [CSE 201 | 2011–12]

**Answer — Two formatted I/O techniques; setup manipulator**

**Two formatted I/O techniques in C++:**

- ios member functions:** Directly call methods on the stream object:

```
cout.width(10), cout.precision(4), cout.fill('*').
```

- 2. Manipulators:** Special functions/objects inserted into the stream with <<: setw(10), setprecision(4), setfill('\*'), fixed, left, etc.

**Custom setup manipulator (width=10, precision=4, fill='\*'):**

```
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  ostream& setup(ostream& out){
6      out << setw(10) << setprecision(4) << setfill('*') <<
7          fixed;
8      return out;
9  }
10
11 int main(){
12     cout << setup << 3.14159    << "\n";    // ****3.1416
13     cout << setup << 123.456    << "\n";    // ***123.4560
14     cout << setup << 0.001234   << "\n";    // ****0.0012
15     return 0;
16 }
```

- Q17.** Create a program that prints the natural logarithm and base-10 logarithm of numbers from 2 to 100. Format: right justified within field width 10, precision 5. **(10 marks)**  
[CSE 201 | 2010–11]

**Answer — Natural log and base-10 log, right-justified width=10 precision=5**

```
1  #include <iostream>
2  #include <iomanip>
3  #include <cmath>
4  using namespace std;
5
6  int main(){
7      cout << right << setw(6) << "n" << setw(12) << "ln(n)"
8          << setw(12) << "log10(n)" << "\n";
9
10     for(int i=2; i<=100; i++){
11         cout << setw(6)<< i << setw(12) << setprecision(5) <<
12             fixed << log((double)i) << setw(12) <<
13             setprecision(5) << fixed << log10((double)i)<< "\n";
14     }
15     return 0;
16 }
```

Q18. Create an inserter and extractor for this class:

```

1  class pwr {
2      int base, exponent; double result;
3  public:
4      pwr(int b, int e) {
5          base = b; exponent = e; result = 1;
6          for (; e; e--) result = result * base;
7      }
8  };

```

(10 marks)

[CSE 201 | 2010–11]

Answer — pwr class inserter and extractor

```

1  #include <iostream>
2  using namespace std;
3
4  class pwr {
5      int base, exponent;
6      double result;
7  public:
8      pwr(int b, int e): base(b), exponent(e), result(1){
9          for(int i=e; i; i--) result *= base;
10     }
11
12     // Extractor: reads base and exponent, computes result
13     friend istream& operator>>(istream& in, pwr& p){
14         cout<<"Base: ";    in>>p.base;
15         cout<<"Exponent: "; in>>p.exponent;
16         p.result=1;
17         for(int i=p.exponent; i; i--) p.result *= p.base;
18         return in;
19     }
20
21     // Inserter: displays base^exponent = result
22     friend ostream& operator<<(ostream& out, const pwr& p){
23         out<<p.base<<"^"<<p.exponent<<" = "<<p.result<<"\n";
24         return out;
25     }
26 };
27
28 int main(){
29     pwr p1(2, 8);
30     cout << p1;           // 2^8 = 256
31
32     pwr p2(3, 0);

```

```
33     cin  >> p2;           // prompt for base and exponent
34     cout << p2;
35     return 0;
36 }
```

BUET · CSE 107 / CSE 201

# Object Oriented Programming

Section S10

---

Java Basics, Arrays & Strings

## S10 — Java Basics, Arrays & Strings

**Q1.** What will be the output of the following code? What is the problem with the loop for efficiency? Rewrite for efficiency (single thread):

```

1  class Q1 {
2      public static void main(String[] args) {
3          String s1 = "Hello";
4          String s2 = new String("Hello");
5          String s3 = "Hello";
6          s2.intern();
7          System.out.println(s1 == s2);
8          System.out.println(s1 == s3);
9          System.out.println(s2 == s3);
10         final String s0 = "Hello";
11         final String t0 = "World";
12         String t1 = s0 + t0;
13         String t2 = new String("HelloWorld");
14         String t3 = "HelloWorld";
15         t3.intern();
16         System.out.println(t1 == t2);
17         System.out.println(t1 == t3);
18         System.out.println(t2 == t3);
19     }
20 }
21 String result = "";
22 for (int i = 0; i < 10000; i++) { result += i; }

```

(15 marks)

[CSE 107 | 2023–24]

### Answer — String == Comparisons & Loop Efficiency

Output of Q1.main():

| Statement | Value | Reason                                                                                           |
|-----------|-------|--------------------------------------------------------------------------------------------------|
| s1 == s2  | false | s2 is a new String() on the heap; different reference from pool s1.                              |
| s1 == s3  | true  | s3 = "Hello" uses the same pool literal as s1.                                                   |
| s2 == s3  | false | s2 is on heap; s3 is in pool.                                                                    |
| t1 == t2  | false | t1 is compile-time constant "HelloWorld" (pool); t2 is new String() (heap).                      |
| t1 == t3  | true  | t1 and t3 both refer to the pool literal "HelloWorld" (final constants inlined at compile time). |
| t2 == t3  | false | t2 is heap; t3 is pool.                                                                          |

*Note: s2.intern() and t3.intern() are called but their return values are not assigned, so they have no effect on the comparisons above.*

**Problem with the loop:**

```

1 String result = "";
2 for (int i = 0; i < 10000; i++) { result += i; }

```

**String is immutable:** each `+=` creates a brand-new **String** object and discards the old one. This produces  $\sim 10,000$  short-lived objects and  $O(n^2)$  character copying, making the loop very slow for large `n`.

**Efficient rewrite using `StringBuilder`:**

```

1 StringBuilder sb = new StringBuilder();
2 for (int i = 0; i < 10000; i++) { sb.append(i); }
3 String result = sb.toString();

```

**StringBuilder** maintains a mutable internal buffer and appends in amortised  $O(1)$  per call — only one final **String** object is created.

**Q2.** Write the Java function `minmax` (only one function):

```

1 int a = minmax("min", 2, 1, 6, 4, 5); // a = 1
2 int b = minmax("min", 3, 0, 6);       // b = 0
3 int c = minmax("max", 1, -2, 6, 5);   // c = 6
4 int d = minmax("max", 1, 3, 7);       // d = 7

```

(10 marks)

[CSE 107 | 2022–23, 2019–20, 2016–17]

↪ Similar: CSE 107 | 2022–23 (as *maxmin*)

### Answer — `minmax()` with Varargs

```

1 public class MinMaxDemo {
2
3     public static int minmax(String op, int... nums) {
4         int result = nums[0];
5         for (int i = 1; i < nums.length; i++) {
6             if (op.equals("min") && nums[i] < result) result
7                 = nums[i];
8             if (op.equals("max") && nums[i] > result) result
9                 = nums[i];
10        }
11        return result;
12    }
13
14    public static void main(String[] args) {
15        int a = minmax("min", 2, 1, 6, 4, 5); // a = 1
16        int b = minmax("min", 3, 0, 6);       // b = 0
17        int c = minmax("max", 1, -2, 6, 5);   // c = 6
18        int d = minmax("max", 1, 3, 7);       // d = 7
19        System.out.println(a + " " + b + " " + c + " " + d);
20    }
21 }

```

*Note: `maxmin()` from CSE 107 | 2022–23 uses the identical implementation — only the method name changes.*

**Q3.** What are the problems with the following Java code?

```
1 public class TestStatic {
2     static int a = 3, b;
3     int c;
4     { c = 10; }
5     static { b = a * 4; c = b; }
6     int f2() { return a * b; }
7     static void f1(int x) {
8         System.out.println("x =" + x);
9         System.out.println("c =" + c);
10    }
11    public static void main(String[] args) {
12        f1(42);
13        System.out.println("Area = " + f2());
14    }
15 }
```

(10 marks)

[CSE 107 | 2019–20]

### Answer — Problems in TestStatic

#### Identified problems:

##### 1. Static initialiser block accesses instance variable `c`:

```
1 static { b = a * 4; c = b; } // ERROR: c is an instance
    variable
```

A `static` block runs before any object is created; it cannot access instance fields like `c`.

##### 2. Static method `f1()` accesses instance variable `c`:

```
1 static void f1(int x) { ... System.out.println("c =" + c);
    } // ERROR
```

`c` is an instance variable; it cannot be referenced from a `static` context without an object reference.

##### 3. `main()` calls instance method `f2()` without an object:

```
1 System.out.println("Area = " + f2()); // ERROR: f2() is
    non-static
```

`f2()` is not `static`; it must be called on an instance (`new TestStatic().f2()`).



Corrected code:

```

1  public class TestStatic {
2      static int a = 3, b;
3      int c;
4      { c = 10; }
5      static { b = a * 4; }           // removed: c = b (c is
                                     non-static)
6
7      int f2() { return a * b; }
8      static void f1(int x) {
9          System.out.println("x = " + x);
10         // removed: cannot print c from static context
11     }
12     public static void main(String[] args) {
13         f1(42);
14         TestStatic obj = new TestStatic();
15         System.out.println("c = " + obj.c);           // access
                                                         via object
16         System.out.println("Area = " + obj.f2()); // call via
                                                         object
17     }
18 }

```

Q4. Identify all the problems in the following code segment:

```

1  static class StaticTest {
2      static int x = 5, y;
3      int z;
4      { x = 5; z = 20; }
5      static { y = x * 4; z = y; }
6      int s1() { return x * y; }
7      static void s2() {
8          System.out.println("z = " + z);
9      }
10     public static void main(String[] args) {
11         s1();
12         System.out.println("z = " + z);
13         s2();
14     }
15 }

```

(10 marks)

[CSE 107 | 2022–23]

### Answer — Problems in StaticTest

Identified problems:

1. Static initialiser accesses instance variable z:

```
1 static { y = x * 4; z = y; } // ERROR: z is an instance
    variable
```

2. Static method s2() accesses instance variable z:

```
1 static void s2() { System.out.println("z = " + z); } //
    ERROR
```

3. main() calls instance method s1() without an object:

```
1 s1(); // ERROR: s1() is non-static; needs an object
```

4. main() accesses instance variable z directly:

```
1 System.out.println("z = " + z); // ERROR: z is non-static
```

5. Top-level class declared **static**: A top-level class cannot be static in Java. Only nested (inner) classes can be static.

Corrected code:

```
1 class StaticTest { // removed: static
    keyword
2     static int x = 5, y;
3     int z;
4     { x = 5; z = 20; }
5     static { y = x * 4; } // removed: z = y
6
7     int s1() { return x * y; }
8
9     static void s2(int zVal) { // pass z as
        parameter
10         System.out.println("z = " + zVal);
11     }
12
13     public static void main(String[] args) {
14         StaticTest obj = new StaticTest();
15         System.out.println("s1() = " + obj.s1()); // call via
            object
16         System.out.println("z = " + obj.z); // access
            via object
17         s2(obj.z);
18     }
19 }
```

**Q5.** Describe 3 different ways to convert an int value to a String. (10 marks) [CSE 107 | 2021–22]

**Answer — Converting int to String**

```

1  int n = 42;
2
3  // Way 1: String.valueOf() -- most common, explicit, no
   boxing
4  String s1 = String.valueOf(n);           // "42"
5
6  // Way 2: Integer.toString() -- static method of wrapper
   class
7  String s2 = Integer.toString(n);         // "42"
8
9  // Way 3: Concatenation with empty string -- simplest but
   implicit
10 String s3 = "" + n;                      // "42"
11 // (creates a StringBuilder internally, then calls toString()
   )

```

| Way | Method                           | Notes                                                              |
|-----|----------------------------------|--------------------------------------------------------------------|
| 1   | <code>String.valueOf(n)</code>   | Preferred; null-safe for objects                                   |
| 2   | <code>Integer.toString(n)</code> | Also accepts radix: <code>Integer.toString(n, 2)</code> for binary |
| 3   | <code>"" + n</code>              | Concise but slightly less readable                                 |

**Q6.** Describe boxing and unboxing in Java. Give examples with primitive `int` and `Integer`. Explain the difference between boxing/unboxing and autoboxing/auto-unboxing with code. When should you not use autoboxing? **(10 marks)** [CSE 107 | 2021–22]

**Answer — Type Wrappers, Autoboxing & Auto-Unboxing****What are type wrappers?**

A **type wrapper** is a class in `java.lang` that encapsulates a primitive data type inside an object. Each primitive has a corresponding wrapper:

| Primitive          | Wrapper              | Primitive            | Wrapper                |
|--------------------|----------------------|----------------------|------------------------|
| <code>byte</code>  | <code>Byte</code>    | <code>float</code>   | <code>Float</code>     |
| <code>short</code> | <code>Short</code>   | <code>double</code>  | <code>Double</code>    |
| <code>int</code>   | <code>Integer</code> | <code>char</code>    | <code>Character</code> |
| <code>long</code>  | <code>Long</code>    | <code>boolean</code> | <code>Boolean</code>   |

**Why are they needed?**

- Collections (e.g. `ArrayList`) require objects, not primitives.
- They provide utility methods such as `Integer.parseInt()`, `Integer.toBinaryString()`, `Double.isNaN()`, etc.

- They allow primitives to be used in generic types.

### Boxing and Unboxing (manual):

```

1 // Boxing: primitive -> wrapper object
2 Integer iObj = Integer.valueOf(42); // explicit boxing
3
4 // Unboxing: wrapper object -> primitive
5 int i = iObj.intValue();           // explicit unboxing

```

### Autoboxing and Auto-unboxing (Java 5+):

The compiler inserts the boxing/unboxing calls automatically.

```

1 Integer iObj = 42;           // autoboxing: compiler calls Integer.
    valueOf(42)
2 int i = iObj;               // auto-unboxing: compiler calls iObj.
    intValue()
3
4 // In collections:
5 ArrayList<Integer> list = new ArrayList<>();
6 list.add(10);               // autoboxing: 10 -> Integer.valueOf
    (10)
7 int x = list.get(0);        // auto-unboxing: Integer.intValue()

```

### When should you NOT use autoboxing?

- In **performance-critical loops** — autoboxing in a tight loop creates many temporary wrapper objects, causing excessive GC pressure.
- When **comparing values with ==** — == on Integer objects compares references, not values (use equals() or intValue() instead).
- When the wrapper variable might be **null** — auto-unboxing a null wrapper throws NullPointerException.

**Q7.** What is the difference between the following two declarations?

- `int c[][], x;`
- `int[][] c, x;`

Write 2 different ways of creating a jagged array in Java. **(10 marks)** [CSE 107 | 2019–20, 2020–21]

### Answer — 2-D Declaration Difference & Jagged Arrays

- `int c[][], x;` — c is a 2-D array of int; x is a plain int scalar.
- `int[][] c, x;` — Both c and x are 2-D arrays of int.

**Two ways to create a jagged array:**

**Way 1 — Allocate rows separately after declaring the array**

```
1  int[][] jagged = new int[3][];    // 3 rows, column size
   unspecified
2  jagged[0] = new int[2];           // row 0 has 2 columns
3  jagged[1] = new int[4];           // row 1 has 4 columns
4  jagged[2] = new int[3];           // row 2 has 3 columns
```

**Way 2 — Initialise with an array initialiser**

```
1  int[][] jagged = {
2      {1, 2},
3      {3, 4, 5, 6},
4      {7, 8, 9}
5  };
6  // Row 0: 2 elements, Row 1: 4 elements, Row 2: 3 elements
```

**Q8.** Consider the following code segment. List with explanation all the auto-boxing and auto-unboxing performed in the main method:

```
1  public class AutoBoxingUnboxingDemo {
2      static int f(Integer v) { return v; }
3      public static void main(String args[]) {
4          Integer a = f(100);
5          Integer i0b, i0b2;
6          int i;
7          i0b = 100;
8          ++i0b;
9          i0b2 = i0b + (i0b / 3);
10         i = i0b + (i0b / 3);
11         Integer int0b = 100;
12         Double double0b = 98.6;
13         double0b = double0b + int0b;
14     }
15 }
```

(10 marks)

[CSE 107 | Jan 2021]

## Answer — Autoboxing / Auto-Unboxing in AutoBoxingUnboxingDemo

| Statement                       | Type          | Explanation                                                                                        |
|---------------------------------|---------------|----------------------------------------------------------------------------------------------------|
| Integer a = f(100)              | Auto-boxing   | Literal 100 (int) is autoboxed to Integer before passing to f(Integer v).                          |
|                                 | Auto-unboxing | Inside f: return v auto-unboxes Integer to int to match return type int.                           |
| Integer a = f(100) (assignment) | Auto-boxing   | The int returned by f is autoboxed to Integer to store in a.                                       |
| i0b = 100                       | Auto-boxing   | 100 (int literal) is autoboxed to Integer.                                                         |
| ++i0b                           | Both          | i0b is auto-unboxed to int, incremented, then autoboxed back to Integer.                           |
| i0b2 = i0b + (i0b / 3)          | Both          | i0b is auto-unboxed (twice) for arithmetic; result is autoboxed into i0b2.                         |
| i = i0b + (i0b / 3)             | Auto-unboxing | i0b is auto-unboxed (twice); result stored in primitive int i (no boxing needed).                  |
| Integer int0b = 100             | Auto-boxing   | 100 autoboxed to Integer.                                                                          |
| Double double0b = 98.6          | Auto-boxing   | 98.6 autoboxed to Double.                                                                          |
| double0b = double0b + int0b     | Both          | double0b and int0b are auto-unboxed; int is widened to double; result is autoboxed back to Double. |

**Q9.** A singleton class is a class that can have only one object at a time. After the first time, if we try to instantiate a singleton class, the new variable also points to the first object created. Write a complete Java code to create a singleton class named MySingleton, where the single instance can be retrieved using a method called getInstance. **(10 marks)** [CSE 107 | 2020–21]

## Answer — MySingleton: Singleton Pattern

```

1 public class MySingleton {
2
3     // Private static field holds the single instance
4     private static MySingleton instance = null;
5
6     // Private constructor prevents direct instantiation
7     private MySingleton() {
8         System.out.println("MySingleton created.");
9     }
10 }

```

```

9      }
10
11     // Public factory method -- creates instance only on
12     // first call
13     public static MySingleton getInstance() {
14         if (instance == null)
15             instance = new MySingleton();
16         return instance;
17     }
18
19     public void showMessage() {
20         System.out.println("Hello from MySingleton!");
21     }
22 }
23 // -- Usage in main
24 -----
25 class TestSingleton {
26     public static void main(String[] args) {
27         MySingleton s1 = MySingleton.getInstance();
28         MySingleton s2 = MySingleton.getInstance();
29
30         s1.showMessage();
31
32         // Both references point to the same object
33         System.out.println(s1 == s2); // true
34     }
35 }

```

**Key design decisions:**

- The constructor is **private** — no external code can call `new MySingleton()`.
- `instance` is **static** — it belongs to the class, not any object.
- `getInstance()` is **static** — callable without an existing object.
- Lazy initialisation: the object is created only when first requested.

**Q10.** Consider the following code segment. How many of the 4 method calls will generate a compiler error and why?

```

1  public class VarArgsTest {
2      static void f(int... v) { }
3      static void f(String msg, boolean... v) { }
4      static void f(String msg, int... v) { }
5      static void f(int n, int... v) { }
6      static void f(double... v) { }
7      public static void main(String[] args) {
8          f("Int", 10, 20, 30);

```

```

9         f("boolean", true, false, false);
10        f();
11        f(1, 2, 3);
12    }
13 }
```

(5 marks)

[CSE 107 | 2020–21]

**Answer — VarArgsTest: Compiler Errors**

The five overloaded methods are:

```

1 static void f(int... v)           // M1
2 static void f(String msg, boolean... v) // M2
3 static void f(String msg, int... v) // M3
4 static void f(int n, int... v)    // M4
5 static void f(double... v)        // M5
```

| Call                             | Error? | Reason                                                                                                                          |
|----------------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------|
| f("Int", 10, 20, 30)             | No     | Matches M3 unambiguously (String, int...).                                                                                      |
| f("boolean", true, false, false) | No     | Matches M2 unambiguously (String, boolean...).                                                                                  |
| f()                              | Yes    | Ambiguous: both M1 (int...) and M5 (double...) match a zero-argument call. Compiler cannot choose.                              |
| f(1, 2, 3)                       | Yes    | Ambiguous: M1 (int...) matches all args as int[]; M4 (int n, int...) also matches with n=1 and v={2,3}. Compiler cannot choose. |

**Answer: 2 out of the 4 calls** — f() and f(1,2,3) — generate a compiler error due to ambiguity.

**Q11.** Consider your own StringTokenizer class - **MyStringTokenizer** with the following methods:

- **MyStringTokenizer(String str, String delimiter)** — the only constructor that takes the main String and the delimiter to tokenize
- **int countTokens()** — returns total number of tokens
- **boolean hasMoreTokens()** — returns false if no more token left, true otherwise
- **String nextToken()** — returns the next token

Write complete Java code for your **MyStringTokenizer** class. You can add necessary instance variables. You can't use the original **StringTokenizer** class. You can



only use the `split` method of `String` class which works as follows:

```
String s = "what is your name?";  
String [] t = s.split(" ");  
t[0] = what, t[1] = is, t[2] = your, t[3] = name
```

(10 marks)

[CSE 107 | 2020–21; CSE 201 | 2014–15]

#### Answer — MyStringTokenizer Class

```
1  public class MyStringTokenizer {  
2  
3      private String[] tokens;  
4      private int currentIndex;  
5  
6      // Constructor: tokenise the string using the given  
7      // delimiter  
8      public MyStringTokenizer(String str, String delimiter) {  
9          // split() uses regex; escape special chars if needed  
10         tokens = str.split(delimiter);  
11         currentIndex = 0;  
12     }  
13  
14     // Returns total number of tokens  
15     public int countTokens() {  
16         return tokens.length;  
17     }  
18  
19     // Returns true if more tokens remain  
20     public boolean hasMoreTokens() {  
21         return currentIndex < tokens.length;  
22     }  
23  
24     // Returns the next token and advances the cursor  
25     public String nextToken() {  
26         if (!hasMoreTokens())  
27             throw new java.util.NoSuchElementException("No  
28             more tokens");  
29         return tokens[currentIndex++];  
30     }  
31 }  
32  
33 // -- Demo  
34 -----  
35 class TokenizerTest {  
36     public static void main(String[] args) {  
37         MyStringTokenizer mst =  
38             new MyStringTokenizer("what is your name?", " ");  
39  
40         System.out.println("Total tokens: " + mst.countTokens
```

```

        ());
38         while (mst.hasMoreTokens())
39             System.out.println(mst.nextToken());
40     }
41 }

```

**Output:**

```

1  Total tokens: 4
2  what
3  is
4  your
5  name?

```

**Q12.** Identify the problems in the following code segment and write the complete corrected code:

```

1  class A {
2      var x;
3      void getX() { return x; }
4      void setX(int x) { x = x; }
5  }
6  public static void main(String[] args) {
7      A[] a = new A[10];
8      for (int i = 0; i < a.length(); i++) { a[i].setX(i); }
9      for (int i = 0; i < a.length(); i++) { System.out.println(
10         a[i].getX()); }

```

(10 marks)

[CSE 107 | 2020–21]

### Answer — Problems in class A & Corrected Code

**Identified problems:**

1. **var x;** — var is only valid for local variables with type inference (Java 10+); it cannot be used as an instance variable type. Must be **int x;** (or another explicit type).
2. **void getX(){ return x; }** — return type is void but the method returns x (an int). Should be **int getX()**.
3. **x = x; inside setX(int x)** — assigns the parameter to itself; the field x is never updated. Must use **this.x = x**.
4. **new A[10] elements are not initialised** — a[i] is null until explicitly assigned. a[i].setX(i) throws **NullPointerException**.
5. **a.length()** — arrays use **length** (a field), not **length()** (a method). Use **a.length**.

6. **main** is outside any class — in Java, **main** must be inside a class.

Corrected code:

```

1  class A {
2      int x;                                // fix 1: explicit
        type
3      int getX() { return x; }              // fix 2: correct
        return type
4      void setX(int x) { this.x = x; }      // fix 3: this.x = x
5  }
6
7  public class TestA {
8      public static void main(String[] args) { // fix 6:
        inside class
9          A[] a = new A[10];
10         for (int i = 0; i < a.length; i++) { // fix 5:
            length not length()
11             a[i] = new A();                  // fix 4:
            initialise objects
12             a[i].setX(i);
13         }
14         for (int i = 0; i < a.length; i++)
15             System.out.println(a[i].getX());
16     }
17 }
```

**Q13.** What do you mean by auto-boxing and auto-unboxing? When should you not use them? (5+5 marks) [CSE 107 | 2016–17, 2019–20]

**Q14.** What is the difference between the following two declarations in Java?

(a) `int c[], x;`

(b) `int[] c, x;`

(6 marks)

[CSE 201 | 2014–15, 2007–08; CSE 107 | 2017–18]

**Answer —** `int c[], x` vs. `int[] c, x`

(a) `int c[], x;`

The `[]` is attached to the *variable name* `c`, not to the type. Only `c` is an array of `int`; `x` is a plain `int`.

```

1  int c[], x;
2  // c is int[] (array of int)
3  // x is int (scalar)
```

(b) `int[] c, x;`

The `[]` is attached to the *type*. Both `c` and `x` are arrays of `int`.

```
1  int[] c, x;  
2  // c is int[] (array of int)  
3  // x is int[] (array of int)
```

**Key takeaway:** In Java, placing `[]` after the type applies the array type to *all* variables in the declaration, whereas placing `[]` after a variable name applies the array type only to *that* variable. The second form (`int[] c, x`) is preferred in Java style guides for clarity.

**Q15.** Write a Java program that takes a String as input and prints the number of tokens (delimiter: space) and the tokens themselves. **(10 marks)** [CSE 107 | 2017–18]

#### Answer — Count Tokens and Print (Delimiter: Space)

```
1  import java.util.Scanner;  
2  
3  public class TokenCounter {  
4      public static void main(String[] args) {  
5          Scanner sc = new Scanner(System.in);  
6          System.out.print("Enter a string: ");  
7          String input = sc.nextLine();  
8          sc.close();  
9  
10         // split on one or more spaces  
11         String[] tokens = input.trim().split("\\s+");  
12  
13         System.out.println("Number of tokens: " + tokens.  
14                             length);  
15         System.out.println("Tokens:");  
16         for (String token : tokens)  
17             System.out.println("    " + token);  
18     }  
19 }
```

#### Sample run:

```
1  Enter a string: the quick brown fox  
2  Number of tokens: 4  
3  Tokens:  
4      the  
5      quick  
6      brown  
7      fox
```

**Q16.** What is a type wrapper? Name any three. Why are type wrappers needed? **(3+3+4)**

marks)

[CSE 107 | 2017–18]

**Q17.** What are boxing and unboxing? Give an example for each. **(10 marks)** [CSE 107 | 2017–18]

**Q18.** What is the full prototype of Java `main` method? Describe the significance of each term. **(5 marks)** [CSE 201 | 2007–08; CSE 107 | 2016–17]

#### Answer — Java main Method Prototype

```
1 public static void main(String[] args)
```

| Term                       | Significance                                                                                                                                                                |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>public</code>        | Access modifier. <code>main</code> must be <code>public</code> so that the JVM (which is outside the class) can call it.                                                    |
| <code>static</code>        | The JVM calls <code>main</code> without creating an object of the class; <code>static</code> makes it callable on the class itself.                                         |
| <code>void</code>          | Return type. <code>main</code> does not return any value to the JVM.                                                                                                        |
| <code>main</code>          | The name the JVM looks for as the program entry point. Any other name is not recognised as the starting method.                                                             |
| <code>String[] args</code> | Command-line arguments passed to the program are received as an array of <code>String</code> objects. <code>args</code> is a conventional name but any identifier is valid. |

#### What happens if the signature is wrong?

If `main` is not declared with the exact signature above (e.g. missing `static`, wrong return type, wrong parameter), the JVM cannot find the entry point and throws:

```
1 Main method not found in class MyClass, please define the
  main method as:
2 public static void main(String[] args)
```

The program does not run and terminates with a runtime error.

**Q19.** What are type wrappers? Why are they used? Explain Autoboxing and Auto-unboxing with examples. **(10 marks)** [CSE 107 | 2015–16]

**Q20.** You need to write a method `editStudentDetails()` that takes name, age, and cgpa as input, modifies them inside the method, and makes changed values available outside. Show in `main()` how to call this method and access the changed values. **(10 marks)** [CSE 201 | 2015–16]

**Answer — editStudentDetails(): Pass-by-Reference Simulation**

Java is strictly **pass-by-value**. To make changes to `name`, `age`, and `cgpa` visible outside the method, we wrap them in a **mutable object** and pass the object reference.

```
1  class Student {
2      String name;
3      int age;
4      double cgpa;
5
6      Student(String name, int age, double cgpa) {
7          this.name = name; this.age = age; this.cgpa = cgpa;
8      }
9  }
10
11 public class EditDemo {
12
13     public static void editStudentDetails(Student s) {
14         s.name = "Alice";
15         s.age = 21;
16         s.cgpa = 3.95;
17     }
18
19     public static void main(String[] args) {
20         Student student = new Student("Bob", 20, 3.50);
21
22         System.out.println("Before: " + student.name
23             + ", " + student.age + ", " + student.cgpa);
24
25         editStudentDetails(student);
26
27         System.out.println("After : " + student.name
28             + ", " + student.age + ", " + student.cgpa);
29     }
30 }
```

**Output:**

```
1  Before: Bob, 20, 3.5
2  After : Alice, 21, 3.95
```

The object reference `student` is passed by value, but both `main` and `editStudentDetails` point to the *same* object on the heap, so modifications inside the method are visible in `main`.

- Q21.** Write Java code that takes  $n$  integers from the command line and prints the maximum and minimum. (10 marks) [CSE 201 | 2014–15; CSE 201 | 2007–08]

**Answer — Max and Min of n Command-Line Integers**

```
1 public class MaxMin {
2     public static void main(String[] args) {
3         if (args.length == 0) {
4             System.out.println("No input provided.");
5             return;
6         }
7         int max = Integer.parseInt(args[0]);
8         int min = Integer.parseInt(args[0]);
9
10        for (int i = 1; i < args.length; i++) {
11            int val = Integer.parseInt(args[i]);
12            if (val > max) max = val;
13            if (val < min) min = val;
14        }
15        System.out.println("Maximum: " + max);
16        System.out.println("Minimum: " + min);
17    }
18 }
```

**Sample run:**

```
1 $ java MaxMin 4 7 -2 15 3
2 Maximum: 15
3 Minimum: -2
```

**Q22.** Consider the following main method. Write the Java function `testf` (only one function allowed):

```
1 public class TestMain{
2     public static void main(String[] args){
3         int a = testf("sum", 1, 2, 4, 5, 6); // a = 18
4         int b = testf("sum", 1, 3, 6);       // b = 10
5         int c = testf("mult", 1, 2, 4, 5, 6); // c = 240
6         int d = testf("mult", 1, 3, 6);      // d = 18
7     }
8 }
```

(10 marks)

[CSE 201 | 2014–15]

**Answer — testf() with Varargs**

```
1 public class TestMain {
2
3     // Single method using varargs to handle any number of
4     // integers
5     public static int testf(String op, int... nums) {
6         if (op.equals("sum")) {
```

```

6         int sum = 0;
7         for (int n : nums) sum += n;
8         return sum;
9     } else if (op.equals("mult")) {
10        int product = 1;
11        for (int n : nums) product *= n;
12        return product;
13    }
14    throw new IllegalArgumentException("Unknown op: " +
        op);
15 }
16
17 public static void main(String[] args) {
18     int a = testf("sum", 1, 2, 4, 5, 6); // a = 18
19     int b = testf("sum", 1, 3, 6);       // b = 10
20     int c = testf("mult", 1, 2, 4, 5, 6); // c = 240
21     int d = testf("mult", 1, 3, 6);      // d = 18
22     System.out.println(a + " " + b + " " + c + " " + d);
23 }
24 }

```

- Q23.** Write down the differences between: (i) String and StringBuffer classes in Java; (ii) method overriding and method overloading. **(6 marks)** [CSE 201 | 2013–14]
- Q24.** Write a method that accepts an array of Strings SearchList and a search String SearchKey. The method determines and returns the number of occurrences of SearchKey in SearchList. **(8 marks)** [CSE 201 | 2012–13]

#### Answer — countOccurrences Method

```

1 public class SearchDemo {
2
3     public static int countOccurrences(String[] searchList,
4                                       String searchKey) {
5         int count = 0;
6         for (String s : searchList)
7             if (s.equals(searchKey)) count++;
8         return count;
9     }
10
11     public static void main(String[] args) {
12         String[] list = {"apple", "banana", "apple", "cherry",
13                          "apple"};
14         System.out.println(countOccurrences(list, "apple"));
15         // 3
16         System.out.println(countOccurrences(list, "mango"));
17     }
18 }

```



```
15         }
16     }
    // 0
```

- Q25.** Write a code segment to create a 2-D array of Integer class which will contain 4 rows. Each row will have 3, 1, 10, and 5 columns respectively. **(7 marks)** [CSE 201 | 2012–13]

#### Answer — 2-D Integer Array with Jagged Rows

```
1  public class JaggedInteger {
2      public static void main(String[] args) {
3
4          // Outer array: 4 rows; column sizes: 3, 1, 10, 5
5          Integer[][] arr = new Integer[4][];
6          arr[0] = new Integer[3];
7          arr[1] = new Integer[1];
8          arr[2] = new Integer[10];
9          arr[3] = new Integer[5];
10
11         // Print to verify (default is null for Integer
12         // objects)
13         for (Integer[] row : arr) {
14             for (Integer elem : row)
15                 System.out.print(elem + " "); // prints null
16             System.out.println();
17         }
18     }
```

*Note: Unlike primitive `int[]`, an `Integer[]` is initialised to `null` (not 0) by default because `Integer` is a reference type.*

- Q26.** Differentiate between primitive types and reference types in Java. **(5 marks)** [CSE 201 | 2010–11]

## Answer — Primitive Types vs. Reference Types

| Primitive Types                                                                                                                                                                         | Reference Types                                                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| 8 built-in types: <code>byte</code> , <code>short</code> , <code>int</code> , <code>long</code> , <code>float</code> , <code>double</code> , <code>char</code> , <code>boolean</code> . | Classes, interfaces, arrays, and enums.                                                 |
| Variable stores the <i>actual value</i> directly.                                                                                                                                       | Variable stores a <i>memory address</i> (reference) pointing to the object on the heap. |
| Stored on the stack (for local variables).                                                                                                                                              | Object is stored on the heap; the reference may be on the stack.                        |
| Cannot be <code>null</code> .                                                                                                                                                           | Can be <code>null</code> (no object referenced).                                        |
| Passed by value; callee gets a copy.                                                                                                                                                    | Passed by value of the reference; callee can modify the object through the reference.   |
| No methods; operations via operators only.                                                                                                                                              | Have methods and fields; support OOP features.                                          |

```

1 int x = 5;           // primitive -- value 5 stored directly
2 String s = "Hello"; // reference -- s holds address of
   String object
3 s = null;           // valid for references
4 // x = null;         // compile error -- primitives cannot be
   null

```

**Q27.** Write two differences between Java reference and C++ pointer. (4 marks) [CSE 201 | 2010–11]

## Answer — Java Reference vs. C++ Pointer

| Java Reference                                                                                                                                   | C++ Pointer                                                                                                                               |
|--------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Cannot perform arithmetic operations (e.g. <code>ref++</code> is illegal). A reference always points to a valid object or is <code>null</code> . | Supports pointer arithmetic ( <code>ptr++</code> , <code>ptr + 2</code> , etc.), allowing raw traversal of memory.                        |
| Automatically managed by the <b>Garbage Collector</b> ; the programmer never calls <code>free</code> or <code>delete</code> .                    | Manually managed; the programmer must call <code>delete</code> / <code>free</code> to release memory, risking leaks or dangling pointers. |

**Q28.** Create a jagged array having three rows with 2, 5, and 3 elements respectively. All elements initialized to default values. Write code to print the contents using Java's for-each statement only. (4+4 marks) [CSE 201 | 2010–11]

**Answer — Jagged Array: Creation & For-Each Print**

```

1 public class JaggedArray {
2     public static void main(String[] args) {
3
4         // Create jagged array: rows with 2, 5, and 3
4         // elements
5         int[][] jagged = new int[3][];
6         jagged[0] = new int[2];    // default: all 0
7         jagged[1] = new int[5];    // default: all 0
8         jagged[2] = new int[3];    // default: all 0
9
10        // Print using for-each only (no index variable)
11        for (int[] row : jagged) {    // outer for-each
12            : each row
13            for (int elem : row) {    // inner for-each
14                : each element
15                System.out.print(elem + " ");
16            }
17            System.out.println();    // newline after
18            : each row
19        }
20    }
21 }

```

**Output:**

```

1 0 0
2 0 0 0 0 0
3 0 0 0

```

**Q29.** What is the role of Garbage Collector in Java? Consider the following code snippet and answer: (i) What will be the output? (ii) How many String objects will be created? Show contents of each and which will be garbage collected.

```

1 String s1 = "It";
2 String s2 = "was";
3 String s3 = s1 + " " + s2;
4 s2 += " roses, ";
5 s3 = s3 + s2 + "roses all the way.";
6 System.out.println(s3);

```

**(2+8 marks)**

[CSE 201 | 2010–11]

**Answer — Garbage Collector & String Object Analysis****Role of Garbage Collector (GC):**

The GC automatically reclaims heap memory occupied by objects that are no longer reachable by any live reference, preventing memory leaks without requiring the

programmer to call `free/delete`.

(i) Output of the code:

```
1 String s1 = "It";
2 String s2 = "was";
3 String s3 = s1 + " " + s2;           // "It was"
4 s2 += " roses, ";                  // s2 = "was roses, "
5 s3 = s3 + s2 + "roses all the way.";
6 System.out.println(s3);
```

Output:

```
1 It was was roses, roses all the way.
```

(ii) String objects created and garbage collected:

| # | Content                                | Created by     | GC?                 |
|---|----------------------------------------|----------------|---------------------|
| 1 | "It"                                   | literal        | No (pool)           |
| 2 | "was"                                  | literal        | No (pool)           |
| 3 | " "                                    | literal (temp) | Yes                 |
| 4 | "It was"                               | s1+" "+s2      | Yes (s3 reassigned) |
| 5 | "was roses, "                          | s2 += ...      | No (s2 holds it)    |
| 6 | "roses all the way."                   | literal        | No (pool)           |
| 7 | "It was was roses, roses all the way." | final s3       | No (s3 holds it)    |

Objects **3** and **4** become unreachable and are eligible for garbage collection: the intermediate " " string and the old value of `s3` ("It was") after `s3` is reassigned.

**Q30.** What do you understand by short-circuit evaluation related to operators in Java? Which operators show this behavior? Explain with an example. **(8 marks)** [CSE 201 | 2010–11]

### Answer — Short-Circuit Evaluation in Java

**Definition:**

Short-circuit evaluation means the second operand of a logical expression is evaluated *only if necessary* to determine the final result.

**Operators that exhibit this behaviour:**

- **&& (logical AND)** — if the left operand is **false**, the overall result is definitely **false**; the right operand is **not evaluated**.
- **|| (logical OR)** — if the left operand is **true**, the overall result is definitely **true**; the right operand is **not evaluated**.

*Note:* The bitwise operators `&` and `|` do **not** short-circuit — both operands are always evaluated.

**Example:**

```

1  int x = 0;
2
3  // Short-circuit AND: x != 0 is false, so (10 / x > 1) is
   NEVER evaluated
4  // Without short-circuit, dividing by x=0 would throw
   ArithmeticException
5  if (x != 0 && (10 / x > 1))
6      System.out.println("Condition true");
7  else
8      System.out.println("Short-circuited -- no division by
   zero");
9
10 // Short-circuit OR: first condition is true, second is
   skipped
11 boolean flag = true;
12 if (flag || (++x > 5))
13     System.out.println("x = " + x); // prints x = 0 (x not
   incremented)

```

- Q31.** Explain two usages of the keyword `static`. (8 marks) [CSE 201 | 2010–11]
- Q32.** What is the use of an `import` declaration in Java? What is the restriction? Mention two cases where import declarations are not required. (6 marks) [CSE 201 | 2010–11]
- Q33.** Write the names, byte size, and default values of the primitive data types in Java. (15 marks) [CSE 201 | 2009–10]

**Answer — Java Primitive Data Types**

| Type    | Size (bytes)      | Default value | Range / Notes                           |
|---------|-------------------|---------------|-----------------------------------------|
| byte    | 1                 | 0             | −128 to 127                             |
| short   | 2                 | 0             | −32,768 to 32,767                       |
| int     | 4                 | 0             | −2 <sup>31</sup> to 2 <sup>31</sup> − 1 |
| long    | 8                 | 0L            | −2 <sup>63</sup> to 2 <sup>63</sup> − 1 |
| float   | 4                 | 0.0f          | IEEE 754 single precision               |
| double  | 8                 | 0.0d          | IEEE 754 double precision               |
| char    | 2                 | '\u0000'      | Unicode 0 to 65,535                     |
| boolean | 1 (JVM-dependent) | false         | true or false                           |

*Note: Default values apply only to instance/class variables. Local variables have no default and must be initialised before use.*

- Q34.** What are the five phases Java programs normally go through? Briefly describe them.

(10 marks)

[CSE 201 | 2008–09]

**Answer — Five Phases of a Java Program**

1. **Edit** — The programmer writes the source code in a text editor and saves the file with a `.java` extension.
2. **Compile** — The Java compiler (`javac`) translates the `.java` source file into **bytecode** and stores it in a `.class` file. Bytecode is platform-independent intermediate code.
3. **Load** — The **Class Loader** (part of the JVM) reads the `.class` file from disk (or network) into memory, ready for execution.
4. **Verify** — The **Bytecode Verifier** checks the loaded bytecode for correctness: it ensures no stack overflows, that operands are of the correct type, and that no illegal data conversions are attempted. This step provides security.
5. **Execute** — The **Java Interpreter** (or Just-In-Time compiler) within the JVM translates bytecode into native machine instructions and runs the program.

`.java`  $\xrightarrow{\text{javac}}$  `.class` (bytecode)  $\xrightarrow{\text{ClassLoader}}$  Memory  $\xrightarrow{\text{Verifier}}$  Verified  $\xrightarrow{\text{JVM}}$  Execution

**Q35.** Write two different usages with examples for each of the following Java keywords: (i) `this`, (ii) `final`, (iii) `super`, (iv) `extends`, (v) `static`. (20 marks) [CSE 201 | 2009–10]

**Answer — Two Usages Each: `this`, `final`, `super`, `extends`, `static`****(i) `this`**

```

1  class Point {
2      int x, y;
3      // Usage 1: distinguish field from parameter
4      Point(int x, int y) { this.x = x; this.y = y; }
5      // Usage 2: call another constructor in the same class
6      Point() { this(0, 0); }
7  }

```

**(ii) `final`**

```

1  // Usage 1: final variable (constant)
2  final double PI = 3.14159;
3
4  // Usage 2: final method (cannot be overridden)
5  class Base {
6      final void display() { System.out.println("Base"); }
7  }

```

```
8 // class Child extends Base { void display(){ } } // error
```

**(iii) super**

```
1 class Animal { String name; Animal(String n) { name=n; } }
2 class Dog extends Animal {
3     // Usage 1: call parent constructor
4     Dog(String n) { super(n); }
5     // Usage 2: call overridden parent method
6     void show() { super.show(); System.out.println("Dog"); }
7 }
```

**(iv) extends**

```
1 // Usage 1: class inheritance
2 class Vehicle { int speed; }
3 class Car extends Vehicle { int doors; }
4
5 // Usage 2: interface extension
6 interface Flyable { void fly(); }
7 interface SuperFlyable extends Flyable { void soar(); }
```

**(v) static**

```
1 class Counter {
2     // Usage 1: static field (shared across all instances)
3     static int count = 0;
4     Counter() { count++; }
5
6     // Usage 2: static method (callable without an object)
7     static int getCount() { return count; }
8 }
9 // Counter.getCount() - no instance needed
```

**Q36.** Write Java code to create the following multidimensional array: [2007–2008]

```
1 0 0 0 0
2 0 0 0
3 0 0
4 0
```

**Answer — Jagged (Ragged) 2-D Array**

```
1 public class JaggedArray {
2     public static void main(String[] args) {
3         // Create a jagged array: row i has (4-i) columns
4         int[][] arr = new int[4][];
5         for (int i = 0; i < arr.length; i++)
```

```

6         arr[i] = new int[4 - i]; // 4, 3, 2, 1 elements
7
8         // All elements default to 0 - print to verify
9         for (int i = 0; i < arr.length; i++) {
10             for (int j = 0; j < arr[i].length; j++)
11                 System.out.print(arr[i][j] + " ");
12             System.out.println();
13         }
14     }
15 }

```

**Output:**

```

0 0 0 0
0 0 0
0 0
0

```

**Explanation:** In Java, a 2-D array is an array of arrays. Allocating each row separately (`arr[i] = new int[4-i]`) creates rows of different lengths (a *jagged* or *ragged* array). All `int` elements are automatically initialised to 0.

- Q37.** Write a Java code that takes  $n$  integers from the command line and outputs the maximum. Example: `java Max 1 2 4 5 -3 6 10 -7` outputs 10. [2007–2008]

**Answer — Maximum of n Command-Line Integers**

```

1 public class Max {
2     public static void main(String[] args) {
3         if (args.length == 0) {
4             System.out.println("Usage: java Max n1 n2 ...");
5             return;
6         }
7
8         int max = Integer.parseInt(args[0]);
9         for (int i = 1; i < args.length; i++) {
10             int n = Integer.parseInt(args[i]);
11             if (n > max) max = n;
12         }
13
14         System.out.println(max);
15     }
16 }

```

**Run:**

```
java Max 1 2 4 5 -3 6 10 -7
```

**Output:**



10

**Key points:**

- `args` is a `String[]`; each argument is parsed to `int` using `Integer.parseInt()`.
- The first element seeds `max`; the loop updates it as needed.
- `args.length` replaces the C-style `argc`.

BUET · CSE 107 / CSE 201

# Object Oriented Programming

Section S11

---

Java OOP: Classes, Inheritance & Polymorphism

## S11 — Java OOP: Classes, Inheritance & Polymorphism

Q1. What will be the output of the following code?

```

1  abstract class X {
2      X() { System.out.println("X Constructor"); draw(); }
3      abstract void draw();
4      abstract void process(int x);
5      void process(Object o) { System.out.println("X process
        Object"); }
6  }
7  class Y extends X {
8      int y = 5;
9      Y() { System.out.println("Y Constructor"); }
10     void draw() { System.out.println("Y with y =" + y); }
11     void process(int x) { System.out.println("Y process int");
        }
12     void process(String s) { System.out.println("Y process
        String"); }
13 }
14 class Test {
15     public static void main(String[] args) {
16         X obj = new Y();
17         obj.process(10);
18         obj.process("hello");
19         obj.process((Object)"hello");
20     }
21 }

```

(10 marks)

[CSE 107 | 2023–24]

### Answer — Output Tracing: Abstract Class, Overriding and Overloading

#### Output:

```

X Constructor
Y with y =0
Y Constructor
Y process int
X process Object
X process Object

```

#### Explanation step by step:

1. `new Y()` first calls `X()`. Inside `X()`, `draw()` is called. Dynamic dispatch calls `Y.draw()`, but `y` is not yet initialised (still 0), so it prints **Y with y =0**.
2. `X()` finishes; `y = 5` is now assigned; `Y()` prints **Y Constructor**.
3. `obj.process(10)` — `obj` is of type `X`. `X` declares `process(int)` as abstract, overridden by `Y` — prints **Y process int**.

4. `obj.process("hello")` — compile-time type is `X`, which has no `process(String)`; the only match is `process(Object)` in `X` — prints **X process Object**.
5. `obj.process((Object)"hello")` — cast forces the argument to `Object`; again matches `X.process(Object)` — prints **X process Object**.

**Key concept:** Overloading is resolved at *compile time* based on the declared type (`X`); overriding is resolved at *run time* based on the actual object type (`Y`).

**Q2.** Write Java code for the following using a `Book` class to achieve the expected output:

```

1  class Book {
2      private String title;
3      private String author;
4      private int year;
5      private int edition;
6
7      public Book(String title, String author, int year, int
          edition) {
8          this.title = title;
9          this.author = author;
10         this.year = year;
11         this.edition = edition;
12     }
13
14     public static void main(String[] args) {
15         Book b1 = new Book("1984", "George Orwell", 1949, 1);
16         Book b2 = new Book("1984", "George Orwell", 1949, 2);
17
18         System.out.println(b1);           // Book (1984,
19             George Orwell, 1949, 1)
20         System.out.println(b1 == b2);     // false
21         System.out.println(b1.equals(b2)); // true
22
23         HashSet<Book> set = new HashSet<>();
24         set.add(b1);
25         set.add(b2);
26         System.out.println(set.size());   // 1
27     }

```

(15 marks)

[CSE 107 | 2023–24]

**Answer — Book Class: toString, equals, hashCode**

```

1  import java.util.HashSet;
2  import java.util.Objects;
3

```

```

4  class Book {
5      private String title;
6      private String author;
7      private int    year;
8      private int    edition;
9
10     Book(String title, String author, int year, int edition)
11     {
12         this.title    = title;
13         this.author   = author;
14         this.year     = year;
15         this.edition  = edition;
16     }
17
18     // Two Books are equal if title, author, and year match
19     // (edition is ignored for equality)
20     @Override
21     public boolean equals(Object obj) {
22         if (this == obj) return true;
23         if (!(obj instanceof Book)) return false;
24         Book o = (Book) obj;
25         return year == o.year
26             && title.equals(o.title)
27             && author.equals(o.author);
28     }
29
30     // hashCode must be consistent with equals
31     @Override
32     public int hashCode() {
33         return Objects.hash(title, author, year);
34     }
35
36     // Format: Book (title, author, year, edition)
37     @Override
38     public String toString() {
39         return "Book (" + title + ", " + author
40             + ", " + year + ", " + edition + ")";
41     }
42 }
43
44 class BookDemo {
45     public static void main(String[] args) {
46         Book b1 = new Book("1984", "George Orwell", 1949, 1);
47         Book b2 = new Book("1984", "George Orwell", 1949, 2);
48
49         System.out.println(b1);           // Book (1984,
50             George Orwell, 1949, 1)
51         System.out.println(b1 == b2);     // false

```

```

50         System.out.println(b1.equals(b2)); // true
51
52         HashSet<Book> set = new HashSet<>();
53         set.add(b1); set.add(b2);
54         System.out.println(set.size());    // 1
55     }
56 }

```

**Note:** `hashCode()` must be overridden alongside `equals()` so that equal objects produce the same hash — a requirement for correct behaviour in `HashSet` and `HashMap`.

**Q3.** Consider the following `Point` and `Circle` classes. Complete them to achieve the expected output:

```

1  import java.util.HashMap;
2
3  class Point {
4      private int x, y;
5
6      Point(int x, int y) {
7          this.x = x;
8          this.y = y;
9      }
10 }
11
12 class Circle {
13     Point center;
14     int radius;
15
16     Circle(int x, int y, int r) {
17         center = new Point(x, y);
18         this.radius = r;
19     }
20 }
21
22 public class Test {
23     public static void main(String[] args) {
24         Circle c1 = new Circle(10, 20, 5);
25         Circle c2 = new Circle(10, 20, 5);
26         System.out.println(c1);
27         System.out.println(c2);
28         System.out.println(c1.equals(c2));
29         HashMap<Circle, String> m = new HashMap<>();
30         m.put(c1, "I am a Circle");
31         System.out.println(m.get(c2));
32     }
33 }
34

```

```

35 // Expected output:
36 // Circle: center (10, 20), radius: 5
37 // Circle: center (10, 20), radius: 5
38 // true
39 // I am a Circle
40 // Note: Point's getters for x and y are private

```

(15 marks)

[CSE 107 | 2021–22]

## Answer — Point and Circle Classes

```

1  import java.util.HashMap;
2  import java.util.Objects;
3
4  class Point {
5      private int x, y;
6
7      Point(int x, int y) {
8          this.x = x;
9          this.y = y;
10     }
11
12     public int getX() { return x; }
13     public int getY() { return y; }
14
15     @Override
16     public boolean equals(Object o) {
17         if (this == o) return true;
18         if (!(o instanceof Point)) return false;
19         Point p = (Point) o;
20         return x == p.x && y == p.y;
21     }
22
23     @Override
24     public int hashCode() {
25         return Objects.hash(x, y);
26     }
27 }
28
29 class Circle {
30     Point center;
31     int radius;
32
33     Circle(int x, int y, int r) {
34         center = new Point(x, y);
35         this.radius = r;
36     }
37
38     @Override

```

```

39     public String toString() {
40         return "Circle: center (" + center.getX() + ", " +
41             center.getY() + "), radius: " + radius;
42     }
43
44     @Override
45     public boolean equals(Object o) {
46         if (this == o) return true;
47         if (!(o instanceof Circle)) return false;
48         Circle c = (Circle) o;
49         return radius == c.radius && center.equals(c.center);
50     }
51
52     @Override
53     public int hashCode() {
54         return Objects.hash(center, radius);
55     }
56 }
57
58 public class Test {
59     public static void main(String[] args) {
60         Circle c1 = new Circle(10, 20, 5);
61         Circle c2 = new Circle(10, 20, 5);
62
63         System.out.println(c1);
64         System.out.println(c2);
65         System.out.println(c1.equals(c2));
66
67         HashMap<Circle, String> m = new HashMap<>();
68         m.put(c1, "I am a Circle");
69
70         System.out.println(m.get(c2));
71     }
72 }

```

**Q4.** Complete the Movie class (with title, releaseYear, productionCompany, runningTime) to achieve the expected output:

```

1  Movie m1 = new Movie("The Lord of the Rings", 2001, "New Line
   Cinema", 178);
2  Movie m2 = new Movie("The Lord of the Rings", 2001, "WingNut
   Films", 178);
3  System.out.println(m1 == m2);           // false
4  System.out.println(m1.equals(m2));      // true
5  HashMap map = new HashMap();
6  map.put(m1, 93);
7  System.out.println(map.get(m2));        // 93

```



(10 marks)

[CSE 107 | 2020–21]

## Answer — Movie Class: equals and hashCode for HashMap

```

1  import java.util.HashMap;
2  import java.util.Objects;
3
4  class Movie {
5      private String title;
6      private int    releaseYear;
7      private String productionCompany;
8      private int    runningTime;
9
10     Movie(String title, int releaseYear,
11           String productionCompany, int runningTime) {
12         this.title           = title;
13         this.releaseYear     = releaseYear;
14         this.productionCompany = productionCompany;
15         this.runningTime     = runningTime;
16     }
17
18     // Equal if title, releaseYear, and runningTime match
19     // (productionCompany is intentionally excluded)
20     @Override
21     public boolean equals(Object obj) {
22         if (this == obj) return true;
23         if (!(obj instanceof Movie)) return false;
24         Movie o = (Movie) obj;
25         return releaseYear == o.releaseYear
26             && runningTime == o.runningTime
27             && title.equals(o.title);
28     }
29
30     @Override
31     public int hashCode() {
32         return Objects.hash(title, releaseYear, runningTime);
33     }
34 }
35
36 class MovieDemo {
37     public static void main(String[] args) {
38         Movie m1 = new Movie("The Lord of the Rings",
39                             2001, "New Line Cinema", 178);
40         Movie m2 = new Movie("The Lord of the Rings",
41                             2001, "WingNut Films", 178);
42
43         System.out.println(m1 == m2);           // false
44         System.out.println(m1.equals(m2));      // true
45     }
46 }

```

```

46         HashMap map = new HashMap();
47         map.put(m1, 93);
48         System.out.println(map.get(m2));        // 93
49     }
50 }

```

**Q5.** Complete the Institute/College/Movie class to achieve the expected output (equals returns true, HashMap.get returns 100):

```

1 // Expected output: true, 100
2 College c1 = new College(135790, 1, 1, 0);
3 College c2 = new College(135790, 1, 1, 0);
4 System.out.println(c1.equals(c2));
5 HashMap map = new HashMap();
6 map.put(c1, 100);
7 System.out.println(map.get(c2));

```

(8+10 marks)

[CSE 107 | 2016–17, 2019–20, 2020–21]

#### Answer — Institute/College: equals and hashCode

```

1 import java.util.HashMap;
2 import java.util.Objects;
3
4 class Institute {
5     protected int id;
6
7     Institute(int id) { this.id = id; }
8
9     @Override
10    public boolean equals(Object obj) {
11        if (this == obj) return true;
12        if (!(obj instanceof Institute)) return false;
13        Institute o = (Institute) obj;
14        return this.id == o.id;
15    }
16
17    @Override
18    public int hashCode() {
19        return Objects.hash(id);
20    }
21 }
22
23 class College extends Institute {
24     private int dept;
25     private int batch;
26     private int section;

```

```

27
28     College(int id, int dept, int batch, int section) {
29         super(id);
30         this.dept    = dept;
31         this.batch    = batch;
32         this.section = section;
33     }
34
35     @Override
36     public boolean equals(Object obj) {
37         if (this == obj) return true;
38         if (!(obj instanceof College)) return false;
39         College o = (College) obj;
40         return super.equals(o)
41             && this.dept    == o.dept
42             && this.batch    == o.batch
43             && this.section == o.section;
44     }
45
46     @Override
47     public int hashCode() {
48         return Objects.hash(id, dept, batch, section);
49     }
50 }
51
52 class CollegeDemo {
53     public static void main(String[] args) {
54         College c1 = new College(135790, 1, 1, 0);
55         College c2 = new College(135790, 1, 1, 0);
56
57         System.out.println(c1.equals(c2)); // true
58
59         HashMap map = new HashMap();
60         map.put(c1, 100);
61         System.out.println(map.get(c2));   // 100
62     }
63 }

```

**Q6.** Write Java code to fix the problems in the main method:

```

1  class P { int x = 1;
2      class Q { int y = 2; }
3      static class R { int z = 3; }
4  }
5  public class NestedClassTest {
6      public static void main(String[] args) {
7          P p = new P(); p.print(); p.show();
8          Q q = new Q(); q.print();

```

```

9         R r = new R(); r.print();
10     }
11 }

```

(5 marks)

[CSE 107 | 2020–21, 2017–18]

### Answer — Fixing Nested Class main() Problems

#### Problems in the original main():

- `p.print()` and `p.show()` — `P` has no such methods; these calls must be removed or added to `P`.
- `Q q = new Q()` — `Q` is an *inner* class of `P`; it cannot be instantiated without a `P` instance.
- `q.print()` — `Q` has no `print()` method.
- `R r = new R()` — `R` is `P.R`, not simply `R` in the outer scope.
- `r.print()` — `R` has no `print()` method.

#### Fixed code:

```

1  class P {
2      int x = 1;
3
4      class Q {
5          int y = 2;
6          void print() { System.out.println("Q.y = " + y); }
7      }
8
9      static class R {
10         int z = 3;
11         void print() { System.out.println("R.z = " + z); }
12     }
13
14     void print() { System.out.println("P.x = " + x); }
15 }
16
17 public class NestedClassTest {
18     public static void main(String[] args) {
19         P p = new P();
20         p.print();                // P.x = 1
21
22         P.Q q = p.new Q();        // inner class needs outer
23                                 // instance
24         q.print();                // Q.y = 2
25
26         P.R r = new P.R();        // static nested class: no
27                                 // outer needed
28         r.print();                // R.z = 3
29     }
30 }

```

```

27     }
28 }

```

**Key rules:**

- **Inner class (Q):** requires an enclosing instance — use `outerRef.new Inner()`.
- **Static nested class (R):** no enclosing instance needed — use `new Outer.Nested()`.

**Q7.** Write Java code for an Animal hierarchy where the Animal class cannot be instantiated:

```

1 Bird alex = new Albatross("Alex", 39);
2 Mammal spot = new Dog("Spot", 7);
3 Mammal fred = new FruitBat("Fred", 10);
4 Reptile steph = new EasternBrownSnake("Steph", 12);
5 Fish bruce = new GreatWhiteShark("Bruce", 40);
6 Arachnid charlotte = new RedBackSpider("Charlotte", 1);
7 Animal[] animals = {alex, spot, fred, steph, bruce, charlotte
    };

```

(10 marks)

[CSE 107 | 2020–21]

**Answer — Animal Hierarchy (Abstract Base Class)**

```

1  // Abstract base -- cannot be instantiated directly
2  abstract class Animal {
3      protected String name;
4      protected int    age;
5
6      Animal(String name, int age) {
7          this.name = name; this.age = age;
8      }
9
10     @Override
11     public String toString() {
12         return getClass().getSimpleName()
13             + "(" + name + "\", " + age + ")";
14     }
15 }
16
17 // Mid-level abstract classes
18 abstract class Bird extends Animal { Bird(String n, int a
19     ) { super(n,a); } }
19 abstract class Mammal extends Animal { Mammal(String n, int
20     a) { super(n,a); } }
20 abstract class Reptile extends Animal { Reptile(String n,

```

```

    int a) { super(n,a); } }
21 abstract class Fish extends Animal { Fish(String n, int a
    ) { super(n,a); } }
22 abstract class Arachnid extends Animal { Arachnid(String n,
    int a) { super(n,a); } }
23
24 // Concrete leaf classes
25 class Albatross extends Bird { Albatross(String n,
    int a) { super(n,a); } }
26 class Dog extends Mammal { Dog(String n, int a)
    { super(n,a); } }
27 class FruitBat extends Mammal { FruitBat(String n,
    int a) { super(n,a); } }
28 class EasternBrownSnake extends Reptile{ EasternBrownSnake(
    String n,int a){ super(n,a); } }
29 class GreatWhiteShark extends Fish { GreatWhiteShark(
    String n, int a) { super(n,a); } }
30 class RedBackSpider extends Arachnid { RedBackSpider(String
    n, int a) { super(n,a); } }
31
32 class AnimalHierarchy {
33     public static void main(String[] args) {
34         Bird alex = new Albatross("Alex", 39);
35         Mammal spot = new Dog("Spot", 7);
36         Mammal fred = new FruitBat("Fred", 10);
37         Reptile steph = new EasternBrownSnake("Steph",
            12);
38         Fish bruce = new GreatWhiteShark("Bruce", 40);
39         Arachnid charlotte= new RedBackSpider("Charlotte", 1)
            ;
40
41         Animal[] animals = {alex, spot, fred, steph, bruce,
            charlotte};
42
43         for (Animal a : animals)
44             System.out.println(a);
45     }
46 }

```

**Output:**

```

Albatross("Alex", 39)
Dog("Spot", 7)
FruitBat("Fred", 10)
EasternBrownSnake("Steph", 12)
GreatWhiteShark("Bruce", 40)
RedBackSpider("Charlotte", 1)

```

**Q8.** Explain briefly dynamic method dispatching. (6 marks) [CSE 107 | 2017–18]

### Answer — Dynamic Method Dispatching

**Dynamic Method Dispatching** is the mechanism by which a call to an overridden method is resolved at **run time** rather than compile time. It is the foundation of run-time polymorphism in Java.

When a superclass reference variable holds a subclass object, and an overridden method is called through that reference, Java determines *which version* of the method to execute based on the **actual type of the object** at run time — not the declared type of the reference.

```

1  class Shape {
2      void draw() { System.out.println("Drawing Shape"); }
3  }
4  class Circle extends Shape {
5      @Override
6      void draw() { System.out.println("Drawing Circle"); }
7  }
8  class Rectangle extends Shape {
9      @Override
10     void draw() { System.out.println("Drawing Rectangle"); }
11 }
12
13 class Demo {
14     public static void main(String[] args) {
15         Shape s;           // superclass reference
16
17         s = new Circle();
18         s.draw();           // "Drawing Circle"  --
                             // dispatched at run time
19
20         s = new Rectangle();
21         s.draw();           // "Drawing Rectangle" --
                             // dispatched at run time
22     }
23 }

```

The JVM uses a *virtual method table* (vtable) per class to look up the correct method implementation at run time. This enables writing general-purpose code against a superclass type that automatically adapts to specific subclass behaviour.

**Q9.** Write Java code to exchange two integer variables using a method named `swap`. The changes inside the method must be visible to the `main` method. Write the `main` method too. (7 marks) [CSE 107 | 2016–17]

**Answer — swap() Visible to main()**

Java is *pass-by-value*, so swapping primitive ints inside a method is **not** visible to the caller. To make the swap visible, wrap the integers in an array (or any mutable object).

```

1  class SwapDemo {
2      // swap using a 2-element array
3      static void swap(int[] a) {
4          int tmp = a[0];
5          a[0]    = a[1];
6          a[1]    = tmp;
7      }
8
9      public static void main(String[] args) {
10         int[] vals = {10, 20};
11         System.out.println("Before: a=" + vals[0] + ", b=" +
12             vals[1]);
13         swap(vals);
14         System.out.println("After:  a=" + vals[0] + ", b=" +
15             vals[1]);
16     }
17 }

```

**Output:**

Before: a=10, b=20

After: a=20, b=10

**Why it works:** The array reference is passed by value, but the array object on the heap is shared, so modifications to `a[0]` and `a[1]` are visible in `main()`.

**Or**

```

1  class NumberWrapper {
2      int value;
3      NumberWrapper(int value) { this.value = value; }
4  }
5
6  public class SwapObjects {
7      public static void main(String[] args) {
8          NumberWrapper objA = new NumberWrapper(5);
9          NumberWrapper objB = new NumberWrapper(10);
10
11         swap(objA, objB);
12
13         System.out.println("objA: " + objA.value); // Outputs
14             10
15         System.out.println("objB: " + objB.value); // Outputs
16             5
17     }
18 }

```



```

16
17     public static void swap(NumberWrapper a, NumberWrapper b)
18     {
19         int temp = a.value;
20         a.value = b.value;
21         b.value = temp;
22     }

```

**Q10.** Write Java code for a TwoDimMatrix class: input, print, determinant, and div(A) methods. (15 marks) [CSE 201 | 2015–16]

#### Answer — TwoDimMatrix Class

```

1  import java.util.Scanner;
2
3  class TwoDimMatrix {
4      private double[][] m = new double[2][2];
5
6      // Input 4 elements row-by-row
7      void input() {
8          Scanner sc = new Scanner(System.in);
9          System.out.println("Enter 4 elements (row by row):");
10         for (int i = 0; i < 2; i++)
11             for (int j = 0; j < 2; j++)
12                 m[i][j] = sc.nextDouble();
13     }
14
15     // Print the 2x2 matrix
16     void print() {
17         for (int i = 0; i < 2; i++)
18             System.out.printf("| %6.2f  %6.2f |%n",
19                             m[i][0], m[i][1]);
20     }
21
22     // Determinant of a 2x2 matrix: ad - bc
23     double determinant() {
24         return m[0][0] * m[1][1] - m[0][1] * m[1][0];
25     }
26
27     // div(A): divide this matrix by A (multiply by A's
28     // inverse)
29     // A^-1 = (1/det(A)) * [[d,-b],[-c,a]]
30     TwoDimMatrix div(TwoDimMatrix A) {
31         double det = A.determinant();
32         if (det == 0)
33             throw new ArithmeticException("Matrix A is

```

```

        singular.");
33
34        TwoDimMatrix result = new TwoDimMatrix();
35        // result = this * A^-1
36        double[][] inv = {
37            { A.m[1][1] / det, -A.m[0][1] / det},
38            {-A.m[1][0] / det, A.m[0][0] / det}
39        };
40        for (int i = 0; i < 2; i++)
41            for (int j = 0; j < 2; j++)
42                for (int k = 0; k < 2; k++)
43                    result.m[i][j] += this.m[i][k] * inv[k][j];
44        return result;
45    }
46
47    public static void main(String[] args) {
48        TwoDimMatrix M = new TwoDimMatrix();
49        M.input();
50        System.out.println("Matrix M:"); M.print();
51        System.out.println("det(M) = " + M.determinant());
52
53        TwoDimMatrix A = new TwoDimMatrix();
54        A.input();
55        System.out.println("Matrix A:"); A.print();
56
57        TwoDimMatrix Q = M.div(A);
58        System.out.println("M / A:"); Q.print();
59    }
60 }

```

**Q11.** Write a Java program to store student records as a collection. **Student** contains Name, Age, CGPA. Sort by: descending CGPA first; if equal, ascending age; if equal, lexicographic order. **(20 marks)** [CSE 201 | 2015–16]

#### Answer — Student Collection with Custom Sort

```

1  import java.util.*;
2
3  class Student implements Comparable<Student> {
4      String name;
5      int age;
6      double cgpa;
7
8      Student(String name, int age, double cgpa) {
9          this.name = name; this.age = age; this.cgpa = cgpa;
10     }

```

```

11
12 // Sort: desc CGPA -> asc age -> lexicographic name
13 @Override
14 public int compareTo(Student o) {
15     int cmp = Double.compare(o.cgpa, this.cgpa); // desc
16         CGPA
17     if (cmp != 0) return cmp;
18     cmp = Integer.compare(this.age, o.age); // asc
19         age
20     if (cmp != 0) return cmp;
21     return this.name.compareTo(o.name); // lex
22         name
23 }
24
25 @Override
26 public String toString() {
27     return name + " (age=" + age + ", cgpa=" + cgpa + ")"
28     ;
29 }
30 }
31
32 class StudentSort {
33     public static void main(String[] args) {
34         List<Student> list = new ArrayList<>();
35         list.add(new Student("Alice", 21, 3.8));
36         list.add(new Student("Bob", 20, 3.9));
37         list.add(new Student("Carol", 22, 3.8));
38         list.add(new Student("Dave", 21, 3.8));
39         list.add(new Student("Eve", 19, 4.0));
40
41         Collections.sort(list);
42
43         System.out.println("Sorted students:");
44         for (Student s : list)
45             System.out.println("    " + s);
46     }
47 }

```

**Output:**

Sorted students:

```

Eve (age=19, cgpa=4.0)
Bob (age=20, cgpa=3.9)
Alice (age=21, cgpa=3.8)
Dave (age=21, cgpa=3.8)
Carol (age=22, cgpa=3.8)

```

**Q12.** Write a program to take input of 120 students, sort them as described above, and

write all student information to a file named `Students.Info`. Write student objects directly to file and read them back as objects. **(15 marks)** [CSE 201 | 2015–16]

### Answer — 120 Students: Sort, Serialize to File and Read Back

```

1  import java.io.*;
2  import java.util.*;
3
4  class Student implements Comparable<Student>, Serializable {
5      String name; int age; double cgpa;
6
7      Student(String name, int age, double cgpa) {
8          this.name = name; this.age = age; this.cgpa = cgpa;
9      }
10
11     @Override
12     public int compareTo(Student o) {
13         int c = Double.compare(o.cgpa, this.cgpa);
14         if (c != 0) return c;
15         c = Integer.compare(this.age, o.age);
16         if (c != 0) return c;
17         return this.name.compareTo(o.name);
18     }
19
20     @Override
21     public String toString() {
22         return name + " age=" + age + " cgpa=" + cgpa;
23     }
24 }
25
26 class StudentFile {
27     public static void main(String[] args) throws Exception {
28         Scanner sc = new Scanner(System.in);
29         List<Student> list = new ArrayList<>();
30
31         // Input 120 students
32         for (int i = 0; i < 120; i++) {
33             System.out.print("Name: ");    String name = sc.
34                 next();
35             System.out.print("Age: ");      int    age  = sc.
36                 nextInt();
37             System.out.print("CGPA: ");    double cgpa = sc.
38                 nextDouble();
39             list.add(new Student(name, age, cgpa));
40         }
41
42         Collections.sort(list);
43
44         // Write objects to file

```

```

42         try (ObjectOutputStream oos = new ObjectOutputStream(
43             new FileOutputStream("Students.Info"))) {
44             oos.writeObject(list);
45             System.out.println("Written to Students.Info");
46         }
47
48         // Read objects back from file
49         try (ObjectInputStream ois = new ObjectInputStream(
50             new FileInputStream("Students.Info"))) {
51             @SuppressWarnings("unchecked")
52             List<Student> loaded = (List<Student>) ois.
53                 readObject();
54             System.out.println("Read back " + loaded.size() +
55                 " students:");
56             for (Student s : loaded)
57                 System.out.println("    " + s);
58         }
59     }

```

**Key points:**

- Student implements Serializable so the entire object (and the List) can be written with ObjectOutputStream.
- The whole sorted list is written as a single object and read back with one readObject() call.
- try-with-resources ensures streams are closed automatically.

**Q13.** What is the difference between declaring a variable with **protected** access modifier versus no modifier? (5 marks) [CSE 201 | 2013–14]

**Answer — protected vs Default (No Modifier)**

| Access from                   | No modifier (package-private) | protected |
|-------------------------------|-------------------------------|-----------|
| Same class                    | ✓                             | ✓         |
| Same package                  | ✓                             | ✓         |
| Subclass in different package | ×                             | ✓         |
| Non-subclass, different pkg   | ×                             | ×         |

The only difference is that **protected** additionally allows access from **subclasses in other packages**, while the default (no modifier) restricts access strictly to the same package.

```

1 package p1;
2 class A {
3     int x = 1;           // package-private

```

```

4     protected int y = 2; // protected
5 }
6
7 package p2;
8 class B extends p1.A {
9     void test() {
10        // System.out.println(x); // ERROR -- different
           package
11        System.out.println(y);    // OK -- protected,
           subclass
12    }
13 }

```

**Q14.** How is multiple inheritance implemented in Java? Can an abstract class be final? Explain briefly. (4+4 marks) [CSE 201 | 2013–14]

#### Answer — Multiple Inheritance in Java and abstract + final

##### Multiple Inheritance in Java:

Java does *not* support multiple inheritance through classes (to avoid the Diamond Problem). Instead, it achieves multiple inheritance of **type** via **interfaces** — a class can implement any number of interfaces.

```

1 interface Flyable { void fly(); }
2 interface Swimmable { void swim(); }
3
4 class Duck implements Flyable, Swimmable {
5     public void fly() { System.out.println("Duck flies"); }
6     public void swim() { System.out.println("Duck swims"); }
7 }

```

##### Can an abstract class be final?

**No.** Declaring a class both **abstract** and **final** is a **compile-time error** in Java. The two keywords are contradictory:

- **abstract** requires the class to be *subclassed* so that its abstract methods can be implemented.
- **final** *prevents* any subclassing.

Together they make the class both “must be extended” and “cannot be extended”, which is logically impossible and the compiler rejects it.

**Q15.** Write a class **Animal** that has name and age as member variables. Write another class that adds 3 animals to an **ArrayList** and removes the animal named “Lion”. (10 marks) [CSE 201 | 2012–13]

## Answer — Animal Class with ArrayList

```

1  import java.util.ArrayList;
2  import java.util.Iterator;
3
4  class Animal {
5      private String name;
6      private int age;
7
8      Animal(String name, int age) { this.name = name; this.age
          = age; }
9
10     public String getName() { return name; }
11     public int    getAge()  { return age; }
12
13     @Override
14     public String toString() {
15         return "Animal{name=" + name + ", age=" + age + "}";
16     }
17 }
18
19 class AnimalDemo {
20     public static void main(String[] args) {
21         ArrayList<Animal> list = new ArrayList<>();
22         list.add(new Animal("Lion", 5));
23         list.add(new Animal("Tiger", 3));
24         list.add(new Animal("Parrot", 2));
25
26         System.out.println("Before: " + list);
27
28         Iterator<Animal> it = list.iterator();
29         while (it.hasNext())
30             if (it.next().getName().equals("Lion")) it.remove
31                 ();
32
33         System.out.println("After:  " + list);
34     }
35 }

```

## Output:

Before: [Animal{name=Lion, age=5}, Animal{name=Tiger, age=3}, ...]  
 After: [Animal{name=Tiger, age=3}, Animal{name=Parrot, age=2}]

- Q16.** A movie club contains many movies. Each movie has a title and one or more directors. A movie may be borrowed and returned. Write `Movie` and `MovieClub` classes with a method to return current number of movies. **(16 marks)** [CSE 201 | 2012–13]

## Answer — Movie and MovieClub Classes

```

1  import java.util.ArrayList;
2
3  class Movie {
4      private String title;
5      private ArrayList<String> directors;
6      private boolean borrowed;
7
8      Movie(String title) {
9          this.title = title;
10         this.directors = new ArrayList<>();
11         this.borrowed = false;
12     }
13
14     void addDirector(String d) { directors.add(d); }
15
16     void borrow() {
17         if (!borrowed) borrowed = true;
18         else System.out.println(title + " is already borrowed
19             .");
20     }
21
22     void returnMovie() {
23         if (borrowed) borrowed = false;
24         else System.out.println(title + " was not borrowed.");
25         ;
26     }
27
28     boolean isBorrowed() { return borrowed; }
29
30     @Override
31     public String toString() {
32         return title + " " + directors
33             + (borrowed ? " [borrowed]" : " [available]");
34     }
35 }
36
37 class MovieClub {
38     private ArrayList<Movie> movies = new ArrayList<>();
39
40     void addMovie(Movie m) { movies.add(m); }
41     void removeMovie(Movie m) { movies.remove(m); }
42     int currentCount() { return movies.size(); }
43
44     void listMovies() {
45         for (Movie m : movies) System.out.println(m);
46     }
47 }

```



```

46
47 class MovieDemo {
48     public static void main(String[] args) {
49         Movie m1 = new Movie("Inception");
50         m1.addDirector("Christopher Nolan");
51
52         Movie m2 = new Movie("The Matrix");
53         m2.addDirector("Lana Wachowski");
54         m2.addDirector("Lilly Wachowski");
55
56         MovieClub club = new MovieClub();
57         club.addMovie(m1);
58         club.addMovie(m2);
59
60         System.out.println("Total movies: " + club.
            currentCount());
61         m1.borrow();
62         club.listMovies();
63         m1.returnMovie();
64         System.out.println("After return: " + club.
            currentCount());
65     }
66 }

```

**Output:**

```

Total movies: 2
Inception [Christopher Nolan] [borrowed]
The Matrix [Lana Wachowski, Lilly Wachowski] [available]
After return: 2

```

- Q17.** Write a student class with: id, name, course, number of units completed. Write `inc()` to increment units by one. Write another class to create a student object, get units, call `inc()`, and display the object's state. **(14 marks)** [CSE 201 | 2012–13]

**Answer — Student Class with `inc()` Method**

```

1 class Student {
2     private int id;
3     private String name;
4     private String course;
5     private int unitsCompleted;
6
7     Student(int id, String name, String course, int units) {
8         this.id = id;
9         this.name = name;
10        this.course = course;
11        this.unitsCompleted = units;

```

```

12     }
13
14     int getUnits() { return unitsCompleted; }
15     void inc()      { unitsCompleted++; }
16
17     @Override
18     public String toString() {
19         return "Student{id=" + id + ", name=" + name
20             + ", course=" + course
21             + ", units=" + unitsCompleted + "}";
22     }
23 }
24
25 class StudentDemo {
26     public static void main(String[] args) {
27         Student s = new Student(1, "Alice", "CSE", 30);
28
29         System.out.println("Units before inc(): " + s.
30             getUnits());
31         s.inc();
32         System.out.println("Units after inc(): " + s.
33             getUnits());
34         System.out.println("State: " + s);
35     }
36 }

```

**Output:**

Units before inc(): 30

Units after inc(): 31

State: Student{id=1, name=Alice, course=CSE, units=31}

**Q18.** Show two examples of using the super keyword. (4 marks) [CSE 201 | 2012–13]

**Answer — Two Uses of the super Keyword****Use 1 — Call the parent constructor:**

```

1 class Animal {
2     String name;
3     Animal(String name) { this.name = name; }
4 }
5
6 class Dog extends Animal {
7     String breed;
8     Dog(String name, String breed) {
9         super(name);           // calls Animal(String)
10        this.breed = breed;

```

```

11     }
12 }

```

### Use 2 — Call an overridden parent method:

```

1  class Shape {
2      void draw() { System.out.println("Drawing Shape"); }
3  }
4
5  class Circle extends Shape {
6      @Override
7      void draw() {
8          super.draw();        // calls Shape's draw()
9          System.out.println("Drawing Circle");
10     }
11 }

```

**Q19.** What are the properties of static variables and methods? Why is `main()` declared as static? (6+5 marks) [CSE 201 | 2012–13]

### Answer — Static Variables, Methods and Why `main()` is Static

#### Properties of static variables:

- Belong to the **class**, not to any individual object.
- Only **one copy** exists in memory, shared by all instances.
- Initialized when the class is loaded (before any object is created).
- Accessed as `ClassName.variable` (no object needed).
- Retain their value across all instances throughout the program.

#### Properties of static methods:

- Belong to the class; called without creating an object.
- Can access **only** static members directly.
- Cannot use `this` or `super` keywords.
- Cannot be overridden (only hidden in subclasses).

```

1  class Counter {
2      static int count = 0;    // shared across all instances
3      Counter() { count++; }
4      static int getCount() { return count; }
5  }
6  // Counter.getCount() -- no object needed

```

**Why is `main()` declared static?**

The JVM must call `main()` *before* any object of the class exists. Since no instance is available at startup, `main()` must be **static** so the JVM can invoke it directly on the class without instantiating it first.

**Q20.** Write a short note on the lifetime of an object in Java. (6 marks) [CSE 201 | 2011–12]

**Answer — Lifetime of an Object in Java**

An object's lifetime spans from its **creation** to its **garbage collection**. The stages are:

1. **Creation** — `new` allocates memory on the heap and calls the constructor to initialise the object.
2. **In use (reachable)** — At least one live reference points to the object. It can be accessed and used freely.
3. **Unreachable** — All references go out of scope or are set to `null`. The object is no longer accessible but still occupies heap memory.
4. **Garbage collection** — The JVM's Garbage Collector (GC) detects the unreachable object and reclaims its memory automatically. The programmer cannot force or predict exactly when this happens.
5. **Finalization (deprecated)** — Before collection, the GC may call `finalize()` (deprecated since Java 9) for any last-resort cleanup, but relying on it is strongly discouraged.

```

1 void demo() {
2     Student s = new Student(1, "Alice", "CSE", 30); //
           created
3     s.inc();                                     // in
           use
4 } // s goes out of scope -- object becomes unreachable
5 // GC will eventually reclaim the heap memory

```

Unlike C/C++, the programmer never explicitly frees memory; the GC handles deallocation, preventing dangling pointers and most memory leaks.

**Q21.** Discuss the access levels of a constructor method of a class in Java. (8 marks) [CSE 201 | 2011–12]

**Answer — Access Levels of Constructors in Java**

A constructor can have any of the four access modifiers:

| Modifier      | Who can call it            | Typical use                |
|---------------|----------------------------|----------------------------|
| public        | Anyone, from any package   | Normal instantiation       |
| protected     | Same package + subclasses  | Abstract-like base classes |
| (no modifier) | Same package only          | Package-level factories    |
| private       | Only inside the same class | Singleton, factory methods |

```

1  class Singleton {
2      private static Singleton instance;
3
4      private Singleton() { }    // private constructor
5
6      public static Singleton getInstance() {
7          if (instance == null) instance = new Singleton();
8          return instance;
9      }
10 }
11
12 class Base {
13     protected Base() { }    // only subclasses or same package
14 }
15
16 class Child extends Base {
17     Child() { super(); }    // OK -- subclass can call
18                             // protected ctor
19 }
    
```

If *no* constructor is written, the compiler inserts a default public no-arg constructor automatically.

**Q22.** What is the purpose of the method `finalize` in `Object`? Why is its use discouraged?  
(8 marks) [CSE 201 | 2011–12]

### Answer — `finalize()` Purpose and Why It Is Discouraged

#### Purpose:

`protected void finalize() throws Throwable` is a method defined in `java.lang.Object`. The Garbage Collector *may* call it on an object just before reclaiming its memory, giving the object a chance to release non-memory resources (e.g., file handles, native memory).

```

1  class Resource {
2      @Override
3      protected void finalize() throws Throwable {
4          System.out.println("Cleaning up resource");
5          super.finalize();
6      }
7  }
    
```

Why its use is discouraged (deprecated since Java 9):

- **No guaranteed timing** — the GC decides *when* (or *whether*) to call it; resources may be held for a very long time.
- **No guaranteed execution** — the JVM may exit without ever calling `finalize()`.
- **Performance overhead** — objects with finalizers take at least two GC cycles to collect.
- **Resurrection risk** — a finalizer can accidentally make the object reachable again, causing subtle bugs.
- **Better alternatives exist** — use `try-with-resources` and `AutoCloseable/Closeable` for deterministic cleanup.

**Q23.** Why does the `main` method not require an `argc`-type parameter for command line arguments? (4 marks) [CSE 201 | 2011–12]

**Answer — Why Java's `main()` Needs No `argc`**

In C/C++, `main(int argc, char* argv[])` requires a separate `argc` because a raw array carries no length information.

In Java, `main(String[] args)` receives a **String array object**, which already knows its own length via `args.length`. Therefore a separate count parameter is unnecessary.

```

1 public class Demo {
2     public static void main(String[] args) {
3         System.out.println("Argument count: " + args.length);
4         for (int i = 0; i < args.length; i++)
5             System.out.println("args[" + i + "] = " + args[i]);
6     }
7 }
8 // Run: java Demo Hello World
9 // Output:
10 //     Argument count: 2
11 //     args[0] = Hello
12 //     args[1] = World

```

The JVM populates `args` automatically with the command-line tokens; `args.length` replaces the role of `argc`.

**Q24.** What is run-time polymorphism? How does Java achieve run-time polymorphism? (6 marks) [CSE 201 | 2011–12]

**Answer — Run-Time Polymorphism in Java**

**Run-time polymorphism** (also called *dynamic polymorphism*) is the ability to call an overridden method through a parent-class reference, with the actual method executed determined at **run time** based on the real type of the object, not the declared type of the reference.

**How Java achieves it — Dynamic Method Dispatch:**

Java uses a *virtual method table* (vtable). Every overridable method call through a reference is resolved at run time by looking up the vtable of the actual object. This is automatic for all non-static, non-final, non-private methods.

```

1  class Animal {
2      void sound() { System.out.println("Some sound"); }
3  }
4  class Dog extends Animal {
5      @Override
6      void sound() { System.out.println("Woof"); }
7  }
8  class Cat extends Animal {
9      @Override
10     void sound() { System.out.println("Meow"); }
11 }
12
13 class Demo {
14     public static void main(String[] args) {
15         Animal a;           // declared type: Animal
16
17         a = new Dog();
18         a.sound();           // "Woof" -- resolved at run
                               time
19
20         a = new Cat();
21         a.sound();           // "Meow" -- resolved at run
                               time
22     }
23 }

```

The same reference `a` calls different implementations depending on the actual object it holds — this is run-time polymorphism.

**Q25.** Override the `equals(Object obj)` method in the following `Student` class so that two students are equal if they have the same `name` and `cgpa`:

```

1  class Student {
2      private int id; private String name; private double cgpa;
3      public Student(int i, String n, double c) { id=i; name=n;
        cgpa=c; }
4  }
5  // s1.equals(s2) -> false (different name), s1.equals(s3) ->

```

```

        true (same name+cgpa)
6 // s1.equals(s4) -> false (same name, different cgpa)

```

(11 marks)

[CSE 201 | 2010–11]

**Answer — Overriding equals() in Student**

```

1  class Student {
2      private int    id;
3      private String name;
4      private double cgpa;
5
6      public Student(int i, String n, double c) {
7          id = i; name = n; cgpa = c;
8      }
9
10     @Override
11     public boolean equals(Object obj) {
12         if (this == obj) return true;
13         if (!(obj instanceof Student)) return false;
14         Student other = (Student) obj;
15         return this.name.equals(other.name)
16             && Double.compare(this.cgpa, other.cgpa) == 0;
17     }
18 }
19
20 class TestStudent {
21     public static void main(String[] args) {
22         Student s1 = new Student(1, "Alice", 3.8);
23         Student s2 = new Student(2, "Bob", 3.8); //
24             different name
25         Student s3 = new Student(3, "Alice", 3.8); // same
26             name+cgpa
27         Student s4 = new Student(4, "Alice", 3.5); // same
28             name, diff cgpa
29
30         System.out.println(s1.equals(s2)); // false
31         System.out.println(s1.equals(s3)); // true
32         System.out.println(s1.equals(s4)); // false
33     }
34 }

```

**Note:** `Double.compare()` is used instead of `==` for double comparison to avoid floating-point precision issues.

**Q26.** Suppose the following class is defined:

```

1  class Point {
2      int x, y;
3      Point(int x1, int y1) { x = x1; y = y1; }

```



```
4 }
```

State the values of Point objects after each statement:

```
1 Point point1; Point point2;
2 point1 = new Point(10, 20);
3 point2 = point1;
4 point1 = new Point(30, 60);
5 point1 = point2;
```

(6 marks)

[CSE 201 | 2007–08]

### Answer — Point Object Reference Tracing

| Statement                   | point1  | point2             |
|-----------------------------|---------|--------------------|
| Point point1; Point point2; | null    | null               |
| point1 = new Point(10, 20); | (10,20) | null               |
| point2 = point1;            | (10,20) | (10,20) (same obj) |
| point1 = new Point(30, 60); | (30,60) | (10,20)            |
| point1 = point2;            | (10,20) | (10,20) (same obj) |

**Key insight:** point2 = point1 copies the *reference*, not the object. After point1 = new Point(30,60), point2 still points to the original (10,20) object. The final point1 = point2 makes both variables refer to the (10,20) object again; the (30,60) object becomes unreachable and eligible for GC.

**Q27.** Write Java code for the following regarding arrays of reference types vs primitive types. What is the difference between int c[], x; and int[] c, x;? (6 marks)  
[CSE 201 | 2007–08]

### Answer — Primitive vs Reference Arrays and Declaration Syntax

**Primitive array vs Reference array:**

```
1 // Primitive array -- each element stores the value directly
2 int[] prims = new int[3];           // [0, 0, 0]
3 prims[0] = 10; prims[1] = 20;
4
5 // Reference array -- each element stores a reference (null
  by default)
6 String[] refs = new String[3];      // [null, null, null]
7 refs[0] = "Hello"; refs[1] = "World";
8
9 // Array of objects
10 Point[] points = new Point[2];      // [null, null]
11 points[0] = new Point(1, 2);        // must create each object
    separately
12 points[1] = new Point(3, 4);
```

Difference between `int c[], x;` and `int[] c, x;`:

| Declaration              | c             | x             |
|--------------------------|---------------|---------------|
| <code>int c[], x;</code> | array (int[]) | plain int     |
| <code>int[] c, x;</code> | array (int[]) | array (int[]) |

In `int c[], x;` the `[]` applies only to `c`; `x` is a simple `int`. In `int[] c, x;` the `[]` is part of the type, so *both* `c` and `x` are `int` arrays. The second form is preferred as it is less ambiguous.

BUET · CSE 107 / CSE 201

# Object Oriented Programming

Section S12

---

Interfaces, Abstract Classes & Nested Classes

## S12 — Interfaces, Abstract Classes & Nested Classes

Q1. Also fix the following:

```

1 interface X1 { void m1(); static void m2(); default void m3();
  }
2 interface X2 { private void m4(); void m5(); }
3 abstract class A1 implements X1 { final void m6(); abstract
  void m7(); }
4 class A2 extends A1 implements X2 { /* your code */ }
```

(10 marks)

[CSE 107 | 2023–24]

### Answer — Fix X1/X2/A1 & Write A2

Errors and fixes:

1. interface X2: `private void m4();` — private interface method must have a body (Java 9+). Fix: `private void m4() { }`
2. abstract class A1: `final void m6();` — A final method cannot be abstract (no body but cannot be overridden — contradiction). Fix: `final void m6() { }`

Corrected code:

```

1 interface X1 { void m1(); static void m2() { } default void
  m3() { } }
2 interface X2 { private void m4() { } void m5(); } // Java
  9+
3 abstract class A1 implements X1 {
4     final void m6() { } // fixed: body added
5     abstract void m7();
6 }
```

Minimum code for A2:

Must implement: `m1()` (from X1), `m5()` (from X2), `m7()` (abstract in A1).

```

1 class A2 extends A1 implements X2 {
2     @Override public void m1() { }
3     @Override public void m5() { }
4     @Override void m7() { }
5 }
```

Q2. What are two types of nested classes? What are the differences between them? List all problems in the following code. Write Java code to fix only the problems in the main method:

```

1 class X {
2     int a = 1;
```

```

3      void display() { Y obY = new Y(); obY.display(); Z obZ =
          new Z(); obZ.display(); }
4      void showNested() { System.out.println(b); System.out.
          println(c); }
5      class Y { int b = 2; void display() { System.out.println(a
          ); System.out.println(b); } }
6      static class Z { int c = 3; void display() { System.out.
          println(a); System.out.println(c); } }
7  }
8  class Test {
9      public static void main(String[] args) {
10         X obX = new X(); obX.display(); obX.showNested();
11         Y obY = new Y(); obY.display();
12         Z obZ = new Z(); obZ.display();
13     }
14 }

```

(20 marks)

[CSE 107 | Jan 2021, 2023–24]

### Answer — Nested Class Types, Differences & Fixed main

#### Two types:

1. **Inner class** (class Y) — non-static; needs outer object; can access all outer members.
2. **Static nested class** (class Z) — static; no outer object needed; can only access outer static members.

#### Problems in the given code:

1. `showNested()` prints `b` and `c` — these belong to `Y` and `Z` respectively and are not directly accessible in `X`.
2. `static class Z` tries to access `a` (non-static member of `X`) — illegal.
3. In main: `Y obY = new Y()` — wrong; must be `X.Y` via outer object.
4. In main: `Z obZ = new Z()` — wrong; must be `X.Z`.

#### Fixed main method:

```

1  class Test {
2      public static void main(String[] args) {
3          X obX = new X();
4          obX.display();
5          obX.showNested();
6
7          X.Y obY = obX.new Y(); // inner class: needs outer
              object
8          obY.display();

```

```

9
10         X.Z obZ = new X.Z();    // static nested: no outer
           object needed
11         obZ.display();
12     }
13 }

```

**Q3.** Outline the errors in the provided code and fix them without removing access modifiers. Then write the minimum code for class c2 so it compiles:

```

1  interface i1 { default void f1(); void f2(); static void f3();
   }
2  interface i2 { void f4(); private void f5(); }
3  abstract class c1 implements i1 {
4      abstract void f6(); final void f7() { }
5  }
6  class c2 extends c1 implements i2 { /* your code */ }

```

(10 marks)

[CSE 107 | 2022–23]

### Answer — Fix Errors in Interfaces & Write c2

#### Errors and fixes:

1. interface i1: default void f1(); — A default method must have a body. Fix: default void f1() { }
2. interface i1: static void f3(); — A static interface method must have a body (Java 8+). Fix: static void f3() { }
3. interface i2: private void f5(); — A private interface method must have a body (Java 9+). Fix: private void f5() { }

#### Corrected declarations:

```

1  interface i1 {
2      default void f1() { }    // fixed: body added
3      void f2();
4      static void f3() { }    // fixed: body added
5  }
6  interface i2 {
7      void f4();
8      private void f5() { }    // fixed: body added (Java 9+)
9  }
10 abstract class c1 implements i1 {
11     abstract void f6();
12     final void f7() { }
13 }

```

**Minimum code for c2:**

c2 must implement: f2() (from i1), f4() (from i2), and f6() (abstract in c1).

```

1 class c2 extends c1 implements i2 {
2     @Override public void f2() { }
3     @Override public void f4() { }
4     @Override void f6() { }
5 }

```

**Q4.** 7 methods: f1()–f7(), 3 interfaces i1–i3, 1 abstract class A. Constraints: (i) each interface  $\leq 2$  methods; (ii) abstract class A can only implement 1 interface and define  $\leq 1$  abstract method; (iii) MyClass cannot implement any interface. Write Java code. (10 marks) [CSE 107 | 2022–23, 2020–21]

**Answer — 7 Methods, 3 Interfaces, Abstract Class, No Interface for MyClass**

**Strategy:**

- Chain i1, i2, i3 so abstract class A implements only i1 (1 interface rule).
- A declares at most 1 abstract method.
- MyClass extends A — inherits everything, implements no interface directly.

```

1 interface i1 { void f1(); void f2(); }
2 interface i2 extends i1 { void f3(); void f4(); }
3 interface i3 extends i2 { void f5(); void f6(); void f7(); }
4
5 abstract class A implements i3 {
6     // implements f1..f6 concretely, declares f7 abstract
7     public void f1() { System.out.println("f1"); }
8     public void f2() { System.out.println("f2"); }
9     public void f3() { System.out.println("f3"); }
10    public void f4() { System.out.println("f4"); }
11    public void f5() { System.out.println("f5"); }
12    public void f6() { System.out.println("f6"); }
13    public abstract void f7(); // <= 1 abstract method in A
14 }
15
16 class MyClass extends A { // implements no interface
17     directly
18     public void f7() { System.out.println("f7"); }
19 }

```

**Q5.** Write minimum code in class c2/B for successful compilation. You cannot define it as abstract:

```

1 interface i1 { default void f1() { } void f2(); }
2 interface i2 { void f3(); void f4(); }
3 abstract class c1 implements i1 {
4     abstract void f5();
5     final void f6() { }
6 }
7 class c2 extends c1 implements i2 { /* your code */ }

```

(6+10 marks)

[CSE 107 | 2016–17, 2019–20, 2021–22, Jan 2021]

↪ Similar: CSE 201 | 2014–15, 2008–09

### Answer — Minimum Code for c2

Analysis of what c2 must provide:

| Method | Source        | Reason                  |
|--------|---------------|-------------------------|
| f2()   | i1 (abstract) | not implemented by c1   |
| f5()   | c1 (abstract) | declared abstract in c1 |
| f3()   | i2 (abstract) | not yet implemented     |
| f4()   | i2 (abstract) | not yet implemented     |

*Note: f1() has a default body in i1, so c2 need not override it. f6() is final in c1 — already implemented, cannot be overridden.*

```

1 class c2 extends c1 implements i2 {
2     @Override public void f2() { } // from i1 (abstract)
3     @Override void f5() { } // from c1 (abstract)
4     @Override public void f3() { } // from i2
5     @Override public void f4() { } // from i2
6 }

```

**Q6.** Explain three uses of the `final` keyword with short examples. Describe the fundamental differences between an interface and an abstract class. (10 marks) [CSE 107 | 2021–22]

### Answer — final Keyword (3 Uses) & Interface vs. Abstract Class

Three uses of `final`:

1. **Final variable** — value cannot be changed after initialisation.

```

1 final int MAX = 100;
2 // MAX = 200; // compile error

```

2. **Final method** — cannot be overridden in a subclass.

```

1 class Base {
2     final void show() { System.out.println("Base"); }

```



```

3 }
4 class Child extends Base {
5     // void show() { } // compile error
6 }

```

3. Final class — cannot be extended (subclassed).

```

1 final class Immutable { }
2 // class Sub extends Immutable { } // compile error

```

### Fundamental differences between Interface and Abstract Class:

| Interface                                         | Abstract Class                                           |
|---------------------------------------------------|----------------------------------------------------------|
| All fields are <code>public static final</code> . | Fields can have any access modifier and be non-static.   |
| A class can implement multiple interfaces.        | A class can extend only one abstract class.              |
| No constructors allowed.                          | Can have constructors.                                   |
| Represents a <i>contract</i> (what to do).        | Represents a <i>partial implementation</i> (what + how). |
| Methods are <code>public</code> by default.       | Methods can have any access modifier.                    |

Q7. What is the problem with the code below? Write two different ways to fix the problem:

```

1 interface A { default void f() { System.out.println("A's f"); } }
2 interface B { default void f() { System.out.println("B's f"); } }
3 public class C implements A, B { }

```

(7 marks)

[CSE 107 | 2016–17, Jan 2021]

### Answer — Default Method Conflict: Two Fixes

**Problem:** Both A and B provide a default `void f()`, so C inherits two conflicting default implementations. The compiler reports an error: “*class C inherits unrelated defaults for f() from its superinterfaces A and B*”.

**Fix 1 — Override `f()` in C with its own implementation:**

```

1 public class C implements A, B {
2     @Override
3     public void f() {
4         System.out.println("C's own f");
5     }
6 }

```

```
6 }
```

**Fix 2 —** Override `f()` in `C` and explicitly delegate to one interface:

```
1 public class C implements A, B {
2     @Override
3     public void f() {
4         A.super.f();    // explicitly call A's default
5     }
6 }
```

The syntax `InterfaceName.super.method()` is used to disambiguate which interface's default method to call.

**Q8.** Suppose there are 7 methods: `f1()`–`f7()`, 4 interfaces `i1`–`i4`, and a class `MyClass`. Constraints: (i) each interface defines at most 2 methods; (ii) `MyClass` can only implement 1 interface. Write Java code for `MyClass`. (10 marks) [CSE 107 | 2019–20, Jan 2021; CSE 201 | 2014–15, 2008–09]

↪ Similar: CSE 201 | 2007–08

**Answer —** 7 Methods, 4 Interfaces, `MyClass` Implements 1

**Strategy:** Chain interfaces using `extends` so that `MyClass` only needs to implement **one** interface (`i4`) yet still gets access to all 7 methods.

```
1 interface i1 { void f1(); void f2(); }           // 2
   methods
2 interface i2 extends i1 { void f3(); void f4(); } // 2 new +
   2 inherited
3 interface i3 extends i2 { void f5(); void f6(); } // 2 new +
   4 inherited
4 interface i4 extends i3 { void f7(); }           // 1 new +
   6 inherited
5
6 class MyClass implements i4 {
7     public void f1() { System.out.println("f1"); }
8     public void f2() { System.out.println("f2"); }
9     public void f3() { System.out.println("f3"); }
10    public void f4() { System.out.println("f4"); }
11    public void f5() { System.out.println("f5"); }
12    public void f6() { System.out.println("f6"); }
13    public void f7() { System.out.println("f7"); }
14 }
```

*MyClass* implements only `i4` but must provide bodies for all 7 methods because interfaces chain via `extends`.

**Q9.** Write three uses of `final` keyword with examples. (6 marks) [CSE 201 |

2007–08, 2008–09; CSE 107 | 2017–18]

### Answer — Three Uses of final with Examples

1. **Final variable** — creates a constant.

```
1 final int MAX_SIZE = 50; // cannot be reassigned
```

2. **Final method** — prevents method overriding.

```
1 class A { final void show() { System.out.println("A"); } }
2 class B extends A { /* void show(){} */ } // not allowed
```

3. **Final class** — prevents inheritance.

```
1 final class Utility { }
2 // class Sub extends Utility { } // compile error
```

**Q10.** What is an inner class? Point out the problem in the following code segment (if any):

```
1 class Outer {
2     int outer_x = 5;
3     void test() { Inner inner = new Inner(); inner.display();
4         }
5     class Inner { int z = 60; void display() { System.out.
6         println(outer_x); } }
7     void showz() { System.out.println(z); }
8 }
```

(3+6 marks)

[CSE 107 | 2017–18]

### Answer — Inner Class & Problem in Outer/Inner Code

**Inner class:** A non-static class defined inside another class. It has access to all members (including private) of the outer class and holds an implicit reference to the outer class instance.

**Problem in the given code:**

`showz()` in `Outer` tries to print `z`, but `z` is a member of `Inner` — not directly accessible from `Outer`'s method.

```
1 class Outer {
2     int outer_x = 5;
3
4     void test() {
5         Inner inner = new Inner();
6         inner.display();
7     }
8 }
```

```

9      class Inner {
10         int z = 60;
11         void display() {
12             System.out.println(outer_x); // OK: inner
13                                     accesses outer member
14         }
15     }
16     void showz() {
17         // System.out.println(z); // ERROR: z not in scope
18         // here
19         Inner inner = new Inner();
20         System.out.println(inner.z); // Fix: access via
21         // object
22     }
23 }

```

**Q11.** What is abstract class? Write the restrictions imposed on an abstract class. Point out and fix the problems in the following code:

```

1  final class X { final void show() { System.out.println("
2  Printed from a method"); } }
3  class Y extends X { void show() { System.out.println("In
4  subclass method"); } }

```

(3+2+6 marks)

[CSE 107 | 2017–18]

### Answer — Abstract Class Restrictions & Fix

#### Restrictions on an abstract class:

1. Cannot be instantiated directly (`new AbstractClass()` is illegal).
2. If a class has even one abstract method, the class must be declared **abstract**.
3. A subclass must implement all abstract methods, or itself be declared **abstract**.
4. Cannot use both **abstract** and **final** on the same class or method.
5. Abstract methods cannot be **private**, **static**, or **final**.

#### Problems in the given code:

1. `final class X` cannot be extended — `class Y extends X` is a compile error.
2. `final void show()` in `X` cannot be overridden — `Y.show()` is a compile error.

**Fix:** Remove **final** from the class and the method.

```

1  class X {                                // removed
    final
2      void show() {                        // removed
        final
3          System.out.println("Printed from a method");
4      }
5  }
6  class Y extends X {
7      void show() { System.out.println("In subclass method"); }
8  }

```

**Q12.** Consider the following Java code and write minimum code for myclass:

```

1  interface Int1 { void f1(); void f2(); }
2  interface Int2 extends Int1 { void f3(); }
3  class MyClass implements Int2 { /* Complete this class */ }
4  public class Test {
5      public static void main(String[] args) {
6          MyClass my = new MyClass();
7          my.f1(); my.f2(); my.f3();
8      }
9  }

```

(9 marks)

[CSE 107 | 2017–18]

### Answer — Minimum Code for MyClass

**Analysis:** Int2 extends Int1, so MyClass must implement f1(), f2() (from Int1) and f3() (from Int2).

```

1  interface Int1 { void f1(); void f2(); }
2  interface Int2 extends Int1 { void f3(); }
3
4  class MyClass implements Int2 {
5      public void f1() { System.out.println("f1"); }
6      public void f2() { System.out.println("f2"); }
7      public void f3() { System.out.println("f3"); }
8  }
9
10 public class Test {
11     public static void main(String[] args) {
12         MyClass my = new MyClass();
13         my.f1(); my.f2(); my.f3();
14     }
15 }

```

**Output:**

```

1  f1

```

```

2  f2
3  f3

```

- Q13.** Write Java code to do the following: (i) Declare a static nested class `Inner1` of `Outer1` and create an object of `Inner1` in `NestedDemo.main()`. (ii) Declare an inner class `Inner2` of `Outer2` and create an object of `Inner2` in `NestedDemo.main()`. (8 marks) [CSE 107 | 2016–17]

#### Answer — Static Nested & Inner Class Object Creation

```

1  // (i) Static nested class
2  class Outer1 {
3      static class Inner1 {
4          void display() { System.out.println("Inner1 (static
5              nested)"); }
6      }
7  }
8  // (ii) Inner class (non-static)
9  class Outer2 {
10     class Inner2 {
11         void display() { System.out.println("Inner2 (inner
12             class)"); }
13     }
14 }
15 public class NestedDemo {
16     public static void main(String[] args) {
17
18         // (i) Static nested: no outer object needed
19         Outer1.Inner1 obj1 = new Outer1.Inner1();
20         obj1.display();
21
22         // (ii) Inner class: outer object required first
23         Outer2 outer2 = new Outer2();
24         Outer2.Inner2 obj2 = outer2.new Inner2();
25         obj2.display();
26     }
27 }

```

#### Output:

```

1  Inner1 (static nested)
2  Inner2 (inner class)

```

- Q14.** What are the two uses of the `super` keyword? What is an abstract class? What are

the restrictions? (10 marks)

[CSE 107 | 2016–17]

**Answer — Two Uses of super & Abstract Class****Two uses of the super keyword:**

1. **Call superclass constructor** — must be the first statement in the subclass constructor.

```

1  class Animal { Animal(String name) { System.out.println(
    name); } }
2  class Dog extends Animal {
3      Dog() { super("Dog"); } // calls Animal's constructor
4  }

```

2. **Access superclass method/variable** — when overridden in the subclass.

```

1  class A { void show() { System.out.println("A"); } }
2  class B extends A {
3      void show() {
4          super.show(); // calls A's show
5          System.out.println("B");
6      }
7  }

```

**Abstract class:** A class declared with the **abstract** keyword that may contain abstract (bodyless) methods.

**Restrictions:**

- Cannot be instantiated.
- Subclass must implement all abstract methods or be abstract itself.
- Abstract methods cannot be **private**, **static**, or **final**.
- A class with an abstract method must be declared abstract.

**Q15.** When we say “a class is final” or “a method is final” — what does it mean? Show example codes. (5 marks) [CSE 107 | 2015–16]

**Answer — Meaning of final Class and final Method**

**Final class:** A class declared **final** cannot be subclassed. Any attempt to extend it causes a compile-time error.

```

1  final class Lock { }
2  // class SpecialLock extends Lock { } // compile error

```

**Final method:** A method declared `final` cannot be overridden in a subclass, but it *can* be inherited and called.

```

1  class Parent {
2      final void display() { System.out.println("Parent display
3      "); }
4  }
5  class Child extends Parent {
6      // void display() { } // compile error: cannot override
7      // final method
8  }
9  public class Main {
10     public static void main(String[] args) {
11         Child c = new Child();
12         c.display(); // inherited fine; Output: Parent
13         display
14     }
15 }

```

Common use of *final* class: `java.lang.String` is a final class — it cannot be subclassed to prevent modification of its behaviour.

**Q16.** What is a static code block? When is it executed? Can there be multiple static code blocks? In what sequence? Show with example. (10 marks) [CSE 107 | 2015–16]

### Answer — Static Code Block

**Static code block:** A block of code preceded by the `static` keyword, declared inside a class but outside any method. It is used to initialise static variables or perform one-time class-level setup.

**When executed:** Once, when the class is *loaded* by the JVM (before any object is created or any static method is called).

**Multiple static blocks:** Allowed. They execute in the **order they appear** in the source file.

```

1  class Demo {
2      static int x;
3      static int y;
4
5      static {
6          x = 10;
7          System.out.println("First static block: x = " + x);
8      }
9
10     static {
11         y = 20;
12         System.out.println("Second static block: y = " + y);
13     }
14 }

```



```

15     Demo() { System.out.println("Constructor called"); }
16 }
17
18 public class Main {
19     public static void main(String[] args) {
20         System.out.println("main() started");
21         Demo d1 = new Demo();
22         Demo d2 = new Demo();
23     }
24 }

```

**Output:**

```

1 First static block: x = 10
2 Second static block: y = 20
3 main() started
4 Constructor called
5 Constructor called

```

The static blocks ran once before *main()*, in top-to-bottom order. The constructor ran twice (once per object).

**Q17.** What are the three uses of Java `final` keyword? (5 marks) [CSE 201 | 2014–15]

**Answer — Three Uses of the final Keyword**

1. **Final variable** — acts as a constant; cannot be reassigned after initialisation.

```

1 final int SPEED_LIMIT = 80;

```

2. **Final method** — cannot be overridden by any subclass.

```

1 class Vehicle {
2     final void fuelType() { System.out.println("Petrol"); }
3 }

```

3. **Final class** — cannot be extended; prevents inheritance.

```

1 final class SecureToken { }

```

**Q18.** Write an interface and a class that implements it. Show how objects can be created using an interface. (10 marks) [CSE 201 | 2012–13]

**Answer — Interface, Implementing Class & Object Creation**

```

1 // Interface declaration
2 interface Shape {

```

```

3     double area();
4     double perimeter();
5 }
6
7 // Class implementing the interface
8 class Circle implements Shape {
9     double radius;
10
11     Circle(double r) { radius = r; }
12
13     @Override
14     public double area() { return Math.PI * radius * radius;
15         }
16
17     @Override
18     public double perimeter() { return 2 * Math.PI * radius;
19         }
20 }
21
22 // Main class
23 public class Main {
24     public static void main(String[] args) {
25
26         // Method 1: object via concrete class reference
27         Circle c = new Circle(5.0);
28         System.out.println("Area: " + c.area());
29
30         // Method 2: object via interface reference (
31             polymorphism)
32         Shape s = new Circle(7.0);
33         System.out.println("Perimeter: " + s.perimeter());
34     }
35 }

```

**Key point:** An interface reference (Shape s) can hold any object whose class implements Shape. Only the methods declared in the interface are accessible through s.

**Q19.** Write the differences between abstract class and interface. (5 marks) [CSE 201 | 2012–13]

### Answer — Abstract Class vs. Interface

| Abstract Class                                     | Interface                                                                                                      |
|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Declared with <code>abstract class</code> .        | Declared with <code>interface</code> .                                                                         |
| Can have both abstract and concrete methods.       | Methods are implicitly abstract (Java 7 and earlier); <code>default/static</code> methods allowed from Java 8. |
| Can have instance variables (any access modifier). | Variables are implicitly <code>public static final</code> (constants).                                         |
| A class can extend only <b>one</b> abstract class. | A class can implement <b>multiple</b> interfaces.                                                              |
| Can have constructors.                             | Cannot have constructors.                                                                                      |
| Used when classes share common state/behaviour.    | Used to define a contract with no shared state.                                                                |

**Q20.** Write the use of `abstract` and `final` keywords in Java with examples. (7 marks)  
[CSE 201 | 2011–12]

### Answer — abstract and final Keywords in Java

#### abstract keyword:

- Abstract class** — cannot be instantiated; may contain abstract and concrete methods.

```

1  abstract class Animal {
2      abstract void sound();           // must be overridden
3      void breathe() { System.out.println("Breathing"); }
4  }
```

- Abstract method** — has no body; subclass must implement it.

```

1  class Dog extends Animal {
2      void sound() { System.out.println("Woof"); }
3  }
```

#### final keyword:

- Final variable** — constant; value fixed after assignment.  
`final double PI = 3.14159;`

- Final method** — cannot be overridden in subclass.  
`final void show() { }`

- Final class** — cannot be subclassed.  
`final class MyClass { }`

*Note: A method cannot be both **abstract** and **final** — **abstract** requires over-riding while **final** prevents it.*

**Q21.** What are the differences between static nested class and inner class? **(5 marks)**  
[CSE 201 | 2010–11]

#### Answer — Static Nested Class vs. Inner Class

| Static Nested Class                                                                                     | Inner Class                                                              |
|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Declared with <code>static</code> keyword.                                                              | Declared without <code>static</code> .                                   |
| Does not hold a reference to outer class instance.                                                      | Holds an implicit reference to outer class instance.                     |
| Can only access <code>static</code> members of outer class.                                             | Can access all members (including <code>private</code> ) of outer class. |
| Instantiated without outer object:<br><code>new Outer.Nested()</code><br><code>outer.new Inner()</code> | Requires outer object:                                                   |
| Can have <code>static</code> members of its own.                                                        | Cannot have <code>static</code> members (except constants).              |

**Q22.** Do you agree or disagree: “All interfaces are abstract classes but not all abstract classes are interfaces”? Give valid reasoning. **(4 marks)** [CSE 201 | 2010–11]

#### Answer — “All interfaces are abstract classes but not all abstract classes are interfaces”

##### Verdict: Disagree.

Conceptually, an interface is *similar* to a fully abstract class, but in Java they are **distinct constructs**:

1. An interface is *not* an abstract class in Java’s type system. `interface` and `abstract class` are separate keywords.
2. An abstract class can have instance variables, constructors, and concrete methods — an interface (before Java 8) had none of these.
3. A class can implement multiple interfaces but extend only one abstract class.
4. Technically, the JVM does treat interfaces as a special kind of abstract type, but the statement “all interfaces *are* abstract classes” is misleading because the two serve different design purposes.

**Correct statement:** Every interface is *implicitly abstract*, but it is *not* an abstract class — it is its own independent type.

**Q23.** What are the two types of nested classes in Java? Briefly describe them. What are the rules regarding access of nested class and outer class? **(6 marks)** [CSE 201 | 2007–08]

### Answer — Types of Nested Classes & Access Rules

Two types of nested classes:

1. **Static nested class** — declared with the `static` keyword inside an outer class. It does *not* hold a reference to an outer class instance.
2. **Inner class (non-static nested class)** — declared without `static`. It holds an implicit reference to the enclosing outer class instance.

Access rules:

| Inner class                                                                                   | Static nested class                                                                           |
|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Can access all members (including <code>private</code> ) of the outer class.                  | Can only access <code>static</code> members of the outer class.                               |
| Outer class can access all members (including <code>private</code> ) of the inner class.      | Outer class can access all members of the static nested class.                                |
| Requires outer class object to instantiate: <code>Outer.Inner obj = outer.new Inner();</code> | Can be instantiated without outer object: <code>Outer.Nested obj = new Outer.Nested();</code> |

BUET · CSE 107 / CSE 201

# Object Oriented Programming

Section S13

---

Exception Handling

## S13 — Exception Handling

**Q1.** What is the problem in the following C++ program? Rewrite using exception handling:

```
1  #include <iostream>
2  using namespace std;
3  int divide() {
4      int d = 0;
5      int a = 10 / d;
6      return a;
7  }
8  int main() { cout << divide() << endl; return 0; }
```

(2+5 marks)

[CSE 107 | 2023–24]

### Answer — Division by Zero: Problem and Fix with Exception Handling

**Problem:**  $d = 0$  causes **division by zero**, which is undefined behaviour in C++. The program may crash or produce garbage output with no graceful error message.

**Fixed version using exception handling:**

```
1  #include <iostream>
2  #include <stdexcept>
3  using namespace std;
4
5  int divide() {
6      int d = 0;
7      if (d == 0)
8          throw runtime_error("Division by zero!");
9      int a = 10 / d;
10     return a;
11 }
12
13 int main() {
14     try {
15         cout << divide() << endl;
16     } catch (const runtime_error& e) {
17         cerr << "Caught exception: " << e.what() << endl;
18     }
19     return 0;
20 }
```

**Output:**

Caught exception: Division by zero!

**Q2.** Consider a class `BankAccount`. Write: (i) `InvalidAmountException`;  
(ii) `InsufficientBalanceException`; (iii) complete `BankAccount/Account` class:  
(15 marks)

[CSE 107 | 2022–23]

**Answer — BankAccount with InvalidAmount and InsufficientBalance Exceptions**

```
1  class InvalidAmountException extends Exception {
2      InvalidAmountException(double amt) {
3          super("Amount must be positive. Got: " + amt);
4      }
5  }
6
7  class InsufficientBalanceException extends Exception {
8      InsufficientBalanceException(double needed, double
9          available) {
10         super("Need " + needed + " but only " + available + "
11             available");
12     }
13 }
14
15 class BankAccount {
16     private String owner;
17     private double balance;
18
19     BankAccount(String owner, double initialBalance)
20         throws InvalidAmountException {
21         if (initialBalance < 0)
22             throw new InvalidAmountException(initialBalance);
23         this.owner = owner;
24         this.balance = initialBalance;
25     }
26
27     void deposit(double amt) throws InvalidAmountException {
28         if (amt <= 0) throw new InvalidAmountException(amt);
29         balance += amt;
30     }
31
32     void withdraw(double amt)
33         throws InvalidAmountException,
34             InsufficientBalanceException {
35         if (amt <= 0) throw new InvalidAmountException(amt);
36         if (amt > balance)
37             throw new InsufficientBalanceException(amt,
38                 balance);
39         balance -= amt;
40     }
41
42     double getBalance() { return balance; }
43
44     public static void main(String[] args) {
45         try {
46             BankAccount acc = new BankAccount("Alice", 500);
```



```

43         acc.deposit(200);
44         System.out.println("Balance: " + acc.getBalance()
45                               ); // 700
46         acc.withdraw(900); // throws
47         InsufficientBalanceException
48     } catch (InvalidAmountException e) {
49         System.out.println("Invalid: " + e.getMessage());
50     } catch (InsufficientBalanceException e) {
51         System.out.println("Insufficient: " + e.
52                               getMessage());
53     }
54 }
55 }

```

**Q3.** Write the name of the five keywords in Java dedicated to exception handling. (15 marks)

Consider a class named `BankAccount` with the following information:

- `double balance` – the balance of the account
- `void init(double amount)` – this method sets the amount as the initial balance
- `void debit(double amount)` – this method subtracts the amount from the balance
- `void credit(double amount)` – this method adds the amount directly to the balance

The amount provided in the above methods can't be negative. The balance of the account can't also be negative.

Write Java code for the following:

- A custom exception (`InvalidAmountException`) that triggers when the given amount is negative and shows the message 'The given amount can't be negative' and the given amount.
- A custom exception (`InsufficientBalanceException`) that triggers when the balance goes negative and shows the message 'The account balance can't be negative' and the current balance.
- The complete `Account` (or `BankAccount`) class to trigger these exceptions when needed.

You don't need to write any main method.

[CSE 107 | 2016–17, 2022–23]

### Answer — BankAccount Class with Custom Exceptions

Five keywords for exception handling in Java: `try`, `catch`, `finally`, `throw`, and `throws`.

```

1 // i. Custom exception for negative amounts

```

```
2  class InvalidAmountException extends Exception {
3      public InvalidAmountException(double amount) {
4          super("The given amount can't be negative: " + amount
5              );
6      }
7  }
8  // ii. Custom exception for negative balance
9  class InsufficientBalanceException extends Exception {
10     public InsufficientBalanceException(double currentBalance
11         ) {
12         super("The account balance can't be negative: " +
13             currentBalance);
14     }
15 }
16 // iii. The complete BankAccount class
17 class BankAccount {
18     private double balance;
19
20     void init(double amount) throws InvalidAmountException {
21         if (amount < 0) {
22             throw new InvalidAmountException(amount);
23         }
24         this.balance = amount;
25     }
26
27     void debit(double amount) throws InvalidAmountException,
28         InsufficientBalanceException {
29         if (amount < 0) {
30             throw new InvalidAmountException(amount);
31         }
32         if (this.balance - amount < 0) {
33             throw new InsufficientBalanceException(this.
34                 balance - amount);
35         }
36         this.balance -= amount;
37     }
38
39     void credit(double amount) throws InvalidAmountException
40     {
41         if (amount < 0) {
42             throw new InvalidAmountException(amount);
43         }
44         this.balance += amount;
45     }
46 }
```

**Q4.** Consider a class named `Inventory` with the following information: (10)

- `double stock` - the stock of the inventory
- `double minStock` - the minimum stock of the inventory
- `void setInitialStock(double amount)` - this method sets the amount as the initial stock
- `void addToInventory(double amount)` - this method adds the amount to the stock
- `void removeFromInventory(double amount)` - this method subtracts the amount from the stock

The amount provided in the above four methods can't be negative. The stock of the inventory can't also be below the minimum stock.

Write Java code for the following:

- (i) A custom exception named `InvalidStockAmountException` with appropriate parameters that triggers when the given amount is negative and shows the message 'The given stock amount can't be negative'.
- (ii) A custom exception named `InvalidInventoryStockException` with appropriate parameters that triggers when the stock goes below the minimum stock and shows the message 'The stock can't be less than the minimum stock'.
- (iii) The complete `Inventory` class to trigger these exceptions when needed. You don't need to write any main method.

[CSE 107 | 2020–21]

#### Answer — Inventory Class with Custom Exceptions

```
1 // (i) Custom exception for negative stock amounts
2 class InvalidStockAmountException extends Exception {
3     public InvalidStockAmountException() {
4         super("The given stock amount can't be negative.");
5     }
6 }
7
8 // (ii) Custom exception for stock dropping below minimum
9 // limits
10 class InvalidInventoryStockException extends Exception {
11     public InvalidInventoryStockException() {
12         super("The stock can't be less than the minimum stock
13             .");
14     }
15 }
16
17 // (iii) The complete Inventory class matching specified
18 // requirements
19 class Inventory {
20     private double stock;
```

```
18     private double minStock;
19
20     // Constructor to set fields
21     public Inventory(double minStock) {
22         this.minStock = minStock;
23         this.stock = 0.0;
24     }
25
26     public void setInitialStock(double amount) throws
        InvalidStockAmountException,
        InvalidInventoryStockException {
27         if (amount < 0) {
28             throw new InvalidStockAmountException();
29         }
30         if (amount < minStock) {
31             throw new InvalidInventoryStockException();
32         }
33         this.stock = amount;
34     }
35
36     public void addToInventory(double amount) throws
        InvalidStockAmountException {
37         if (amount < 0) {
38             throw new InvalidStockAmountException();
39         }
40         this.stock += amount;
41     }
42
43     public void removeFromInventory(double amount) throws
        InvalidStockAmountException,
        InvalidInventoryStockException {
44         if (amount < 0) {
45             throw new InvalidStockAmountException();
46         }
47         if (this.stock - amount < minStock) {
48             throw new InvalidInventoryStockException();
49         }
50         this.stock -= amount;
51     }
52
53     // Getters for inspecting the states
54     public double getStock() {
55         return stock;
56     }
57
58     public double getMinStock() {
59         return minStock;
60     }
```

```
61 }
```

**Q5.** Explain the use of `try` and `catch` in C++ with appropriate examples. (10 marks)  
[CSE 107 | 2015–16]

#### Answer — try and catch in C++

**try block:** Encloses code that might throw an exception. **catch block:** Handles a specific type of exception thrown from the try block.

```
1  #include <iostream>
2  #include <stdexcept>
3  using namespace std;
4
5  double safeDivide(double a, double b) {
6      if (b == 0)
7          throw runtime_error("Cannot divide by zero");
8      return a / b;
9  }
10
11 int main() {
12     // Example 1: catching a runtime_error
13     try {
14         cout << safeDivide(10, 2) << endl;    // 5
15         cout << safeDivide(10, 0) << endl;    // throws
16     } catch (const runtime_error& e) {
17         cout << "runtime_error: " << e.what() << endl;
18     }
19
20     // Example 2: multiple catch blocks
21     try {
22         int choice = 2;
23         if (choice == 1) throw 42;            // throw int
24         if (choice == 2) throw string("err"); // throw string
25     } catch (int i) {
26         cout << "Caught int: " << i << endl;
27     } catch (const string& s) {
28         cout << "Caught string: " << s << endl;
29     } catch (...) {
30         cout << "Caught unknown exception" << endl;
31     }
32
33     return 0;
34 }
```

**Output:**

5

```
runtime_error: Cannot divide by zero
Caught string: err
```

**Q6.** Show the hierarchical relation among `Throwable`, `Exception`, `RuntimeException` and `Error` classes. Write a custom exception `AuthFailureException`. Also write an `Authenticate()` method that will throw `AuthFailureException` on failure. (10 marks) [CSE 107 | 2015–16]

### Answer — Java Exception Hierarchy and Custom Exception Handling

#### Hierarchical Relation:

In Java, the exception handling framework is structured as a class hierarchy rooted at `java.lang.Object`. The four core classes are related as shown below:

```
java.lang.Object
|
+-- java.lang.Throwable
    |
    +-- java.lang.Error (Unchecked/Fatal JVM errors)
        |   |-- OutOfMemoryError
        |   +-- StackOverflowError
        |
    +-- java.lang.Exception (Checked/Application exceptions)
        |-- IOException
        |-- SQLException
        |
        +-- java.lang.RuntimeException (Unchecked/Logic errors)
            |-- NullPointerException
            +-- ArrayIndexOutOfBoundsException
```

- **Throwable:** The ultimate superclass of all errors and exceptions in the Java language. Only objects that are instances of this class (or one of its subclasses) can be thrown by the JVM or via the `throw` statement.
- **Error:** A subclass of `Throwable` that indicates serious, unrecoverable problems that a reasonable application should not try to catch (e.g., system crashes or resource exhaustion).
- **Exception:** A subclass of `Throwable` that indicates conditions that a reasonable application might want to catch and handle. These are typically compile-time "checked" exceptions.
- **RuntimeException:** A major subclass of `Exception` that represents "unchecked" exceptions or programming flaws (such as null pointer dereferences or array index violations) occurring at runtime.

#### Java Implementation:

Below is the implementation of the custom exception `AuthFailureException` and the required `Authenticate()` validation method.

```
1  // Custom checked exception extending the base Exception
   class
2  class AuthFailureException extends Exception {
3      // Constructor that accepts a detailed error message
4      public AuthFailureException(String message) {
5          super(message);
6      }
7  }
8
9  class UserAuthenticator {
10     // Authenticate method that validates credentials and
       declares throwing the custom exception
11     public void Authenticate(String username, String password
       ) throws AuthFailureException {
12         // Simple mock credentials check
13         String correctUsername = "admin";
14         String correctPassword = "password123";
15
16         if (!correctUsername.equals(username) || !
           correctPassword.equals(password)) {
17             // Throwing the custom exception on failure
18             throw new AuthFailureException("Authentication
               failed: Invalid credentials for user '" +
               username + "'.");
19         }
20
21         System.out.println("Authentication successful for
           user: " + username);
22     }
23 }
```

**Q7.** Write your own `TwoDimException` that is thrown when the determinant of a matrix is zero. (10 marks) [CSE 201 | 2015–16]

#### Answer — TwoDimException for Zero Determinant

```
1  class TwoDimException extends Exception {
2      TwoDimException() {
3          super("Matrix is singular: determinant is zero.");
4      }
5  }
6
7  class Matrix2x2 {
8      private double[][] m;
```

```

9
10 Matrix2x2(double a, double b, double c, double d) {
11     m = new double[][]{{a, b}, {c, d}};
12 }
13
14 double determinant() {
15     return m[0][0] * m[1][1] - m[0][1] * m[1][0];
16 }
17
18 // Returns inverse; throws TwoDimException if det == 0
19 Matrix2x2 inverse() throws TwoDimException {
20     double det = determinant();
21     if (det == 0) throw new TwoDimException();
22     return new Matrix2x2(
23         m[1][1] / det, -m[0][1] / det,
24         -m[1][0] / det, m[0][0] / det);
25 }
26
27 @Override
28 public String toString() {
29     return String.format("[[%.2f, %.2f], [%.2f, %.2f]]",
30         m[0][0], m[0][1], m[1][0], m
31         [1][1]);
32 }
33
34 public static void main(String[] args) {
35     Matrix2x2 m1 = new Matrix2x2(2, 1, 5, 3);
36     Matrix2x2 m2 = new Matrix2x2(1, 2, 2, 4); // det = 0
37
38     try {
39         System.out.println("Inverse of m1: " + m1.inverse
40             ());
41         System.out.println("Inverse of m2: " + m2.inverse
42             ());
43     } catch (TwoDimException e) {
44         System.out.println("Error: " + e.getMessage());
45     }
46 }

```

**Output:**

Inverse of m1: [[3.00, -1.00], [-5.00, 2.00]]  
 Error: Matrix is singular: determinant is zero.

- Q8.** Consider a `MyStack` class with `push(Object)`, `pop()`, `top()`, `isEmpty()`. `pop()` and `top()` throw `StackEmptyException`; `push()` throws `StackFullException`. Write the custom exceptions and the `MyStack` class. (10 marks) [CSE 201 | 2014–15]



**Answer — MyStack with StackEmptyException and StackFullException**

```
1  class StackEmptyException extends Exception {
2      StackEmptyException() { super("Stack is empty"); }
3  }
4
5  class StackFullException extends Exception {
6      StackFullException() { super("Stack is full"); }
7  }
8
9  class MyStack {
10     private Object[] data;
11     private int top;
12     private int capacity;
13
14     MyStack(int capacity) {
15         this.capacity = capacity;
16         data = new Object[capacity];
17         top = -1;
18     }
19
20     boolean isEmpty() { return top == -1; }
21     boolean isFull() { return top == capacity - 1; }
22
23     void push(Object obj) throws StackFullException {
24         if (isFull()) throw new StackFullException();
25         data[++top] = obj;
26     }
27
28     Object pop() throws StackEmptyException {
29         if (isEmpty()) throw new StackEmptyException();
30         return data[top--];
31     }
32
33     Object top() throws StackEmptyException {
34         if (isEmpty()) throw new StackEmptyException();
35         return data[top];
36     }
37
38     public static void main(String[] args) {
39         MyStack s = new MyStack(3);
40         try {
41             s.push("A"); s.push("B"); s.push("C");
42             System.out.println("Top: " + s.top()); // C
43             System.out.println("Pop: " + s.pop()); // C
44             s.push("D");
45             s.push("E"); // StackFullException
46         } catch (StackFullException e) {
47             System.out.println("Full: " + e.getMessage());
```

```
48         } catch (StackEmptyException e) {  
49             System.out.println("Empty: " + e.getMessage());  
50         }  
51     }  
52 }
```

**Q9.** Write a program that throws and catches an exception when the value of a variable is greater than 1000. (10 marks) [CSE 201 | 2012–13]

**Answer — Throw and Catch When Value > 1000**

```
1  class ValueTooLargeException extends Exception {  
2      ValueTooLargeException(int val) {  
3          super("Value " + val + " exceeds 1000");  
4      }  
5  }  
6  
7  class ThrowDemo {  
8      static void check(int val) throws ValueTooLargeException  
9      {  
10         if (val > 1000)  
11             throw new ValueTooLargeException(val);  
12         System.out.println("Value OK: " + val);  
13     }  
14  
15     public static void main(String[] args) {  
16         int[] tests = {500, 1001, 999, 2000};  
17         for (int v : tests) {  
18             try {  
19                 check(v);  
20             } catch (ValueTooLargeException e) {  
21                 System.out.println("Exception: " + e.  
22                     getMessage());  
23             }  
24         }  
25     }  
26 }
```

**Output:**

```
Value OK: 500  
Exception: Value 1001 exceeds 1000  
Value OK: 999  
Exception: Value 2000 exceeds 1000
```

**Q10.** Write a program that illustrates re-throwing an exception. (4 marks) [CSE 201 | 2011–12]

**Answer — Re-throwing an Exception**

```
1  class RethrowDemo {
2      static void level2() throws Exception {
3          throw new Exception("Original exception from level2")
4          ;
5      }
6
7      static void level1() throws Exception {
8          try {
9              level2();
10             } catch (Exception e) {
11                 System.out.println("level1 caught: " + e.
12                     getMessage());
13                 System.out.println("level1 re-throwing...");
14                 throw e;    // re-throw the same exception
15             }
16         }
17
18         public static void main(String[] args) {
19             try {
20                 level1();
21             } catch (Exception e) {
22                 System.out.println("main caught: " + e.getMessage
23                     ());
24             }
25         }
26     }
```

**Output:**

```
level1 caught: Original exception from level2
level1 re-throwing...
main caught: Original exception from level2
```

Re-throwing allows a method to partially handle an exception (e.g., log it) and then pass it up to a higher-level handler.

**Q11.** What are the keywords associated with exception handling in Java? Write the use of each keyword. (10 marks) [CSE 201 | 2011–12]

**Answer — Java Exception Handling Keywords**

Java has **five** keywords for exception handling:

| Keyword | Use                                                      |
|---------|----------------------------------------------------------|
| try     | Wraps code that may throw an exception                   |
| catch   | Handles a specific exception type                        |
| finally | Runs always, whether or not an exception occurred        |
| throw   | Explicitly throws an exception object                    |
| throws  | Declares which checked exceptions a method may propagate |

```

1  class KeywordsDemo {
2      // 'throws' declares checked exception
3      static int divide(int a, int b) throws
        ArithmeticException {
4          if (b == 0)
5              throw new ArithmeticException("div by zero"); //
                'throw'
6          return a / b;
7      }
8
9      public static void main(String[] args) {
10         try {                                // 'try'
11             System.out.println(divide(10, 2));
12             System.out.println(divide(10, 0));
13         } catch (ArithmeticException e) { // 'catch'
14             System.out.println("Caught: " + e.getMessage());
15         } finally {                            // 'finally'
16             System.out.println("Finally block always runs");
17         }
18     }
19 }

```

**Output:**

```

5
Caught: div by zero
Finally block always runs

```

**Q12.** In JVM, when is a `ClassCastException` thrown? How can we avoid it? (5+2 marks) [CSE 201 | 2010–11]

**Answer — ClassCastException: When and How to Avoid****When is it thrown?**

A `ClassCastException` is thrown at run time when the JVM attempts to cast an object to a type it is *not* an instance of.

```

1  Object obj = "Hello";           // obj actually holds a String
2  Integer i  = (Integer) obj; // ClassCastException at run time
    !

```

**How to avoid it:**

Use the instanceof operator to check the actual type before casting.

```
1 Object obj = "Hello";
2
3 if (obj instanceof Integer) {
4     Integer i = (Integer) obj;
5     System.out.println("Integer: " + i);
6 } else {
7     System.out.println("Not an Integer: " + obj.getClass().
8         getName());
9 }
10 // Output: Not an Integer: java.lang.String
```

Using generics also prevents ClassCastException at compile time by enforcing type safety without explicit casts.

**Q13.** Explain three different usages of the keyword finally with examples. (9 marks)  
[CSE 201 | 2010–11]

**Answer — Three Usages of finally****Usage 1 — Resource cleanup (always close resources):**

```
1 FileReader fr = null;
2 try {
3     fr = new FileReader("file.txt");
4     // read file ...
5 } catch (IOException e) {
6     System.out.println("IO Error: " + e.getMessage());
7 } finally {
8     if (fr != null) try { fr.close(); } catch (IOException
9         ignored) {}
10    System.out.println("FileReader closed.");
11 }
```

**Usage 2 — Runs even when no exception occurs:**

```
1 try {
2     System.out.println("No exception here");
3 } catch (Exception e) {
4     System.out.println("This won't run");
5 } finally {
6     System.out.println("Finally runs anyway"); // always
7     executes
8 }
```

**Usage 3 — Runs even after a return statement in try:**

```
1 static int getValue() {
2     try {
```

```
3         return 10;                // return is deferred
4     } finally {
5         System.out.println("finally before actual return");
6     }
7 }
8 // Output: "finally before actual return", then returns 10
```

- Q14.** Write down the syntax used for Java’s ‘catch or declare’ strategy of exception handling. (7 marks) [CSE 201 | 2010–11]

#### Answer — Catch or Declare Strategy

Java’s “catch or declare” rule applies to *checked* exceptions: any method that may throw a checked exception must either (a) **catch** it locally or (b) **declare** it with **throws**.

**Option A — Catch it (handle locally):**

```
1 void readFile(String path) {
2     try {
3         FileReader fr = new FileReader(path);
4         // ... use fr ...
5     } catch (IOException e) {
6         System.out.println("Handled: " + e.getMessage());
7     }
8 }
```

**Option B — Declare it (propagate to caller):**

```
1 void readFile(String path) throws IOException {
2     FileReader fr = new FileReader(path); // not caught here
3     // ... use fr ...
4 }
5
6 // Caller must then catch or declare IOException
7 void caller() throws IOException {
8     readFile("data.txt");
9 }
```

Unchecked exceptions (`RuntimeException` and its subclasses) are *exempt* from this rule — they may be thrown without declaration.

- Q15.** How can we create a user-defined exception type? Give an example. (4 marks) [CSE 201 | 2010–11]

**Answer — Creating a User-Defined Exception**

Extend `Exception` (for checked) or `RuntimeException` (for unchecked) and provide a constructor that passes a message to `super()`.

```

1  // User-defined checked exception
2  class AgeOutOfRangeException extends Exception {
3      AgeOutOfRangeException(int age) {
4          super("Age " + age + " is out of valid range (0-150)"
5              );
6      }
7  }
8  class Person {
9      private int age;
10
11     void setAge(int age) throws AgeOutOfRangeException {
12         if (age < 0 || age > 150)
13             throw new AgeOutOfRangeException(age);
14         this.age = age;
15     }
16
17     public static void main(String[] args) {
18         Person p = new Person();
19         try {
20             p.setAge(25);    // OK
21             p.setAge(200);  // throws
22         } catch (AgeOutOfRangeException e) {
23             System.out.println("Caught: " + e.getMessage());
24         }
25     }
26 }

```

**Output:**

Caught: Age 200 is out of valid range (0-150)

**Q16.** Write a program that illustrates the application of multiple `catch` statements. (5 marks) [CSE 201 | 2007–08]

**Answer — Multiple catch Statements**

```

1  class MultiCatchDemo {
2      static void test(int choice) {
3          try {
4              if (choice == 1)
5                  throw new ArithmeticException("Arithmetic
6                      error");
7              else if (choice == 2)

```

```

7         throw new ArrayIndexOutOfBoundsException("Bad
            index");
8     else if (choice == 3)
9         throw new NullPointerException("Null ref");
10    else {
11        int[] arr = new int[3];
12        arr[5] = 10; // causes
                       ArrayIndexOutOfBoundsException
13    }
14    } catch (ArithmeticException e) {
15        System.out.println("Caught ArithmeticException: "
16                            + e.getMessage());
17    } catch (ArrayIndexOutOfBoundsException e) {
18        System.out.println("Caught
19                            ArrayIndexOutOfBoundsException: "
20                            + e.getMessage());
21    } catch (NullPointerException e) {
22        System.out.println("Caught NullPointerException:
23                            "
24                            + e.getMessage());
25    } catch (Exception e) {
26        System.out.println("Caught general Exception: "
27                            + e.getMessage());
28    }
29    }
30
31    public static void main(String[] args) {
32        test(1); test(2); test(3); test(4);
33    }

```

**Output:**

```

Caught ArithmeticException: Arithmetic error
Caught ArrayIndexOutOfBoundsException: Bad index
Caught NullPointerException: Null ref
Caught ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3

```

**Q17.** Suppose you are asked to write `MyException` that takes a `String` parameter. When displayed, it shows the `String`. Write necessary code. (8 marks) [CSE 201 | 2007–08]

**Answer — MyException with String Parameter**

```

1  class MyException extends Exception {
2      private String message;
3
4      MyException(String msg) {

```



```
5         super(msg);           // pass to Exception
6         this.message = msg;
7     }
8
9     @Override
10    public String toString() {
11        return "MyException: " + message;
12    }
13 }
14
15 class MyExceptionDemo {
16     static void riskyMethod(int x) throws MyException {
17         if (x < 0)
18             throw new MyException("Negative value not allowed
19                                     : " + x);
20         System.out.println("Value is: " + x);
21     }
22
23     public static void main(String[] args) {
24         try {
25             riskyMethod(5);
26             riskyMethod(-3);
27         } catch (MyException e) {
28             System.out.println(e);           // calls
29             toString()
30             System.out.println(e.getMessage()); // from super
31         }
32     }
33 }
```

**Output:**

```
Value is: 5
MyException: Negative value not allowed: -3
Negative value not allowed: -3
```

BUET · CSE 107 / CSE 201

# Object Oriented Programming

Section S14

---

Multithreading & Concurrency

## S14 — Multithreading & Concurrency

**Q1.** Complete the following Java code that finds the summation of a 10000-element integer array using an array of 10 threads in parallel. You cannot write any more classes, but you can add variables, constructors, and other methods. **(15 marks)**

```
1  class ParallelSum implements Runnable {
2      // your code here
3  }
4
5  public class Main {
6      public static void main(String[] args) {
7          java.util.Random random = new java.util.Random();
8          int[] numbers = new int[10000];
9
10         for (int i = 0; i < numbers.length; i++) {
11             numbers[i] = random.nextInt(10000);
12         }
13
14         ParallelSum[] parallelSum = new ParallelSum[10];
15         // your code here
16     }
17 }
```

[CSE 107 | 2023–24]

### Answer — Parallel Sum of 10,000-Element Array (10 Threads)

To achieve parallel summation, each `ParallelSum` instance should be responsible for a subset (chunk) of the array. After the threads finish executing (`join`), the main thread aggregates the partial sums.

```
1  class ParallelSum implements Runnable {
2      private int[] numbers;
3      private int start;
4      private int end;
5      private long partialSum;
6
7      // Constructor to assign the array and the bounds for
8      // this thread
9      public ParallelSum(int[] numbers, int start, int end) {
10         this.numbers = numbers;
11         this.start = start;
12         this.end = end;
13         this.partialSum = 0;
14     }
15
16     @Override
17     public void run() {
18         for (int i = start; i < end; i++) {
```

```
18         partialSum += numbers[i];
19     }
20 }
21
22 // Method to retrieve the calculated sum for this chunk
23 public long getPartialSum() {
24     return partialSum;
25 }
26 }
27
28 public class Main {
29     public static void main(String[] args) {
30         java.util.Random random = new java.util.Random();
31         int[] numbers = new int[10000];
32
33         for (int i = 0; i < numbers.length; i++) {
34             numbers[i] = random.nextInt(10000);
35         }
36
37         ParallelSum[] parallelSum = new ParallelSum[10];
38         Thread[] threads = new Thread[10];
39
40         int chunkSize = numbers.length / 10; // 1000 elements
41                                         // per thread
42
43         // 1. Initialize Runnables and start Threads
44         for (int i = 0; i < 10; i++) {
45             int start = i * chunkSize;
46             int end = start + chunkSize;
47
48             parallelSum[i] = new ParallelSum(numbers, start,
49                                             end);
50             threads[i] = new Thread(parallelSum[i]);
51             threads[i].start();
52         }
53
54         long totalSum = 0;
55
56         // 2. Wait for threads to finish and aggregate
57         // results
58         for (int i = 0; i < 10; i++) {
59             try {
60                 threads[i].join(); // Ensure the thread has
61                                     // finished
62                 totalSum += parallelSum[i].getPartialSum();
63             } catch (InterruptedException e) {
64                 e.printStackTrace();
65             }
66         }
67     }
68 }
```

```

62         }
63
64         System.out.println("Total sum = " + totalSum);
65     }
66 }

```

**Q2.** Consider the following code. Determine all possible outputs assuming T1 starts first. If two threads execute f0 and f1, will they run in parallel? Why or why not?

```

1  class TestClass {
2      public void f0() { synchronized(this) { for(int i=0;i<5;i
        ++ ) System.out.println(Thread.currentThread().getName()
        +" "+i); } }
3      public void f1() { synchronized(TestClass.class) { for(int
        i=0;i<5;i++) System.out.println(Thread.currentThread()
        .getName()+" "+i); } }
4      public synchronized static void f2() { for(int i=0;i<5;i
        ++ ) System.out.println(Thread.currentThread().getName()
        +" "+i); }
5      public synchronized static void f3() { for(int i=0;i<5;i
        ++ ) System.out.println(Thread.currentThread().getName()
        +" "+i); }
6  }
7  class SynchronizedTest {
8      public static void main(String[] args) {
9          new Thread(new TestClass()::f1, "T1").start();
10         new Thread(TestClass::f2, "T2").start();
11         new Thread(TestClass::f3, "T3").start();
12     }
13 }

```

(10 marks)

[CSE 107 | 2023–24]

### Answer — Lock Analysis: f0, f1, f2, f3

Lock objects used by each method:

| Method | Lock object                   | Type          |
|--------|-------------------------------|---------------|
| f0()   | this (instance)               | instance lock |
| f1()   | TestClass.class               | class lock    |
| f2()   | TestClass.class (static sync) | class lock    |
| f3()   | TestClass.class (static sync) | class lock    |

The three threads in main:

- **T1** calls f1() — acquires the **class lock** (TestClass.class).
- **T2** calls static f2() — also acquires the **class lock**.

- **T3** calls `static f3()` — also acquires the **class lock**.

All three threads compete for the *same* lock (`TestClass.class`), so **they cannot run in parallel**. They execute one at a time in some order.

**Possible outputs** (T1 starts first; T2 and T3 may interleave after T1 finishes — all orderings of T2 and T3 are valid):

- (i) T1 runs completely, then T2, then T3.
- (ii) T1 runs completely, then T3, then T2.

Within each thread's turn the output is always its 5 lines in order (T1 0 ...T1 4, etc.). No interleaving of individual lines between threads is possible because each holds the class lock for its entire loop.

**Would `f0` and `f1` run in parallel?**

**Yes** — if two threads called `f0` and `f1` on the *same* instance, they would still run in parallel because `f0` locks on **this** (the *instance* lock) while `f1` locks on `TestClass.class` (the *class* lock). These are two *different* objects, so neither thread blocks the other.

**Q3.** What is the problem with the following code? Write 2 different ways to fix it:

```

1  class MyThread implements Runnable {
2      List<String> list;
3      Thread t;
4      public MyThread(List<String> list) { this.list = list; t =
          new Thread(this); t.start(); }
5      public void run() { for(int i=1;i<=10000;i++) list.add(
          String.valueOf(i)); }
6  }
7  public class TestSyncArrayList {
8      public static void main(String[] args) throws Exception {
9          List<String> list = new ArrayList<>();
10         MyThread t1 = new MyThread(list), t2 = new MyThread(
            list);
11         t1.t.join(); t2.t.join();
12         System.out.println(list.size());
13     }
14 }
```

(10 marks)

[CSE 107 | 2023–24]

### Answer — Race Condition on ArrayList: Problem & Two Fixes

**The problem:**

ArrayList is **not thread-safe**. When two threads call `list.add()` concurrently, internal array resizing and index updates can interleave, causing:

- lost updates (final size less than 20,000), or
- an `ArrayIndexOutOfBoundsException` during resize.

The printed size is therefore unpredictable and almost never 20,000.

### Fix 1 — Replace `ArrayList` with `CopyOnWriteArrayList`

`CopyOnWriteArrayList` is a thread-safe variant from `java.util.concurrent`. Every write creates a fresh copy of the internal array, so concurrent `add()` calls are safe without explicit locking.

```

1  import java.util.*;
2  import java.util.concurrent.CopyOnWriteArrayList;
3
4  class MyThread implements Runnable {
5      List<String> list;
6      Thread t;
7      public MyThread(List<String> list) {
8          this.list = list;
9          t = new Thread(this);
10         t.start();
11     }
12     public void run() {
13         for (int i = 1; i <= 10000; i++)
14             list.add(String.valueOf(i));
15     }
16 }
17
18 public class Fix1 {
19     public static void main(String[] args) throws Exception {
20         // Use thread-safe list instead of ArrayList
21         List<String> list = new CopyOnWriteArrayList<>();
22         MyThread t1 = new MyThread(list);
23         MyThread t2 = new MyThread(list);
24         t1.t.join(); t2.t.join();
25         System.out.println(list.size()); // always 20000
26     }
27 }
```

### Fix 2 — Synchronise the `add()` call manually

Keep `ArrayList` but wrap every `add()` inside a synchronized block on the shared list object.

```

1  class MyThreadSync implements Runnable {
2      List<String> list;
3      Thread t;
4      public MyThreadSync(List<String> list) {
```

```

5         this.list = list;
6         t = new Thread(this);
7         t.start();
8     }
9     public void run() {
10         for (int i = 1; i <= 10000; i++)
11             synchronized (list) {           // lock on the
12                 shared list
13                 list.add(String.valueOf(i));
14             }
15     }
16
17     public class Fix2 {
18         public static void main(String[] args) throws Exception {
19             List<String> list = new ArrayList<>();
20             MyThreadSync t1 = new MyThreadSync(list);
21             MyThreadSync t2 = new MyThreadSync(list);
22             t1.t.join(); t2.t.join();
23             System.out.println(list.size()); // always 20000
24         }
25     }

```

*Note: `Collections.synchronizedList(new ArrayList<>())` is also a valid Fix 2 alternative — it wraps every method with a synchronized block automatically.*

- Q4.** You are given a task to find the total count of prime numbers in a randomly generated array of 10,000 integers (values 1–5000). Divide work equally among 5 threads. Main thread waits and prints final count. **(15 marks)** [CSE 107 | 2022–23]

#### Answer — Parallel Prime Count in 10,000-Element Array (5 Threads)

Each thread inspects 2,000 elements of the array for primality. Partial counts are stored in a shared `counts[]` array.

```

1     public class PrimeCount {
2
3         static boolean isPrime(int n) {
4             if (n < 2) return false;
5             for (int i = 2; i * i <= n; i++)
6                 if (n % i == 0) return false;
7             return true;
8         }
9
10        static class PrimeTask implements Runnable {
11            private final int[] arr;
12            private final int start, end, id;
13            private final int[] counts;

```



```

14
15     PrimeTask(int[] arr, int start, int end,
16               int[] counts, int id) {
17         this.arr = arr; this.start = start;
18         this.end = end; this.counts = counts;
19         this.id = id;
20     }
21
22     @Override
23     public void run() {
24         int c = 0;
25         for (int i = start; i < end; i++)
26             if (isPrime(arr[i])) c++;
27         counts[id] = c;
28     }
29 }
30
31 public static void main(String[] args) throws
    InterruptedException {
32     int[] arr = new int[10000];
33     int[] counts = new int[5];
34     java.util.Random rnd = new java.util.Random();
35     for (int i = 0; i < arr.length; i++)
36         arr[i] = rnd.nextInt(5000) + 1;
37
38     Thread[] threads = new Thread[5];
39     int chunk = arr.length / 5; // 2000 each
40
41     for (int i = 0; i < 5; i++) {
42         threads[i] = new Thread(new PrimeTask(
43             arr, i * chunk, (i + 1) * chunk, counts, i));
44         threads[i].start();
45     }
46     for (Thread t : threads) t.join();
47
48     int total = 0;
49     for (int c : counts) total += c;
50     System.out.println("Total prime count = " + total);
51 }
52 }

```

**Q5.** You are given the following class. Create threads T1–T5 from main to execute f1, f2, f0, fs1, and fs2. (i) Which threads run in parallel and why? (ii) Which do not? Can restriction R2 be relaxed?

```

1 class ThreadTest {
2     public void f0() { for(int i=1;i<500;i++) System.out.
        println(Thread.currentThread().getName()+" "+i); }

```

```

3     public synchronized void f1() { for(int i=1;i<1000;i++)
        System.out.println(Thread.currentThread().getName()+" "+
        +i); }
4     public synchronized void f2() { for(int i=1;i<1500;i++)
        System.out.println(Thread.currentThread().getName()+" "+
        +i); }
5     public synchronized static void fs1() { for(int i=1;i
        <2000;i++) System.out.println(Thread.currentThread().
        getName()+" "+i); }
6     public synchronized static void fs2() { for(int i=1;i
        <2500;i++) System.out.println(Thread.currentThread().
        getName()+" "+i); }
7 }

```

R1: Cannot change ThreadTest. R2: Can create only one object. **(15 marks)** [CSE 107 | 2022–23]

### Answer — ThreadTest: Parallel Execution Analysis

#### Thread creation (one object, R2):

```

1  public class Main {
2      public static void main(String[] args) throws
        InterruptedException {
3          ThreadTest obj = new ThreadTest(); // only ONE object
            (R2)
4
5          Thread t1 = new Thread(() -> obj.f1(),      "T1");
6          Thread t2 = new Thread(() -> obj.f2(),      "T2");
7          Thread t3 = new Thread(() -> obj.f0(),      "T3");
8          Thread t4 = new Thread(ThreadTest::fs1,     "T4");
9          Thread t5 = new Thread(ThreadTest::fs2,     "T5");
10
11         t1.start(); t2.start(); t3.start(); t4.start(); t5.
            start();
12         t1.join(); t2.join(); t3.join(); t4.join(); t5.
            join();
13     }
14 }

```

#### Lock analysis:

| Thread | Method            | Lock             | Kind     |
|--------|-------------------|------------------|----------|
| T1     | f1() synchronized | obj (this)       | instance |
| T2     | f2() synchronized | obj (this)       | instance |
| T3     | f0() plain        | none             | —        |
| T4     | fs1() static sync | ThreadTest.class | class    |
| T5     | fs2() static sync | ThreadTest.class | class    |

## (i) Which threads run in parallel?

- **T3 runs in parallel with everyone.** `f0()` acquires no lock, so it never blocks and is never blocked by T1, T2, T4, or T5.
- **T1 or T2 runs in parallel with T4 or T5.** The instance lock (`obj`) and the class lock (`ThreadTest.class`) are *different* objects, so an instance-method thread does not block a static-method thread and vice versa.

## (ii) Which threads do NOT run in parallel?

- **T1 and T2 cannot run in parallel** — both compete for the instance lock `obj`; only one executes at a time.
- **T4 and T5 cannot run in parallel** — both compete for the class lock `ThreadTest.class`.

Can R2 (*only one object*) be relaxed?

Yes. If two separate instances `obj1` and `obj2` are created, T1 (on `obj1`) and T2 (on `obj2`) acquire *different* instance locks and therefore **run in parallel**. Relaxing R2 increases concurrency between T1 and T2 without affecting the class-level synchronisation of T4 and T5.

**Q6.** Describe two different ways of creating threads in Java. Which one is better and why? Explain the difference between a synchronized method and a synchronized statement. (10 marks) [CSE 107 | 2021–22]

## Answer — Thread Creation, Preference &amp; Synchronization

Java provides two fundamental ways to create threads.

## Way 1: Extending Thread

```

1  class CountThread extends Thread {
2      public void run() {
3          for (int i = 0; i < 5; i++)
4              System.out.println(getName() + " -> " + i);
5      }
6  }
7  // new CountThread().start();

```

## Way 2: Implementing Runnable

```

1  class CountRunnable implements Runnable {
2      public void run() {
3          for (int i = 0; i < 5; i++)
4              System.out.println(Thread.currentThread().getName
5                  () + " -> " + i);
6      }

```

```
7 // new Thread(new CountRunnable()).start();
```

### Which is better?

Implementing `Runnable` is the preferred approach because:

- Java supports only single inheritance; a class that already inherits from another class *cannot* extend `Thread`. Implementing `Runnable` avoids this limitation.
- It cleanly separates task logic from thread management (separation of concerns).
- The same `Runnable` object can be reused across multiple threads or passed to an `ExecutorService`.

### Synchronized method vs. synchronized statement

A **synchronized method** places the lock on the entire method body. The implicit lock object is always `this`:

```
1 public synchronized void deposit(int amount) { balance +=
    amount; }
```

A **synchronized statement** (block) locks only a specific section of code and allows the programmer to choose the lock object explicitly, enabling finer-grained concurrency:

```
1 public void deposit(int amount) {
2     doSomeNonCriticalWork(); // runs without holding any
    lock
3     synchronized (this) {
4         balance += amount; // only this block is mutually
        exclusive
5     }
6 }
```

Synchronized statements are preferred when only a small portion of the method requires mutual exclusion, thereby reducing lock contention and improving throughput.

**Q7.** Find the minimum of a 1000-integer array using 5 threads in parallel:

```
1 class ParallelMin implements Runnable { /* your code */ }
2 class Main {
3     public static void main(String[] args) {
4         int[] numbers = new int[1000];
5         ParallelMin[] parallelMin = new ParallelMin[5];
6         // your code
```

```

7     }
8 }

```

(15 marks)

[CSE 107 | 2021–22]

**Answer — Parallel Minimum of 1000-Element Array (5 Threads)**

Each thread finds the minimum of its 200-element sub-array. The main thread waits for all threads, then finds the global minimum across the five partial results.

```

1  class ParallelMin implements Runnable {
2      private final int[] numbers;
3      private final int start, end;
4      int localMin; // read by main after join()
5
6      ParallelMin(int[] numbers, int start, int end) {
7          this.numbers = numbers;
8          this.start = start;
9          this.end = end;
10         this.localMin = Integer.MAX_VALUE;
11     }
12
13     @Override
14     public void run() {
15         for (int i = start; i < end; i++)
16             if (numbers[i] < localMin) localMin = numbers[i];
17     }
18 }
19
20 class Main {
21     public static void main(String[] args) throws
22         InterruptedException {
23         int[] numbers = new int[1000];
24         // Fill with sample data
25         java.util.Random rnd = new java.util.Random();
26         for (int i = 0; i < numbers.length; i++)
27             numbers[i] = rnd.nextInt(10000);
28
29         ParallelMin[] parallelMin = new ParallelMin[5];
30         Thread[] threads = new Thread[5];
31         int chunk = numbers.length / 5; // 200 each
32
33         for (int i = 0; i < 5; i++) {
34             parallelMin[i] = new ParallelMin(
35                 numbers, i * chunk, (i + 1) * chunk);
36             threads[i] = new Thread(parallelMin[i]);
37             threads[i].start();
38         }
39         for (Thread t : threads) t.join();

```

```

40         int globalMin = Integer.MAX_VALUE;
41         for (ParallelMin pm : parallelMin)
42             if (pm.localMin < globalMin) globalMin = pm.
                localMin;
43
44         System.out.println("Global minimum = " + globalMin);
45     }
46 }

```

**Q8.** What are the two common ways to achieve synchronization between Threads? Which one gives more control? (5 marks) [CSE 107 | Jan 2021]

### Answer — Thread Synchronisation

The two common ways to achieve synchronisation in Java are:

#### 1. Synchronized methods

Add the `synchronized` keyword to a method declaration. Java automatically acquires the intrinsic lock (*monitor*) of `this` before executing the method body and releases it when the method returns.

```

1  class Counter {
2      private int count = 0;
3      public synchronized void increment() { count++; }
4      public synchronized int getCount()    { return count; }
5  }

```

#### 2. Synchronized statements (blocks)

Wrap only the *critical section* with `synchronized(obj)` where `obj` is the object whose monitor you want to acquire.

```

1  class Counter {
2      private int count = 0;
3      private final Object lock = new Object();
4
5      public void increment() {
6          // non-critical work here (no lock held)
7          synchronized (lock) {
8              count++;          // only this line needs mutual
                                exclusion
9          }
10     }
11 }

```

**Which gives more control?**

**Synchronized statements** give more control because:

- The programmer can choose *exactly which object* to lock on — it does not have to be `this`.

- The lock is held for the *minimum necessary duration*, reducing contention and improving concurrency.
- Multiple independent critical sections inside one class can use *different* lock objects, allowing them to be entered concurrently by different threads.

**Q9.** Write complete Java code to compute the summation of 1 to 10,000 by dividing work equally among 5 different threads. Main thread waits and only prints the final summation. **(10 marks)** [CSE 107 | 2020–21]

↪ Similar: CSE 201 | 2014–15

### Answer — Summation of 1 to 10,000 Using 5 Threads

The range [1, 10000] is divided into 5 equal sub-ranges of 2000 numbers each. Each thread computes the partial sum of its sub-range. The main thread waits for all worker threads to finish via `join()`, then sums the partial results and prints the total.

```

1  public class ParallelSum {
2
3      static final int TOTAL    = 10000;
4      static final int THREADS = 5;
5      static final int RANGE    = TOTAL / THREADS;    // 2000 per
           thread
6
7      // Array to store each thread's partial sum
8      static long[] partialSum = new long[THREADS];
9
10     static class SumTask implements Runnable {
11         private final int id;        // thread index 0..4
12         private final int start;    // inclusive
13         private final int end;      // inclusive
14
15         SumTask(int id, int start, int end) {
16             this.id = id;
17             this.start = start;
18             this.end   = end;
19         }
20
21         @Override
22         public void run() {
23             long sum = 0;
24             for (int i = start; i <= end; i++) sum += i;
25             partialSum[id] = sum;
26             System.out.println("Thread-" + id +
27                 " summed [" + start + ", " + end + "] = " +
28                 sum);
29         }
30     }
31 }

```

```

29     }
30
31     public static void main(String[] args) throws
        InterruptedException {
32         Thread[] workers = new Thread[THREADS];
33
34         // Create and start each worker
35         for (int i = 0; i < THREADS; i++) {
36             int start = i * RANGE + 1;
37             int end    = start + RANGE - 1;
38             workers[i] = new Thread(new SumTask(i, start, end
39                                     ));
40             workers[i].start();
41         }
42
43         // Main thread waits for all workers
44         for (Thread t : workers) t.join();
45
46         // Aggregate partial sums
47         long total = 0;
48         for (long ps : partialSum) total += ps;
49
50         System.out.println("\nFinal Summation (1 to 10000) =
51                             " + total);
52         // Expected: 50_050_000
53     }
54 }

```

How work is divided:

| Thread       | Range        | Partial Sum       |
|--------------|--------------|-------------------|
| Thread-0     | 1 – 2000     | 2,001,000         |
| Thread-1     | 2001 – 4000  | 6,001,000         |
| Thread-2     | 4001 – 6000  | 10,001,000        |
| Thread-3     | 6001 – 8000  | 14,001,000        |
| Thread-4     | 8001 – 10000 | 18,045,000        |
| <b>Total</b> |              | <b>50,050,000</b> |

Note: `partialSum` is written by each thread to a *distinct index*, so no synchronisation is needed for the write. The main thread reads only after all `join()` calls complete, ensuring full visibility.

**Q10.** Write Java code to compute the summation of 1 to 10,000,000 by dividing the work equally among 10 threads (no formula allowed, only loops). The main thread waits for all threads to finish and prints the final summation. (10 marks) [CSE 107 | Jan 2021]



## Answer — Sum 1 to 10,000,000 (10 Threads, Loops Only)

```

1  public class SumLoop10M {
2
3      static final int TOTAL    = 10000000;
4      static final int THREADS = 10;
5      static final int CHUNK    = TOTAL / THREADS; // 1,000,000
6
7      static long[] partial = new long[THREADS];
8
9      static class SumTask implements Runnable {
10         private final int id, start, end;
11         SumTask(int id, int start, int end) {
12             this.id = id; this.start = start; this.end = end;
13         }
14         @Override
15         public void run() {
16             long sum = 0;
17             for (int i = start; i <= end; i++) // loop only,
18                 // no formula
19                 sum += i;
20             partial[id] = sum;
21         }
22     }
23
24     public static void main(String[] args) throws
25         InterruptedException {
26         Thread[] threads = new Thread[THREADS];
27         for (int i = 0; i < THREADS; i++) {
28             int s = i * CHUNK + 1;
29             int e = s + CHUNK - 1;
30             threads[i] = new Thread(new SumTask(i, s, e));
31             threads[i].start();
32         }
33         for (Thread t : threads) t.join(); // main waits
34
35         long total = 0;
36         for (long p : partial) total += p;
37         System.out.println("Final sum = " + total);
38         // Expected: 50,000,005,000,000
39     }
40 }

```

**Q11.** Write Java code to compute the summation of 1 to 10,000 by dividing the work equally among 5 threads (no formula allowed, only loops). The main thread waits for all threads to finish and prints the final summation. (10 marks) [CSE 107 | 2020–21]

## Answer — Sum 1 to 10,000 (5 Threads, Loops Only)

```

1  public class SumLoop10K {
2
3      static final int TOTAL    = 10000;
4      static final int THREADS = 5;
5      static final int CHUNK    = TOTAL / THREADS; // 2000 each
6
7      static long[] partial = new long[THREADS];
8
9      static class SumTask implements Runnable {
10         private final int id, start, end;
11         SumTask(int id, int start, int end) {
12             this.id = id; this.start = start; this.end = end;
13         }
14         @Override
15         public void run() {
16             long sum = 0;
17             for (int i = start; i <= end; i++) sum += i;
18             partial[id] = sum;
19         }
20     }
21
22     public static void main(String[] args) throws
23         InterruptedException {
24         Thread[] threads = new Thread[THREADS];
25         for (int i = 0; i < THREADS; i++) {
26             int s = i * CHUNK + 1;
27             int e = s + CHUNK - 1;
28             threads[i] = new Thread(new SumTask(i, s, e));
29             threads[i].start();
30         }
31         for (Thread t : threads) t.join();
32
33         long total = 0;
34         for (long p : partial) total += p;
35         System.out.println("Final sum = " + total); //
36         50,005,000

```

**Q12.** Consider the Producer-Consumer problem with buffer of size 1. The producer generates letters A–Z circularly. The reader reads and displays them. Use wait/notifyAll or ArrayBlockingQueue. (15 marks) [CSE 107 | 2019–20, Jan 2021]

↪ Similar: CSE 201 | 2013–14

## Answer — Producer-Consumer: A-Z Circular, Buffer Size 1

## Solution 1 — Using wait / notifyAll

```

1  class Buffer {
2      private char data;
3      private boolean hasData = false;
4
5      public synchronized void put(char c) throws
        InterruptedException {
6          while (hasData) wait();          // wait if buffer is
            full
7          data = c;
8          hasData = true;
9          System.out.println("Produced: " + c);
10         notifyAll();
11     }
12
13     public synchronized char get() throws
        InterruptedException {
14         while (!hasData) wait();          // wait if buffer is
            empty
15         hasData = false;
16         System.out.println("Consumed: " + data);
17         notifyAll();
18         return data;
19     }
20 }
21
22 class Producer extends Thread {
23     private final Buffer buf;
24     Producer(Buffer buf) { this.buf = buf; }
25     @Override
26     public void run() {
27         try {
28             for (int cycle = 0; cycle < 2; cycle++) // 2 full
                cycles
29                 for (char c = 'A'; c <= 'Z'; c++)
30                     buf.put(c);
31             } catch (InterruptedException e) { e.printStackTrace
                (); }
32     }
33 }
34
35 class Consumer extends Thread {
36     private final Buffer buf;
37     Consumer(Buffer buf) { this.buf = buf; }
38     @Override
39     public void run() {

```

```

40         try {
41             for (int i = 0; i < 52; i++) // 2 x 26 letters
42                 buf.get();
43         } catch (InterruptedException e) { e.printStackTrace
44             (); }
45     }
46
47     public class ProducerConsumerAZ {
48         public static void main(String[] args) {
49             Buffer buf = new Buffer();
50             new Producer(buf).start();
51             new Consumer(buf).start();
52         }
53     }

```

### Solution 2 — Using ArrayBlockingQueue (simpler)

```

1  import java.util.concurrent.ArrayBlockingQueue;
2
3  public class ProducerConsumerAZBQ {
4      public static void main(String[] args) {
5          ArrayBlockingQueue<Character> queue =
6              new ArrayBlockingQueue<>(1);
7
8          Thread producer = new Thread(() -> {
9              try {
10                 for (int cycle = 0; cycle < 2; cycle++)
11                     for (char c = 'A'; c <= 'Z'; c++) {
12                         queue.put(c);
13                         System.out.println("Produced: " + c);
14                     }
15                 } catch (InterruptedException e) {}
16            });
17
18            Thread consumer = new Thread(() -> {
19                try {
20                    for (int i = 0; i < 52; i++) {
21                        char c = queue.take();
22                        System.out.println("Consumed: " + c);
23                    }
24                } catch (InterruptedException e) {}
25            });
26
27            producer.start();
28            consumer.start();
29        }
30    }

```

**Q13.** Mr. Chandler and Ms. Phoebe worked in the same place. Their work is to put and get product in a shared inventory array of size 3. Ms. Phoebe puts a single product in the inventory and Mr. Chandler gets a single product from the inventory at a time. Ms. Phoebe cannot put a product to the inventory if the inventory array is full with 4 products. Mr. Chandler cannot get a product from a inventory if the inventory array contains no product. Ms. Phoebe can put a product to the inventory if any one of the place of the inventory array is empty. Mr. Chandler can get a product from the inventory if any one of the place of the inventory array is full with a product. Write a java code to solve the above mentioned problem using the concepts of inter thread communication. You can use **wait/notifyAll** or **ArrayBlockingQueue**. (10 marks) [CSE 107 | Jan 2021, 2019–20]

↪ Similar: CSE 201 | 2014–15, 2008–09, 2009–10

### Answer — Chandler & Phoebe: Shared Inventory of Size 3

#### Solution 1 — Using wait / notifyAll

```

1  class Inventory {
2      private final int[] slots;
3      private int count = 0;
4
5      Inventory(int size) { slots = new int[size]; }
6
7      public synchronized void put(int product)
8          throws InterruptedException {
9          while (count == slots.length) wait(); // full --
10             Phoebe waits
11             slots[count++] = product;
12             System.out.println("Phoebe put product " + product
13                 + " [count=" + count + "]");
14             notifyAll();
15         }
16
17         public synchronized int get() throws InterruptedException
18             {
19             while (count == 0) wait(); // empty -- Chandler waits
20             int product = slots[--count];
21             System.out.println("Chandler got product " + product
22                 + " [count=" + count + "]");
23             notifyAll();
24             return product;
25         }
26     }
27
28     class Phoebe extends Thread {
29         private final Inventory inv;
30         Phoebe(Inventory inv) { this.inv = inv; }
31         @Override

```

```

30     public void run() {
31         try {
32             for (int i = 1; i <= 10; i++) inv.put(i);
33         } catch (InterruptedException e) {}
34     }
35 }
36
37 class Chandler extends Thread {
38     private final Inventory inv;
39     Chandler(Inventory inv) { this.inv = inv; }
40     @Override
41     public void run() {
42         try {
43             for (int i = 0; i < 10; i++) inv.get();
44         } catch (InterruptedException e) {}
45     }
46 }
47
48 public class InventoryDemo {
49     public static void main(String[] args) {
50         Inventory inv = new Inventory(3);
51         new Phoebe(inv).start();
52         new Chandler(inv).start();
53     }
54 }

```

### Solution 2 — Using ArrayBlockingQueue

```

1  import java.util.concurrent.ArrayBlockingQueue;
2
3  public class InventoryBQ {
4      public static void main(String[] args) {
5          ArrayBlockingQueue<Integer> inv = new
              ArrayBlockingQueue<>(3);
6
7          Thread phoebe = new Thread(() -> {
8              try {
9                  for (int i = 1; i <= 10; i++) {
10                     inv.put(i);
11                     System.out.println("Phoebe put " + i);
12                 }
13             } catch (InterruptedException e) {}
14         });
15
16         Thread handler = new Thread(() -> {
17             try {
18                 for (int i = 0; i < 10; i++) {
19                     int p = inv.take();

```

```

20         System.out.println("Chandler got " + p);
21     }
22     } catch (InterruptedException e) {}
23 });
24
25     phoebe.start();
26     chandler.start();
27 }
28 }

```

- Q14.** Mr. P puts products and Mr. C gets products from a shared inventory of size 4. P cannot put if full (4 products); C cannot get if empty. Write complete Java code using **wait** and **notifyAll** only — you are not allowed to use **BlockingQueue**. (20 marks) [CSE 107 | Jan 2021]

#### Answer — Mr. P & Mr. C: Inventory Size 4, wait/notifyAll Only

**BlockingQueue** is **not** used. All synchronisation is implemented with Java's built-in monitor (**synchronized** + **wait** + **notifyAll**).

```

1  class SharedInventory {
2      private final int MAX = 4;
3      private final int[] slots = new int[MAX];
4      private int count = 0;
5
6      // Called by P (Producer)
7      public synchronized void put(int item)
8          throws InterruptedException {
9          while (count == MAX) {          // inventory full -- P
10             waits
11             System.out.println("P waiting (inventory full)");
12             wait();
13         }
14         slots[count++] = item;
15         System.out.println("P put item " + item
16             + " [size=" + count + "]");
17         notifyAll();                    // wake C if it was
18         waiting
19     }
20
21     // Called by C (Consumer)
22     public synchronized int get() throws InterruptedException
23     {
24         while (count == 0) {            // inventory empty -- C
25             waits
26             System.out.println("C waiting (inventory empty)");
27         }
28     }
29 }

```

```
23         wait();
24     }
25     int item = slots[--count];
26     System.out.println("C got item " + item
27                       + " [size=" + count + "]);
28     notifyAll();           // wake P if it was
29     waiting
30     return item;
31 }
32
33 class Producer extends Thread {
34     private final SharedInventory inv;
35     Producer(SharedInventory inv) { this.inv = inv; }
36     @Override
37     public void run() {
38         try {
39             for (int i = 1; i <= 12; i++) {
40                 inv.put(i);
41                 Thread.sleep(50); // simulate production time
42             }
43         } catch (InterruptedException e) {}
44     }
45 }
46
47 class Consumer extends Thread {
48     private final SharedInventory inv;
49     Consumer(SharedInventory inv) { this.inv = inv; }
50     @Override
51     public void run() {
52         try {
53             for (int i = 0; i < 12; i++) {
54                 inv.get();
55                 Thread.sleep(150); // simulate consumption
56                                     time
57             }
58         } catch (InterruptedException e) {}
59     }
60 }
61
62 public class InventoryWaitNotify {
63     public static void main(String[] args) {
64         SharedInventory inv = new SharedInventory();
65         new Producer(inv).start();
66         new Consumer(inv).start();
67     }
68 }
```



**Key points:**

- `while` (not `if`) guards each `wait()` to handle spurious wake-ups correctly.
- `notifyAll()` is used instead of `notify()` to avoid the risk of notifying the wrong thread.
- Both `put` and `get` are **synchronized** on the same object (`inv`), so only one thread executes either method at a time.

**Q15.** Consider the `TestClass` with `f1()` (1s sleep) and `f2()` (2s sleep). Create two threads without changing `TestClass` or extending/implementing any class:

```

1  class TestClass {
2      public void f1() { for(int i=0;i<5;i++) { System.out.
        println(i); try { Thread.sleep(1000); } catch(
        InterruptedException e){} } }
3      public static void f2() { for(int i=0;i<5;i++) { System.
        out.println(i); try { Thread.sleep(2000); } catch(
        InterruptedException e){} } }
4  }
```

(10 marks)

[CSE 107 | Jan 2021]

### Answer — Two Threads via Lambda (No Subclassing)

Because we cannot extend or implement any class, we use **lambda expressions** as `Runnable` instances. `TestClass` is used as-is; a single object is created for the instance method `f1()`, while the static method `f2()` is referenced directly.

```

1  public class Main {
2      public static void main(String[] args) throws
        InterruptedException {
3          TestClass obj = new TestClass();
4
5          // Thread for f1() -- instance method, needs an
            object
6          Thread t1 = new Thread(() -> obj.f1(), "T1");
7
8          // Thread for f2() -- static method, no object needed
9          Thread t2 = new Thread(TestClass::f2, "T2");
10
11         t1.start();
12         t2.start();
13
14         t1.join();
15         t2.join();
16
17         System.out.println("Both threads finished.");
18     }
```

```
19 }
```

Why this satisfies all constraints:

- `TestClass` is not modified.
- No new class extends or implements anything — `() -> obj.f1()` is an anonymous lambda that the compiler treats as a `Runnable` without requiring an explicit `implements Runnable` declaration.
- Two separate threads (`t1` for `f1()`, `t2` for `f2()`) are created and run concurrently.

**Q16.** Explain with code how to fix the `SharedCounter` class so that multiple threads can correctly operate on it:

```
1 class SharedCounter {
2     private int counter;
3     SharedCounter() { this.counter = 0; }
4     private void increment() { this.counter++; }
5     public int get() { return this.counter; }
6     public void count() { for(int i=1;i<=1000;i++) increment()
7                          ; }
7 }
```

(5 marks)

[CSE 107 | 2020–21]

### Answer — Fixing SharedCounter for Thread Safety

#### The problem:

`counter++` is *not atomic* — it compiles to three operations: read, increment, write. Two threads can both read the same value, both increment it, and both write back, losing one update (a *lost update* race condition).

**Fix — make `increment()` synchronized:**

```
1 class SharedCounter {
2     private int counter;
3     SharedCounter() { this.counter = 0; }
4
5     // Fix: add synchronized so only one thread increments at
6     // a time
7     private synchronized void increment() { this.counter++; }
8
9     public int get() { return this.counter; }
10
11    public void count() {
12        for (int i = 1; i <= 1000; i++) increment();
13    }
13 }
```

```

14
15 public class TestSharedCounter {
16     public static void main(String[] args) throws
        InterruptedException {
17         SharedCounter sc = new SharedCounter();
18
19         Thread t1 = new Thread(sc::count, "T1");
20         Thread t2 = new Thread(sc::count, "T2");
21         Thread t3 = new Thread(sc::count, "T3");
22
23         t1.start(); t2.start(); t3.start();
24         t1.join();  t2.join();  t3.join();
25
26         System.out.println("Final count: " + sc.get()); //
            always 3000
27     }
28 }

```

Alternative fix — use AtomicInteger:

```

1 import java.util.concurrent.atomic.AtomicInteger;
2
3 class SharedCounterAtomic {
4     private AtomicInteger counter = new AtomicInteger(0);
5     private void increment() { counter.incrementAndGet(); }
6     public int get()         { return counter.get(); }
7     public void count()      { for (int i=1;i<=1000;i++)
        increment(); }
8 }

```

`AtomicInteger.incrementAndGet()` is a single hardware-level atomic operation and avoids the overhead of acquiring a monitor lock.

- Q17.** Write three different ways to create Threads in Java with short code examples. What is the difference between synchronized method and synchronized statement? (10 marks) [CSE 107 | 2019–20]

### Answer — Three Ways to Create Threads & Synchronization

#### Way 1: Extending Thread

```

1 class T1 extends Thread {
2     public void run() { System.out.println("Thread via
        extension"); }
3 }
4 // Usage: new T1().start();

```

#### Way 2: Implementing Runnable

```

1 class T2 implements Runnable {

```

```

2     public void run() { System.out.println("Thread via
           Runnable"); }
3 }
4 // Usage: new Thread(new T2()).start();

```

### Way 3: Using a Lambda expression (Java 8+)

Since `Runnable` is a functional interface, it can be expressed as a lambda — the most concise modern approach.

```

1 Thread t = new Thread(() -> System.out.println("Thread via
           Lambda"));
2 t.start();

```

### Synchronized method vs. synchronized statement

| Synchronized Method                                                                                                                                                                                                                  | Synchronized (Block)                                                                                                                                                                                                                                          | Statement |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| The <code>synchronized</code> keyword is placed in the method signature. The lock acquired is always <code>this</code> (or the <code>Class</code> object for <code>static</code> methods). Entire method body is mutually exclusive. | The <code>synchronized</code> keyword wraps only a <i>critical section</i> inside a method, and the programmer specifies <i>which object</i> to lock on. Only the enclosed block is mutually exclusive; the rest of the method runs without holding the lock. |           |
| Less flexible — always locks the current object.                                                                                                                                                                                     | More flexible — can lock on any object, enabling finer-grained concurrency.                                                                                                                                                                                   |           |

```

1 // Synchronized method
2 public synchronized void increment() {
3     count++;
4 }
5
6 // Synchronized statement -- locks only on the shared '
   counter' object
7 public void increment() {
8     // ... non-critical code runs here without holding a lock
9     ...
10    synchronized (counter) {
11        count++;
12    }
13    // ... more non-critical code ...
14 }

```

- Q18.** Divide summation of 1 to 10,000,000 equally among 10 different threads. Main thread waits and prints final sum. (10 marks) [CSE 107 | 2019–20]

### Answer — Summation of 1 to 10,000,000 Using 10 Threads

The range [1, 10,000,000] is divided into 10 equal sub-ranges of 1,000,000 numbers each. Each thread computes its partial sum and stores it in a shared array. The main thread calls `join()` on every worker before aggregating the results.

```
1 public class ParallelSum {
2
3     static final int TOTAL    = 10000000;
4     static final int THREADS = 10;
5     static final int RANGE    = TOTAL / THREADS; // 1,000,000
6         each
7
8     static long[] partial = new long[THREADS];
9
10    static class SumTask implements Runnable {
11        private final int id, start, end;
12        SumTask(int id, int start, int end) {
13            this.id = id; this.start = start; this.end = end;
14        }
15        @Override
16        public void run() {
17            long sum = 0;
18            for (int i = start; i <= end; i++) sum += i;
19            partial[id] = sum;
20        }
21    }
22
23    public static void main(String[] args) throws
24        InterruptedException {
25        Thread[] workers = new Thread[THREADS];
26        for (int i = 0; i < THREADS; i++) {
27            int s = i * RANGE + 1;
28            int e = s + RANGE - 1;
29            workers[i] = new Thread(new SumTask(i, s, e));
30            workers[i].start();
31        }
32        for (Thread t : workers) t.join(); // main waits
33
34        long total = 0;
35        for (long p : partial) total += p;
36        System.out.println("Sum = " + total); //
37            50,000,005,000,000
38    }
```

- Q19.** What are the advantages of multithreading over process-based multitasking? (6 marks) [CSE 107 | 2017–18; CSE 201 | 2012–13]

#### Answer — Multithreading vs. Process-Based Multitasking

**Process-based multitasking** (multiprocessing) runs multiple independent programs concurrently; each process has its own address space and system resources. **Thread-based multitasking** (multithreading) runs multiple threads *within the same process*, all sharing the same memory and resources.

#### Advantages of multithreading over process-based multitasking:

1. **Lower overhead:** Creating a thread is far cheaper than creating a process because threads share the heap, code segment, and open file descriptors of their parent process. No new virtual address space needs to be set up.
2. **Faster context switching:** Switching between threads of the same process is faster than switching between processes because the operating system does not need to swap the entire memory map — only the CPU registers and stack pointer change.
3. **Shared memory — easier communication:** Threads in the same process share the same address space, so data can be exchanged through shared variables without expensive inter-process communication (IPC) mechanisms such as pipes, sockets, or shared memory segments.
4. **Responsiveness:** In a multithreaded GUI application, one thread can handle user interaction while another performs a long computation, keeping the application responsive.
5. **Better CPU utilisation:** When one thread is blocked on I/O, the CPU can execute another thread of the same process, improving throughput without spawning new heavy-weight processes.
6. **Simplified program structure:** Tasks that are naturally concurrent (e.g. reading input and writing output simultaneously) can be modelled as separate threads within a single program, rather than as separate communicating processes.

- Q20.** Describe the methods Java uses for inter-thread communication. (9 marks) [CSE 107 | 2017–18]

#### Answer — Inter-Thread Communication in Java

Java's inter-thread communication is built on three methods defined in the `Object` class. Because every object has a monitor, these methods can be called on *any* object; however, they must always be called from within a `synchronized` block or method (otherwise Java throws `IllegalMonitorStateException`).

| Method                   | Description                                                                                                                                                                                                                                       |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>wait()</code>      | Causes the current thread to release the monitor lock and enter the <i>Waiting</i> state. The thread remains waiting until another thread calls <code>notify()</code> or <code>notifyAll()</code> on the same object, or until it is interrupted. |
| <code>notify()</code>    | Wakes up <i>one</i> arbitrary thread that is waiting on this object's monitor. The awakened thread must re-acquire the lock before it can proceed.                                                                                                |
| <code>notifyAll()</code> | Wakes up <i>all</i> threads waiting on this object's monitor. All awakened threads compete to reacquire the lock; only one succeeds at a time.                                                                                                    |

### Classic Producer–Consumer example:

```

1  class SharedBuffer {
2      private int data;
3      private boolean hasData = false;
4
5      // Called by Producer thread
6      public synchronized void produce(int val) throws
           InterruptedException {
7          while (hasData)           // wait if buffer is full
8              wait();
9          data = val;
10         hasData = true;
11         System.out.println("Produced: " + val);
12         notify();                 // wake up Consumer
13     }
14
15     // Called by Consumer thread
16     public synchronized int consume() throws
           InterruptedException {
17         while (!hasData)           // wait if buffer is empty
18             wait();
19         hasData = false;
20         System.out.println("Consumed: " + data);
21         notify();                 // wake up Producer
22         return data;
23     }
24 }

```

### Key rules:

- Always call `wait()` inside a `while` loop (not `if`), because of *spurious wake-ups* — a thread may wake up even without a `notify()`.
- Prefer `notifyAll()` over `notify()` when multiple threads may be waiting, to avoid potential deadlock from notifying the wrong thread.

**Q21.** What are the two ways of creating a thread? Give an example for each kind. Which one is better and why? (8+5 marks) [CSE 201 | 2011–12; CSE 107 | 2016–17]

→ Similar: CSE 201 | 2009–10, 2012–13

### Answer — Two Ways to Create a Thread in Java

Java provides two mechanisms to create a thread.

#### Way 1: Extending the Thread class

Create a subclass of Thread and override its run() method. The thread is started by calling start() on an instance of the subclass.

```
1  class MyThread extends Thread {
2      private String name;
3      MyThread(String name) { this.name = name; }
4
5      @Override
6      public void run() {
7          for (int i = 1; i <= 3; i++) {
8              System.out.println(name + " - count: " + i);
9          }
10     }
11 }
12
13 public class ThreadDemo1 {
14     public static void main(String[] args) throws
15         InterruptedException {
16         MyThread t1 = new MyThread("Thread-A");
17         MyThread t2 = new MyThread("Thread-B");
18         t1.start();
19         t2.start();
20         t1.join();    // main waits for t1 to finish
21         t2.join();    // main waits for t2 to finish
22         System.out.println("Main thread ends.");
23     }
24 }
```

#### Way 2: Implementing the Runnable interface

Implement the Runnable interface and pass an instance of it to a Thread object. The class is free to extend any other class simultaneously.

```
1  class MyRunnable implements Runnable {
2      private String name;
3      MyRunnable(String name) { this.name = name; }
4
5      @Override
6      public void run() {
7          for (int i = 1; i <= 3; i++) {
8              System.out.println(name + " - count: " + i);
9          }
10     }
11 }
```



```

10     }
11 }
12
13 public class ThreadDemo2 {
14     public static void main(String[] args) throws
        InterruptedException {
15         Thread t1 = new Thread(new MyRunnable("Thread-A"));
16         Thread t2 = new Thread(new MyRunnable("Thread-B"));
17         t1.start();
18         t2.start();
19         t1.join();
20         t2.join();
21         System.out.println("Main thread ends.");
22     }
23 }

```

**Which is better and why?**

Implementing `Runnable` is *generally preferred* for the following reasons:

1. **Single Inheritance constraint:** Java does not support multiple inheritance. If a class already extends another class, it *cannot* also extend `Thread`. Implementing `Runnable` imposes no such restriction.
2. **Separation of concerns:** `Runnable` separates the task logic from the thread mechanism, making code more modular and reusable.
3. **Resource efficiency:** The same `Runnable` object can be submitted to thread pools (e.g. `ExecutorService`), whereas a `Thread` subclass cannot be reused after termination.
4. **Better OOP design:** Extending `Thread` is appropriate only when you need to modify the threading behaviour itself, which is rarely required in practice.

**Q22.** Find the minimum of a 500-integer array using 5 threads in parallel. You are not allowed to write any more classes or arrays, but you can add normal variables, constructors, and other methods:

```

1  class ParallelMin implements Runnable { /* your code */ }
2  class Main {
3      public static void main(String[] args) {
4          Random random = new Random();
5          int[] numbers = new int[500];
6          for (int i = 0; i < numbers.length; i++)
7              numbers[i] = random.nextInt();
8          // your code
9      }
10 }

```

(10 marks)

[CSE 107 | 2016–17]

**Answer — Parallel Minimum of 500-Element Array (No Extra Arrays)**

The constraint forbids additional classes or arrays. The partial minimum of each thread is stored as an instance field of `ParallelMin` and accessed directly through the object reference after `join()`.

```

1  class ParallelMin implements Runnable {
2      private final int[] numbers;
3      private final int start, end;
4      int localMin;
5
6      ParallelMin(int[] numbers, int start, int end) {
7          this.numbers = numbers;
8          this.start = start;
9          this.end = end;
10         this.localMin = Integer.MAX_VALUE;
11     }
12
13     @Override
14     public void run() {
15         for (int i = start; i < end; i++)
16             if (numbers[i] < localMin) localMin = numbers[i];
17     }
18 }
19
20 class Main {
21     public static void main(String[] args) throws
22         InterruptedException {
23         Random random = new Random();
24         int[] numbers = new int[500];
25         for (int i = 0; i < numbers.length; i++)
26             numbers[i] = random.nextInt();
27
28         // 5 objects hold the partial results -- no extra
29         // array needed
30         int chunk = numbers.length / 5; // 100 each
31         ParallelMin pm0 = new ParallelMin(numbers, 0, 100);
32         ParallelMin pm1 = new ParallelMin(numbers, 100, 200);
33         ParallelMin pm2 = new ParallelMin(numbers, 200, 300);
34         ParallelMin pm3 = new ParallelMin(numbers, 300, 400);
35         ParallelMin pm4 = new ParallelMin(numbers, 400, 500);
36
37         Thread t0=new Thread(pm0), t1=new Thread(pm1),
38             t2=new Thread(pm2), t3=new Thread(pm3),
39             t4=new Thread(pm4);
40
41         t0.start(); t1.start(); t2.start(); t3.start(); t4.
42             start();
43         t0.join(); t1.join(); t2.join(); t3.join(); t4.

```

```

        join();
41
42         int globalMin = Math.min(pm0.localMin,
43                                 Math.min(pm1.localMin,
44                                 Math.min(pm2.localMin,
45                                 Math.min(pm3.localMin, pm4.localMin))
46                                 ));
47         System.out.println("Global minimum = " + globalMin);
48     }
49 }

```

**Q23.** You need to simulate the traffic behavior of different vehicles moving inside BUET campus. There are four types, i.e., Car, Microbus, AutoRickshaw, and Rickshaw, of vehicles run in the campus. Each vehicle type should have necessary properties, e.g., number of wheels, maximum speed, etc., which represent real vehicles that you see in real life. In the simulated environment you need to instantiate 10 vehicles of each type, where each vehicle has its current position and speed. Their initial positions and speeds will be randomly generated while you instantiate these vehicles. Vehicles will run along the road segments which are represented as a fixed set of rectangles. In every second, each vehicle updates its position based on the previous position and current speed. You need to make sure that a vehicle does not go outside the road segments at any point of time. Two vehicles must not collide with each other, i.e., if during a position update a vehicle finds another vehicle within a threshold range it cannot move and must need to decrease its speed.

Write a Java program to implement the above simulator. You have to run each vehicle as a thread. Please make your own assumption to build the whole scenario as realistic as possible to reflect the real-world scenario. **(35 marks)** [CSE 201 | 2015–16]

### Answer — BUET Campus Traffic Simulation

Each vehicle type is a subclass of an abstract `Vehicle` thread and carries type-specific properties (number of wheels, maximum speed). Initial positions and speeds are randomly generated. A shared `Road` object serialises position updates: if another vehicle is found within a configurable *threshold* range the vehicle stays put and its speed is decreased; if the road boundary would be exceeded the vehicle is clamped to the boundary.

```

1  import java.util.Random;
2
3  // -- Shared road segment
   -----
4  class Road {
5      static final int LENGTH    = 500;  // road length in
        metres

```

```

6      static final int THRESHOLD = 10;    // collision-detection
          range
7
8      // Coarse-grained lock: one position-update at a time
9      public synchronized int tryMove(int oldPos, int newPos,
10                                     int[] allPositions, int
                                         myIdx) {
11          // 1. Clamp to road boundaries
12          newPos = Math.max(0, Math.min(LENGTH - 1, newPos));
13
14          // 2. Check threshold range against every other
              vehicle
15          for (int i = 0; i < allPositions.length; i++) {
16              if (i == myIdx) continue;
17              if (Math.abs(allPositions[i] - newPos) <
                  THRESHOLD)
18                  return -1;    // blocked - caller must
                      decrease speed
19          }
20          return newPos;
21      }
22  }
23
24  // -- Abstract vehicle thread
      -----
25  abstract class Vehicle extends Thread {
26      protected int    position;
27      protected int    speed;           // metres per second
28      protected final int    maxSpeed;
29      protected final int    numWheels;
30      protected final Road    road;
31      protected final int[]  allPositions; // shared position
          array
32      protected final int    myIdx;
33      private    static final Random rng = new Random();
34
35      Vehicle(String name, int maxSpeed, int numWheels,
36              Road road, int[] allPositions, int idx) {
37          super(name);
38          this.maxSpeed    = maxSpeed;
39          this.numWheels    = numWheels;
40          this.road        = road;
41          this.allPositions = allPositions;
42          this.myIdx        = idx;
43          // Random initial position and speed
44          this.position    = rng.nextInt(Road.LENGTH);
45          this.speed        = 1 + rng.nextInt(maxSpeed);
46          allPositions[idx] = this.position;

```

```

47     }
48
49     @Override
50     public void run() {
51         try {
52             for (int step = 0; step < 30; step++) {
53                 int next = position + speed;
54                 int result = road.tryMove(position, next,
55                                         allPositions, myIdx
56                                         );
57
58                 if (result == -1) {
59                     // Blocked by nearby vehicle: decrease
60                     // speed
61                     speed = Math.max(1, speed - 1);
62                     System.out.printf("%-22s BLOCKED pos=%d
63                                     speed->%d%n",
64                                     getName(), position,
65                                     speed);
66
67                 } else {
68                     position = result;
69                     allPositions[myIdx] = position;
70                     // Gradually recover speed up to maximum
71                     if (speed < maxSpeed) speed++;
72                     System.out.printf("%-22s pos=%d speed=%d%
73                                     n",
74                                     getName(), position,
75                                     speed);
76
77                 }
78                 Thread.sleep(1000); // update every second
79             }
80         } catch (InterruptedException e) {
81             Thread.currentThread().interrupt();
82         }
83     }
84 }
85
86 // -- Concrete vehicle types (wheels, max speed are
87 // assumptions) ---
88 class Car extends Vehicle {
89     // 4 wheels, max speed 60 m/s (assumed scaled for
90     // simulation)
91     Car(Road r, int[] pos, int idx) {
92         super("Car-" + idx, 60, 4, r, pos, idx);
93     }
94 }
95
96 class Microbus extends Vehicle {
97     // 4 wheels, max speed 40 m/s
98     Microbus(Road r, int[] pos, int idx) {

```

```

87         super("Microbus-" + idx, 40, 4, r, pos, idx);
88     }
89 }
90 class AutoRickshaw extends Vehicle {
91     // 3 wheels, max speed 25 m/s
92     AutoRickshaw(Road r, int[] pos, int idx) {
93         super("AutoRickshaw-" + idx, 25, 3, r, pos, idx);
94     }
95 }
96 class Rickshaw extends Vehicle {
97     // 3 wheels, max speed 10 m/s
98     Rickshaw(Road r, int[] pos, int idx) {
99         super("Rickshaw-" + idx, 10, 3, r, pos, idx);
100    }
101 }
102
103 // -- Main
104 -----
104 public class BUETTraffic {
105     static final int PER_TYPE = 10;
106     static final int TOTAL    = PER_TYPE * 4;    // 40
107     vehicles
108
109     public static void main(String[] args) throws
110     InterruptedException {
111         Road road      = new Road();
112         int[] allPositions = new int[TOTAL];
113         Vehicle[] vehicles = new Vehicle[TOTAL];
114
115         int idx = 0;
116         for (int i = 0; i < PER_TYPE; i++)
117             vehicles[idx] = new Car(road, allPositions, idx
118                                     ++);
119         for (int i = 0; i < PER_TYPE; i++)
120             vehicles[idx] = new Microbus(road, allPositions,
121                                         idx++);
122         for (int i = 0; i < PER_TYPE; i++)
123             vehicles[idx] = new AutoRickshaw(road,
124                                             allPositions, idx++);
125         for (int i = 0; i < PER_TYPE; i++)
126             vehicles[idx] = new Rickshaw(road, allPositions,
127                                         idx++);
128
129         for (Vehicle v : vehicles) v.start();
130         for (Vehicle v : vehicles) v.join();
131
132         System.out.println("Simulation complete.");
133     }

```

```
128 }
```

### Key design decisions and guarantees:

- **Vehicle properties:** Each subclass fixes `numWheels` and `maxSpeed` to realistic values (Car: 4 wheels / 60; Microbus: 4 / 40; AutoRickshaw: 3 / 25; Rickshaw: 3 / 10).
- **Random initialisation:** Both `position` and `speed` are drawn from a uniform random distribution at construction time, satisfying the problem statement.
- **Boundary enforcement:** `Math.max/min` clamps `newPos` to `[0, LENGTH - 1]` so no vehicle ever leaves the road segment.
- **Threshold-based collision:** `tryMove()` scans all other vehicles; if any is within `THRESHOLD` cells of the intended new position the move is refused and the caller decrements its speed by 1 (minimum 1), matching the specification exactly.
- **Thread safety:** `tryMove()` is synchronized on the `Road` object, so the shared `allPositions[]` array is read and written atomically, preventing race conditions.
- **Periodic update:** Each vehicle calls `Thread.sleep(1000)` between steps, implementing the “every second” update cycle required by the problem.

**Q24.** Why are `suspend()`, `resume()` and `stop()` methods of `Thread` class deprecated? Demonstrate how you can achieve suspend, resume and stop behaviors in a thread that you have implemented, without using the deprecated methods. Write necessary code and provide explanations where appropriate. Do not use unnecessary/redundant `sleep`, `wait` or `notify` method calls. **(5+10 marks)** [CSE 107 | 2015–16]

### Answer — Deprecated Thread Methods and Safe Implementation

#### Why These Methods Are Deprecated:

- `stop()`:** It is inherently unsafe. It stops a thread abruptly by throwing a `ThreadDeath` exception up the stack, causing the thread to instantly release all locks/monitors it holds. If any shared objects protected by those locks were in an incomplete or transitioning state, they are exposed to other threads in a corrupted, inconsistent state.
- `suspend()` / `resume()`:** They are inherently deadlock-prone. When a thread is frozen via `suspend()`, it does **not** release any locks it currently holds. If the managing thread trying to call `resume()` requires access to any of those same locked resources before it can proceed, the application reaches an unrecoverable deadlock.

**Safe Design Pattern:** To accurately replace these behaviors without creating redundant or resource-heavy busy-waiting loops, state flags (**active** and **paused**) are declared as **volatile** to ensure visibility across threads. Inter-thread communication primitives (**wait()** and **notify()**) are deployed explicitly and selectively to suspend execution without hoarding monitors.

```

1  class SafeRunnable implements Runnable {
2      private volatile boolean active = true;
3      private volatile boolean paused = false;
4
5      @Override
6      public void run() {
7          while (active) {
8              // Synchronized block used strictly to safely
9              // park the thread
10             synchronized (this) {
11                 while (paused && active) {
12                     try {
13                         wait(); // Releases the monitor lock
14                             // while waiting
15                     } catch (InterruptedException e) {
16                         active = false;
17                     }
18                 }
19             }
20
21             if (!active) break;
22
23             // --- Actual core work of the thread goes here
24             ---
25             System.out.println(Thread.currentThread().getName
26                 () + " is running...");
27
28             try {
29                 Thread.sleep(200); // Standard processing
30                 // simulation interval
31             } catch (InterruptedException e) {
32                 active = false;
33             }
34
35             System.out.println(Thread.currentThread().getName() +
36                 " clean termination complete.");
37         }
38
39         // Safe substitute for suspend()
40         public synchronized void safeSuspend() {
41             paused = true;
42         }
43     }

```



```
37
38 // Safe substitute for resume()
39 public synchronized void safeResume() {
40     if (paused) {
41         paused = false;
42         notify(); // Target execution guard; only signals
                     // when thread is paused
43     }
44 }
45
46 // Safe substitute for stop()
47 public void safeStop() {
48     active = false;
49     safeResume(); // Unblocks the thread immediately if
                     // it was parked in wait()
50 }
51 }
52
53 public class Main {
54     public static void main(String[] args) throws
55         InterruptedException {
56         SafeRunnable runnable = new SafeRunnable();
57         Thread worker = new Thread(runnable, "WorkerThread");
58         worker.start();
59
60         Thread.sleep(600);
61         System.out.println("[Main] Suspending thread...");
62         runnable.safeSuspend();
63
64         Thread.sleep(1000);
65         System.out.println("[Main] Resuming thread...");
66         runnable.safeResume();
67
68         Thread.sleep(600);
69         System.out.println("[Main] Stopping thread cleanly...");
70         runnable.safeStop();
71
72         worker.join();
73 }
```

**Q25.** “Oracle recommends that calls to `wait()` should take place within a loop that checks the conditions” — Why? (5 marks) [CSE 107 | 2015–16]

**Answer — Why wait() Must Be Inside a Loop**

Oracle recommends placing `wait()` inside a `while` loop that re-checks the condition for two reasons:

**1. Spurious wake-ups**

The Java specification explicitly permits a thread to wake up from `wait()` *without any call to `notify()` or `notifyAll()`*. This is called a **spurious wake-up** and can occur due to low-level OS signal interruptions. If `wait()` is inside an `if`, the thread proceeds even though the condition it was waiting for may still be false.

**2. notifyAll() wakes too many threads**

When `notifyAll()` is used, *all* waiting threads wake up. Only one of them may actually find the condition satisfied; the rest must go back to waiting. An `if` check causes those threads to proceed incorrectly, while a `while` loop sends them back to `wait()`.

**Illustration:**

```
1  // WRONG -- if-based check
2  public synchronized void consume() throws
    InterruptedException {
3      if (!hasData) wait();    // spurious wake-up or notifyAll
        can
4                                  // bypass this check incorrectly
5      process(data);
6  }
7
8  // CORRECT -- while-based check
9  public synchronized void consume() throws
    InterruptedException {
10     while (!hasData) wait(); // always re-checks after waking
        up
11     process(data);
12 }
```

The `while` loop turns `wait()` into a self-correcting mechanism: no matter why the thread woke up — real notification, spurious wake, or `notifyAll()` — it re-evaluates the condition and waits again if the condition is still false.

**Q26.** You are using a 3rd party package called `Sophisticated` that defines the following classes:

- `SophisticatedProblemsSolver`
- `SophisticatedProblem`
- `SimpleResult`

`SophisticatedProblemsSolver` has a public method with the following signature:

```
public SimpleResult Solve(SophisticatedProblem problem)
```

Your scenario is multi-threaded but you are unsure if `Solve()` is thread-safe. The following classes written by you need to use this method:

- `CompetativeProgrammingProblemsSolver`
- `SoftwareDevelopmentProblemSolver`
- `PassInTheExamsProblemSolver`

Each of the above classes needs to interact with the same instance of `SophisticatedProblemsSolver`. The constructors of these classes are passed in a reference of `SophisticatedProblemsSolver` type. Demonstrate a solution to guarantee that your program will be thread-safe. Write necessary code and provide explanations where appropriate. (Do not use static variables in your solution. Note: You do not have source code for the package `Sophisticated`) **(15 marks)** [CSE 107 | 2015–16]

### Answer — Thread-Safe Cooperation on a Shared Object Reference

**Synchronization Strategy:** Since the problem states that the constructors of our classes must directly accept a reference of type `SophisticatedProblemsSolver`, we cannot wrap it in an external container class. Instead, because all three classes share the exact same object reference in memory, we can use a `**synchronized block**` anchored directly onto that shared reference (`synchronized(solver)`). This approach acts as a mutual-exclusion lock, ensuring that only one thread can invoke `Solve()` at any given time, thus eliminating data races without utilizing any prohibited static variables.

```

1  // --- Mock placeholders for the unmodifiable 3rd party
   package ---
2  class SophisticatedProblem {}
3  class SimpleResult {}
4  class SophisticatedProblemsSolver {
5      public SimpleResult Solve(SophisticatedProblem problem) {
6          // Unknown implementation detail -- assumed unsafe
7          return new SimpleResult();
8      }
9  }
10
11 // --- Class 1: Competitive Programming Problems Solver ---
12 class CompetativeProgrammingProblemsSolver implements
   Runnable {
13     private final SophisticatedProblemsSolver solver;
14
15     // Constructor accepts standard reference as specified
16     public CompetativeProgrammingProblemsSolver(
17         SophisticatedProblemsSolver solver) {
18         this.solver = solver;
19     }
20
21     @Override
22     public void run() {

```

```
22     SophisticatedProblem problem = new
23         SophisticatedProblem();
24     SimpleResult result;
25     // Synchronize on the shared instance to serialize
26     // calls to Solve()
27     synchronized (solver) {
28         result = solver.Solve(problem);
29     }
30     System.out.println("CP Solver finished processing.");
31 }
32
33 // --- Class 2: Software Development Problem Solver ---
34 class SoftwareDevelopmentProblemSolver implements Runnable {
35     private final SophisticatedProblemsSolver solver;
36
37     public SoftwareDevelopmentProblemSolver(
38         SophisticatedProblemsSolver solver) {
39         this.solver = solver;
40     }
41
42     @Override
43     public void run() {
44         SophisticatedProblem problem = new
45             SophisticatedProblem();
46         SimpleResult result;
47
48         synchronized (solver) {
49             result = solver.Solve(problem);
50         }
51         System.out.println("SD Solver finished processing.");
52     }
53 }
54
55 // --- Class 3: Pass In The Exams Problem Solver ---
56 class PassInTheExamsProblemSolver implements Runnable {
57     private final SophisticatedProblemsSolver solver;
58
59     public PassInTheExamsProblemSolver(
60         SophisticatedProblemsSolver solver) {
61         this.solver = solver;
62     }
63
64     @Override
65     public void run() {
66         SophisticatedProblem problem = new
67             SophisticatedProblem();
```

```
64         SimpleResult result;
65
66         synchronized (solver) {
67             result = solver.Solve(problem);
68         }
69         System.out.println("Exam Solver finished processing."
70                             );
71     }
72 }
73 // --- Demonstration Main Class ---
74 public class Main {
75     public static void main(String[] args) {
76         // Step 1: Create a single shared instance of the un-
77             // thread-safe solver
78         SophisticatedProblemsSolver sharedSolver = new
79             SophisticatedProblemsSolver();
80
81         // Step 2: Pass the same instance to all consumer
82             // constructors
83         Thread t1 = new Thread(new
84             CompetativeProgrammingProblemsSolver(sharedSolver)
85             );
86         Thread t2 = new Thread(new
87             SoftwareDevelopmentProblemSolver(sharedSolver));
88         Thread t3 = new Thread(new
89             PassInTheExamsProblemSolver(sharedSolver));
90
91         // Step 3: Execute tasks concurrently
92         t1.start();
93         t2.start();
94         t3.start();
95     }
96 }
```

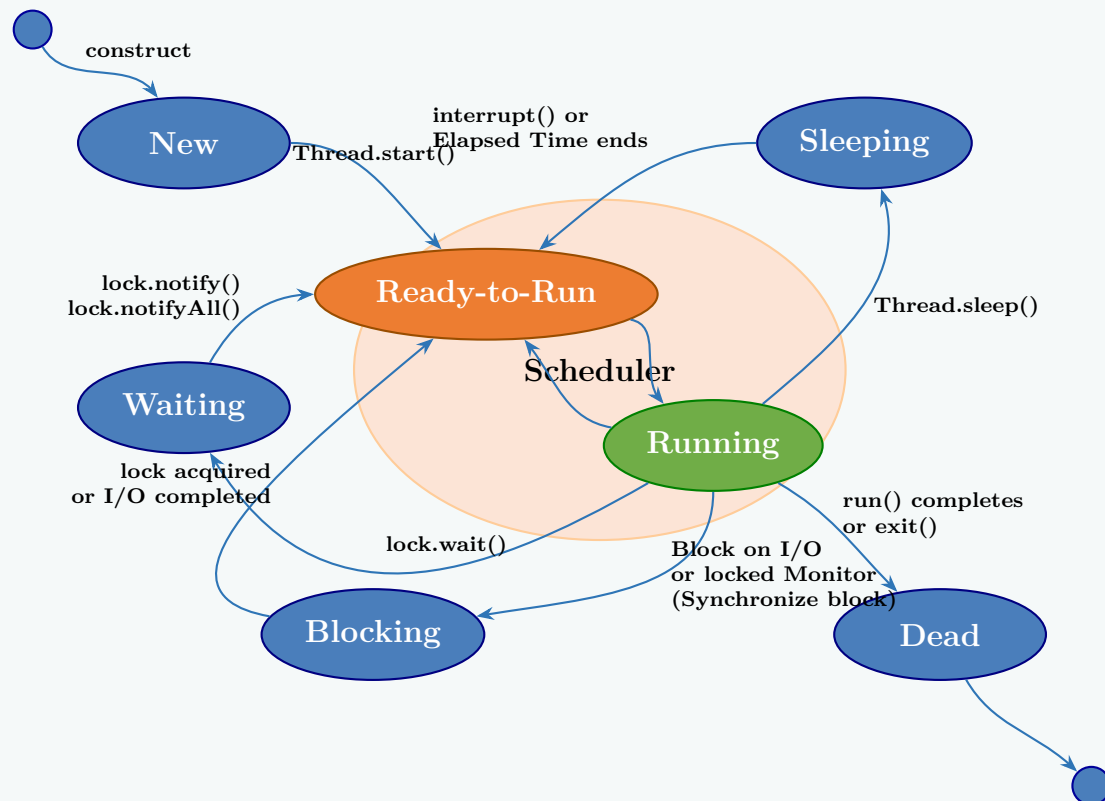
### Why this guarantees Thread Safety:

- **Object Monitor Locking:** Java objects have an intrinsic lock. By wrapping the invocation within `synchronized(solver)`, a thread must successfully acquire the monitor lock belonging to that specific `sharedSolver` instance before executing `Solve()`.
- **No Static Leakage:** The synchronization context relies entirely on the heap lifecycle instance passed through standard object-oriented instantiation, fully avoiding static state variables.
- **Black-Box Compliant:** It requires zero access to or changes inside the unmodifiable `Sophisticated` package source code.

**Q27.** Draw the life-cycle / schematic diagram of different states of a Thread. (9 marks)  
[CSE 201 | 2013–14, 2008–09, 2007–08]

### Answer — Thread Life-Cycle (State Diagram)

A Java thread passes through the following states during its lifetime:



#### Description of each state:

- New** — The thread object has been created (constructed), but `Thread.start()` has not yet been called.
- Ready-to-Run** — `Thread.start()` has been called. The thread is ready to execute and is waiting in a pool for the **Scheduler** to assign it CPU time.
- Running** — The **Scheduler** has selected the thread from the Ready-to-Run state and it is currently executing its `run()` method.
- Sleeping** — The thread transitions here when `Thread.sleep()` is called. It returns to the Ready-to-Run state once the specified elapsed time ends or if `interrupt()` is called.
- Waiting** — The thread goes into this state by calling `lock.wait()`. It remains here until another thread wakes it up by calling `lock.notify()` or `lock.notifyAll()`, after which it goes back to Ready-to-Run.

- 6. Blocking** — A thread is blocked when it is waiting for an I/O operation to complete or waiting to acquire a locked Monitor (e.g., entering a `synchronized` block). It returns to Ready-to-Run when the I/O is completed or the lock is acquired.
- 7. Dead** — The thread enters this final state when its `run()` method completes or it forcibly exits. A dead thread cannot be restarted.

**Q28.** Discuss “runnable” and “waiting” states of Thread briefly. **(10 marks)** [CSE 201 | 2013–14]

### Answer — Runnable and Waiting States

#### Runnable State

A thread enters the *Runnable* state as soon as `start()` is invoked on it. In this state the thread is *eligible to run* but is not necessarily executing at this instant; it is queued and awaiting CPU time from the JVM thread scheduler.

- A thread also returns to *Runnable* after waking up from a `sleep()`, after being notified out of a `wait()`, or after a blocking I/O operation completes.
- Multiple threads can be in the *Runnable* state simultaneously; the scheduler picks one to move to *Running*.
- A *Running* thread may be pre-empted (moved back to *Runnable*) at any time by the scheduler to give another thread a turn.

#### Waiting State

A thread enters the *Waiting* state when it voluntarily suspends itself and cannot proceed until some external condition is satisfied:

- `Object.wait()` — thread releases the monitor lock and waits until another thread calls `notify()` or `notifyAll()` on the same object.
- `Thread.join()` — the calling thread waits indefinitely for the target thread to terminate.
- `LockSupport.park()` — low-level parking used internally by `java.util.concurrent` utilities.

A *Waiting* thread holds **no** lock and consumes no CPU. It transitions back to *Runnable* only when:

- (i) another thread calls `notify()` / `notifyAll()`,
- (ii) the joined thread terminates, or
- (iii) the waiting thread is interrupted.

| Aspect      | Runnable           | Waiting                                |
|-------------|--------------------|----------------------------------------|
| CPU usage   | May use CPU        | No CPU usage                           |
| Lock held   | Possible           | No lock held                           |
| How to exit | Scheduler picks it | <code>notify()</code> / join completes |

**Q29.** Why may deadlocks occur while developing a user interface? Write a program to show how this can be mitigated. (4+7 marks) [CSE 201 | 2013–14]

### Answer — Deadlocks in UI and How to Mitigate Them

#### Why deadlocks occur in UI programs:

Java Swing (and AWT) is **single-threaded**: all UI operations must run on the *Event Dispatch Thread* (EDT). A deadlock occurs when:

- (i) A background thread holds a lock **A** and tries to update the UI, so it calls `SwingUtilities.invokeLater()` to schedule work on the EDT and then *blocks waiting* for the EDT to finish.
- (ii) The EDT is simultaneously trying to acquire lock **A** (e.g. to read shared data before a repaint).
- (iii) Both threads wait for each other — **deadlock**.

**Mitigation** — use `invokeLater` instead of `invokeAndWait`, and keep locks short:

```

1  import javax.swing.*;
2  import java.awt.*;
3
4  class SharedModel {
5      private String value = "initial";
6
7      public synchronized void setValue(String v) { value = v;
8          }
9      public synchronized String getValue() { return
10         value; }
11 }
12
13 public class UIDeadlockDemo {
14     public static void main(String[] args) {
15         SharedModel model = new SharedModel();
16         JLabel label = new JLabel(model.getValue());
17         JFrame frame = new JFrame("Safe UI Update");
18         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
19         frame.add(label, BorderLayout.CENTER);
20         frame.setSize(300, 100);
21         frame.setVisible(true);

```



```

21      // Background thread updates the model, then
        schedules
22      // UI refresh on EDT using invokeLater (non-blocking)
        .
23      // It does NOT hold any lock while calling
        invokeLater,
24      // so the EDT can freely acquire the model's lock.
        Thread worker = new Thread(() -> {
25          for (int i = 1; i <= 10; i++) {
26              model.setValue("Step " + i);    // short lock
27
28              // invokeLater: background thread does NOT
                block
29              String snapshot = model.getValue(); // read
                outside EDT
30              SwingUtilities.invokeLater(
31                  () -> label.setText(snapshot) // EDT
                    updates UI
32              );
33
34              try { Thread.sleep(500); }
35              catch (InterruptedException e) {}
36          }
37      });
38      worker.setDaemon(true);
39      worker.start();
40  }
41  }
42  }

```

#### Key rules to prevent UI deadlocks:

- Never call `invokeAndWait()` while holding a lock that the EDT might need.
- Prefer `invokeLater()` (fire-and-forget) over `invokeAndWait()` (blocking).
- Keep lock scopes as small as possible — read shared data, release the lock, then schedule the UI update.

**Q30.** Write a Java code to solve the Producer-Consumer problem where two threads share a common buffer of size 1. Use inter-thread communication with synchronized monitor. **(18 marks)** [CSE 201 | 2011–12]

↪ Similar: CSE 201 | 2007–08

#### Answer — Producer-Consumer: Buffer Size 1, Synchronized Monitor

```

1  class SharedBuffer {
2      private int data;

```

```
3     private boolean hasData = false;
4
5     // Producer calls this
6     public synchronized void produce(int value)
7         throws InterruptedException {
8         while (hasData) wait();    // wait until Consumer
9         reads
10        data    = value;
11        hasData = true;
12        System.out.println("Produced: " + value);
13        notifyAll();                // wake Consumer
14    }
15
16    // Consumer calls this
17    public synchronized int consume()
18        throws InterruptedException {
19        while (!hasData) wait();    // wait until Producer
20        writes
21        hasData = false;
22        System.out.println("Consumed: " + data);
23        notifyAll();                // wake Producer
24        return data;
25    }
26
27    class Producer extends Thread {
28        private final SharedBuffer buf;
29        Producer(SharedBuffer buf) { this.buf = buf; }
30        @Override
31        public void run() {
32            try {
33                for (int i = 1; i <= 10; i++) buf.produce(i);
34            } catch (InterruptedException e) {}
35        }
36    }
37
38    class Consumer extends Thread {
39        private final SharedBuffer buf;
40        Consumer(SharedBuffer buf) { this.buf = buf; }
41        @Override
42        public void run() {
43            try {
44                for (int i = 0; i < 10; i++) buf.consume();
45            } catch (InterruptedException e) {}
46        }
47    }
48
49    public class ProducerConsumerDemo {
```

```

49     public static void main(String[] args) {
50         SharedBuffer buf = new SharedBuffer();
51         new Producer(buf).start();
52         new Consumer(buf).start();
53     }
54 }

```

**How it works:**

- The buffer holds exactly one integer at a time (`hasData` flag acts as the full/empty indicator).
- `produce()` blocks with `wait()` if `hasData` is true; `consume()` blocks if `hasData` is false.
- After each read/write, `notifyAll()` unblocks the waiting partner thread.

**Q31.** Give an example of a multithreaded Java program using the `Runnable` interface. You must ensure that the main thread terminates last. Mention two different ways to achieve synchronization among threads in Java. **(6+4 marks)** [CSE 201 | 2010–11]

**Answer — Multithreaded Program with `Runnable`; Main Terminates Last**

```

1  class PrintTask implements Runnable {
2      private final String name;
3      PrintTask(String name) { this.name = name; }
4
5      @Override
6      public void run() {
7          for (int i = 1; i <= 4; i++) {
8              System.out.println(name + " -> step " + i);
9              try { Thread.sleep(100); } catch (
                InterruptedException e) {}
10         }
11         System.out.println(name + " finished.");
12     }
13 }
14
15 public class RunnableDemo {
16     public static void main(String[] args) {
17         Thread t1 = new Thread(new PrintTask("Worker-1"));
18         Thread t2 = new Thread(new PrintTask("Worker-2"));
19         Thread t3 = new Thread(new PrintTask("Worker-3"));
20
21         t1.start();
22         t2.start();
23         t3.start();

```

```

24
25         // join() ensures main waits until each worker
           finishes
26     try {
27         t1.join();
28         t2.join();
29         t3.join();
30     } catch (InterruptedException e) {
31         e.printStackTrace();
32     }
33
34     System.out.println("Main thread terminates last.");
35 }
36 }

```

**How main terminates last:** `Thread.join()` causes the calling thread (here, `main`) to suspend until the specified thread completes. By calling `join()` on every worker before printing the final message, we guarantee that all workers have finished before `main` exits.

#### Two ways to achieve synchronisation:

1. **Synchronized method** — declare a method with the `synchronized` keyword; the entire method body becomes a critical section locked on `this`.
2. **Synchronized statement (block)** — wrap only the shared-resource access inside `synchronized(obj){}`; allows a more targeted lock with a programmer-chosen object.

**Q32.** When is it a must to implement multithreading with the `Runnable` interface instead of extending the `Thread` class? Give a simple example of a multithreaded Java program using the `Runnable` interface. (15 marks) [CSE 201 | 2009–10]

#### Answer — When `Runnable` is Mandatory

##### When is `Runnable` mandatory?

It is *compulsory* to implement `Runnable` (rather than extending `Thread`) when the class that is to be run as a thread **already extends another class**.

Because Java supports only *single inheritance*, a class cannot extend both `Thread` and another base class simultaneously. Implementing `Runnable` is the only option in this case.

##### Example — a class that must extend another class:

```

1  // Suppose we have an existing base class
2  class Animal {

```

```
3     String sound;
4     Animal(String sound) { this.sound = sound; }
5     public void makeSound() { System.out.println(sound); }
6 }
7
8 // Dog must extend Animal AND run as a thread.
9 // Extending Thread is NOT possible here because Java has
10 // single inheritance. Implementing Runnable is the only way
11
12 class Dog extends Animal implements Runnable {
13     private int times;
14
15     Dog(String sound, int times) {
16         super(sound);
17         this.times = times;
18     }
19
20     @Override
21     public void run() {
22         for (int i = 0; i < times; i++) {
23             makeSound();
24             try { Thread.sleep(200); } catch (
25                 InterruptedException e) {}
26         }
27     }
28 }
29
30 public class RunnableMustDemo {
31     public static void main(String[] args) throws
32         InterruptedException {
33         Dog dog1 = new Dog("Woof!", 3);
34         Dog dog2 = new Dog("Bark!", 3);
35
36         Thread t1 = new Thread(dog1);
37         Thread t2 = new Thread(dog2);
38
39         t1.start();
40         t2.start();
41
42         t1.join();
43         t2.join();
44
45         System.out.println("All dogs done. Main exits last.");
46         ;
47     }
48 }
```

Other reasons to prefer Runnable even when not strictly required:

- Separates the task (what to do) from the thread (how to run it).
- The same `Runnable` instance can be submitted to thread pools (`ExecutorService`) without modification.
- Promotes better OOP design — extend a class only when you are specialising its behaviour.

**Q33.** Define a circular Queue class that can be safely shared by multiple threads. Define `Producer` and `Consumer` classes extending `Thread`. Write a `Test` class with one Queue but several producers and consumers. (20 marks) [CSE 201 | 2009–10]

### Answer — Thread-Safe Circular Queue with Multiple Producers & Consumers

```

1  class Queue {
2      private final int[] buffer;
3      private int head = 0, tail = 0, count = 0;
4      private final int capacity;
5
6      Queue(int capacity) {
7          this.capacity = capacity;
8          this.buffer = new int[capacity];
9      }
10
11     public synchronized void enqueue(int item)
12         throws InterruptedException {
13         while (count == capacity) wait(); // full -- producer
14             waits
15         buffer[tail] = item;
16         tail = (tail + 1) % capacity; // wrap around
17         count++;
18         System.out.println(Thread.currentThread().getName()
19             + " enqueued " + item);
20         notifyAll();
21     }
22
23     public synchronized int dequeue() throws
24         InterruptedException {
25         while (count == 0) wait(); // empty --
26             consumer waits
27         int item = buffer[head];
28         head = (head + 1) % capacity; // wrap around
29         count--;
30         System.out.println(Thread.currentThread().getName()
31             + " dequeued " + item);
32         notifyAll();
33         return item;
34     }

```

```
31     }
32 }
33
34 class Producer extends Thread {
35     private final Queue queue;
36     private final int id;
37     Producer(Queue queue, int id) {
38         super("Producer-" + id);
39         this.queue = queue; this.id = id;
40     }
41     @Override
42     public void run() {
43         try {
44             for (int i = 0; i < 5; i++) {
45                 queue.enqueue(id * 100 + i);
46                 Thread.sleep(100);
47             }
48         } catch (InterruptedException e) {}
49     }
50 }
51
52 class Consumer extends Thread {
53     private final Queue queue;
54     Consumer(Queue queue, int id) {
55         super("Consumer-" + id);
56         this.queue = queue;
57     }
58     @Override
59     public void run() {
60         try {
61             for (int i = 0; i < 5; i++) {
62                 queue.dequeue();
63                 Thread.sleep(150);
64             }
65         } catch (InterruptedException e) {}
66     }
67 }
68
69 class Test {
70     public static void main(String[] args) {
71         Queue q = new Queue(5); // circular buffer of size 5
72
73         // Multiple producers and consumers
74         Producer p1 = new Producer(q, 1);
75         Producer p2 = new Producer(q, 2);
76         Consumer c1 = new Consumer(q, 1);
77         Consumer c2 = new Consumer(q, 2);
78     }
```

```

79         p1.start(); p2.start();
80         c1.start(); c2.start();
81     }
82 }

```

**Design notes:**

- The circular buffer uses `head`, `tail`, and `count` to track the queue state; modular arithmetic (`% capacity`) wraps the pointers.
- All shared state is accessed inside `synchronized` methods, ensuring mutual exclusion.
- `notifyAll()` wakes all waiting threads; each then re-checks its `while` condition before proceeding.

**Q34.** What do you know about Thread Synchronization? What are the two ways to achieve synchronization? (8 marks) [CSE 201 | 2008–09]

**Answer — Thread Synchronisation****What is Thread Synchronisation?**

When two or more threads share access to a common resource (e.g. a shared variable, file, or data structure), concurrent modifications can leave the resource in an inconsistent state — a situation called a *race condition*. **Thread synchronisation** is the mechanism by which only *one thread at a time* is permitted to access a critical section of code, thereby preventing race conditions and ensuring data integrity.

Java implements synchronisation using *monitors*: every Java object has an implicit monitor (intrinsic lock). A thread that enters a `synchronized` region must first acquire the object's monitor; all other threads trying to enter the same region block until the lock is released.

**Two ways to achieve synchronisation in Java:****Way 1 — Synchronized method**

Place the `synchronized` keyword in the method signature. The lock is the `this` reference (or the `Class` object for `static` methods).

```

1  class SharedBank {
2      private double balance;
3
4      public synchronized void deposit(double amount) {
5          balance += amount;
6      }
7      public synchronized double getBalance() {
8          return balance;
9      }
10 }

```

**Way 2 — Synchronized statement (block)**



Enclose only the critical section with `synchronized(lockObj)`. The programmer explicitly specifies the lock object.

```
1  class SharedBank {
2      private double balance;
3      private final Object lock = new Object();
4
5      public void deposit(double amount) {
6          // logging, validation, etc. run here -- no lock held
7          synchronized (lock) {
8              balance += amount;
9          }
10     }
11 }
```

### Comparison:

| Synchronized method                     | Synchronized block                  |
|-----------------------------------------|-------------------------------------|
| Entire method is locked                 | Only the critical section is locked |
| Lock object is always <code>this</code> | Any object can be the lock          |
| Simpler to write                        | More flexible and fine-grained      |
| Higher contention                       | Lower contention                    |

BUET · CSE 107 / CSE 201

# Object Oriented Programming

Section S15

---

Generics & Collections

## S15 — Generics & Collections

**Q1.** Implement the `MyMax` class to achieve the provided output for the main method with the following restrictions:

- (a) `MyMax` class can only be created for Java number classes.
- (b) The implementation of methods `sameMax1` and `sameMax2` will be different.

```

1  public class MyMaxTest {
2      public static void main(String[] args) {
3          Long[] n1 = {10L, 20L, 30L, 40L, 50L};
4          MyMax<Long> mx1 = new MyMax<>(n1);
5          System.out.println(mx1.max()); // 50.0
6
7          Float[] n2 = {50.0f, 20.0f, 40.0f, 10.0f, 30.0f};
8          MyMax<Float> mx2 = new MyMax<>(n2);
9          System.out.println(mx2.max()); // 50.0
10
11         Double[] n3 = {50.0, 20.0, 40.0, 10.0, 30.0};
12         MyMax<Double> mx3 = new MyMax<>(n3);
13         System.out.println(mx3.max()); // 50.0
14
15         System.out.println(mx1.sameMax1(mx2)); // true
16         System.out.println(mx2.sameMax2(mx3)); // true
17     }
18 }

```

(10 marks)

[CSE 107 | 2023–24]

### Answer — MyMax with sameMax1 and sameMax2

```

1  class MyMax<T extends Number & Comparable<T>> {
2      private T[] data;
3
4      MyMax(T[] data) { this.data = data; }
5
6      double max() {
7          T m = data[0];
8          for (T t : data)
9              if (t.compareTo(m) > 0) m = t;
10         return m.doubleValue();
11     }
12
13     // sameMax1: compares via Double.compare (exact equality
14     // check)
15     boolean sameMax1(MyMax<? extends Number> other) {
16         return Double.compare(this.max(), other.max()) == 0;
17     }
18 }

```

```

18      // sameMax2: compares via subtraction with epsilon
           tolerance
19      boolean sameMax2(Object o) {
20          if(!(o instanceof MyMax)) return false;
21          MyMax<? extends Number> other = (MyMax<? extends
                Number>) o;
22
23          return Math.abs(this.max() - other.max()) < 1e-9;
24      }
25  }

```

**Q2.** Write two/three differences between `ArrayList` and `Vector`. (10 marks) [CSE 107 | 2016–17, 2021–22, 2022–23]

↪ Similar: CSE 107 / Jan 2021, 2020–21

#### Answer — ArrayList vs Vector

| Feature         | ArrayList                                              | Vector                                 |
|-----------------|--------------------------------------------------------|----------------------------------------|
| Synchronization | Not synchronized (not thread-safe)                     | Synchronized (thread-safe)             |
| Performance     | Faster (no sync overhead)                              | Slower due to locking                  |
| Growth          | Grows by 50% when full                                 | Doubles in size when full              |
| Legacy          | Introduced in Java 1.2 (modern)                        | Legacy class (Java 1.0)                |
| Iterator        | Uses <code>Iterator</code> / <code>ListIterator</code> | Also supports <code>Enumeration</code> |

**Recommendation:** Prefer `ArrayList` for single-threaded use. For thread safety, use `Collections.synchronizedList()` or `CopyOnWriteArrayList` instead of `Vector`.

**Q3.** Write three differences between `Hashtable` and `HashMap`. (10 marks) [CSE 107 | 2016–17, 2017–18, 2019–20, 2022–23]

#### Answer — Hashtable vs HashMap

| Feature          | Hashtable                         | HashMap                                    |
|------------------|-----------------------------------|--------------------------------------------|
| Null keys/values | Neither key nor value can be null | One null key, multiple null values allowed |
| Synchronization  | Synchronized (thread-safe)        | Not synchronized                           |
| Performance      | Slower due to locking             | Faster                                     |
| Legacy           | Legacy class (Java 1.0)           | Modern (Java 1.2)                          |
| Iteration order  | No guaranteed order               | No guaranteed order                        |

**Example showing null:**

```

1  HashMap<String, Double> map = new HashMap<>();

```

```

2 map.put(null, 100.0);           // OK
3 map.put("Alice", null);        // OK
4
5 Hashtable<String, Double> ht = new Hashtable<>();
6 // ht.put(null, 100.0);        // NullPointerException!
7 // ht.put("Alice", null);      // NullPointerException!

```

**Q4.** Create an appropriate Hash-based collection to store IPL players (Pat Cummins: 17,300,000, Kane Williamson: 16,000,000, MS Dhoni: null). Iterate and print values. Increase Pat Cummins's salary by 500,000. **(10 marks)** [CSE 107 | 2022–23]

↪ *Similar: CSE 107 | 2016–17, 2017–18*

### Answer — IPL Players HashMap (Supports null Values)

Since one salary is null, HashMap is required (Hashtable does not allow null values).

```

1 import java.util.HashMap;
2 import java.util.Map;
3
4 class IPLDemo {
5     public static void main(String[] args) {
6         HashMap<String, Integer> players = new HashMap<>();
7         players.put("Pat Cummins", 17_300_000);
8         players.put("Kane Williamson", 16_000_000);
9         players.put("MS Dhoni", null);           // null
10            salary
11
12        // Iterate and print
13        System.out.println("--- IPL Salaries ---");
14        for (Map.Entry<String, Integer> e : players.entrySet())
15            System.out.println(e.getKey() + ": " + e.getValue());
16
17        // Increase Pat Cummins's salary by 500,000
18        players.put("Pat Cummins",
19            players.get("Pat Cummins") + 500_000);
20
21        System.out.println("\nAfter raise:");
22        System.out.println("Pat Cummins: " + players.get("Pat
23            Cummins"));
24    }
25 }

```

**Output:**

```

--- IPL Salaries ---
Pat Cummins: 17300000

```

Kane Williamson: 16000000  
MS Dhoni: null

After raise:  
Pat Cummins: 17800000

- Q5.** Write a generic interface `iQueue` with `enqueue`, `dequeue`, `isEmpty`. Write a generic class `Queue` implementing `iQueue`. The interface only supports numeric types. **(10 marks)** [CSE 107 | 2019–20, 2022–23]

#### Answer — Generic `iQueue` Interface and `Queue` Class (Numeric Only)

```

1  import java.util.LinkedList;
2
3  // Bounded to Number subclasses (numeric types only)
4  interface iQueue<T extends Number> {
5      void enqueue(T item);
6      T dequeue();
7      boolean isEmpty();
8  }
9
10 class Queue<T extends Number> implements iQueue<T> {
11     private LinkedList<T> list = new LinkedList<>();
12
13     @Override
14     public void enqueue(T item) { list.addLast(item); }
15
16     @Override
17     public T dequeue() {
18         if (isEmpty())
19             throw new RuntimeException("Queue is empty");
20         return list.removeFirst();
21     }
22
23     @Override
24     public boolean isEmpty() { return list.isEmpty(); }
25
26     @Override
27     public String toString() { return list.toString(); }
28 }
29
30 class QueueDemo {
31     public static void main(String[] args) {
32         Queue<Integer> q = new Queue<>();
33         q.enqueue(10); q.enqueue(20); q.enqueue(30);
34         System.out.println("Queue: " + q);
35         System.out.println("Dequeue: " + q.dequeue());

```

```

36         System.out.println("Queue: " + q);
37
38         Queue<Double> dq = new Queue<>();
39         dq.enqueue(1.5); dq.enqueue(2.5);
40         System.out.println("Double Queue: " + dq);
41     }
42 }

```

**Output:**

```

Queue: [10, 20, 30]
Dequeue: 10
Queue: [20, 30]
Double Queue: [1.5, 2.5]

```

**Q6.** Implement the `MyStats` class taking `List` as input. Can only be created for Java number classes. `sameAvg1` and `sameAvg2` must have different implementations:

```

1  Integer[] n1 = {1, 2, 3, 4, 5};
2  List<Integer> list1 = Arrays.asList(n1);
3  MyStats<Integer> s1 = new MyStats<>(list1);
4  System.out.println(s1.average()); // 3.0
5  Long[] n2 = {5L, 2L, 4L, 1L, 3L};
6  List<Long> list2 = Arrays.asList(n2);
7  MyStats<Long> s2 = new MyStats<>(list2);
8  System.out.println(s2.average()); // 3.0
9  Double[] n3 = {5.0, 4.0, 3.0, 2.0, 1.0};
10 List<Double> list3 = Arrays.asList(n3);
11 MyStats<Double> s3 = new MyStats<>(list3);
12 System.out.println(s1.sameAvg1(s2)); // true
13 System.out.println(s2.sameAvg2(s3)); // true

```

**(10 marks)**

[CSE 107 | Jan 2021, 2022–23]

**Answer — MyStats Taking List as Input**

```

1  import java.util.*;
2
3  class MyStats<T extends Number> {
4      private List<T> data;
5
6      MyStats(List<T> data) {
7          this.data = data;
8      }
9
10     double average() {
11         double sum = 0.0;
12         for (T t : data) {
13             sum += t.doubleValue();

```

```

14         }
15         return sum / data.size();
16     }
17
18     // sameAvg1: Implementation using a Wildcard (?)
19     boolean sameAvg1(MyStats<?> other) {
20         return Math.abs(this.average() - other.average()) < 1
21             e-9;
22     }
23
24     // sameAvg2: Implementation using a Generic Method
25     // Parameter (V)
26     <V extends Number> boolean sameAvg2(MyStats<V> other) {
27         return Math.abs(this.average() - other.average()) < 1
28             e-9;
29     }
30 }
31
32 class MyStatsList {
33     public static void main(String[] args) {
34         List<Integer> list1 = Arrays.asList(1, 2, 3, 4, 5);
35         MyStats<Integer> s1 = new MyStats<>(list1);
36         System.out.println(s1.average()); // 3.0
37
38         List<Long> list2 = Arrays.asList(5L, 2L, 4L, 1L, 3L);
39         MyStats<Long> s2 = new MyStats<>(list2);
40         System.out.println(s2.average()); // 3.0
41
42         List<Double> list3 = Arrays.asList(5.0, 4.0, 3.0,
43             2.0, 1.0);
44         MyStats<Double> s3 = new MyStats<>(list3);
45
46         System.out.println(s1.sameAvg1(s2)); // true
47         System.out.println(s2.sameAvg2(s3)); // true
48     }
49 }

```

**Q7.** Write two differences between `ArrayList` and `Vector`. Consider `Employee` (attributes: `id`, `name`, `age`). Write necessary Java code so that `Collections.sort(listEmployees)` sorts by ascending `id`; if equal, by lexicographic `name`; if both equal, by ascending `age`. (10 marks) [CSE 107 | 2022–23]

**Answer — Employee Sort: `id`, `name`, `age`**

**Two `ArrayList` vs `Vector` differences:** `ArrayList` is not synchronized (faster); `Vector` is. `ArrayList` grows 50%; `Vector` doubles.



```

1  import java.util.*;
2
3  class Employee implements Comparable<Employee> {
4      int id; String name; int age;
5      Employee(int id, String name, int age) {
6          this.id=id; this.name=name; this.age=age;
7      }
8
9      @Override
10     public int compareTo(Employee o) {
11         int c = Integer.compare(this.id, o.id);           // asc
12         id
13         if (c != 0) return c;
14         c = this.name.compareTo(o.name);                 // lex
15         name
16         if (c != 0) return c;
17         return Integer.compare(this.age, o.age);          // asc
18         age
19     }
20
21     @Override
22     public String toString() {
23         return id + "-" + name + "(" + age + ")";
24     }
25 }
26
27 class EmployeeSort {
28     public static void main(String[] args) {
29         ArrayList<Employee> list = new ArrayList<>();
30         list.add(new Employee(2, "Zara", 25));
31         list.add(new Employee(1, "Bob", 22));
32         list.add(new Employee(1, "Alice", 30));
33         list.add(new Employee(1, "Alice", 25));
34
35         Collections.sort(list);
36         list.forEach(System.out::println);
37     }
38 }

```

**Output:**

```

1-Alice(25)
1-Alice(30)
1-Bob(22)
2-Zara(25)

```

**Q8.** Implement the `MyAvg` class. Can only be created for Java number classes. It must support `average()`, `sameAvg()` (comparing two same-type objects), and `sameAvgAny()`

(comparing two different-type objects):

```

1 Integer[] n1 = {10, 20, 30, 40, 50};
2 MyAvg<Integer> s1 = new MyAvg<>(n1);
3 System.out.println(s1.average()); // 30.0
4 Integer[] n2 = {50, 20, 40, 10, 30};
5 MyAvg<Integer> s2 = new MyAvg<>(n2);
6 System.out.println(s2.average()); // 30.0
7 System.out.println(s1.sameAvg(s2)); // true
8 Double[] n3 = {50.0, 40.0, 30.0, 20.0, 10.0};
9 MyAvg<Double> s3 = new MyAvg<>(n3);
10 System.out.println(s3.average()); // 30.0
11 System.out.println(s2.sameAvgAny(s3)); // true

```

(10 marks)

[CSE 107 | 2021–22]

### Answer — MyAvg with sameAvg and sameAvgAny

```

1 class MyAvg<T extends Number> {
2     private T[] data;
3
4     MyAvg(T[] data) { this.data = data; }
5
6     double average() {
7         double sum = 0;
8         for (T t : data) sum += t.doubleValue();
9         return sum / data.length;
10    }
11
12    // sameAvg: same type T
13    boolean sameAvg(MyAvg<T> other) {
14        return Double.compare(this.average(), other.average())
15            == 0;
16    }
17
18    // sameAvgAny: any numeric type (wildcard)
19    boolean sameAvgAny(MyAvg<? extends Number> other) {
20        return Math.abs(this.average() - other.average()) < 1
21            e-9;
22    }
23 }
24
25 class MyAvgDemo {
26     public static void main(String[] args) {
27         Integer[] n1 = {10, 20, 30, 40, 50};
28         MyAvg<Integer> s1 = new MyAvg<>(n1);
29         System.out.println(s1.average()); // 30.0
30
31         Integer[] n2 = {50, 20, 40, 10, 30};
32         MyAvg<Integer> s2 = new MyAvg<>(n2);

```

```

31         System.out.println(s2.average());           // 30.0
32         System.out.println(s1.sameAvg(s2));         // true
33
34         Double[] n3 = {50.0, 40.0, 30.0, 20.0, 10.0};
35         MyAvg<Double> s3 = new MyAvg<>(n3);
36         System.out.println(s3.average());           // 30.0
37         System.out.println(s2.sameAvgAny(s3));       // true
38     }
39 }

```

**Q9.** Given the following Product class, write Java code to: (i) define `ArrayList<Product> myProducts`, (ii) generate 4 Products with names 'A' to 'D' and random prices, (iii) sort `myProducts` by name in ascending order (you may modify the Product class if necessary):

```

1  class Product {
2      private String name; private double price;
3      Product(String name, double price) { this.name = name;
4          this.price = price; }
5      public String getName() { return this.name; }
6      public double getPrice() { return this.price; }
7  }

```

(10 marks)

[CSE 107 | Jan 2021]

### Answer — Product Sort by Name Ascending

```

1  import java.util.*;
2
3  class Product implements Comparable<Product> {
4      private String name; private double price;
5      Product(String name, double price) { this.name=name; this
6          .price=price; }
7      public String getName() { return name; }
8      public double getPrice() { return price; }
9
10     @Override
11     public int compareTo(Product o) {
12         return this.name.compareTo(o.name); // ascending by
13         name
14     }
15
16     @Override
17     public String toString() {
18         return name + "($ " + String.format("%.2f", price) + "
19         )";
20     }
21 }

```

```

18 }
19
20 class ProductSortByName {
21     public static void main(String[] args) {
22         Random rand = new Random();
23
24         // (i) Define ArrayList
25         ArrayList<Product> myProducts = new ArrayList<>();
26
27         // (ii) Generate 4 products D, B, A, C with random
28             prices
29         String[] names = {"D", "B", "A", "C"};
30         for (String n : names)
31             myProducts.add(new Product(n, 10 + rand.
32                 nextDouble() * 90));
33
34         System.out.println("Before: " + myProducts);
35
36         // (iii) Sort by name ascending
37         Collections.sort(myProducts);
38
39         System.out.println("After: " + myProducts);
40     }
41 }

```

**Output (example):**

Before: [D(\$45.23), B(\$78.10), A(\$12.50), C(\$67.89)]

After: [A(\$12.50), B(\$78.10), C(\$67.89), D(\$45.23)]

**Q10.** Write a generic interface `iStack` with `push`, `pop`, `isEmpty`. Write a generic class `Stack` implementing `iStack`. Interface only supports numeric types. **(10 marks)**  
[CSE 107 | 2020–21, 2016–17]

↪ Similar: CSE 107 | 2022–23

#### Answer — Generic `iStack` Interface and `Stack` Class

```

1  import java.util.ArrayList;
2
3  interface iStack<T extends Number> {
4      void    push(T item);
5      T      pop();
6      boolean isEmpty();
7  }
8
9  class Stack<T extends Number> implements iStack<T> {
10     private ArrayList<T> data = new ArrayList<>();
11

```

```

12     @Override
13     public void push(T item) { data.add(item); }
14
15     @Override
16     public T pop() {
17         if (isEmpty()) throw new RuntimeException("Stack is
18             empty");
19         return data.remove(data.size() - 1);
20     }
21
22     @Override
23     public boolean isEmpty() { return data.isEmpty(); }
24
25     @Override
26     public String toString() { return data.toString(); }
27 }
28
29 class StackDemo {
30     public static void main(String[] args) {
31         Stack<Integer> s = new Stack<>();
32         s.push(1); s.push(2); s.push(3);
33         System.out.println("Stack: " + s);
34         System.out.println("Pop: " + s.pop());
35         System.out.println("Pop: " + s.pop());
36         System.out.println("Stack: " + s);
37         System.out.println("Empty: " + s.isEmpty());
38     }
39 }

```

**Output:**

```

Stack: [1, 2, 3]
Pop: 3
Pop: 2
Stack: [1]
Empty: false

```

**Q11.** What can you say about the following statement?

```

1 public class ABC<S extends Z & Y & X> { }

```

(5 marks)

[CSE 107 | 2020–21, 2016–17]

### Answer — Bounded Type Parameter with Multiple Bounds

The statement declares a generic class ABC with a type parameter S that is **bounded by multiple types**: S extends Z & Y & X.

**Rules for multiple bounds:**

- S must be a subtype of **all** listed types.
- **At most one** bound can be a class; the rest must be interfaces. The class bound must come **first**.
- If Z is a class, Y and X must be interfaces. If all are interfaces, any order is valid.

```

1 interface Y { void doY(); }
2 interface X { void doX(); }
3 class Z      { void doZ() {} }
4
5 // S must extend Z and implement both Y and X
6 public class ABC<S extends Z & Y & X> {
7     void use(S s) {
8         s.doZ(); s.doY(); s.doX(); // all accessible
9     }
10 }
11
12 class Concrete extends Z implements Y, X {
13     public void doY() {}
14     public void doX() {}
15 }
16
17 // ABC<Concrete> ok = new ABC<>(); // valid

```

**Q12.** Write three differences between `HashMap` and `Hashtable`. Create an appropriate Hash-based collection to store the following monthly salaries: Sakib Al Hasan: 3,00,000; Mashrafe Bin Mortaza: null; Mushfiqur Rahim: 2,50,000. Then increase Sakib Al Hasan's salary by 50,000. **(3+10 marks)** [CSE 107 | 2017–18]

### Answer — Cricketer Salaries with `HashMap`

**Three differences:** `HashMap` allows null key/value; `Hashtable` does not. `HashMap` is unsynchronized (faster); `Hashtable` is synchronized. `HashMap` is modern (Java 1.2); `Hashtable` is legacy (Java 1.0).

```

1 import java.util.HashMap;
2 import java.util.Map;
3
4 class CricketerSalary {
5     public static void main(String[] args) {
6         // HashMap chosen because Mashrafe's salary is null
7         HashMap<String, Integer> salaries = new HashMap<>();
8         salaries.put("Sakib Al Hasan", 300_000);
9         salaries.put("Mashrafe Bin Mortaza", null);
10        salaries.put("Mushfiqur Rahim", 250_000);
11    }
12 }

```

```

12         System.out.println("--- Monthly Salaries ---");
13         for (Map.Entry<String, Integer> e : salaries.entrySet())
14             System.out.println(e.getKey() + ": " + e.getValue());
15
16         // Increase Sakib Al Hasan's salary by 50,000
17         salaries.put("Sakib Al Hasan",
18                     salaries.get("Sakib Al Hasan") + 50_000);
19
20         System.out.println("\nSakib after raise: "
21                             + salaries.get("Sakib Al Hasan"));
22     }
23 }

```

**Output:**

```

--- Monthly Salaries ---
Sakib Al Hasan: 300000
Mashrafe Bin Mortaza: null
Mushfiqur Rahim: 250000

Sakib after raise: 350000

```

**Q13.** Write the general syntax for: (i) declaring a generic class in Java, (ii) declaring a reference to a generic class and creating an instance. **(10 marks)** [CSE 107 | 2017–18]

**Answer — Java generic class syntax**

```

1  // (i) Declaring a generic class in Java:
2  class MyGeneric<T> {
3      private T data;
4      public MyGeneric(T val) { data = val; }
5      public T getData()      { return data; }
6  }
7
8  // Bounded type parameter:
9  class Pair<T extends Comparable<T>> {
10     T first, second;
11     Pair(T a, T b){ first=a; second=b; }
12     T max(){ return first.compareTo(second)>=0 ? first :
13             second; }
14 }
15
16 // (ii) Declaring a reference and creating an instance:
17 MyGeneric<Integer> intObj = new MyGeneric<>(42);

```

```

17 MyGeneric<String>    strObj  = new MyGeneric<>("Hello");
18 Pair<Double>         pair    = new Pair<>(3.14, 2.71);
19
20 System.out.println(intObj.getData()); // 42
21 System.out.println(strObj.getData()); // Hello
22 System.out.println(pair.max());       // 3.14

```

- Q14.** Write two differences between `ArrayList` and `Vector`. Write three differences between `Hashtable` and `HashMap`. Create an appropriate Hash-based collection to store the following weekly salaries: Lionel Messi: 350,000; Cristiano Ronaldo: 365,000; Neymar: null. Then increase Lionel Messi's balance by 100,000. **(10 marks)** [CSE 107 | 2016–17]

#### Answer — Footballer Salaries with HashMap

**ArrayList vs Vector** (2 differences): `ArrayList` is not synchronized (faster); `Vector` is synchronized. `ArrayList` grows by 50%; `Vector` doubles in size.

**HashMap vs Hashtable** (3 differences): `HashMap` allows null key/value; `Hashtable` does not. `HashMap` is unsynchronized; `Hashtable` is synchronized. `HashMap` is modern; `Hashtable` is legacy.

```

1  import java.util.HashMap;
2  import java.util.Map;
3
4  class FootballerSalary {
5      public static void main(String[] args) {
6          // HashMap needed because Neymar's salary is null
7          HashMap<String, Integer> salaries = new HashMap<>();
8          salaries.put("Lionel Messi", 350_000);
9          salaries.put("Cristiano Ronaldo", 365_000);
10         salaries.put("Neymar", null);
11
12         System.out.println("--- Weekly Salaries ---");
13         for (Map.Entry<String, Integer> e : salaries.entrySet())
14             System.out.println(e.getKey() + ": " + e.getValue());
15
16         // Increase Messi's salary by 100,000
17         salaries.put("Lionel Messi",
18             salaries.get("Lionel Messi") + 100_000);
19
20         System.out.println("\nMessi after raise: "
21             + salaries.get("Lionel Messi"));
22     }
23 }

```



**Output:**

```

--- Weekly Salaries ---
Lionel Messi: 350000
Cristiano Ronaldo: 365000
Neymar: null

Messi after raise: 450000

```

- Q15.** Write a Java program to store student records as a collection of **Student** objects. The **Student** class contains **Name**, **Age**, and **CGPA**. Write the class so that the collection can be sorted using Java Collection's **sort()** method with the following priority: (i) descending CGPA; (ii) if CGPAs are equal, ascending age; (iii) if ages are also equal, lexicographic order of name. **(20 marks)** [CSE 201 | 2015–16]

**Answer — Student Collection Sorted by CGPA, Age, Name**

```

1  import java.util.*;
2
3  class Student implements Comparable<Student> {
4      String name; int age; double cgpa;
5
6      Student(String name, int age, double cgpa) {
7          this.name=name; this.age=age; this.cgpa=cgpa;
8      }
9
10     @Override
11     public int compareTo(Student o) {
12         int c = Double.compare(o.cgpa, this.cgpa); // desc
13             CGPA
14         if (c != 0) return c;
15         c = Integer.compare(this.age, o.age);      // asc
16             age
17         if (c != 0) return c;
18         return this.name.compareTo(o.name);        // lex
19             name
20     }
21
22     @Override
23     public String toString() {
24         return name + "(age=" + age + ",cgpa=" + cgpa + ")";
25     }
26 }
27
28 class StudentCollectionSort {
29     public static void main(String[] args) {
30         ArrayList<Student> list = new ArrayList<>();

```

```

28         list.add(new Student("Alice", 21, 3.8));
29         list.add(new Student("Bob", 20, 3.9));
30         list.add(new Student("Carol", 22, 3.8));
31         list.add(new Student("Dave", 21, 3.8));
32         list.add(new Student("Eve", 19, 4.0));
33
34         Collections.sort(list);
35         list.forEach(System.out::println);
36     }
37 }

```

**Output:**

```

Eve(age=19,cgpa=4.0)
Bob(age=20,cgpa=3.9)
Alice(age=21,cgpa=3.8)
Dave(age=21,cgpa=3.8)
Carol(age=22,cgpa=3.8)

```

- Q16.** Write five differences between `Hashtable` and `HashMap`. Consider the following bank account information. Write Java code to: (i) create a `Hashtable`, (ii) store the five entries, (iii) increase John Doe's balance by 100:

| Name        | Balance |
|-------------|---------|
| John Doe    | 3434.34 |
| Tom Smith   | 123.22  |
| Jane Baker  | 1378.00 |
| Tod Hall    | 99.22   |
| Ralph Smith | 19.08   |

**(10 marks)**

[CSE 201 | 2014–15]

**Answer — Hashtable with Bank Accounts**

**Five differences** (see Q2 table above; additional points): null keys/values forbidden in `Hashtable`; `Hashtable` is legacy/synchronized; `HashMap` is modern/unsynchronized; `HashMap` allows one null key and multiple null values; `HashMap` is generally preferred in new code.

```

1  import java.util.Hashtable;
2
3  class BankDemo {
4      public static void main(String[] args) {
5          // (i) Create Hashtable
6          Hashtable<String, Double> accounts = new Hashtable
              <>();
7
8          // (ii) Store five entries

```

```

9      accounts.put("John Doe",    3434.34);
10     accounts.put("Tom Smith",   123.22);
11     accounts.put("Jane Baker",  1378.00);
12     accounts.put("Tod Hall",    99.22);
13     accounts.put("Ralph Smith",  19.08);
14
15     System.out.println("Before: " + accounts.get("John
16         Doe"));
17
18     // (iii) Increase John Doe's balance by 100
19     accounts.put("John Doe", accounts.get("John Doe") +
20         100);
21
22     System.out.println("After:  " + accounts.get("John
23         Doe"));
24
25     // Print all
26     for (String name : accounts.keySet())
27         System.out.printf("%-15s %.2f\n",
28             name, accounts.get(name));
29 }

```

**Output:**

Before: 3434.34  
After: 3534.34

**Q17.** Consider the following class. Write Java code to: (i) define `ArrayList<Product>` `myProducts`, (ii) generate 5 Products A–E with random prices, (iii) add Product F with price 1000 at index 1, (iv) sort by price ascending:

```

1  class Product {
2      private String name; private double price;
3      Product(String name, double price) { this.name = name;
4          this.price = price; }
5      public String getName() { return this.name; }
6      public double getPrice() { return this.price; }
7  }

```

**(10 marks)**

[CSE 201 | 2014–15]

**Answer — Product ArrayList: Generate, Insert, Sort by Price**

```

1  import java.util.*;
2
3  class Product {
4      private String name; private double price;
5      Product(String name, double price) { this.name=name; this

```

```

        .price=price; }
6    public String getName() { return name; }
7    public double getPrice() { return price; }
8    @Override
9    public String toString() {
10        return name + "($ " + String.format("%.2f", price) + "
        )";
11    }
12 }
13
14 class ProductDemo {
15     public static void main(String[] args) {
16         Random rand = new Random();
17
18         // (i) Define ArrayList
19         ArrayList<Product> myProducts = new ArrayList<>();
20
21         // (ii) Generate 5 products A-E with random prices
22         for (char c = 'A'; c <= 'E'; c++)
23             myProducts.add(new Product(String.valueOf(c),
24                                     10 + rand.nextDouble()
25                                     * 90));
26
27         System.out.println("Before insert: " + myProducts);
28
29         // (iii) Add Product F with price 1000 at index 1
30         myProducts.add(1, new Product("F", 1000));
31
32         System.out.println("After insert: " + myProducts);
33
34         // (iv) Sort by price ascending
35         myProducts.sort(Comparator.comparingDouble(Product::
36             getPrice));
37
38         System.out.println("After sort: " + myProducts);
39     }
40 }

```

**Q18.** A class `Employee` has two member variables: `name` (`String`) and `age` (`int`). Write the class so an `ArrayList` of `Employee` objects can be sorted by either variable. (14 marks) [CSE 201 | 2013–14]

**Answer — Employee Sorted by Name or Age**

```

1  import java.util.*;
2
3  class Employee implements Comparable<Employee> {

```

```

4      String name; int age;
5      Employee(String name, int age) { this.name=name; this.age
      =age; }
6
7      // Natural order: by name
8      @Override
9      public int compareTo(Employee o) { return this.name.
      compareTo(o.name); }
10
11     // Comparator for age
12     static Comparator<Employee> byAge() {
13         return Comparator.comparingInt(e -> e.age);
14     }
15
16     @Override
17     public String toString() { return name + "(" + age + ")";
      }
18 }
19
20 class EmployeeSortDemo {
21     public static void main(String[] args) {
22         ArrayList<Employee> list = new ArrayList<>();
23         list.add(new Employee("Zara", 25));
24         list.add(new Employee("Alice", 30));
25         list.add(new Employee("Bob", 22));
26
27         Collections.sort(list);           // sort by name
28         System.out.println("By name: " + list);
29
30         list.sort(Employee.byAge());      // sort by age
31         System.out.println("By age:  " + list);
32     }
33 }

```

**Output:**

By name: [Alice(30), Bob(22), Zara(25)]

By age: [Bob(22), Zara(25), Alice(30)]

- Q19.** Write a class `Student` containing a `HashMap` with `Integer` key and `List of String` as values. Add 3 studentIds as keys with [firstname, lastname] values. Write a method to print all. (13 marks) [CSE 201 | 2013–14]

**Answer — Student HashMap with List Values**

```

1  import java.util.*;
2
3  class StudentMap {

```

```
4     private HashMap<Integer, List<String>> students = new
        HashMap<>();
5
6     void addStudent(int id, String firstName, String lastName
    ) {
7         List<String> names = new ArrayList<>();
8         names.add(firstName); names.add(lastName);
9         students.put(id, names);
10    }
11
12    void printAll() {
13        for (Map.Entry<Integer, List<String>> e : students.
            entrySet())
14            System.out.println("ID: " + e.getKey()
15                               + " -> " + e.getValue());
16    }
17
18    public static void main(String[] args) {
19        StudentMap sm = new StudentMap();
20        sm.addStudent(1001, "Alice", "Smith");
21        sm.addStudent(1002, "Bob", "Jones");
22        sm.addStudent(1003, "Carol", "White");
23        sm.printAll();
24    }
25 }
```

**Output:**

```
ID: 1001 -> [Alice, Smith]
ID: 1002 -> [Bob, Jones]
ID: 1003 -> [Carol, White]
```

**Q20.** Consider a file named ‘products.txt’ containing following information.

|    |     |     |    |
|----|-----|-----|----|
| 1, | P1, | T1, | 10 |
| 2, | P2, | T2, | 20 |
| 3, | P3, | T1, | 30 |
| 4, | P4, | T2, | 20 |
| 5, | P5, | T1, | 30 |

This file represents information of products where the attributes are (id, name, type, price). A single product’s information is contained in a single line. You need to do the following.

- (i) Write a class in Java named Product with necessary attributes.
- (ii) Write a function in Java to store the products information from this ‘products.txt’ file to an ArrayList of Product named ‘myProducts’.

- (iii) Write a function in Java to store the contents of an ArrayList of Product named 'myProducts' to a file named 'products2.txt' in the same format as given in the 'products.txt' file.

(4+8+8 marks)

[CSE 201 | 2008–09]

### Answer — Product Read/Write from File

```
1  import java.io.*;
2  import java.util.*;
3
4  // (i) Product class with necessary attributes
5  class Product {
6      int id;
7      String name, type;
8      int price; // using int based on the sample data provided
9
10     Product(int id, String name, String type, int price) {
11         this.id = id;
12         this.name = name;
13         this.type = type;
14         this.price = price;
15     }
16
17     @Override
18     public String toString() {
19         return id + ", " + name + ", " + type + ", " + price;
20     }
21 }
22
23 class ProductFileDemo {
24     // (ii) Function to read products.txt into an ArrayList
25     static ArrayList<Product> readProducts(String filename)
26         throws IOException {
27         ArrayList<Product> myProducts = new ArrayList<>();
28         try (BufferedReader br = new BufferedReader(new
29             FileReader(filename))) {
30             String line;
31             while ((line = br.readLine()) != null) {
32                 if(line.trim().isEmpty()) continue;
33                 String[] p = line.split(",");
34                 // trimming is necessary because of spaces
35                 // after commas
36                 myProducts.add(new Product(
37                     Integer.parseInt(p[0].trim()),
38                     p[1].trim(),
39                     p[2].trim(),
40                     Integer.parseInt(p[3].trim())
41                 ));
42             }
43         }
44     }
45 }
```

```
39         }
40     }
41     return myProducts;
42 }
43
44 // (iii) Function to write ArrayList to products2.txt in
45 // the same format
46 static void writeProducts(ArrayList<Product> myProducts,
47 String filename)
48     throws IOException {
49     try (BufferedWriter bw = new BufferedWriter(new
50         FileWriter(filename))) {
51         for (Product p : myProducts) {
52             bw.write(p.toString());
53             bw.newLine();
54         }
55     }
56 }
57
58 public static void main(String[] args) {
59     try {
60         ArrayList<Product> myProducts = readProducts("
61             products.txt");
62         writeProducts(myProducts, "products2.txt");
63         System.out.println("Products successfully copied
64             to products2.txt");
65     } catch (IOException e) {
66         e.printStackTrace();
67     }
68 }
```



BUET · CSE 107 / CSE 201

# Object Oriented Programming

Section S16

---

Stream API, Lambda & Enums

## S16 — Stream API, Lambda & Enums

**Q1.** Write Java code for the following: (i) Declare an ArrayList of Employee, add 6 employees; (ii) Rewrite two code segments using Java Stream; (iii) Sort by ascending age; if equal, by ascending name:

```

1 // Segment 1
2 List<String> names = new ArrayList<>();
3 for (Employee e : list) names.add(e.getName());
4 // Segment 2
5 List<Employee> employees = new ArrayList<>();
6 for (Employee e : list) if (e.getAge() <= 20) employees.add(e)
    ;

```

(15 marks)

[CSE 107 | 2023–24]

### Answer — Employee Stream Rewrite and Sort

*(Uses the same Employee class as Q2.)*

```

1 import java.util.*;
2 import java.util.stream.*;
3
4 public class EmployeeStreamDemo {
5     public static void main(String[] args) {
6
7         // (i) ArrayList<Employee> with 6 employees
8         ArrayList<Employee> list = new ArrayList<>();
9         list.add(new Employee(1, "Alice", 22));
10        list.add(new Employee(2, "Bob", 19));
11        list.add(new Employee(3, "Carol", 20));
12        list.add(new Employee(4, "Dave", 20));
13        list.add(new Employee(5, "Eve", 17));
14        list.add(new Employee(6, "Frank", 25));
15
16        // -- Segment 1 (original for-each loop) -----
17        List<String> names1 = new ArrayList<>();
18        for (Employee e : list) names1.add(e.getName());
19
20        // (ii-a) Segment 1 using Stream
21        List<String> names2 = list.stream()
22            .map(Employee::getName)
23            .collect(Collectors.toList());
24
25        System.out.println("All names: " + names2);
26
27        // -- Segment 2 (original for-each loop) -----
28        List<Employee> filtered1 = new ArrayList<>();
29        for (Employee e : list)
30            if (e.getAge() <= 20) filtered1.add(e);

```

```

31
32     // (ii-b) Segment 2 using Stream
33     List<Employee> filtered2 = list.stream()
34         .filter(e -> e.getAge() <= 20)
35         .collect(Collectors.toList());
36
37     System.out.println("Age <= 20: " + filtered2);
38
39     // (iii) Sort: ascending age, then ascending name
40     list.sort(Comparator.comparingInt(Employee::getAge)
41         .thenComparing(Employee::getName)
42         );
43
44     System.out.println("Sorted: " + list);
45 }

```

**Output:**

All names: [Alice, Bob, Carol, Dave, Eve, Frank]

Age <= 20: [Bob, Carol, Dave, Eve]

Sorted: [5:Eve(17), 2:Bob(19), 3:Carol(20), 4:Dave(20),  
1:Alice(22), 6:Frank(25)]

- Q2.** Develop Java code for the following using an `Employee` class (id, name, age): (i) Declare `ArrayList<Employee>` and add 6 employees; (ii) Store names of employees aged > 15 in `nameList`; (iii) Sort by ascending age; if equal, by ascending name.  
(15 marks) [CSE 107 | 2021–22]

**Answer — Employee ArrayList, Filter and Sort**

```

1  import java.util.*;
2  import java.util.stream.*;
3
4  class Employee {
5      private int    id;
6      private String name;
7      private int    age;
8
9      Employee(int id, String name, int age) {
10         this.id = id; this.name = name; this.age = age;
11     }
12     public int    getId()    { return id;    }
13     public String getName() { return name; }
14     public int    getAge()   { return age;   }
15
16     @Override
17     public String toString() {

```

```
18         return id + ":" + name + "(" + age + ")";
19     }
20 }
21
22 public class EmployeeDemo {
23     public static void main(String[] args) {
24
25         // (i) ArrayList<Employee> with 6 employees
26         ArrayList<Employee> list = new ArrayList<>();
27         list.add(new Employee(1, "Alice", 22));
28         list.add(new Employee(2, "Bob", 14));
29         list.add(new Employee(3, "Carol", 19));
30         list.add(new Employee(4, "Dave", 30));
31         list.add(new Employee(5, "Eve", 16));
32         list.add(new Employee(6, "Frank", 25));
33
34         // (ii) Store names of employees aged > 15
35         List<String> nameList = list.stream()
36             .filter(e -> e.getAge() > 15)
37             .map(Employee::getName)
38             .collect(Collectors.toList());
39
40         System.out.println("Names (age > 15): " + nameList);
41
42         // (iii) Sort by ascending age; if equal, ascending
43             name
44         list.sort(Comparator.comparingInt(Employee::getAge)
45             .thenComparing(Employee::getName)
46             );
47
48         System.out.println("Sorted employees: " + list);
49     }
50 }
```

**Output:**

```
Names (age > 15): [Alice, Carol, Eve, Dave, Frank]
Sorted employees: [2:Bob(14), 5:Eve(16), 3:Carol(19),
                  1:Alice(22), 6:Frank(25), 4:Dave(30)]
```

**Q3.** Create a Java enum named `HttpStatus` to represent the following HTTP status codes: **(10 marks)**

OK (200, "OK"), NOT\_FOUND (404, "Not Found"),

INTERNAL\_ERROR (500, "Internal Server Error")

Each enum constant should store two fields: an `int` status code and a `String` message.

Your enum should also implement the following methods:

- (a) `isError()`: Returns false for OK, and true for both NOT\_FOUND and INTERNAL\_ERROR
- (b) `toString()`: Returns a string in the format "200 OK", "404 Not Found", "500 Internal Server Error"
- (c) `fromCode(int code)` (static method): Takes an `int` code and returns the corresponding `HttpStatus` constant. If the code doesn't match any defined status, return null.

[CSE 107 | 2023–24]

### Answer — HttpStatus Enum

```
1 public enum HttpStatus {
2     OK(200, "OK"),
3     NOT_FOUND(404, "Not Found"),
4     INTERNAL_ERROR(500, "Internal Server Error");
5
6     private final int code;
7     private final String message;
8
9     // Constructor to store the two fields
10    HttpStatus(int code, String message) {
11        this.code = code;
12        this.message = message;
13    }
14
15    // i. isError() method
16    public boolean isError() {
17        return this != OK;
18    }
19
20    // ii. toString() method
21    @Override
22    public String toString() {
23        return code + " " + message;
24    }
25
26    // iii. fromCode(int code) static method
27    public static HttpStatus fromCode(int code) {
28        for (HttpStatus status : HttpStatus.values()) {
29            if (status.code == code) {
30                return status;
31            }
32        }
33        return null;
34    }
35
```

```

36     // Demonstration (Optional but helpful for testing)
37     public static void main(String[] args) {
38         System.out.println(HttpStatus.OK);
39         // 200 OK
40         System.out.println(HttpStatus.NOT_FOUND);
41         // 404 Not Found
42         System.out.println(HttpStatus.OK.isError());
43         // false
44         System.out.println(HttpStatus.NOT_FOUND.isError());
45         // true
46         System.out.println(HttpStatus.fromCode(500));
47         // 500 Internal Server Error
48     }
49 }

```

**Q4.** Consider the following `Person` class. Write Java code for the following: (i) Declare `ArrayList<Person> list` and add 6 persons; (ii) Rewrite the two segments using Java Stream; (iii) Sort the list by ascending age, then ascending name if age is equal:

```

1  // Segment 1
2  List<String> names = new ArrayList<>();
3  for (int i = 0; i < list.size(); i++) { Person mc = list.get(i);
4      names.add(mc.getName()); }
5  // Segment 2
6  List<Person> persons = new ArrayList<>();
7  for (int i = 0; i < list.size(); i++) { Person mc = list.get(i);
8      if (mc.getAge() <= 20) persons.add(mc); }

```

(20 marks)

[CSE 107 | Jan 2021]

↪ Similar: CSE 107 | 2021-22, 2022-23, 2023-24

### Answer — Person ArrayList, Streams and Sorting

```

1  import java.util.*;
2  import java.util.stream.*;
3
4  class Person {
5      private String name;
6      private int age;
7
8      Person(String name, int age) {
9          this.name = name; this.age = age;
10     }
11     public String getName() { return name; }
12     public int getAge() { return age; }
13
14     @Override

```

```
15     public String toString() {
16         return name + "(" + age + ")";
17     }
18 }
19
20 public class PersonDemo {
21     public static void main(String[] args) {
22
23         // (i) Declare ArrayList<Person> and add 6 persons
24         ArrayList<Person> list = new ArrayList<>();
25         list.add(new Person("Alice", 22));
26         list.add(new Person("Bob", 19));
27         list.add(new Person("Carol", 20));
28         list.add(new Person("Dave", 20));
29         list.add(new Person("Eve", 17));
30         list.add(new Person("Frank", 25));
31
32         // -- Original Segment 1 (loop) -----
33         List<String> names1 = new ArrayList<>();
34         for (int i = 0; i < list.size(); i++) {
35             Person mc = list.get(i);
36             names1.add(mc.getName());
37         }
38
39         // (ii-a) Segment 1 rewritten using Stream
40         List<String> names2 = list.stream()
41             .map(Person::getName)
42             .collect(Collectors.toList());
43
44         System.out.println("All names (stream): " + names2);
45
46         // -- Original Segment 2 (loop) -----
47         List<Person> persons1 = new ArrayList<>();
48         for (int i = 0; i < list.size(); i++) {
49             Person mc = list.get(i);
50             if (mc.getAge() <= 20) persons1.add(mc);
51         }
52
53         // (ii-b) Segment 2 rewritten using Stream
54         List<Person> persons2 = list.stream()
55             .filter(p -> p.getAge() <= 20)
56             .collect(Collectors.toList());
57
58         System.out.println("Age <= 20 (stream): " + persons2);
59         ;
60
61         // (iii) Sort: ascending age, then ascending name
62         list.sort(Comparator.comparingInt(Person::getAge)
```

```

62                                     .thenComparing(Person::getName));
63
64         System.out.println("Sorted list: " + list);
65     }
66 }

```

**Output:**

All names (stream): [Alice, Bob, Carol, Dave, Eve, Frank]

Age <= 20 (stream): [Bob, Carol, Dave, Eve]

Sorted list: [Eve(17), Bob(19), Carol(20), Dave(20),  
Alice(22), Frank(25)]

**Key stream operations used:**

- `.map(Person::getName)` — transforms each `Person` to its name (method reference).
- `.filter(p -> p.getAge() <= 20)` — keeps only persons aged  $\leq 20$ .
- `Comparator.comparingInt(...).thenComparing(...)` — chains two comparators for multi-key sorting.

**Q5.** Consider the `Fruit` class with name and quantity. Write Java code so that `Collections.sort(listFruits)` sorts by descending quantity; if equal, by lexicographic name. **(7 marks)** [CSE 107 | 2016–17, 2020–21]

**Answer — Fruit: Descending Quantity, Then Lexicographic Name**

`Collections.sort()` uses the natural ordering defined by `Comparable<Fruit>`. We override `compareTo()` to sort by **descending quantity** first, then **lexicographic (ascending) name**.

```

1  import java.util.*;
2
3  class Fruit implements Comparable<Fruit> {
4      private String name;
5      private int    quantity;
6
7      Fruit(String name, int quantity) {
8          this.name = name; this.quantity = quantity;
9      }
10     public String getName()    { return name;    }
11     public int    getQuantity(){ return quantity; }
12
13     @Override
14     public int compareTo(Fruit other) {
15         // 1. Descending quantity
16         int c = Integer.compare(other.quantity, this.quantity

```



```

    );
17     if (c != 0) return c;
18     // 2. Ascending (lexicographic) name
19     return this.name.compareTo(other.name);
20 }
21
22 @Override
23 public String toString() {
24     return name + ":" + quantity;
25 }
26 }
27
28 public class FruitDemo {
29     public static void main(String[] args) {
30         List<Fruit> listFruits = new ArrayList<>();
31         listFruits.add(new Fruit("Mango", 10));
32         listFruits.add(new Fruit("Apple", 25));
33         listFruits.add(new Fruit("Banana", 25));
34         listFruits.add(new Fruit("Grape", 8));
35         listFruits.add(new Fruit("Cherry", 15));
36
37         Collections.sort(listFruits);
38
39         System.out.println(listFruits);
40     }
41 }

```

**Output:**

[Apple:25, Banana:25, Cherry:15, Mango:10, Grape:8]

**Explanation:** Apple and Banana both have quantity 25; they are tied on quantity so they are sorted alphabetically (Apple before Banana). Cherry (15), Mango (10), Grape (8) follow in descending quantity order.

- Q6.** Consider the following FIFA World Cup data. Declare Enumeration `FWCWinners`. Print all countries. Print the most recent winner (cannot simply write Argentina):

| Country   | Titles | Last Title |
|-----------|--------|------------|
| Argentina | 3      | 2022       |
| France    | 2      | 2018       |
| Uruguay   | 2      | 1950       |
| England   | 1      | 1966       |
| Spain     | 1      | 2010       |

(10 marks)

[CSE 107 | 2021–22]

## Answer — FWCWinners Enumeration

```
1  enum FWCWinners {
2      ARGENTINA(3, 2022),
3      FRANCE(2, 2018),
4      URUGUAY(2, 1950),
5      ENGLAND(1, 1966),
6      SPAIN(1, 2010);
7
8      private final int titles;
9      private final int lastTitle;
10
11     FWCWinners(int titles, int lastTitle) {
12         this.titles = titles;
13         this.lastTitle = lastTitle;
14     }
15
16     public int getTitles()    { return titles; }
17     public int getLastTitle() { return lastTitle; }
18 }
19
20 public class Main {
21     public static void main(String[] args) {
22
23         // Print all countries
24         System.out.println("All FIFA World Cup Winners:");
25         for (FWCWinners w : FWCWinners.values())
26             System.out.println(w.name()
27                 + " | Titles: " + w.getTitles()
28                 + " | Last: " + w.getLastTitle());
29
30         // Most recent winner (cannot simply write Argentina)
31         FWCWinners recent = FWCWinners.values()[0];
32         for (FWCWinners w : FWCWinners.values())
33             if (w.getLastTitle() > recent.getLastTitle())
34                 recent = w;
35
36         System.out.println("\nMost recent winner: " + recent.
37             name()
38             + " (" + recent.getLastTitle() + ")");
39     }
40 }
```

## Output:

```
1  All FIFA World Cup Winners:
2  ARGENTINA | Titles: 3 | Last: 2022
3  FRANCE | Titles: 2 | Last: 2018
4  URUGUAY | Titles: 2 | Last: 1950
5  ENGLAND | Titles: 1 | Last: 1966
```

```

6  SPAIN | Titles: 1 | Last: 2010
7
8  Most recent winner: ARGENTINA (2022)

```

**Q7.** Consider the following EPL data. Write Java code to: (i) Declare a single Enumeration named EPL; (ii) Print all clubs with points and position:

| Club              | Point | Position |
|-------------------|-------|----------|
| Manchester United | 90    | 1        |
| Chelsea           | 80    | 2        |
| Arsenal           | 75    | 4        |
| Manchester City   | 30    | 19       |
| Liverpool         | 25    | 20       |

(5 marks)

[CSE 107 | Jan 2021]

#### Answer — EPL Enumeration

```

1  enum EPL {
2      MANCHESTER_UNITED(90, 1),
3      CHELSEA(80, 2),
4      ARSENAL(75, 4),
5      MANCHESTER_CITY(30, 19),
6      LIVERPOOL(25, 20);
7
8      private final int points;
9      private final int position;
10
11     EPL(int points, int position) {
12         this.points = points;
13         this.position = position;
14     }
15
16     public int getPoints() { return points; }
17     public int getPosition() { return position; }
18 }
19
20 public class Main {
21     public static void main(String[] args) {
22         // Print all clubs with points and position
23         for (EPL club : EPL.values()) {
24             System.out.println(club.name()
25                 + " | Points: " + club.getPoints()
26                 + " | Position: " + club.getPosition());
27         }
28     }
29 }

```

**Output:**

```

1 MANCHESTER_UNITED | Points: 90 | Position: 1
2 CHELSEA | Points: 80 | Position: 2
3 ARSENAL | Points: 75 | Position: 4
4 MANCHESTER_CITY | Points: 30 | Position: 19
5 LIVERPOOL | Points: 25 | Position: 20

```

**Q8.** Declare Enumeration GOT for Game of Thrones characters. Find the character active for the least number of seasons (cannot simply write Ned Stark):

| Name               | Status | NoOfSeasons |
|--------------------|--------|-------------|
| Jon Snow           | Alive  | 7           |
| Daenerys Targaryen | Alive  | 7           |
| Ned Stark          | Dead   | 1           |
| Joffrey Baratheon  | Dead   | 4           |
| Tyrion Lannister   | Alive  | 7           |

(7 marks)

[CSE 107 | 2016–17]

**Answer — GOT Enumeration: Least Active Character**

```

1 enum GOT {
2     JON_SNOW("Alive", 7),
3     DAENERYS_TARGARYEN("Alive", 7),
4     NED_STARK("Dead", 1),
5     JOFFREY_BARATHEON("Dead", 4),
6     TYRION_LANNISTER("Alive", 7);
7
8     private final String status;
9     private final int seasons;
10
11     GOT(String status, int seasons) {
12         this.status = status;
13         this.seasons = seasons;
14     }
15
16     public int getSeasons() { return seasons; }
17 }
18
19 public class Main {
20     public static void main(String[] args) {
21
22         // Find character with least seasons (cannot simply
23         // write Ned Stark)
24         GOT least = GOT.values()[0];

```

```

24         for (GOT c : GOT.values())
25             if (c.getSeasons() < least.getSeasons())
26                 least = c;
27
28         System.out.println("Least active: " + least.name()
29             + " (" + least.getSeasons() + " season(s))");
30     }
31 }

```

**Output:**

```

1 Least active: NED_STARK (1 season(s))

```

**Q9.** What is a functional interface in Java? Give an example to implement the functional interface as an Anonymous Inner class. **(10 marks)** [CSE 201 | 2015–16]

**Answer — Functional Interface and Anonymous Inner Class****What is a functional interface?**

A **functional interface** is an interface that contains **exactly one abstract method**. It may have any number of default or static methods, but only one abstract method. The `@FunctionalInterface` annotation enforces this contract at compile time.

Functional interfaces are the foundation of **lambda expressions** and **method references** in Java 8+. Examples from the standard library include `Runnable`, `Comparator<T>`, `Predicate<T>`, and `Function<T,R>`.

```

1 // Define a custom functional interface
2 @FunctionalInterface
3 interface MathOperation {
4     int operate(int a, int b); // single abstract method
5
6     // default methods are allowed
7     default void describe() {
8         System.out.println("A math operation");
9     }
10 }

```

**Example 1: Implement as an Anonymous Inner Class**

```

1 public class FunctionalDemo {
2     public static void main(String[] args) {
3
4         // Anonymous inner class implementing MathOperation
5         MathOperation addition = new MathOperation() {
6             @Override
7             public int operate(int a, int b) {
8                 return a + b;
9             }
10         };
11     }
12 }

```

```
9         }
10    };
11
12    MathOperation multiplication = new MathOperation() {
13        @Override
14        public int operate(int a, int b) {
15            return a * b;
16        }
17    };
18
19    System.out.println("5 + 3 = " + addition.operate(5,
20    3));
21    System.out.println("5 * 3 = " + multiplication.
22    operate(5, 3));
23    addition.describe();
24 }
```

**Output:**

5 + 3 = 8

5 \* 3 = 15

A math operation

**Example 2: Equivalent lambda expression (Java 8+)**

```
1 // Lambda is syntactic sugar for the anonymous class above
2 MathOperation subtraction = (a, b) -> a - b;
3 System.out.println("5 - 3 = " + subtraction.operate(5, 3));
```

**Key points:**

- An anonymous inner class provides an instance of the interface without explicitly naming a class. It is verbose but works on all Java versions.
- A lambda expression is a concise replacement for a single-method anonymous class and is only available for functional interfaces.
- Both approaches are interchangeable for functional interfaces; the lambda is preferred in modern Java for readability.

**Q10.** What is a functional interface? Using lambda expression, implement the following interface to compute factorial:

```
1 interface SomeFunc<T> { T func(T t); }
```

(10 marks)

[CSE 107 | 2015–16]

**Answer — Functional Interface & Lambda: Factorial**

A **functional interface** has exactly one abstract method. The `@FunctionalInterface` annotation is optional but recommended.

```

1  interface SomeFunc<T> { T func(T t); }
2
3  public class Main {
4      public static void main(String[] args) {
5
6          // Lambda expression implementing SomeFunc<Integer>
7          SomeFunc<Integer> factorial = n -> {
8              int result = 1;
9              for (int i = 2; i <= n; i++)
10                 result *= i;
11             return result;
12         };
13
14         System.out.println("5! = " + factorial.func(5)); //
15                                     120
16         System.out.println("6! = " + factorial.func(6)); //
17                                     720
18     }
19 }

```

**Output:**

```

1  5! = 120
2  6! = 720

```

- Q11.** Declare a single Enumeration named EPL. Find out the difference between league position of ManUnited and Liverpool (cannot simply write 18). Also declare Enumeration for football clubs from EPL table. **(5 marks)** [CSE 201 | 2014–15]

**Answer — EPL Enum: Position Difference**

```

1  enum EPL {
2      MANCHESTER_UNITED(90, 1),
3      CHELSEA(80, 2),
4      ARSENAL(75, 4),
5      MANCHESTER_CITY(30, 19),
6      LIVERPOOL(25, 20);
7
8      private final int points;
9      private final int position;
10
11      EPL(int points, int position) {
12          this.points = points;

```

```
13         this.position = position;
14     }
15     public int getPosition() { return position; }
16 }
17
18 public class Main {
19     public static void main(String[] args) {
20         // Cannot simply write 18 --- compute from enum data
21         int diff = EPL.LIVERPOOL.getPosition()
22                 - EPL.MANCHESTER_UNITED.getPosition();
23         System.out.println("Position difference: " + diff);
24         // 19
25     }
26 }
```

**Output:**

```
1 Position difference: 19
```



BUET · CSE 107 / CSE 201

# Object Oriented Programming

Section S17

---

File I/O & Serialization

## S17 — File I/O & Serialization

**Q1.** Consider a Java program that copies `src.txt` to `dst.txt` line by line. Implement using try-with-resources. You are not allowed to throw Exception anywhere. (10 marks) [CSE 107 | 2023–24]

### Answer — Copy `src.txt` to `dst.txt` Line by Line (try-with-resources)

```
1  import java.io.*;
2
3  public class FileCopy {
4      public static void main(String[] args) {
5          try (BufferedReader reader = new BufferedReader(
6              new FileReader("src.txt"));
7              BufferedWriter writer = new BufferedWriter(
8                  new FileWriter("dst.txt"))) {
9
10             String line;
11             while ((line = reader.readLine()) != null) {
12                 writer.write(line);
13                 writer.newLine();
14             }
15             System.out.println("File copied successfully.");
16
17             } catch (FileNotFoundException e) {
18                 System.out.println("File not found: " + e.
19                     getMessage());
20             } catch (IOException e) {
21                 System.out.println("IO error: " + e.getMessage())
22                     ;
23             }
24             // reader and writer closed automatically -- no throw
25             anywhere
26         }
27     }
```

#### Key points:

- try-with-resources closes both reader and writer automatically in reverse order.
- All exceptions are caught locally — no `throws` clause is used anywhere.

**Q2.** Consider a Java program that copies binary file `src.mkv` to `dst.mkv` using chunks of 8 KB. Implement using try-with-resources. Not allowed to throw Exception anywhere. (10 marks) [CSE 107 | 2021–22]

**Answer — Copy Binary File in 8 KB Chunks (try-with-resources)**

```
1  import java.io.*;
2
3  public class BinaryFileCopy {
4      private static final int CHUNK = 8 * 1024; // 8 KB
5
6      public static void main(String[] args) {
7          try (InputStream in = new BufferedInputStream(
8              new FileInputStream("src.mkv"), CHUNK);
9              OutputStream out = new BufferedOutputStream(
10                 new FileOutputStream("dst.mkv"), CHUNK)) {
11
12              byte[] buffer = new byte[CHUNK];
13              int bytesRead;
14              long total = 0;
15
16              while ((bytesRead = in.read(buffer)) != -1) {
17                  out.write(buffer, 0, bytesRead);
18                  total += bytesRead;
19              }
20
21              System.out.printf("Copied %.2f MB%n",
22                               total / (1024.0 * 1024));
23
24              } catch (FileNotFoundException e) {
25                  System.out.println("File not found: " + e.
26                      getMessage());
27              } catch (IOException e) {
28                  System.out.println("IO error: " + e.getMessage())
29                      ;
30              }
31          }
32      }
33      // no throws anywhere -- all caught locally
34  }
```

**Key points:**

- `byte[] buffer = new byte[8192]` reads/writes exactly 8 KB at a time for efficiency.
- `in.read(buffer)` returns -1 at end of file.
- `out.write(buffer, 0, bytesRead)` writes only the bytes actually read — important for the last chunk.

**Q3.** Write necessary Java code to write `object1` to a file named `serial/test` and read the object from the same file and assign it to `object2`. (8+9 marks)

```

1 class MyClass {
2     String s;
3     int i;
4     public MyClass(String s,
5         int i) {
6         this.s = s;
7         this.i = i;
8     }
9 }

```

```

1 public class FileDemo {
2     public static void main(
3         String args[]) {
4         MyClass object1 = new
5             MyClass("James Bond"
6                 , 7);
7         // your code
8         MyClass object2;
9         // your code
10    }
11 }

```

[CSE 107 | 2016–17, 2017–18]

**Answer — Serialize object1 to File and Deserialize to object2**

```

1 import java.io.*;
2
3 class MyClass implements Serializable {
4     int id; String name;
5     MyClass(int id, String name) { this.id=id; this.name=name
6         ; }
7     @Override
8     public String toString() { return "MyClass{id="+id+",name
9         ="+name+"}"; }
10 }
11
12 class SerialDemo {
13     public static void main(String[] args) {
14         MyClass object1 = new MyClass(42, "Alice");
15
16         // Write object1 to file "serial"
17         try (ObjectOutputStream oos = new ObjectOutputStream(
18             new FileOutputStream("serial"))) {
19             oos.writeObject(object1);
20             System.out.println("Written: " + object1);
21         } catch (IOException e) { e.printStackTrace(); }
22
23         // Read object from "serial" and assign to object2
24         MyClass object2 = null;
25         try (ObjectInputStream ois = new ObjectInputStream(
26             new FileInputStream("serial"))) {
27             object2 = (MyClass) ois.readObject();
28             System.out.println("Read: " + object2);
29         } catch (IOException | ClassNotFoundException e) {
30             e.printStackTrace();
31         }
32     }
33 }

```

```
31 }
```

**Output:**

Written: MyClass{id=42,name=Alice}

Read: MyClass{id=42,name=Alice}

**Q4.** You are given a text file called `MyAnalysisOfStudentsPerformance.txt`. Write necessary code to read each line from the file and print out to the console. However, in your output each line should be prefixed with the line number. Your I/O operations should be efficient. Once you have run your code, you observe (to your surprise) that the first line in the console output is:

1. আমার ভয় হচ্ছে তোমরা হয়তো পাস করতে পারবে না!

Here, the “1.” is the line number you added from your code, and the remaining part was the line read from the file. How were you able to read the Bangla text? (Hint: By opening the text file in Notepad program, you have noticed that the encoding of the file is UTF-8). **(10+5 marks)** [CSE 107 | 2015–16]

**Answer — Efficient File I/O with UTF-8 Encoding****Part 1: Efficient Java Program (10 Marks)**

To perform file reading efficiently while ensuring character encoding compatibility, we wrap a `FileInputStream` inside an `InputStreamReader` configured with explicit UTF-8 decoding, and then buffer the stream using `BufferedReader`.

```
1  import java.io.BufferedReader;
2  import java.io.FileInputStream;
3  import java.io.InputStreamReader;
4  import java.io.IOException;
5  import java.nio.charset.StandardCharsets;
6
7  public class PerformanceReader {
8      public static void main(String[] args) {
9          String filePath = "MyAnalysisOfStudentsPerformance.
10             txt";
11
12         // Try-with-resources statement ensures proper,
13         // automatic resource closure
14         try (BufferedReader reader = new BufferedReader(
15             new InputStreamReader(
16                 new FileInputStream(filePath),
17                 StandardCharsets.UTF_8))) {
18
19             String line;
20             int lineNumber = 1;
21
22             // Efficiently read line-by-line via the internal
```

```

        character buffer
21         while ((line = reader.readLine()) != null) {
22             System.out.println(lineNumber + ". " + line);
23             lineNumber++;
24         }
25
26     } catch (IOException e) {
27         System.err.println("An error occurred while
28             reading the file: " + e.getMessage());
29     }
30 }

```

## Part 2: Explanation of Bangla Text Reading (5 Marks)

- (a) **Limitations of Default FileReaders:** Standard file utilities like legacy `FileReader` inherently rely on the host platform's default native character encoding (e.g., Windows-1252 or extended ASCII). These legacy single-byte charsets are fundamentally incapable of parsing complex scripts, rendering them as garbled symbols or question marks (???).
- (b) **The UTF-8 Multi-byte Standard:** Bangla characters are non-ASCII and fall within the extended Unicode spectrum, requiring 3 bytes per character under UTF-8 encoding scheme specifications.
- (c) **Explicit Character Set Configuration:** By explicitly instantiating an `InputStreamReader` combined with `StandardCharsets.UTF_8`, we override system defaults. This directly instructs the low-level stream engine to combine sequential raw byte groups correctly and map them directly into Java's internal 16-bit UTF-16 code units (`char`), allowing Bangla text to display natively in a compatible terminal console.
- (d) **I/O Efficiency Handling:** Wrapping this decoder loop inside a `BufferedReader` limits expensive underlying hardware disk reads by retrieving substantial data blocks directly into memory buffers upfront, meeting the target system requirement.

**Q5.** Now, instead of writing each line to the console, you need to write to another file. There is no need to prefix the lines with line number. Modify your code written in 8(a) so that it reads the destination file name from command line and copies the content of `MyAnalysisOfStudentsPerformance.txt` line by line. Keep in mind that the input file uses UTF-8 encoding. Identify what additional code you had to write to account for UTF-8 encoding and why? If you did not write any additional code, then explain why. (10 marks) [CSE 107 | 2015–16]

**Answer — Stream-Based File Copying with Explicit Target Encoding****Part 1: Modified Java Program**

The program reads the destination file name dynamically via command-line arguments (`args[0]`), extracts data line-by-line using a buffered reader loop, and safely pushes it into an explicitly encoded buffered output writer stream.

```
1  import java.io.BufferedReader;
2  import java.io.BufferedWriter;
3  import java.io.FileInputStream;
4  import java.io.FileOutputStream;
5  import java.io.InputStreamReader;
6  import java.io.OutputStreamWriter;
7  import java.io.IOException;
8  import java.nio.charset.StandardCharsets;
9
10 public class FileCopier {
11     public static void main(String[] args) {
12         // Validation check to prevent
13         // ArrayIndexOutOfBoundsException
14         if (args.length < 1) {
15             System.err.println("Error: Please provide the
16                                 destination file name as a command-line
17                                 argument.");
18             return;
19         }
20
21         String srcFile = "MyAnalysisOfStudentsPerformance.txt";
22         String destFile = args[0];
23
24         // Try-with-resources manages clean multi-stream
25         // lifecycles
26         try (BufferedReader reader = new BufferedReader(
27             new InputStreamReader(
28                 new FileInputStream(srcFile),
29                 StandardCharsets.UTF_8));
30             BufferedWriter writer = new BufferedWriter(
31                 new OutputStreamWriter(
32                     new FileOutputStream(destFile),
33                     StandardCharsets.UTF_8))) {
34
35             String line;
36             while ((line = reader.readLine()) != null) {
37                 writer.write(line);
38                 writer.newLine(); // Writes platform-
39                                 // appropriate line separator
40             }
41             System.out.println("File content successfully
```

```

        copied to: " + destFile);
37
38     } catch (IOException e) {
39         System.err.println("An I/O error occurred during
        the copying process: " + e.getMessage());
40     }
41 }
42 }

```

## Part 2: Analysis of UTF-8 Target Encoding Requirement

- **What Additional Code Was Added:** An explicit encoding parameter, `StandardCharsets.UTF_8`, was introduced into the constructor chain of the output stream writer tier:

```

        new OutputStreamWriter(new FileOutputStream(destFile),
                                StandardCharsets.UTF_8)

```

- **Why This Code Is Essential:**
  - (a) **Platform Default Behavior:** In older standard Java platform specifications (prior to JDK 18), text stream implementations like legacy `FileWriter` or unconfigured character streams default to the host environment's native default charset (e.g., `Windows-1252` on standard Western systems).
  - (b) **Preventing Character Corruption (Mojibake):** The input source contains complex multi-byte scripts (such as Bangla). While the input stream successfully parses these into Java's internal 16-bit Unicode format via its own `UTF-8` reader configuration, writing them back to disk without an explicit output encoding will force the system to translate those broad characters through a narrow local system single-byte encoder.
  - (c) **Result of Skipping:** Leaving out this additional instruction will structurally compress the distinct 3-byte character groups into localized ANSI/ASCII maps, resulting in corrupted files dominated by unreadable text elements or block questions marks (???). Explicitly setting `StandardCharsets.UTF_8` preserves the structural integrity of the output.

**Q6.** You are writing a networked 2-person shooter game. Here, a bullet is modeled by a class containing positional coordinates:

```

1  class Bullet implements Serializable, Cloneable {
2      int _posX, _posY;
3      public Bullet(int x, int y) { _posX = x; _posY = y; }
4      public Object Clone() { return new Bullet(_posX, _posY); }
5  }

```



When one person shoots, the bullet leaves his weapon and follows a certain calculation model to traverse through the scene. You do not need to worry about the calculation model. During animation, the model continually updates the position of the bullet and sends the updated coordinates to the other person's machine over the network. This is done by calling the `updateBullet()` method of the `TwoPersonShooterGame` class:

```

1  public class TwoPersonShooterGame {
2      ObjectOutputStream _oos;
3      ObjectInputStream _ois;
4
5      public void updateBullet(Bullet bullet, int x, int y)
6          throws IOException {
7          bullet._posX = x;
8          bullet._posY = y;
9          _oos.writeObject(bullet);
10     }
11 }
```

On the other player's machine, the bullet is rendered accordingly.

- (a) Why does `Bullet` implement `Serializable`?
- (b) Write a constructor for `TwoPersonShooterGame`. It should have a `Socket` as a parameter and hook up the `_ois` and `_oos` variables to the socket's input and output streams respectively. The signature of the constructor should not have a `throws` clause.
- (c) Add a method in `TwoPersonShooterGame` class to read a bullet object from the network. The method should have the following signature:

`public Bullet receiveBulletFromNetwork()`

- (d) Once you have completed the tasks of (i) and (ii), the project is complete. However, you observe that in the first person's machine the bullet is moving through the screen, yet it remains stationary on the second person's machine. Why is this happening? Show three different solutions for the problem (with code and explanation).
- (e) Finally, notice that `_posX` and `_posY` are not encapsulated. If you make them `private`, `Bullet` objects can no longer be cloned as the `Clone()` method is currently written. Why? Correct the `Clone()` method so that `Bullet` objects can still be cloned.

**(3+3+3+11+4 marks)**

**[CSE 107 | 2015–16]**

### Answer — Object Serialization and Stream Caching Dynamics

#### **(i) Requirement for Serializable (3 Marks)**

The `ObjectOutputStream.writeObject()` method requires that any object passed into it must implement the `java.io.Serializable` marker interface. If a class lacks this interface, the JVM will halt execution and throw a

NotSerializableException. Implementing it enables the serialization engine to convert the object's state into a sequential sequence of bytes suitable for transmission across a network socket.

**(ii) Constructor without Throws Clause (3 Marks)**

```

1 public TwoPersonShooterGame(Socket socket) {
2     try {
3         // Crucial: ObjectOutputStream must always be
4         // initialized BEFORE ObjectInputStream
5         // to flush the stream header and prevent both
6         // endpoints from deadlocking on startup.
7         this._oos = new ObjectOutputStream(socket.
8             getOutputStream());
9         this._ois = new ObjectInputStream(socket.
10             getInputStream());
11     } catch (IOException e) {
12         System.err.println("Failed to initialize game network
13             streams: " + e.getMessage());
14     }
15 }

```

**(iii) Implementation of receiveBulletFromNetwork() (3 Marks)**

```

1 public Bullet receiveBulletFromNetwork() {
2     try {
3         return (Bullet) _ois.readObject();
4     } catch (IOException | ClassNotFoundException e) {
5         System.err.println("Error reading bullet from stream:
6             " + e.getMessage());
7         return null;
8     }
9 }

```

**(iv) Root Cause Analysis & Three Solutions (11 Marks)**

**The Problem:** By default, an `ObjectOutputStream` maintains an internal `**wire handle lookup table (cache)**` of references for every object it serializes during its lifetime. When the exact same object reference (`bullet`) is sequentially passed to `writeObject()`, the serialization algorithm optimizes bandwidth consumption by writing an ultra-compact `**back-reference handle**` instead of re-parsing the object's fields. Consequently, even though the internal values of `_posX` and `_posY` change on the sender side, the receiver side ignores them and simply links back to the original cached instance data from the very first frame.

```

1 // --- Solution 1: Explicit Stream Cache Clearing via reset()
2 // ---
3 // Explanation: Calling reset() discards the reference
4 // history table inside the stream layout,
5 // forcing a fresh, full deep serialization of the fields of
6 // any objects passed next.

```

```
4 public void updateBullet(Bullet bullet, int x, int y) throws
   IOException {
5     bullet._posX = x;
6     bullet._posY = y;
7     _oos.reset(); // Purges the object tracking table to
                   // ensure deep transmission
8     _oos.writeObject(bullet);
9 }
10
11 // --- Solution 2: Bypassing Cache via writeUnshared() ---
12 // Explanation: writeUnshared() instructs the stream to treat
   // this write operation as an
13 // entirely isolated entity, ignoring prior reference history
   // mapping and bypassing the cache.
14 public void updateBullet(Bullet bullet, int x, int y) throws
   IOException {
15     bullet._posX = x;
16     bullet._posY = y;
17     _oos.writeUnshared(bullet); // Bypasses standard back-
   // reference caching
18 }
19
20 // --- Solution 3: Emitting Deep Heap Copies (Clones) ---
21 // Explanation: Instantiating a new heap allocation address
   // breaks the reference lookup entirely.
22 // The stream treats the new reference address as a fresh
   // object graph entity and re-sends its data.
23 public void updateBullet(Bullet bullet, int x, int y) throws
   IOException {
24     bullet._posX = x;
25     bullet._posY = y;
26
27     // Invokes the clone method to pass a unique instance
   // pointer over the stream
28     _oos.writeObject(bullet.Clone());
29 }
```

#### (v) Encapsulation and Cloning Correction (4 Marks)

**Why cloning fails with the current method:** While the Java compiler allows a class to access its own private fields, implementing a manual `Clone()` method via the constructor `new Bullet(_posX, _posY)` creates issues with **inheritance**. If a subclass extends `Bullet` and attempts to utilize or inherit this manual cloning logic, it will not have access to the private fields of its parent. Furthermore, standard Java cloning should hook into the intrinsic JVM clone mechanism rather than manually passing fields to constructors.

**Correction:** We correct this by adhering to the standard `java.lang.Cloneable` contract. By overriding the intrinsic `clone()` method from `Object` and utilizing

`super.clone()`, the JVM automatically handles field-by-field memory duplication (a shallow copy) completely bypassing access modifier restrictions.

```

1  // Replaces the manual Clone() method inside the Bullet class
2  @Override
3  public Object clone() {
4      try {
5          // Native JVM bitwise copy; safely accesses all
6              fields regardless of private modifiers
7          return super.clone();
8      } catch (CloneNotSupportedException e) {
9          // This will never be reached as Bullet implements
10             Cloneable
11         throw new AssertionError("Cloning failed");
12     }
13 }

```

**Q7.** Write Java code to copy a file `src.txt` to `dst.txt` line by line using `BufferedReader` and `BufferedWriter`. What are the Java classes and methods that can read/write objects of a class to I/O stream? (10 marks) [CSE 201 | 2014–15]

#### Answer — Copy Text File and Object Stream Classes

```

1  import java.io.*;
2
3  public class TextFileCopy {
4      public static void main(String[] args) {
5          try (BufferedReader br = new BufferedReader(
6              new FileReader("src.txt"));
7              BufferedWriter bw = new BufferedWriter(
8                  new FileWriter("dst.txt"))) {
9              String line;
10             while ((line = br.readLine()) != null) {
11                 bw.write(line);
12                 bw.newLine();
13             }
14             System.out.println("Copied successfully.");
15         } catch (IOException e) {
16             System.out.println("Error: " + e.getMessage());
17         }
18     }
19 }

```

Classes and methods for reading/writing objects:

- `ObjectOutputStream` — wraps an `OutputStream`; method `writeObject(Object)` serializes an object.

- `ObjectInputStream` — wraps an `InputStream`; method `readObject()` deserializes an object.
- The class being serialized must implement `java.io.Serializable`.

**Q8.** What do you understand by serialization in Java I/O? What are the uses of serialization and how is it implemented? (6 marks) [CSE 201 | 2013–14]

### Answer — Serialization in Java

**Serialization** is the process of converting a Java object into a **byte stream** so it can be saved to a file, sent over a network, or stored in a database. The reverse process is called **deserialization**.

**Uses:**

- Persisting object state to disk (save/load).
- Transmitting objects over a network (e.g., RMI, sockets).
- Caching objects in memory or a database.
- Deep cloning of objects.

**Implementation:**

```
1  import java.io.*;
2
3  // Step 1: implement Serializable
4  class Person implements Serializable {
5      String name; int age;
6      Person(String n, int a) { name=n; age=a; }
7  }
8
9  class SerialExample {
10     public static void main(String[] args) throws Exception {
11         Person p = new Person("Alice", 25);
12
13         // Serialize
14         ObjectOutputStream oos = new ObjectOutputStream(
15             new FileOutputStream("person.dat"));
16         oos.writeObject(p); oos.close();
17
18         // Deserialize
19         ObjectInputStream ois = new ObjectInputStream(
20             new FileInputStream("person.dat"));
21         Person p2 = (Person) ois.readObject(); ois.close();
22
23         System.out.println(p2.name + ", " + p2.age); // Alice
24         , 25
```

```

24     }
25 }

```

**Q9.** There is a file named `text.txt`. Write a Java program that reads each line of the file one at a time and prints the line number of the first occurrence of the word “Bangladesh”. (10 marks) [CSE 201 | 2013–14]

#### Answer — Find First Occurrence of “Bangladesh”

```

1  import java.io.*;
2
3  public class FindWord {
4      public static void main(String[] args) {
5          String target = "Bangladesh";
6          int foundAt = -1;
7
8          try (BufferedReader br = new BufferedReader(
9              new FileReader("text.txt"))) {
10             String line;
11             int lineNum = 1;
12             while ((line = br.readLine()) != null) {
13                 if (line.contains(target)) {
14                     foundAt = lineNum;
15                     break;          // stop at first occurrence
16                 }
17                 lineNum++;
18             }
19         } catch (IOException e) {
20             System.out.println("Error: " + e.getMessage());
21         }
22
23         if (foundAt != -1)
24             System.out.println("\"" + target + "\" first
25                               found at line "
26                               + foundAt);
27         else
28             System.out.println("\"" + target + "\" not found.
29                               ");
30     }
31 }

```

**Sample output:**

"Bangladesh" first found at line 3

**Q10.** Why is `BufferedReader` a preferred reader? Write a program that appends a string in an existing file named `a.txt`. (3+8 marks) [CSE 201 | 2012–13]

**Answer — BufferedReader and Appending to a File**

**Why BufferedReader is preferred:** BufferedReader wraps another Reader and maintains an internal buffer (default 8192 chars). Instead of performing a system call for each character, it reads a large block at once, drastically reducing I/O overhead. It also provides the convenient `readLine()` method.

```

1  import java.io.*;
2
3  public class AppendDemo {
4      public static void main(String[] args) {
5          // true = append mode
6          try (BufferedWriter bw = new BufferedWriter(
7              new FileWriter("a.txt", true))) {
8              bw.write("This line is appended.");
9              bw.newLine();
10             System.out.println("String appended to a.txt");
11         } catch (IOException e) {
12             System.out.println("Error: " + e.getMessage());
13         }
14     }
15 }

```

Passing `true` to `FileWriter` opens the file in append mode so existing content is preserved.

- Q11.** The members of class `Record` are a `String id`, `String recordDate`, and `double amount`. Give the class definition. Assume `recordList.dat` contains 10 instances. Write code to read them into a `Record` array. (11 marks) [CSE 201 | 2011–12]

**Answer — Record Class and Reading 10 Objects from File**

```

1  import java.io.*;
2
3  class Record implements Serializable {
4      private String id;
5      private String recordDate;
6      private double amount;
7
8      Record(String id, String recordDate, double amount) {
9          this.id = id; this.recordDate = recordDate; this.
10             amount = amount;
11     }
12
13     @Override
14     public String toString() {
15         return "Record{id=" + id + ", date=" + recordDate
16             + ", amount=" + amount + "}";
17     }
18 }

```

```

17 }
18
19 class RecordReader {
20     public static void main(String[] args) {
21         Record[] records = new Record[10];
22
23         try (ObjectInputStream ois = new ObjectInputStream(
24             new FileInputStream("recordList.dat"))) {
25             for (int i = 0; i < 10; i++)
26                 records[i] = (Record) ois.readObject();
27         } catch (IOException | ClassNotFoundException e) {
28             e.printStackTrace();
29         }
30
31         for (Record r : records)
32             System.out.println(r);
33     }
34 }

```

**Q12.** Explain Object Serialization-Deserialization with an example. (8 marks) [CSE 201 | 2010–11]

### Answer — Object Serialization-Deserialization Example

**Serialization** converts an object to bytes; **Deserialization** converts bytes back to an object.

```

1  import java.io.*;
2
3  class Student implements Serializable {
4      private static final long serialVersionUID = 1L;
5      String name; int roll; double cgpa;
6
7      Student(String name, int roll, double cgpa) {
8          this.name=name; this.roll=roll; this.cgpa=cgpa;
9      }
10     @Override
11     public String toString() {
12         return name + " Roll:" + roll + " CGPA:" + cgpa;
13     }
14 }
15
16 class SerialDesDemo {
17     public static void main(String[] args) {
18         Student s1 = new Student("Bob", 101, 3.75);
19
20         // --- Serialization ---

```



```

21         try (ObjectOutputStream oos = new ObjectOutputStream(
22             new FileOutputStream("student.ser"))) {
23             oos.writeObject(s1);
24             System.out.println("Serialized: " + s1);
25         } catch (IOException e) { e.printStackTrace(); }
26
27         // --- Deserialization ---
28         try (ObjectInputStream ois = new ObjectInputStream(
29             new FileInputStream("student.ser"))) {
30             Student s2 = (Student) ois.readObject();
31             System.out.println("Deserialized: " + s2);
32         } catch (IOException | ClassNotFoundException e) {
33             e.printStackTrace();
34         }
35     }
36 }

```

**Output:**

Serialized: Bob Roll:101 CGPA:3.75

Deserialized: Bob Roll:101 CGPA:3.75

serialVersionUID is recommended to ensure version compatibility during deserialization.

- Q13.** Write Java code to copy a binary file `src.dat` to `dest.dat` using byte stream technique. (7 marks) [CSE 201 | 2008–09]

**Answer — Copy Binary File Using Byte Streams**

```

1  import java.io.*;
2
3  public class BinaryCopy {
4      public static void main(String[] args) {
5          try (FileInputStream fis = new FileInputStream("src.
6              dat");
7              FileOutputStream fos = new FileOutputStream("
8                  dest.dat")) {
9              byte[] buf = new byte[4096];
10             int n;
11             while ((n = fis.read(buf)) != -1)
12                 fos.write(buf, 0, n);
13             System.out.println("Binary file copied.");
14         } catch (IOException e) {
15             System.out.println("Error: " + e.getMessage());
16         }
17     }
18 }

```

```
16 }
```

**Q14.** Write a Java program that counts the total number of characters of the alternating lines of a file named `input.txt` (starting from the 1st line). Use Character Streams.  
[CSE 201 | 2007–2008]

#### Answer — Count Characters of Alternating Lines Using Character Streams

```
1  import java.io.*;
2
3  public class AltLineCount {
4      public static void main(String[] args) {
5          int totalChars = 0;
6          int lineNumber = 0;
7
8          // FileReader is a character stream (not byte stream)
9          try (BufferedReader br = new BufferedReader(
10              new FileReader("input.txt"))) {
11
12              String line;
13              while ((line = br.readLine()) != null) {
14                  lineNumber++;
15                  // Count characters only from odd lines
16                  // (1st, 3rd, 5th, ...)
17                  if (lineNumber % 2 != 0)
18                      totalChars += line.length();
19              }
20
21              } catch (FileNotFoundException e) {
22                  System.err.println("File not found: input.txt");
23              } catch (IOException e) {
24                  System.err.println("I/O error: " + e.getMessage()
25                      );
26              }
27
28              System.out.println("Total characters in alternating"
29                  + " lines (starting from line 1): " + totalChars)
30              ;
31          }
32      }
```

#### How it works:

- `FileReader` is the character-stream equivalent of `FileInputStream`; it reads characters (Unicode) rather than raw bytes.
- `BufferedReader` wraps it to provide efficient `readLine()`.

- `line.length()` gives the number of characters in the line (excluding the new-line).
- Lines are counted from 1; odd-numbered lines (1, 3, 5, ...) are the alternating lines starting from the first.

BUET · CSE 107 / CSE 201

# Object Oriented Programming

Section S18

---

Networking

## S18 — Networking

**Q1.** A Fibonacci server accepts multiple clients. Client sends two integers; server calculates count of Fibonacci numbers between them and sends result. Communicate only through objects. Write complete Java client-server program. **(15 marks)** [CSE 107 | 2022–23]

### Answer — Fibonacci Server: Object-Based Multi-Client TCP

**Protocol:** Client sends a serializable `RangeRequest` object (two integers); server replies with a serializable `FibResult` object (the count of Fibonacci numbers in that range).

```

1  // Shared serializable classes (compile separately)
2  import java.io.Serializable;
3
4  class RangeRequest implements Serializable {
5      int lo, hi;
6      RangeRequest(int lo, int hi) { this.lo=lo; this.hi=hi; }
7  }
8
9  class FibResult implements Serializable {
10     int count;
11     FibResult(int count) { this.count = count; }
12 }

1  // FibServer.java
2  import java.io.*;
3  import java.net.*;
4
5  public class FibServer {
6      public static void main(String[] args) throws Exception {
7          ServerSocket ss = new ServerSocket(5555);
8          System.out.println("Fibonacci server ready on port
9                               5555");
10         while (true)
11             new Thread(new FibHandler(ss.accept())).start();
12     }
13
14     class FibHandler implements Runnable {
15         private final Socket socket;
16         FibHandler(Socket s) { socket = s; }
17
18         @Override
19         public void run() {
20             try (ObjectInputStream in = new ObjectInputStream(
21                     socket.getInputStream());
22                  ObjectOutputStream out = new ObjectOutputStream(

```

```

23         socket.getOutputStream())) {
24
25         RangeRequest req = (RangeRequest) in.readObject()
26         ;
27         int count = countFibInRange(req.lo, req.hi);
28         out.writeObject(new FibResult(count));
29         out.flush();
30     } catch (Exception e) {
31         e.printStackTrace();
32     } finally {
33         try { socket.close(); } catch (IOException
34             ignored) {}
35     }
36
37     // Count Fibonacci numbers in [lo, hi]
38     private int countFibInRange(int lo, int hi) {
39         int count = 0;
40         int a = 0, b = 1;
41         while (a <= hi) {
42             if (a >= lo) count++;
43             int tmp = a + b; a = b; b = tmp;
44         }
45         return count;
46     }
47 }

```

```

1  // FibClient.java
2  import java.io.*;
3  import java.net.*;
4  import java.util.Scanner;
5
6  public class FibClient {
7      public static void main(String[] args) throws Exception {
8          Scanner sc = new Scanner(System.in);
9          System.out.print("Enter lo: "); int lo = sc.nextInt()
10         ;
11          System.out.print("Enter hi: "); int hi = sc.nextInt()
12         ;
13
14         try (Socket socket = new Socket("localhost", 5555);
15             // ObjectOutputStream must be created BEFORE
16             // ObjectInputStream to avoid deadlock
17             ObjectOutputStream out = new ObjectOutputStream(
18                 socket.getOutputStream());
19             ObjectInputStream in = new ObjectInputStream(
20                 socket.getInputStream())) {

```

```

19
20         out.writeObject(new RangeRequest(lo, hi));
21         out.flush();
22
23         FibResult result = (FibResult) in.readObject();
24         System.out.println("Fibonacci numbers in ["
25             + lo + ", " + hi + "]: " + result.count);
26     }
27 }
28 }

```

**Note:** `ObjectOutputStream` must be constructed *before* `ObjectInputStream` on the same socket to avoid a deadlock caused by the stream header exchange during initialisation.

- Q2.** A server program accepts multiple clients. Client sends two integers; server calculates count of prime numbers between them and sends result. Write complete Java server-client program. (15 marks) [CSE 107 | 2021–22]

#### Answer — Prime-Count Server: Multi-Client TCP

```

1  // PrimeServer.java
2  import java.io.*;
3  import java.net.*;
4
5  public class PrimeServer {
6      public static void main(String[] args) throws Exception {
7          ServerSocket ss = new ServerSocket(6666);
8          System.out.println("Prime server ready on port 6666");
9          ;
10         while (true)
11             new Thread(new PrimeHandler(ss.accept())).start();
12         ;
13     }
14 }
15
16 class PrimeHandler implements Runnable {
17     private final Socket socket;
18     PrimeHandler(Socket s) { socket = s; }
19
20     @Override
21     public void run() {
22         try (BufferedReader in = new BufferedReader(
23             new InputStreamReader(socket.getInputStream())
24         )) {
25             PrintWriter out = new PrintWriter(
26                 socket.getOutputStream(), true
27             ) {

```

```

24
25         // Receive two integers on one line: "lo hi"
26         String line = in.readLine();
27         String[] parts = line.trim().split("\\s+");
28         int lo = Integer.parseInt(parts[0]);
29         int hi = Integer.parseInt(parts[1]);
30
31         int count = 0;
32         for (int n = lo; n <= hi; n++)
33             if (isPrime(n)) count++;
34
35         out.println(count);    // send result
36
37     } catch (Exception e) {
38         e.printStackTrace();
39     } finally {
40         try { socket.close(); } catch (IOException
41             ignored) {}
42     }
43
44     private boolean isPrime(int n) {
45         if (n < 2) return false;
46         if (n == 2) return true;
47         if (n % 2 == 0) return false;
48         for (int i = 3; (long) i * i <= n; i += 2)
49             if (n % i == 0) return false;
50         return true;
51     }
52 }

```

```

1  // PrimeClient.java
2  import java.io.*;
3  import java.net.*;
4  import java.util.Scanner;
5
6  public class PrimeClient {
7      public static void main(String[] args) throws Exception {
8          Scanner sc = new Scanner(System.in);
9          System.out.print("Enter lo: "); int lo = sc.nextInt()
10             ;
11          System.out.print("Enter hi: "); int hi = sc.nextInt()
12             ;
13
14          try (Socket socket = new Socket("localhost", 6666);
15              PrintWriter out = new PrintWriter(
16                  socket.getOutputStream(), true);
17              BufferedReader in = new BufferedReader(

```



```

16         new InputStreamReader(socket.getInputStream
17                               ()))) {
18
19             out.println(lo + " " + hi);    // send range
20
21             String result = in.readLine();
22             System.out.println("Prime count in ["
23                               + lo + ", " + hi + "]: " + result);
24         }
25     }

```

Sample output (lo=1, hi=20):

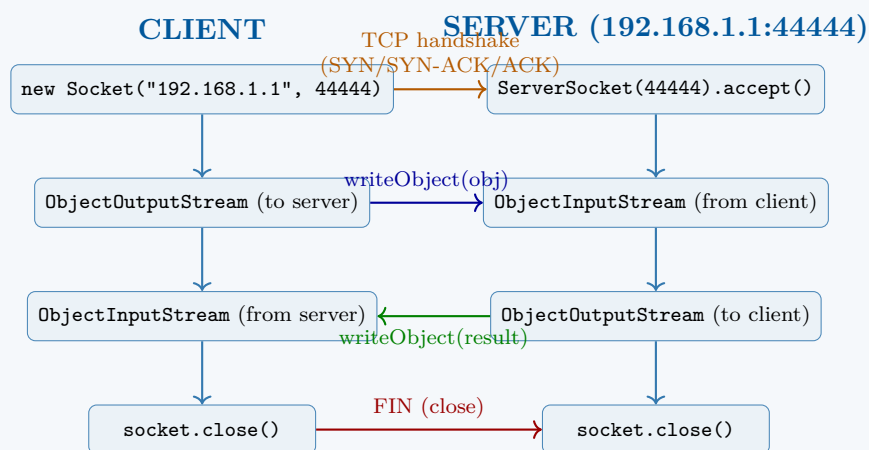
Prime count in [1, 20]: 8

(Primes: 2, 3, 5, 7, 11, 13, 17, 19)

**Q3.** Suppose a client connects to a server at 192.168.1.1, port 44444. Draw the scenario of connection and data transfer. What classes are used to read and write objects in networking? What restriction do they impose? **(10 marks)** [CSE 107 | 2020–21]

### Answer — Connection Scenario, Object Stream Classes and Their Restriction

**Connection and Data Transfer Scenario** (Client → Server at 192.168.1.1:44444):



**Classes used to read and write objects in networking:**

| Class                           | Purpose                                             |
|---------------------------------|-----------------------------------------------------|
| <code>ObjectOutputStream</code> | Serialises Java objects and writes them to a stream |
| <code>ObjectInputStream</code>  | Reads and deserialises Java objects from a stream   |

**Restriction they impose:**

Any class whose objects are sent over the network **must implement the Serializable interface**. If a class does not implement `Serializable`, calling `writeObject()` on its instance throws a `NotSerializableException` at runtime. Additionally, every field of the class (and all referenced objects) must also be serialisable, or must be declared `transient` to be excluded from serialisation.

```

1  // Correct: implements Serializable
2  class Point implements java.io.Serializable {
3      int x, y;
4      Point(int x, int y) { this.x = x; this.y = y; }
5  }
6
7  // Usage
8  ObjectOutputStream oos = new ObjectOutputStream(
9      socket.getOutputStream());
10 oos.writeObject(new Point(3, 4));    // OK
11
12 // Wrong: no Serializable -> throws NotSerializableException
13 class BadPoint { int x, y; }
14 oos.writeObject(new BadPoint());    // runtime exception!

```

**Q4.** Briefly describe the types of TCP Sockets. Write a Java Program that opens a connection to the “whois” port (port 43) on the InterNIC server, sends a website address, and prints the returned data. **(4+15 marks)** [CSE 107 | 2017–18]

**Answer — TCP Socket Types and WHOIS Client****Types of TCP Sockets in Java:**

- 1. Stream Socket (Socket)** — Represents one end of a two-way reliable TCP connection between client and server. Used for continuous byte-stream data transfer. Both client and server use this type once a connection is established.
- 2. Server Socket (ServerSocket)** — Listens on a specified port for incoming TCP connection requests. It is *not* used for data transfer itself; calling `accept()` returns a new `Socket` for each client.

**WHOIS Client (connects to port 43 on InterNIC’s whois.internic.net):**

```

1  import java.io.*;
2  import java.net.*;
3
4  public class WhoisClient {
5      static final String WHOIS_HOST = "whois.internic.net";

```

```

6      static final int      WHOIS_PORT = 43;
7
8      public static void main(String[] args) throws Exception {
9          // Website address to query
10         String domain = "google.com";
11
12         try (Socket socket = new Socket(WHOIS_HOST,
13             WHOIS_PORT);
14             PrintWriter out = new PrintWriter(
15                 socket.getOutputStream(), true);
16             BufferedReader in = new BufferedReader(
17                 new InputStreamReader(
18                     socket.getInputStream())) {
19
20             // WHOIS protocol: send domain name + CRLF
21             out.print(domain + "\r\n");
22             out.flush();
23
24             // Read and print the response line by line
25             String line;
26             while ((line = in.readLine()) != null)
27                 System.out.println(line);
28         }
29     }

```

**How it works:**

- A Socket is opened to `whois.internic.net` on port 43.
- The domain name is sent as plain text followed by a carriage-return + line-feed (`\r\n`), which is the WHOIS protocol.
- The server streams back the registration information; the client reads and prints each line until the server closes the connection.

**Q5.** Develop a chat application where a server can handle maximum 100 clients. Clients can talk to any other client through the server. A client can form a group of maximum 5. Write using Java TCP/IP socket programming. **(25 marks)** [CSE 201 | 2015–16]

### Answer — Chat Application: Server Handling 100 Clients with Group Support

**Design Overview:**

- The server maintains a list of all connected clients (max 100) and a list of groups (each max 5 members).

- Each connected client is handled by a dedicated `ClientHandler` thread.
- A client can send a message to a specific client (unicast) or to a group.
- Groups are identified by a group name stored in a shared `Map`.

Server code:

```
1  import java.io.*;
2  import java.net.*;
3  import java.util.*;
4  import java.util.concurrent.*;
5
6  public class ChatServer {
7      static final int PORT          = 9999;
8      static final int MAX_CLIENTS   = 100;
9      static final int MAX_GROUP     = 5;
10
11     // username -> handler
12     static Map<String, ClientHandler> clients =
13         new ConcurrentHashMap<>();
14     // groupName -> set of usernames
15     static Map<String, Set<String>> groups =
16         new ConcurrentHashMap<>();
17
18     public static void main(String[] args) throws IOException
19     {
20         ServerSocket ss = new ServerSocket(PORT);
21         System.out.println("Chat server started on port " +
22             PORT);
23         while (true) {
24             if (clients.size() >= MAX_CLIENTS) continue;
25             Socket s = ss.accept();
26             new Thread(new ClientHandler(s)).start();
27         }
28     }
29 }
```

```
1  class ClientHandler implements Runnable {
2      private Socket socket;
3      private PrintWriter out;
4      private BufferedReader in;
5      private String username;
6
7      ClientHandler(Socket s) { this.socket = s; }
8
9      @Override
10     public void run() {
11         try {
```

```

12         in = new BufferedReader(
13             new InputStreamReader(socket.
14                 getInputStream()));
15         out = new PrintWriter(socket.getOutputStream(),
16             true);
17
18         // First message is the username
19         username = in.readLine();
20         ChatServer.clients.put(username, this);
21         broadcast "[" + username + " joined the chat");
22
23         String msg;
24         while ((msg = in.readLine()) != null) {
25             // Protocol:
26             // "@user message"      -> private message
27             // "#group message"     -> group message
28             // "/create groupName" -> create group
29             // "/join groupName"    -> join group
30             if (msg.startsWith("@")) {
31                 // private message: @targetUser actual
32                 // message
33                 int sp = msg.indexOf(' ');
34                 String target = msg.substring(1, sp);
35                 String text = msg.substring(sp + 1);
36                 sendTo(target, "[PM from " + username + "
37                     ]: " + text);
38             } else if (msg.startsWith("#")) {
39                 int sp = msg.indexOf(' ');
40                 String grp = msg.substring(1, sp);
41                 String text = msg.substring(sp + 1);
42                 sendToGroup(grp, username, text);
43             } else if (msg.startsWith("/create ")) {
44                 String grp = msg.substring(8).trim();
45                 ChatServer.groups.putIfAbsent(grp,
46                     ConcurrentHashMap.newKeySet());
47                 ChatServer.groups.get(grp).add(username);
48                 out.println("Group '" + grp + "' created.
49                     ");
50             } else if (msg.startsWith("/join ")) {
51                 String grp = msg.substring(6).trim();
52                 Set<String> members = ChatServer.groups.
53                     get(grp);
54                 if (members == null) {
55                     out.println("Group not found.");
56                 } else if (members.size() >= ChatServer.
57                     MAX_GROUP) {
58                     out.println("Group is full (max 5).");
59                 }
60             }
61         }

```

```

52         } else {
53             members.add(username);
54             out.println("Joined group '" + grp +
55                 "'.");
56         }
57     } else {
58         broadcast "[" + username + "]: " + msg);
59     }
60 } catch (IOException e) {
61     System.out.println(username + " disconnected.");
62 } finally {
63     ChatServer.clients.remove(username);
64     broadcast "[" + username + " left the chat]");
65     try { socket.close(); } catch (IOException
66         ignored) {}
67 }
68
69 void sendMessage(String msg) { out.println(msg); }
70
71 private void sendTo(String target, String msg) {
72     ClientHandler h = ChatServer.clients.get(target);
73     if (h != null) h.sendMessage(msg);
74     else out.println("User '" + target + "' not found.");
75 }
76
77 private void sendToGroup(String grp, String sender,
78     String msg) {
79     Set<String> members = ChatServer.groups.get(grp);
80     if (members == null) { out.println("Group not found.");
81         }; return; }
82     for (String m : members) {
83         if (!m.equals(sender)) {
84             ClientHandler h = ChatServer.clients.get(m);
85             if (h != null) h.sendMessage(
86                 "[Group " + grp + " | " + sender + "]: "
87                 + msg);
88         }
89     }
90 }
91
92 private void broadcast(String msg) {
93     for (ClientHandler h : ChatServer.clients.values())
94         h.sendMessage(msg);
95 }

```

```

1  // Client code
2  import java.io.*;
3  import java.net.*;
4
5  public class ChatClient {
6      public static void main(String[] args) throws Exception {
7          Socket socket = new Socket("localhost", 9999);
8          BufferedReader in = new BufferedReader(
9              new InputStreamReader(socket.getInputStream()));
10         PrintWriter out = new PrintWriter(
11             socket.getOutputStream(), true);
12         BufferedReader kbd = new BufferedReader(
13             new InputStreamReader(System.in));
14
15         System.out.print("Enter username: ");
16         String username = kbd.readLine();
17         out.println(username);    // send username first
18
19         // Thread to read messages from server
20         new Thread(() -> {
21             try {
22                 String line;
23                 while ((line = in.readLine()) != null)
24                     System.out.println(line);
25             } catch (IOException ignored) {}
26         }).start();
27
28         // Main thread reads keyboard input and sends to
29         // server
30         String input;
31         while ((input = kbd.readLine()) != null)
32             out.println(input);
33
34         socket.close();
35     }
36 }

```

#### Message Protocol Summary:

| Command      | Effect                             |
|--------------|------------------------------------|
| @alice Hello | Private message to user alice      |
| #team Hi all | Group message to group team        |
| /create team | Create group named team            |
| /join team   | Join group team (max 5 members)    |
| (plain text) | Broadcast to all connected clients |

**Q6.** Write a Java program to establish a connection to [www.google.com](http://www.google.com) using

URLConnection. Output: (i) request method, (ii) response code, (iii) response message, (iv) keys and values in the response header. **(10 marks)** [CSE 107 | 2015–16]

#### Answer — HttpURLConnection to www.google.com

```
1  import java.io.*;
2  import java.net.*;
3  import java.util.*;
4
5  public class GoogleHttpDemo {
6      public static void main(String[] args) throws Exception {
7          URL url = new URL("https://www.google.com");
8          HttpURLConnection conn =
9              (HttpURLConnection) url.openConnection();
10
11         // Set request method (GET by default)
12         conn.setRequestMethod("GET");
13         conn.connect();
14
15         // (i) Request method
16         System.out.println("Request Method : "
17             + conn.getRequestMethod());
18
19         // (ii) Response code
20         System.out.println("Response Code : "
21             + conn.getResponseCode());
22
23         // (iii) Response message
24         System.out.println("Response Message: "
25             + conn.getResponseMessage());
26
27         // (iv) Response header keys and values
28         System.out.println("\n--- Response Headers ---");
29         Map<String, List<String>> headers =
30             conn.getHeaderFields();
31
32         for (Map.Entry<String, List<String>> entry
33             : headers.entrySet()) {
34             // Key can be null for the status line
35             System.out.println(entry.getKey()
36                 + " : " + entry.getValue());
37         }
38
39         conn.disconnect();
40     }
41 }
```

Sample output:



```

Request Method : GET
Response Code  : 200
Response Message: OK

--- Response Headers ---
null           : [HTTP/1.1 200 OK]
Content-Type   : [text/html; charset=UTF-8]
Cache-Control  : [private, max-age=0]
Date           : [Mon, 01 Jan 2024 10:00:00 GMT]
Server         : [gws]
...

```

#### Key classes:

- `URL` — represents the resource address.
- `URLConnection` — subclass of `URLConnection`; provides HTTP-specific methods such as `getResponseCode()` and `getHeaderFields()`.
- `getHeaderFields()` returns a `Map<String, List<String>>` where `null` key holds the HTTP status line.

**Q7.** Why does `Socket` implement `AutoCloseable`? Explain with a sample code snippet. (5 marks) [CSE 107 | 2015–16]

#### Answer — Why Socket Implements AutoCloseable

##### Why Socket implements AutoCloseable:

`Socket` holds an underlying OS-level file descriptor (a network handle). If the socket is not closed after use, the OS resource is **leaked**: the file descriptor stays open even after the Java object is garbage-collected (because GC timing is non-deterministic). By implementing `AutoCloseable` (and its sub-interface `Closeable`), `Socket` can be used in a **try-with-resources** block, which guarantees that `close()` is called *automatically* when the block exits — whether normally or due to an exception. This eliminates the need for a `finally` block and prevents resource leaks.

```

1  // WITHOUT AutoCloseable (error-prone)
2  Socket s = null;
3  try {
4      s = new Socket("localhost", 8080);
5      // ... use socket ...
6  } catch (IOException e) {
7      e.printStackTrace();
8  } finally {
9      if (s != null)
10         try { s.close(); } catch (IOException ignored) {}
11  }
12

```

```

13 // WITH AutoCloseable (try-with-resources -- clean and safe)
14 try (Socket s = new Socket("localhost", 8080);
15     BufferedReader in = new BufferedReader(
16         new InputStreamReader(s.getInputStream()));
17     PrintWriter out = new PrintWriter(
18         s.getOutputStream(), true)) {
19
20     out.println("Hello server");
21     System.out.println("Server says: " + in.readLine());
22
23 } // s, in, out all closed automatically here,
24 // even if an exception is thrown above

```

**Key benefit:** The try-with-resources block calls `s.close()` (and also closes the `BufferedReader` and `PrintWriter` declared in the same block) in **reverse declaration order**, ensuring no descriptor is leaked regardless of how the block exits.

**Q8.** Which protocol uses Datagram packet? Write a client program that receives a Datagram packet using port 1111. (10 marks) [CSE 201 | 2012–13]

#### Answer — Datagram Protocol and UDP Client

**Which protocol uses Datagram packets?**

UDP (User Datagram Protocol) uses `DatagramPacket`. Unlike TCP, UDP is:

- **Connectionless** — no handshake before sending data.
- **Unreliable** — packets may be lost, duplicated, or arrive out of order.
- **Fast** — no connection setup overhead; suitable for streaming, DNS, and gaming.

In Java, UDP is handled by `DatagramSocket` and `DatagramPacket`.

**Client program that receives a `DatagramPacket` on port 1111:**

```

1  import java.net.*;
2
3  public class UDPClient {
4      public static void main(String[] args) throws Exception {
5          // Bind a DatagramSocket to the local port 1111
6          DatagramSocket socket = new DatagramSocket(1111);
7
8          byte[] buffer = new byte[1024];
9
10         // Prepare an empty packet to receive data into
11         DatagramPacket packet =
12             new DatagramPacket(buffer, buffer.length);
13

```

```

14         System.out.println("Waiting for datagram on port
15                               1111...");
16
17         // Blocks until a datagram arrives
18         socket.receive(packet);
19
20         // Convert received bytes to a String
21         String received = new String(
22             packet.getData(), 0, packet.getLength());
23
24         System.out.println("Received from "
25             + packet.getAddress() + ":" + packet.getPort());
26         System.out.println("Message: " + received);
27
28         socket.close();
29     }
30 }

```

Key classes used:

| Class          | Role                                            |
|----------------|-------------------------------------------------|
| DatagramSocket | UDP socket; can send and receive datagrams      |
| DatagramPacket | Wraps the byte array payload, address, and port |

**Q9.** Write a Java program to exchange messages between two peer computers using byte stream with stream socket. (20 marks) [CSE 201 | 2009–10]

### Answer — Byte-Stream Message Exchange Between Two Peers

**Design:** Peer A acts as server (listens on a port); Peer B connects as client. Both then simultaneously read from the network and write from the keyboard using two threads each.

```

1  // PeerServer.java (Peer A -- listens first)
2  import java.io.*;
3  import java.net.*;
4
5  public class PeerServer {
6      public static void main(String[] args) throws Exception {
7          ServerSocket ss = new ServerSocket(7777);
8          System.out.println("Waiting for peer on port 7777...");
9
10         Socket socket = ss.accept();
11         System.out.println("Peer connected: "
12             + socket.getInetAddress());
13
14         // Wrap byte streams in convenient text readers/

```

```

14         writers
15         BufferedReader netIn = new BufferedReader(
16             new InputStreamReader(socket.getInputStream()));
17         PrintWriter netOut = new PrintWriter(
18             socket.getOutputStream(), true);
19         BufferedReader kbd = new BufferedReader(
20             new InputStreamReader(System.in));
21
22         // Thread: continuously read messages from peer
23         Thread reader = new Thread(() -> {
24             try {
25                 String line;
26                 while ((line = netIn.readLine()) != null)
27                     System.out.println("[Peer]: " + line);
28             } catch (IOException e) {
29                 System.out.println("Peer disconnected.");
30             }
31         });
32         reader.setDaemon(true);
33         reader.start();
34
35         // Main thread: read keyboard and send to peer
36         String msg;
37         while ((msg = kbd.readLine()) != null) {
38             netOut.println(msg);
39             if (msg.equalsIgnoreCase("bye")) break;
40         }
41
42         socket.close();
43         ss.close();
44     }

```

```

1  // PeerClient.java (Peer B -- connects to Peer A)
2  import java.io.*;
3  import java.net.*;
4
5  public class PeerClient {
6      public static void main(String[] args) throws Exception {
7          Socket socket = new Socket("localhost", 7777);
8          System.out.println("Connected to peer.");
9
10         BufferedReader netIn = new BufferedReader(
11             new InputStreamReader(socket.getInputStream()));
12         PrintWriter netOut = new PrintWriter(
13             socket.getOutputStream(), true);
14         BufferedReader kbd = new BufferedReader(
15             new InputStreamReader(System.in));

```

```

16
17      // Thread: read from peer
18      Thread reader = new Thread(() -> {
19          try {
20              String line;
21              while ((line = netIn.readLine()) != null)
22                  System.out.println("[Peer]: " + line);
23          } catch (IOException e) {
24              System.out.println("Peer disconnected.");
25          }
26      });
27      reader.setDaemon(true);
28      reader.start();
29
30      // Main: type and send
31      String msg;
32      while ((msg = kbd.readLine()) != null) {
33          netOut.println(msg);
34          if (msg.equalsIgnoreCase("bye")) break;
35      }
36
37      socket.close();
38  }
39 }

```

**Key points:**

- Byte streams (InputStream/ OutputStream) are wrapped in BufferedReader / PrintWriter for convenient line-oriented text exchange.
- A daemon reader thread handles incoming messages concurrently so neither peer blocks the other.
- Typing bye closes the connection gracefully.

**Q10.** What do you know about Socket and ServerSocket? Draw the scenario of operations for connection and data transfer between a client (IP 192.168.0.63, port 55555) and a server. No code needed. (8 marks) [CSE 201 | 2008–09]

**Answer — Socket, ServerSocket and Client-Server Scenario****Socket**

A **Socket** represents **one endpoint** of a two-way TCP connection between two machines. It encapsulates:

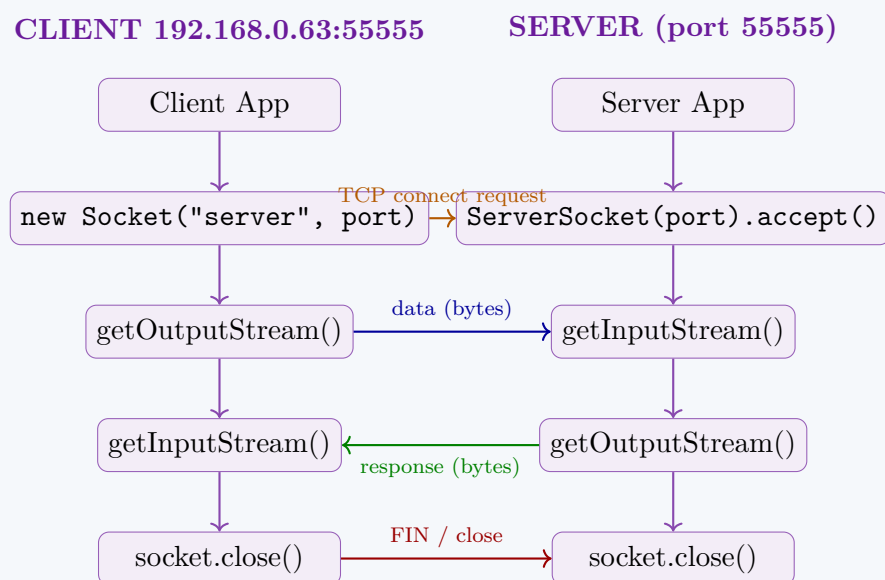
- The remote IP address and port.
- An **InputStream** to read data from the remote end.
- An **OutputStream** to write data to the remote end.

The client creates a `Socket` by specifying the server's IP and port; the JVM completes the TCP three-way handshake automatically.

### ServerSocket

A `ServerSocket` **listens** on a specified port for incoming connection requests. It does *not* itself transfer data; calling `accept()` blocks until a client connects and then returns a new `Socket` dedicated to that client. The `ServerSocket` then goes back to listening.

**Scenario: Client (192.168.0.63:55555) connects to Server**



### Step-by-step description:

1. Server creates `ServerSocket` on port 55555 and calls `accept()` — it blocks, waiting for a client.
2. Client creates `new Socket("server-IP", 55555)`. The JVM performs the TCP three-way handshake (SYN, SYN-ACK, ACK).
3. `accept()` returns a new `Socket` on the server side dedicated to this client. The `ServerSocket` resumes listening for more clients.
4. Client writes data through `getOutputStream()`; server reads it via `getInputStream()`.
5. Server writes a response through its `getOutputStream()`; client reads via its `getInputStream()`.
6. Either side calls `close()` to terminate the connection (TCP FIN sequence).

**Q11.** A multithreaded server at IP 172.20.0.3, port 33333 handles multiple clients. Client sends a String; server returns its reverse. Client displays both strings. Write Java

code for the complete scenario. (15 marks)

[CSE 201 | 2007–08]

### Answer — Multithreaded String-Reverse Server at 172.20.0.3:33333

#### Design:

- Server binds to port 33333 and spawns a new thread per client.
- Client sends one `String`; server returns the reversed string.
- Client displays both the original and the reversed string.

```
1  // ReverseServer.java
2  import java.io.*;
3  import java.net.*;
4
5  public class ReverseServer {
6      public static void main(String[] args) throws Exception {
7          ServerSocket ss = new ServerSocket(33333);
8          System.out.println("Reverse server ready on port
9                               33333");
10
11         while (true) {
12             Socket client = ss.accept();
13             System.out.println("Client connected: "
14                               + client.getInetAddress());
15             new Thread(new ReverseHandler(client)).start();
16         }
17     }
18
19     class ReverseHandler implements Runnable {
20         private final Socket socket;
21         ReverseHandler(Socket s) { socket = s; }
22
23         @Override
24         public void run() {
25             try (BufferedReader in = new BufferedReader(
26                 new InputStreamReader(socket.getInputStream())
27             );
28                 PrintWriter out = new PrintWriter(
29                     socket.getOutputStream(), true)) {
30
31                 String original = in.readLine();
32                 if (original == null) return;
33
34                 String reversed =
35                     new StringBuilder(original).reverse().
36                         toString();
```

```

36         out.println(reversed);
37         System.out.println("Served: \""
38             + original + "\" -> \"" + reversed + "\"");
39
40     } catch (IOException e) {
41         e.printStackTrace();
42     } finally {
43         try { socket.close(); } catch (IOException
44             ignored) {}
45     }
46 }

```

```

1  // ReverseClient.java
2  import java.io.*;
3  import java.net.*;
4  import java.util.Scanner;
5
6  public class ReverseClient {
7      public static void main(String[] args) throws Exception {
8          // Connect to server at 172.20.0.3, port 33333
9          try (Socket socket = new Socket("172.20.0.3", 33333);
10              PrintWriter out = new PrintWriter(
11                  socket.getOutputStream(), true);
12              BufferedReader in = new BufferedReader(
13                  new InputStreamReader(
14                      socket.getInputStream())) {
15
16              Scanner kbd = new Scanner(System.in);
17              System.out.print("Enter a string: ");
18              String original = kbd.nextLine();
19
20              out.println(original);           // send to
21                                              server
22
23              String reversed = in.readLine(); // receive
24                                              reversed
25
26              System.out.println("Original : " + original);
27              System.out.println("Reversed : " + reversed);
28          }
29      }
30  }

```

### Sample interaction:

```

Enter a string: BUET
Original : BUET

```



Reversed : TEUB

**Multithreading role:** Each call to `ss.accept()` is immediately handed off to a new `ReverseHandler` thread, so the server can serve multiple clients simultaneously without any client waiting for another to finish.

BUET · CSE 107 / CSE 201

# Object Oriented Programming

Section S19

---

Miscellaneous Java

## S19 — Miscellaneous Java

**Q1.** What are the improvements in Swing over AWT? (5 marks) [CSE 201 | 2013–14]

### Answer — Improvements of Swing Over AWT

- 1. Lightweight components:** AWT components are *heavyweight* — each is a native OS widget and depends on the underlying platform. Swing components are *lightweight* — drawn entirely by Java in a single native window, giving consistent appearance across all platforms.
- 2. Pluggable Look and Feel (PLAF):** Swing supports changeable look-and-feel themes (Metal, Nimbus, Windows, GTK+) at runtime. AWT always uses the native platform style.
- 3. Richer component set:** Swing provides many components not in AWT: JTable, JTree, JList, JTabbedPane, JSlider, JSpinner, JToolBar, and more.
- 4. MVC architecture:** Swing components are built on Model-View-Controller, separating data (model) from display (view) and input (controller). This makes components highly customisable.
- 5. More powerful features:**
  - Borders, icons, tooltips, and mnemonics built in.
  - Support for Action objects to share functionality between buttons and menu items.
  - Double-buffering enabled by default, reducing flicker.
- 6. Accessibility support:** Swing has built-in hooks for screen readers and assistive technologies via the Java Accessibility API.

**Q2.** You have to develop a user interface where the user can save data or cancel using either two buttons or from a Menu. Each button and its corresponding menu item will have the same caption, mnemonic, tool-tip text, and event-handling code. Implement this requirement using the most appropriate feature of Java. (15 marks) [CSE 201 | 2013–14]

### Answer — Save/Cancel Buttons and Menu Items Sharing Action

The **most appropriate feature** is `javax.swing.Action` (specifically `AbstractAction`). A single `Action` object stores the caption, mnemonic, tooltip text, and event-handling code. Both the `JButton` and the `JMenuItem` are constructed from the same `Action`, automatically sharing all properties.

```
1 import javax.swing.*;  
2 import java.awt.*;  
3 import java.awt.event.*;
```

```

4
5 public class SaveCancelDemo extends JFrame {
6
7     // -- Define Actions -----
8     private final Action saveAction = new AbstractAction("
9         Save") {
10         { // instance initialiser - set shared properties
11             putValue(SHORT_DESCRIPTION, "Save the current
12                 data");
13             putValue(MNEMONIC_KEY, KeyEvent.VK_S);
14         }
15         @Override
16         public void actionPerformed(ActionEvent e) {
17             JOptionPane.showMessageDialog(
18                 SaveCancelDemo.this, "Data saved!");
19         }
20     };
21
22     private final Action cancelAction =
23         new AbstractAction("Cancel") {
24         {
25             putValue(SHORT_DESCRIPTION, "Cancel and discard")
26             ;
27             putValue(MNEMONIC_KEY, KeyEvent.VK_C);
28         }
29         @Override
30         public void actionPerformed(ActionEvent e) {
31             JOptionPane.showMessageDialog(
32                 SaveCancelDemo.this, "Cancelled.");
33         }
34     };
35
36     SaveCancelDemo() {
37         setTitle("Save / Cancel Demo");
38         setDefaultCloseOperation(EXIT_ON_CLOSE);
39         setLayout(new FlowLayout());
40
41         // -- Buttons built from Actions -----
42         add(new JButton(saveAction));
43         add(new JButton(cancelAction));
44
45         // -- Menu built from the same Actions -----
46         JMenuBar menuBar = new JMenuBar();
47         JMenu fileMenu = new JMenu("File");
48         fileMenu.add(new JMenuItem(saveAction));
49         fileMenu.add(new JMenuItem(cancelAction));
50         menuBar.add(fileMenu);
51         setJMenuBar(menuBar);

```

```

49
50         setSize(300, 100);
51         setVisible(true);
52     }
53
54     public static void main(String[] args) {
55         SwingUtilities.invokeLater(SaveCancelDemo::new);
56     }
57 }

```

**Why Action is the right choice:**

- Caption, mnemonic, tooltip and handler are defined **once** in the **Action** and automatically applied to every widget built from it.
- Disabling the **Action** (`setEnabled(false)`) simultaneously disables both the button and the menu item.

**Q3.** Which one is preferred: semantic or low-level events? Mention two examples of each type. (6 marks) [CSE 201 | 2013–14]

#### Answer — Semantic vs Low-Level Events

**Semantic events** represent a high-level user action that has a clear meaning independent of the input device. **Low-level events** represent raw, device-level interactions.

| Semantic Events                                                                                                | Low-Level Events                                                                                             |
|----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Represent a logical user action.                                                                               | Represent raw input device actions.                                                                          |
| Device-independent.                                                                                            | Tied to a specific device (keyboard, mouse).                                                                 |
| Higher abstraction; easier to use.                                                                             | More verbose; require more handling code.                                                                    |
| <b>Examples:</b> <code>ActionEvent</code> (button click, Enter key), <code>ItemEvent</code> (checkbox toggled) | <b>Examples:</b> <code>KeyEvent</code> (key pressed/released), <code>MouseEvent</code> (mouse moved/clicked) |

**Which is preferred?**

**Semantic events** are preferred wherever possible because they are device-independent, require less code, and clearly express the user's intent. Low-level events are used only when fine-grained device control is needed (e.g. detecting which specific key was pressed, or tracking mouse coordinates).

**Q4.** Design a user interface where a `TextArea` and a button are added using `FlowLayout`. Every key the user types will be added to the `TextArea` and the program will exit when ~1 is pressed. (7 marks) [CSE 201 | 2012–13]

## Answer — TextArea and Button with FlowLayout; Exit on ~1

```
1  import java.awt.*;
2  import java.awt.event.*;
3
4  public class TextAreaDemo extends Frame
5                          implements KeyListener {
6      private TextArea textArea;
7      private Button    button;
8
9      TextAreaDemo() {
10         setLayout(new FlowLayout());
11
12         textArea = new TextArea(5, 30);
13         button   = new Button("Clear");
14
15         add(textArea);
16         add(button);
17
18         // Listen for key events on the TextArea
19         textArea.addKeyListener(this);
20
21         button.addActionListener(e -> textArea.setText(""));
22
23         setTitle("TextArea Demo");
24         setSize(400, 200);
25         setVisible(true);
26
27         addWindowListener(new WindowAdapter() {
28             public void windowClosing(WindowEvent e) {
29                 System.exit(0);
30             }
31         });
32     }
33
34     @Override
35     public void keyTyped(KeyEvent e) {
36         // Exit when '~' followed by '1' combination
37         // Simple approach: exit when user types tilde (~)
38         // The question says "~1" as a sequence trigger;
39         // here we treat the literal character '~' as exit.
40         if (e.getKeyChar() == '~') {
41             System.out.println("Exiting...");
42             System.exit(0);
43         }
44         // All other characters are added automatically
45         // by the TextArea's default behaviour.
46     }
47 }
```

```

48     @Override public void keyPressed(KeyEvent e) {}
49     @Override public void keyReleased(KeyEvent e) {}
50
51     public static void main(String[] args) {
52         new TextAreaDemo();
53     }
54 }

```

**Note:** The `TextArea` handles its own typing automatically; the `KeyListener` is added only to detect the exit key combination (~1). All other keystrokes are passed through to the component.

**Q5.** What are the different ways of event handling in Java? (5 marks) [CSE 201 | 2011–12]

### Answer — Different Ways of Event Handling in Java

Java provides **three** common ways to handle events using the Delegation Event Model:

#### 1. Separate listener class

A dedicated class implements the listener interface and is registered on the component. Best for reusable, complex logic.

```

1  class MyListener implements ActionListener {
2      public void actionPerformed(ActionEvent e) {
3          System.out.println("Button clicked!");
4      }
5  }
6  button.addActionListener(new MyListener());

```

#### 2. The enclosing class implements the listener

The frame or panel class itself implements the listener interface, making **this** the listener. Convenient for small programs.

```

1  class MyFrame extends JFrame
2      implements ActionListener {
3      public void actionPerformed(ActionEvent e) {
4          System.out.println("Handled by frame");
5      }
6      MyFrame() { button.addActionListener(this); }
7  }

```

#### 3. Anonymous inner class

The listener is defined inline at the point of registration. Concise for single-use handlers.

```

1  button.addActionListener(new ActionListener() {

```

```

2     public void actionPerformed(ActionEvent e) {
3         System.out.println("Anonymous handler");
4     }
5 });
6 // Or with a lambda (Java 8+):
7 button.addActionListener(
8     e -> System.out.println("Lambda handler"));

```

**Q6.** Explain the roles of the entities involved in Java's Delegation Event Handling model.  
(5 marks) [CSE 201 | 2010–11]

### Answer — Java Delegation Event Handling Model

#### Entities in the Delegation Event Model:

- 1. Event Source** — The component that generates an event (e.g., a JButton). It maintains a list of registered listeners and notifies them when an event occurs.
- 2. Event Object** — An object that encapsulates information about the event (e.g., ActionEvent, MouseEvent). It is created by the source and passed to the listener.
- 3. Event Listener** — An object that implements a listener interface (e.g., ActionListener) and defines the handler method (e.g., actionPerformed()). It is registered with the source via addXxxListener().

**Flow:** Source generates event → creates Event Object → calls listener's handler method passing the Event Object.

```

1  import javax.swing.*;
2  import java.awt.event.*;
3
4  public class DelegationDemo {
5      public static void main(String[] args) {
6          JFrame frame = new JFrame("Demo");
7          JButton btn = new JButton("Click Me");
8
9          // Register listener (delegation)
10         btn.addActionListener(new ActionListener() {
11             @Override
12             public void actionPerformed(ActionEvent e) {
13                 // e = Event Object, this = Listener
14                 System.out.println("Button clicked! "
15                                     + e.getActionCommand());
16             }
17         });
18
19         frame.add(btn);

```



```
20         frame.setSize(200, 100);
21         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
22         frame.setVisible(true);
23     }
24 }
```

The **key idea** is that event handling is *delegated* from the source to a separate listener object — the source does not handle its own events. This cleanly separates UI components from business logic.

**Q7.** What is an adapter class? What are the advantages and disadvantages of using adapter classes? (5 marks) [CSE 201 | 2010–11]

### Answer — Adapter Classes: Definition, Advantages and Disadvantages

#### What is an adapter class?

An **adapter class** is an abstract class that provides *empty default implementations* of all methods in a corresponding listener interface. For example, `MouseAdapter` implements `MouseListener` with seven empty methods. A programmer extends the adapter and overrides *only* the methods they care about, rather than being forced to implement every method of the interface.

```
1  // Without adapter: must implement ALL 7 MouseListener
   methods
2  // With adapter: override only what you need
3  component.addMouseListener(new MouseAdapter() {
4      @Override
5      public void mouseClicked(MouseEvent e) {
6          System.out.println("Clicked at "
7              + e.getX() + "," + e.getY());
8      }
9      // mouseMoved, mouseEntered, etc. not needed here
10 });
```

#### Advantages:

- **Less boilerplate:** You only override the methods you need; the adapter provides empty bodies for the rest.
- **Readable code:** Intent is clear — only the relevant handler method appears.

#### Disadvantages:

- **Consumes the single inheritance slot:** Because Java does not support multiple inheritance, extending an adapter class prevents the class from extending any other class.
- **Silent errors:** If a method name is misspelled when overriding, the compiler does not warn (no `@Override` error if the method signature does not match); the empty adapter method runs instead.

- Q8.** Write Java and HTML code for an Applet that draws the string “Welcome to Java”.  
(6 marks) [CSE 201 | 2010–11]

#### Answer — Applet Displaying “Welcome to Java”

Java code:

```
1  import java.applet.*;
2  import java.awt.*;
3
4  public class WelcomeApplet extends Applet {
5      @Override
6      public void paint(Graphics g) {
7          g.setFont(new Font("Arial", Font.BOLD, 20));
8          g.setColor(Color.BLUE);
9          g.drawString("Welcome to Java", 50, 80);
10     }
11 }
```

HTML code to embed the applet:

```
1  <html>
2      <body>
3          <applet code="WelcomeApplet.class"
4              width="300" height="150">
5              Your browser does not support Java Applets.
6          </applet>
7      </body>
8  </html>
```

**Note:** Java Applets are deprecated since Java 9 and removed in Java 17. In modern practice, the same result is achieved with a JFrame/JPanel application or a web-based technology.

- Q9.** How do you specify a layout manager to a container object? Write the names of all layout managers. Describe any 3 layout managers with appropriate layout pictures.  
(15 marks) [CSE 201 | 2009–10]

#### Answer — Layout Managers: Names and Descriptions

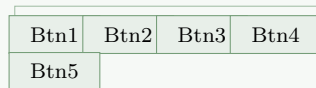
Setting a layout manager:

```
1  container.setLayout(new FlowLayout()); // example
```

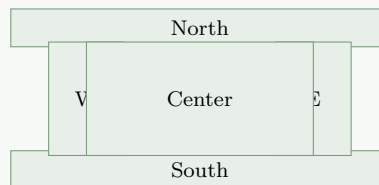
All standard layout managers: FlowLayout, BorderLayout, GridLayout, GridBagLayout, CardLayout, BoxLayout, SpringLayout, GroupLayout (Swing only), null (absolute positioning).

Description of 3 layout managers:

1. **FlowLayout** — places components left-to-right in a row; wraps to the next row when the row is full. Default for JPanel.



2. **BorderLayout** — divides the container into five regions: NORTH, SOUTH, EAST, WEST, CENTER. Default for JFrame's content pane.



3. **GridLayout** — arranges components in a regular grid of equal-sized cells; components fill cells row by row.



- Q10.** Create a GUI using **JFrame** with specific dimensions. Add an input field with a label to take a number from the user and a button labelled “Click on Me”. Display a click counter that increments at each click. Stop incrementing once the number of clicks exceeds the number entered by the user; show an appropriate message thereafter.

(20 marks)

[CSE 201 | 2009–10]

### Answer — JFrame Click Counter with User-Defined Limit

```

1  import javax.swing.*;
2  import java.awt.*;
3  import java.awt.event.*;
4
5  public class ClickCounter extends JFrame {
6      private JLabel    promptLabel    = new JLabel("Enter limit
7          :");
8      private JTextField limitField    = new JTextField(5);
9      private JButton   clickButton   = new JButton("Click on
10         Me");
11      private JLabel    countLabel    = new JLabel("Clicks: 0")
12          ;
13
14      private int clicks = 0;
15      private int limit  = Integer.MAX_VALUE;
16
17      ClickCounter() {
18          setTitle("Click Counter");
19      }
20  }

```

```
16      setLayout(new FlowLayout());
17      setDefaultCloseOperation(EXIT_ON_CLOSE);
18
19      add(promptLabel);
20      add(limitField);
21      add(clickButton);
22      add(countLabel);
23
24      clickButton.addActionListener(e -> {
25          // Read the limit from the text field if not yet
           set
26          try {
27              limit = Integer.parseInt(limitField.getText()
                                     .trim());
28          } catch (NumberFormatException ex) {
29              limit = Integer.MAX_VALUE;
30          }
31
32          if (clicks < limit) {
33              clicks++;
34              countLabel.setText("Clicks: " + clicks);
35              if (clicks == limit)
36                  JOptionPane.showMessageDialog(this,
37                                                  "Limit of " + limit
38                                                  + " clicks reached!");
39          } else {
40              JOptionPane.showMessageDialog(this,
41                                                  "Click limit already reached. No more
           counting.");
42          }
43      });
44
45      setSize(350, 120);
46      setVisible(true);
47  }
48
49  public static void main(String[] args) {
50      SwingUtilities.invokeLater(ClickCounter::new);
51  }
52 }
```

**Q11.** What does Java Applet not have that regular Java classes do? Which Applet method is the substitute for a constructor? Show the order of method calls from an Applet object by a web browser. What is the use of the paint() method? (15 marks)  
[CSE 201 | 2009–10]

**Answer — Applet vs Regular Class; Life-Cycle; paint()****What does a Java Applet not have?**

A Java Applet does **not** have a `main()` method. Regular Java classes use `main()` as the entry point; the JVM calls it to start execution. An Applet's lifecycle is instead controlled by the **browser/applet viewer**.

**Substitute for a constructor:**

The `init()` method serves as the substitute for a constructor. It is called *once* when the applet is first loaded and is used for initialisation tasks (setting up components, reading parameters). Constructors in applets should not be used for initialisation because the applet context is not fully set up when the constructor runs.

**Order of method calls by the browser:**

`init()` → `start()` → `paint()` → `stop()` → `destroy()`

- `stop()` and `start()` may be called multiple times (page hidden/revisited).
- `init()` and `destroy()` are called exactly once.

**Use of `paint(Graphics g)`:**

`paint()` is called by the AWT whenever the applet's display area needs to be (re)drawn — on first display, when the window is uncovered, or when `repaint()` is called programmatically. The `Graphics` object `g` provides methods to draw text, shapes, images, and colours onto the applet's surface.

**Q12.** Write down the name of any six components of the Java Swing package. **(6 marks)**  
[CSE 201 | 2008–09]

**Answer — Six Components of the Java Swing Package**

| Component  | Class                   | Purpose                                                 |
|------------|-------------------------|---------------------------------------------------------|
| Button     | <code>JButton</code>    | Clickable button that fires an <code>ActionEvent</code> |
| Text field | <code>JTextField</code> | Single-line text input                                  |
| Label      | <code>JLabel</code>     | Non-editable text or icon display                       |
| Check box  | <code>JCheckBox</code>  | Toggle on/off option                                    |
| Combo box  | <code>JComboBox</code>  | Drop-down selection list                                |
| Table      | <code>JTable</code>     | Displays data in rows and columns                       |

**Other notable Swing components:** `JTextArea`, `JRadioButton`, `JSlider`, `JSpinner`, `JList`, `JTree`, `JTabbedPane`, `JMenuBar`.

**Q13.** What do you know about Event Handling in Java? What is the name of the interface mainly used for this purpose? Write three different ways for a component to register itself to handle events using this interface, with necessary Java code. **(2+1+15 marks)**  
[CSE 201 | 2008–09]

**Answer — Event Handling in Java; Three Ways Using ActionListener****What is Event Handling?**

Event handling is the mechanism by which a Java program responds to user actions (button clicks, key presses, mouse movements). Java uses the **Delegation Event Model**: the event *source* delegates processing to a registered *listener* object. The **main interface** used for action events is `java.awt.event.ActionListener`, which declares one method:

```
1 void actionPerformed(ActionEvent e);
```

Three ways to register a component as its own handler using `ActionListener`:

**Way 1 — Separate (external) class**

```
1 class Handler implements ActionListener {
2     public void actionPerformed(ActionEvent e) {
3         System.out.println("Separate class handles it");
4     }
5 }
6 JButton b = new JButton("Click");
7 b.addActionListener(new Handler());
```

**Way 2 — The enclosing frame implements `ActionListener`**

```
1 class MyFrame extends JFrame implements ActionListener {
2     JButton b = new JButton("Click");
3     MyFrame() {
4         b.addActionListener(this); // 'this' is the listener
5         add(b); setSize(200,100); setVisible(true);
6     }
7     public void actionPerformed(ActionEvent e) {
8         System.out.println("Frame handles the event");
9     }
10 }
```

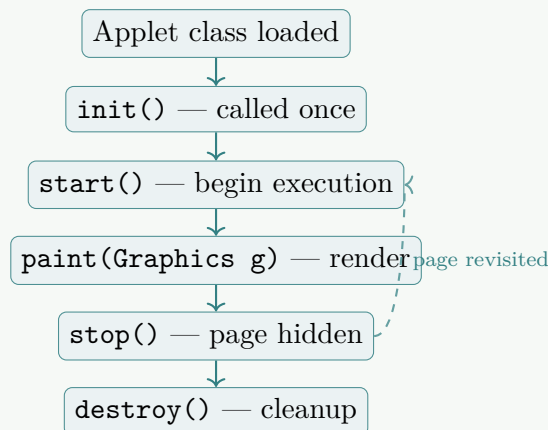
**Way 3 — Anonymous inner class (or lambda)**

```
1 JButton b = new JButton("Click");
2
3 // Anonymous inner class
4 b.addActionListener(new ActionListener() {
5     public void actionPerformed(ActionEvent e) {
6         System.out.println("Anonymous class handles it");
7     }
8 });
9
10 // Equivalent lambda (Java 8+)
11 b.addActionListener(
12     e -> System.out.println("Lambda handles it");
13 );
```

- Q14.** What are the differences between Java Applets and Java Applications? [2007–2008]
- Q15.** Draw the flow of control for a Java Applet. How can you pass parameters to a Java Applet? [2007–2008]

### Answer — Applet Flow of Control and Passing Parameters

**Flow of control (life-cycle):**



### Passing parameters to an Applet:

Parameters are passed via `<param>` tags in the HTML page and read with `getParameter(String name)` inside the applet.

```

1  // HTML
2  // <applet code="MyApplet.class" width="300" height="150">
3  //   <param name="color" value="blue">
4  //   <param name="message" value="Hello World">
5  // </applet>
6
7  import java.applet.*;
8  import java.awt.*;
9
10 public class MyApplet extends Applet {
11     private String color, message;
12
13     @Override
14     public void init() {
15         color = getParameter("color");
16         message = getParameter("message");
17         if (message == null) message = "No message";
18     }
19
20     @Override
21     public void paint(Graphics g) {
22         g.drawString("Color param: " + color, 20, 40);
23         g.drawString(message, 20, 60);
  
```

```
24     }  
25 }
```